Live Markdown

JONAH GAO

Contents

Contents 1

1. Analysis 5

1.1. Background 5

1.1.1 Markdown 5

1.1.2 Markdown Examples 6

1.2. Problem Area 6

1.3. Client and End User 6

1.4. Research Methodology 7

1.4.1 Client Survey 7

1.4.2 Existing Solutions 9

1.5. Features of Proposed Solution 10

1.5.1 Initial Concept 10

1.5.2 Limitations 11

1.6. Critical Path 11

1.6.1 Asynchronous SignalR Connections 11

1.6.2 Asynchronous UI Rendering in the Frontend 12

1.6.3 Asynchronous Task Execution in the Kernel 12

1.6.4 Kernel Isolation 12

1.6.5 Failure and Recovery 12

1.7. Problem Modelling 12

1.7.1 System Data Flow 12

1.7.2 State Transition Diagram for Code Blocks 13

1.7.3 Communication Model (SignalR) 13

1.8. Success Criteria 13

2. Design 15

2.1. High-Level Structure 15

2.2. Kernel Design 16
2.2.1 Language Choice 16
2.2.2 Program.cs 18
2.2.3 Services 18
2.2.4 SignalR Hubs 26
2.3. Frontend Design 29
2.3.1 Application Framework Choice 29
2.3.2 Frontend UI Framework Choice 30
2.3.3 Electron Main Process 31
2.3.4 Preload Context Bridge 32
2.3.5 Text Editor 33
2.3.6 Markdown Pipeline 33
2.4. Human-Computer Interaction (HCI) 37
3. Implementation 39
3.1. Project File Structure 40
3.2. Kernel 41
3.2.1 Program.cs 42
3.2.2 FileHub 44
3.2.3 Other Hub Implementation 49
3.2.4 FileService 49
3.2.5 FileWatcherService 61
3.3. Client 67
3.3.1 Service Connection 67
4. Source Code 68
electron-env.d.ts 68
electron\main.ts 70
preload.ts 77

splash.ts 79
ContextMenu.vue 80
Editor.vue 82
FileExplorer.vue 84
FileTreeNode.vue 91
MarkdownCodeBlock.vue 95
MarkdownIndentedCodeBlock.vue 98
NavigationBar.vue 99
Preview.vue 106
SearchPanel.vue 106
Splash.vue 110
TabsBar.vue 111
Terminal.vue 115
lexer.ts 123
markdown.ts 143
markdownVue3.ts 144
parser.ts 145
preprocessor.ts 146
renderer.ts 147
rendererVue3.ts 150
types.ts 152
fileService.ts 156
pythonService.ts 159
serviceConnection.ts 160
terminalService.ts 162
kernel.ts 163
workspace.ts 164

github-markdown.css 190
GithubTerminalTheme.json 190
styles.css 190
types\workspace.ts 203
App.vue 205
main.ts 209
shims-vue.d.ts 213
splash.ts 213
vite-env.d.ts 213
markdown.stress.test.ts 214
markdown.test.ts 216
tests\types.ts 216
electron-builder.json5 217
index.html 218
package.json 218
splash.html 219
tsconfig.json 219
tsconfig.node.ts 220
vite.config.ts 220
FileHub.cs 222
PythonHub.cs 227
TerminalHub.cs 229
launchSettings.json 230
FileService.cs 230
FileWatcherService.cs 246
PythonVenvRunner.cs 252
TerminalService.cs 258

LogFormatter.cs 263
appsettings.Development.json 264
appsettings.json 264
kernel.csproj 265
Program.cs 265
FileServiceTests.cs 268
FileWatcherServiceTests.cs 272
PythonVenvRunnerTests.cs 275
TerminalServiceTests.cs 278
TestDoubles.cs 281
kernel.Tests.csproj 284
5. Testing 285
6. Evaluation 290
6.1. What I Have Achieved 290
6.2. What Can Be Improved 290
6.3. User Feed Back 291
Use of AI 294
Bibliography 295
Appendix A – Source Code Repository 297

1. Analysis

1.1. Background

1.1.1 Markdown

Markdown is a lightweight markup language for creating formatted text using a plain-text editor. John Gruber created Markdown in 2004 as an easy-to-read markup language. Markdown is widely used for blogging, instant messaging, and large language models, and also

used elsewhere in online forums, documentation pages, and readme file. The several main variants of Markdown specifications are CommonMark [1], GitHub Flavoured Markdown (GFM) [2], and Obsidian Flavoured Markdown (OFM) [3].

1.1.2 Markdown Examples

Text using Markdown syntax	Corresponding HTML produced	Text viewed in a browser

`# Heading`
`## Sub-heading`
`Text in paragraphs can be formatted as`
`*italic*, **bold**, and`
`` `monospace` ``.
```` ``` ````
`a = "this is a code block"`
```` ``` ````
	`<h1>Heading</h1>`
	`<h2>Sub-heading</h2>`
	`<p>Text in paragraphs can be formatted as`
	`<em>italic</em>,`
	`<strong>bold</strong>, and`
	`<code>monospace</code>.</p>`
	`<pre><code>a = "this is a code block"</pre></code>`
		Heading
		Sub-heading
		Text in paragraphs can be formatted as
		italic, bold, and monospace.
		`a = "this is a code block"`

1.2. Problem Area

Although Markdown is commonly used to share technical knowledge, it is essentially just plain text: code snippets are treated as plain text rather than actual code. This means they are isolated from real execution environment. Developers who want to try out a snippet have to leave the current context (e.g. opening an IDE or terminal) because they need to copy the code, create a new file, configure the runtime/compiler, install dependencies and then run the code. These processes of switching context consume a lot of cognitive loads and interrupt the developer's workflow.

Therefore, Markdown editors cannot facilitate efficient rapid prototyping, teaching demonstrations or technical verification, as they do not relate what is written to real code execution. Learning and practice are separated.

1.3. Client and End User

The goal of this project was to bridge the gap between static documentation and live execution by providing a high-performance Markdown editor. This gives the application an IDE-like experience within the document, turning static code snippets into interactive, executable syntax-highlighted code blocks. Prioritising a responsive user interface and optimised throughput, this solution took a 'zero-configuration' approach, providing a sophisticated development environment without the need for installation or configuration.

The solution is designed for:

- Software Developers: Who need to test snippets while documenting APIs or logic.
- Technical Writers: Who want to ensure that the code examples provided in tutorials are functional and up to date.
- Educators and Students: Who require an environment where code explanation (Markdown) and execution live in one cohesive space for better teaching/learning process.

1.4. Research Methodology

1.4.1 Client Survey

To gauge users' interest in the project's potential features, I distributed a questionnaire to computer science students.

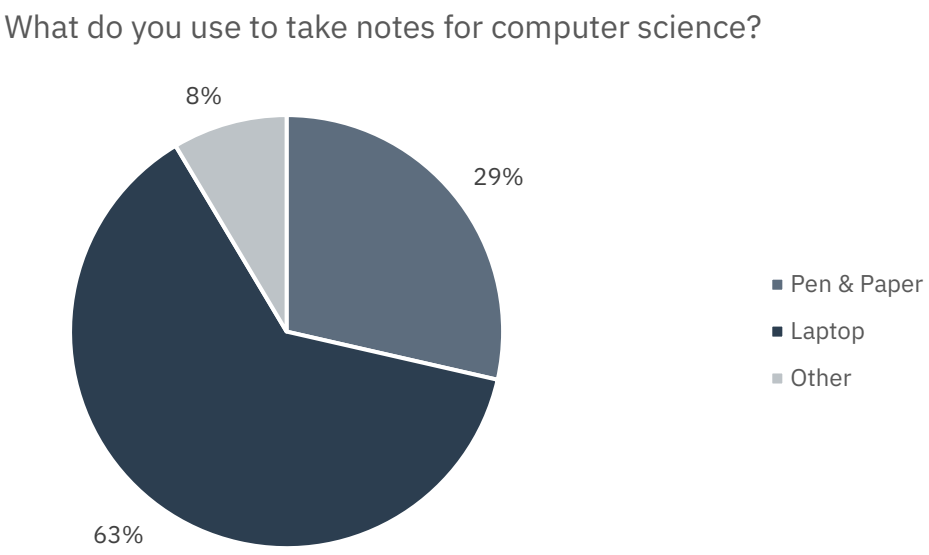


Figure 1.1 This chart illustrates the proportion of students taking notes using different devices.

From this, we can see that most students use laptops for taking notes. Therefore, my project will target laptop users.

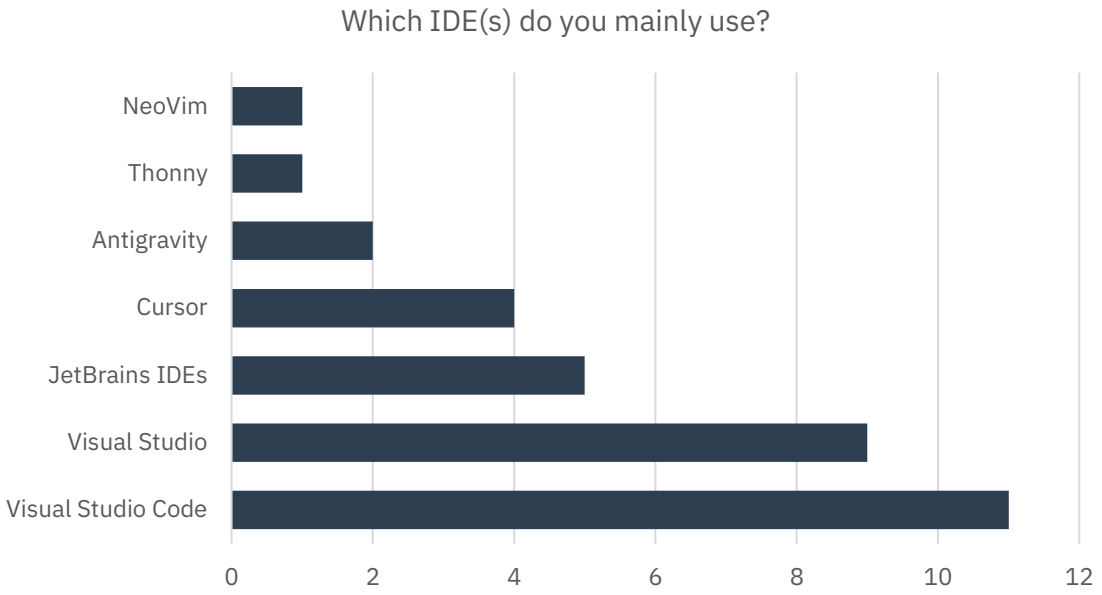


Figure 1.2 This chart illustrates the distribution of students using different IDEs for coding.

From this, we can see that the majority of students use IDEs from the Visual Studio Code [4] family. Therefore, the interaction logic of my project should align with that of Visual Studio Code.

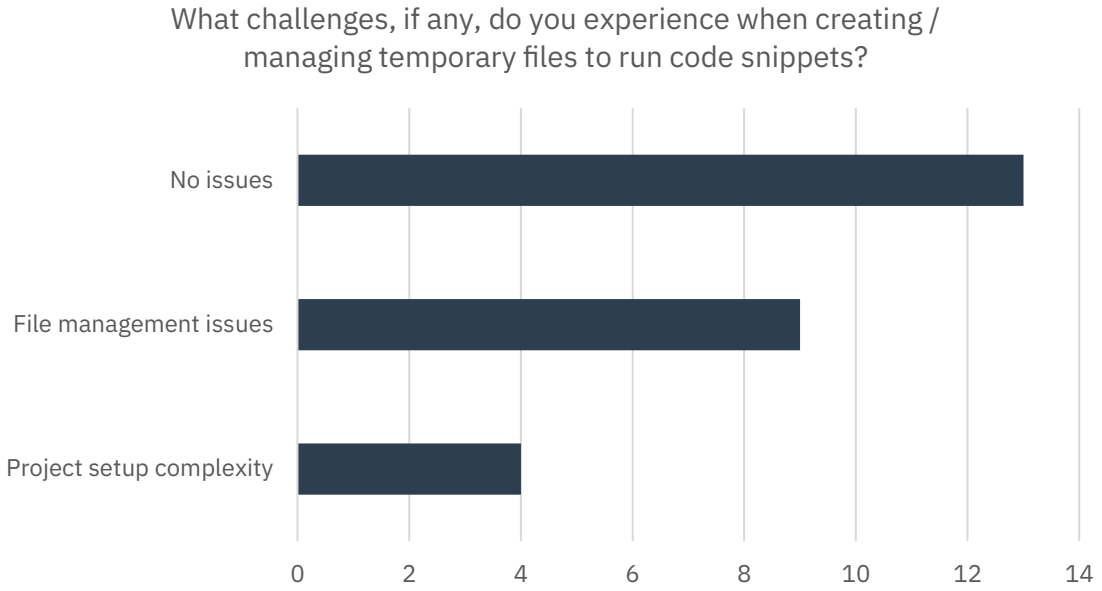


Figure 1.3 This diagram illustrates the distribution of issues encountered by students when managing code snippets.

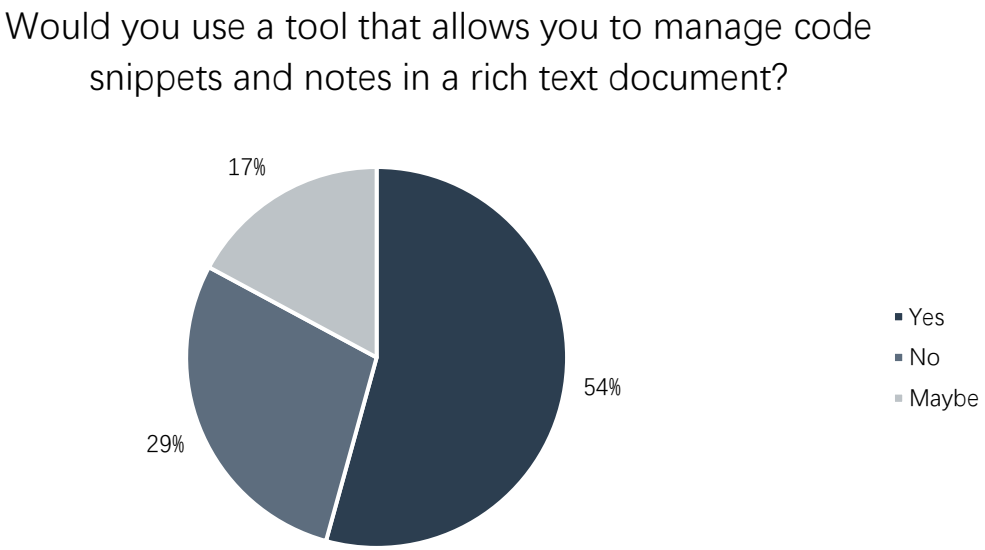


Figure 1.4 This chart illustrates the proportion of students intending to use software with code and note management capabilities.

It appears that a significant proportion of people struggle with managing code snippets, and the fact that most are willing to try this integrated note-and-code management solution proves there is market potential for my project.

1.4.2 Existing Solutions

Based on internet research, the following solutions are commonly used:

- Jupyter Notebook [5]
- JupyterLab [5]
- Obsidian with JupyMD [6]
- VS Code [4] with Codebraid Preview [7]

Solution	Markdown as main format	Markdown compatibility	Implementation model	Suitable scenarios
Jupyter		CommonMark [1]	Cell → Kernel	Research & Experiment
Obsidian with JupyMD	✓	GFM [2] / Obsidian MD [3]	Markdown → ipynb → Execute	Notes & Quick Execution
VS Code with Codebraid Preview	✓	Pandoc MD [8]	Markdown → ipynb → Execute	Notes & Quick Execution

As shown in the table, all of the above solutions share a common limitation: they all require

the use of a format other than Markdown as an intermediate layer before the code can be executed. So, in my project, I'm going to try removing this intermediate layer and connecting the Markdown directly to the code execution backend.

1.5. Features of Proposed Solution

1.5.1 Initial Concept

The solution will be based on a client-server architecture with a local execution backend. The frontend provides a Graphical User Interface (GUI) that supports IDE-style Markdown viewing, editing, and execution, while the backend kernel manages the lifecycle of code execution, file I/O operations, and terminal services. The SignalR [9] protocol will be used to establish real-time, bi-directional communication between the frontend and the kernel.

To achieve this, the application needs to have:

Kernel:

1. **SignalR Hub** – an interface exposed to the frontend to allow remote procedure call.
2. **Code Execution Engine –** manages code execution by automatically configuring execution environments, creating temporary files, executing code blocks, and streaming standard output and standard error to the frontend in real time.
3. **Terminal Service –** manages the lifecycle of pseudo-terminal instances, handles user input, and asynchronously streams standard output and standard error to the frontend in real time.
4. **File Service –** responsible for directory indexing, general file I/O operations, e.g. reading, writing, creating, deleting, renaming, etc.
5. **File Watcher Service** – responsible for managing multiple file watchers, merge and debounce file changes events in the directory.

Frontend:

1. **SignalR Connection Service** – manages the states and lifecycles of multiple connection instances.
2. **Markdown Parser –** parses Markdown documents into an abstract syntax tree (AST) and subsequently renders them into HTML using a custom Markdown renderer.
3. **Code Block Execution** – extracts a code snippet from a code block and embeds the resulting terminal output within the rendered document.

4. **File-tree Management** – performs atomic changes on file tree to keep the it up to date, also performs project-wide search.
5. **Terminal Rendering** – implements a pseudo-terminal interface that supports interactive command-line sessions and handles user input, displaying output asynchronously in real time.
6. **Code editor** – basic syntax highlighting and hinting.
7. **Last State Caching –** persists the user interface state using local storage. This includes panel sizes, file trees, expanded directories, opened files and editor layout. This enables the previous session to be restored.
8. **Tab Management –** maintains an in-memory cache of open tabs.
9. **Kernel Management** – manages life cycle of kernel process.

1.5.2 Limitations

However, after some investigation, it was found that some of the functionalities were excessively complex to complete within the time constraints of the coursework. Therefore, the following functionalities are simplified in this project:

Simplified:

1. **Markdown Parser** – Implementing full support for GitHub Flavoured Markdown [2] introduces significant technical complexity; therefore, the parser will support only a subset of the CommonMark [1] specification.
2. **Code Execution Engine** – Due to the differing environment setup requirements across programming languages, the execution engine will be limited to supporting Python [10] in order to reduce implementation complexity.
3. **Graphical User Interface (GUI)** – In consideration of the time-intensive nature of UI design, the project's primary objective is the implementation of a basic workspace GUI.

1.6. Critical Path

As outlined in Chapter 1.5.1 above, the application is subdivided into two constituent components: the frontend and the kernel. The frontend is responsible solely for user interface rendering, while the backend handles all business logic.

1.6.1 Asynchronous SignalR Connections

1. Investigate different sending methods in SignalR [9] and their effects.

2. Investigate differences between Fire-and-forget, Async RPC, Async Streaming.
3. Implement Data Transfer Objects in both frontend and kernel.
4. Automatically dispose the connection at the end of its lifecycle.
5. Allow user-initiated interruption without restarting the kernel.

1.6.2 Asynchronous UI Rendering in the Frontend

1. UI components render progressively when receiving streaming / async data, thereby avoiding blocking the UI rendering thread.
2. Non-critical UI components and data are loaded and rendered lazily to reduce unnecessary rendering overhead.

1.6.3 Asynchronous Task Execution in the Kernel

1. All tasks should be running asynchronously, thereby avoiding blocking the whole kernel thread.
2. Support cancellation tokens for all long-running kernel tasks.

1.6.4 Kernel Isolation

1. The kernel must run in a separate process to isolate execution from the frontend UI.
2. The kernel is accessed exclusively through a well-defined communication interface.
3. Kernel failures do not propagate to the frontend and are handled independently.

1.6.5 Failure and Recovery

1. Design explicit behaviour for kernel crash, hang, or partial failure, e.g. application should automatically reconnect if SignalR [9] is disconnected unexpectedly.
2. Distinguish recoverable vs non-recoverable kernel errors.
3. Support kernel hot-restart where feasible.

1.7. Problem Modelling

1.7.1 System Data Flow

The model below describes how data moves between the user and the different layers of the application:

- Level 0 (User Input): User inputs Markdown text → Frontend.
- Level 1 (Parsing): Frontend extracts code block → Kernel

- Level 2 (Execution): Kernel executes code → Frontend Output Display.



Figure 1.5 Data Flow Diagram of the application.

1.7.2 State Transition Diagram for Code Blocks

The following states are modelled for every executable block:

1. Idle: Default state with syntax-highlighted code.
2. Pending: User has initiated execution; code is being transmitted to the kernel.
3. Executing: The kernel is running the task; awaiting streamed output.
4. Stream Consumption: The frontend incrementally consumes and renders streamed execution output in its terminal.
5. Completed: Execution has finished successfully.
6. Cancelled: Execution was interrupted by the user.
7. Failed: Execution terminated due to an error.

1.7.3 Communication Model (SignalR)

The interaction is modelled as a Remote Procedure Call (RPC) and Streaming hybrid:

- Inbound (Frontend to Kernel): RPC-based requests, optionally carrying input data.
- Outbound (Kernel to Frontend): Streamed responses for incremental output, or RPC responses for discrete results.

1.8. Success Criteria

#	Description	Success Criteria
1	Async SignalR connections [9]	Frontend and kernel are able to receive multiple RPC / streams asynchronously.
2	Code execution engine	Code executed with correct interpreter / compiler; Stdout / Stderr returned as a stream.
3	Terminal service	Pseudo terminal functioning, stream output, supports multiples terminals.

4	File service	I/O operations functioning, file autosaving at set time intervals and before closing, live asynchronous directory indexing on change and supports project-wide search.
5	Markdown parser	Markdown text correctly parsed into html.
6	Markdown renderer	Markdown correctly rendered.
7	Directory file tree	The file tree of the directory is being built and updated correctly.
8	Code editor	Basic editing features functioning.
9	Tab management	Tabs can be opened / closed.
10	Code block	Code execution button functioning, a new terminal instance will be created to render Stdout.
11	Cancellation token	Code execution can be interrupted by user.
12	Last state caching	Restoration of the previous session UI state when relaunching the application.
13	Graphical user interface	All buttons functioning, able to open / close and resize the panels.

2. Design

2.1. High-Level Structure

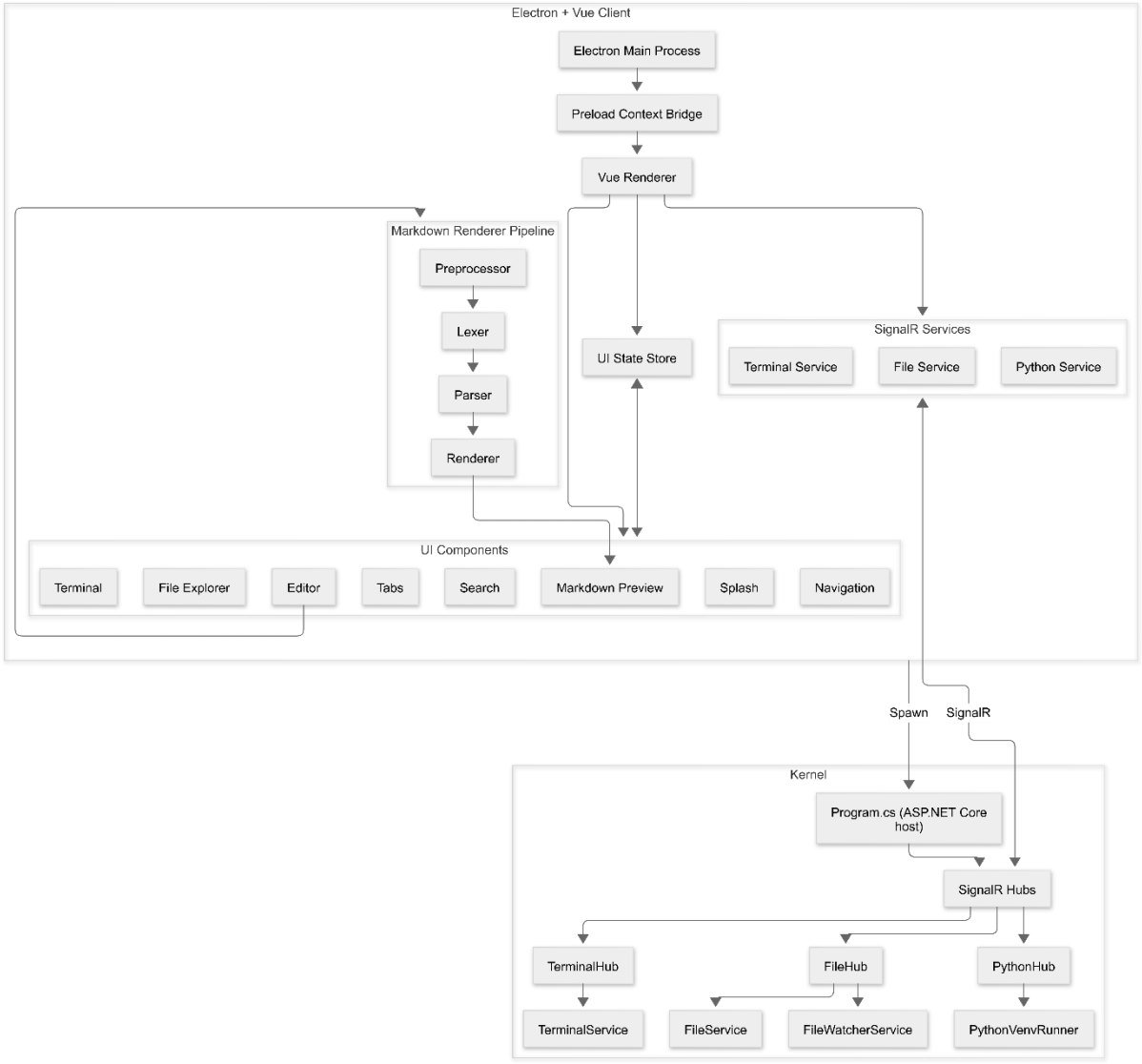


Figure 2.1 High-level architecture of the project.

Overall, the project could be split into two parts as shown in Figure 2.1: frontend and kernel. Frontend and kernel should be two isolated and independent processes, SignalR [9] will act as the bridge between them to allow data transmission and remote procedure call.

2.2. Kernel Design

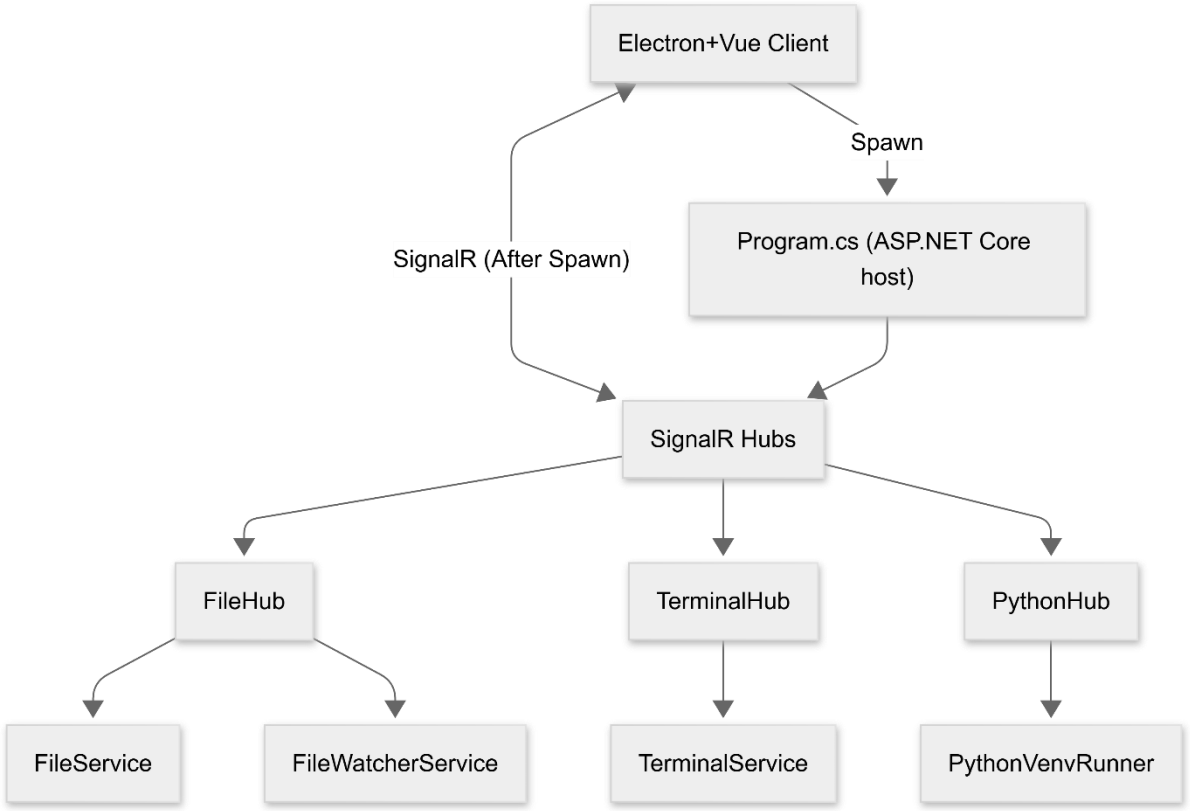


Figure 2.2 High-level architecture of the kernel.

2.2.1 Language Choice

The project is designed as a Windows-only application, which significantly narrows the range of programming language choices.

2.2.1.1. Python

Python is a widely used, high-level dynamically typed and strongly typed programming language with strong support for scripting and rapid prototyping. Python integrates well with the operating system through its standard library, making it suitable for tasks such as automation and web development.

However, as Python is an interpreted language without Just-in-Time (JIT) compilation in its standard implementation (CPython) [11], its runtime performance is generally lower than compiled languages like C++. Therefore, it is not suitable for implementing a high-performance kernel. The standard Python implementation employs a Global Interpreter Lock (GIL) to simplify memory management, which prevents true parallel threading on multi-core systems. As a result, Python does not meet the requirements of low runtime overhead, true

multi-threading demanded by this project.

2.2.1.2. Rust

Rust is a system programming language designed for high performance and strong memory safety guarantees without the use of a garbage collector. It provides true multi-threading and powerful asynchronous programming, making it well suited for low-level and performance-critical systems.

However, Rust's strict ownership and lifetime model with its async model introduces a steep learning curve and increases development complexity.

For this project, which prioritises rapid development and maintainability over low-level control, Rust is considered unsuitable due to its relatively high development difficulty.

2.2.1.3. C++

C++ is a mature system programming language that offers control over memory management and excellent runtime performance. It supports native multi-threading and asynchronous programming, which can be used to implement highly efficient low-level components.

Nevertheless, manual memory management without memory safety guarantees and the absence of a unified environment management significantly increase implementation complexity and development time.

As a result, C++ is not selected, as its low-level nature conflicts with the project's requirement for fast iteration.

2.2.1.4. Node.js

Node.js is a JavaScript runtime built on the V8 engine and is widely used for developing scalable, event-driven applications. It provides an efficient asynchronous I/O model and performs well in APIs and web services.

However, Node.js primarily relies on a single-threaded event loop for execution. Although worker threads are available, CPU-bound tasks cannot be parallelised efficiently without introducing additional complexity and overhead. Furthermore, the V8 engine that Node.js depends on, whose design prioritises execution speed over memory usage. Its complicated multi-tier JIT compilation and garbage collection result in a relatively high baseline memory overhead. Given the requirement for true multi-threaded execution and low memory usage in the kernel layer of this system, Node.js is not considered a suitable choice.

2.2.1.5. C#

C# is a Windows-native language within the .NET ecosystem, it provides first-class support for filesystem monitoring, process control, threading, and asynchronous I/O on Windows.

Modern .NET achieves high performance through JIT compilation and an optimised garbage collector, the Native AOT also allows it to compile applications into fully native executables at build time, enabling standalone deployment with faster startup and reduced runtime overhead.

As a statically typed language, C# delivers strong compile-time guarantees. Native async I/O features enable efficient asynchronous and parallel programming, making C# well suited for building a high-performance kernel layer on Windows.

2.2.1.6. Conclusion

Based on the evaluation above, C# is selected as the implementation language for this project. Its strong Windows native support and rich .NET ecosystem offers a balance between performance and development efficiency that best fits the requirements of this system.

2.2.2 Program.cs

In this desktop app, the C# 'kernel' is a standalone process. Program.cs serves as the entry point and lifetime owner of the kernel process. It's designed to configure Kestrel, CORS, JSON options, SignalR [9] hubs mapping, and logging.

2.2.3 Services

The service layer forms the operational core of the kernel process. Each service encapsulates a specific capability and manages all interactions.

2.2.3.1. FileService

FileService provides structured access to the filesystem, abstracting common file-related operations required by the application.

2.2.3.1.1. File Tree Traversal. In the Windows filesystem, nodes have two types: Directories or files. Therefore, a node can be represented as a record object as follows:

```
public record FileNode(
    string Name,
    string Path,
    string Extension,
    string ParentPath,
    bool IsDirectory,
    bool Expanded = false
);
```

Figure 2.3 record object of the file node

There are two common methods of tree traversal: breadth-first search (BFS) and depth-first search (DFS). From a user psychology perspective, users tend to exhibit a breadth-first navigation preference, favouring wider and shallower hierarchies over deep, nested structures. Therefore, BFS will be used for file tree traversal, as implemented in the `ReadDirAsync` function.

```
var queue = new Queue<string>();
queue.Enqueue(dirPath);

while (queue.Count > 0)
{
    var currentDir = queue.Dequeue();
    IEnumerable<FileSystemInfo> entries;
    entries = new DirectoryInfo(currentDir).EnumerateFileSystemInfos()
    foreach (var entry in entries)
    {
        yield return new FileNode(
            Name: entry.Name,
            Path: entry.FullName,
            Extension: entry is FileInfo file ?
                file.Extension : string.Empty,
            IsDirectory: entry is DirectoryInfo,
            ParentPath: currentDir,
            Expanded: false
        );

        if (entry is DirectoryInfo)
        {
            queue.Enqueue(entry.FullName);
        }
    }
}
```

Figure 2.4 BFS for file tree traversal.

2.2.3.1.2. Streaming File Content. When reading a file, returning only its raw content would result in the loss of essential metadata. Therefore, a `StreamTabAsync` function is introduced. This function invokes `StreamFileContentAsync` internally and returns a tab record stream in which file metadata is delivered first, followed by the file content streamed incrementally to the client.

```
public record TabChunk(
    string Id,
    string Name = "",
    string Path = "",
    string Content = "",
    string Encoding = "utf-8"
    bool IsMetadata = false,
    bool IsError = false,
);
```

Figure 2.5 record object of the tab

2.2.3.1.3. Manage Workspace Settings. In order to restore the workspace state after relaunching the application, the workspace settings need to be saved as a JSON object when exiting, and these settings need to be loaded as a record object when launching the application again. Therefore, the application will require two functions: `SaveWorkspaceSettingsAsync` and `LoadWorkspaceSettingsAsync`.

```
public enum ViewMode
{
    Code,
    Split,
    Preview
}


public enum SidebarPanel
{
    Explorer,
    Search,
    Terminal
}

public record PanelLayout(
    double ExplorerWidth,
    double TerminalHeight,
    double EditorPreviewRatio,
    ViewMode PreferredViewMode,
    string[] OpenedFiles,
    string[] ExpandedDirectories,
    string ActiveFile = "",
    bool ExplorerVisible = true,
    bool SearchVisible = false,
    bool TerminalVisible = false,
    SidebarPanel? ActiveTopPanel = SidebarPanel.Explorer,
    SidebarPanel? ActiveBottomPanel = null
);
```

Figure 2.6 record object of the workspace settings

2.2.3.1.4. Other File I/O Operations. For security reasons, the client must not directly modify files. Consequently, all file input and output operations must be implemented through encapsulated functions.

In Windows, file I/O operations primarily include opening or creating files, reading, writing, and closing files. Higher-level file system operations include renaming, moving, copying, saving and deleting files. Because of C#'s `using(…)` clause and built-in functions automatically handle file closure, an explicit file close function is not implemented in

`FileService.cs`. Additionally, since file and directory operations follow different procedures, each operation is implemented using dedicated functions for files and directories.

The list of functions are as follows:

- `CreateDirectory`
- `CreateFile`
- `DeleteDirectory`
- `DeleteFile`
- `MoveDirectory`
- `MoveFile`
- `CopyDirectory`
- `CopyFile`
- `SaveFileAsync`

In conclusion, the `FileService.cs` will contain a class with the following structures:

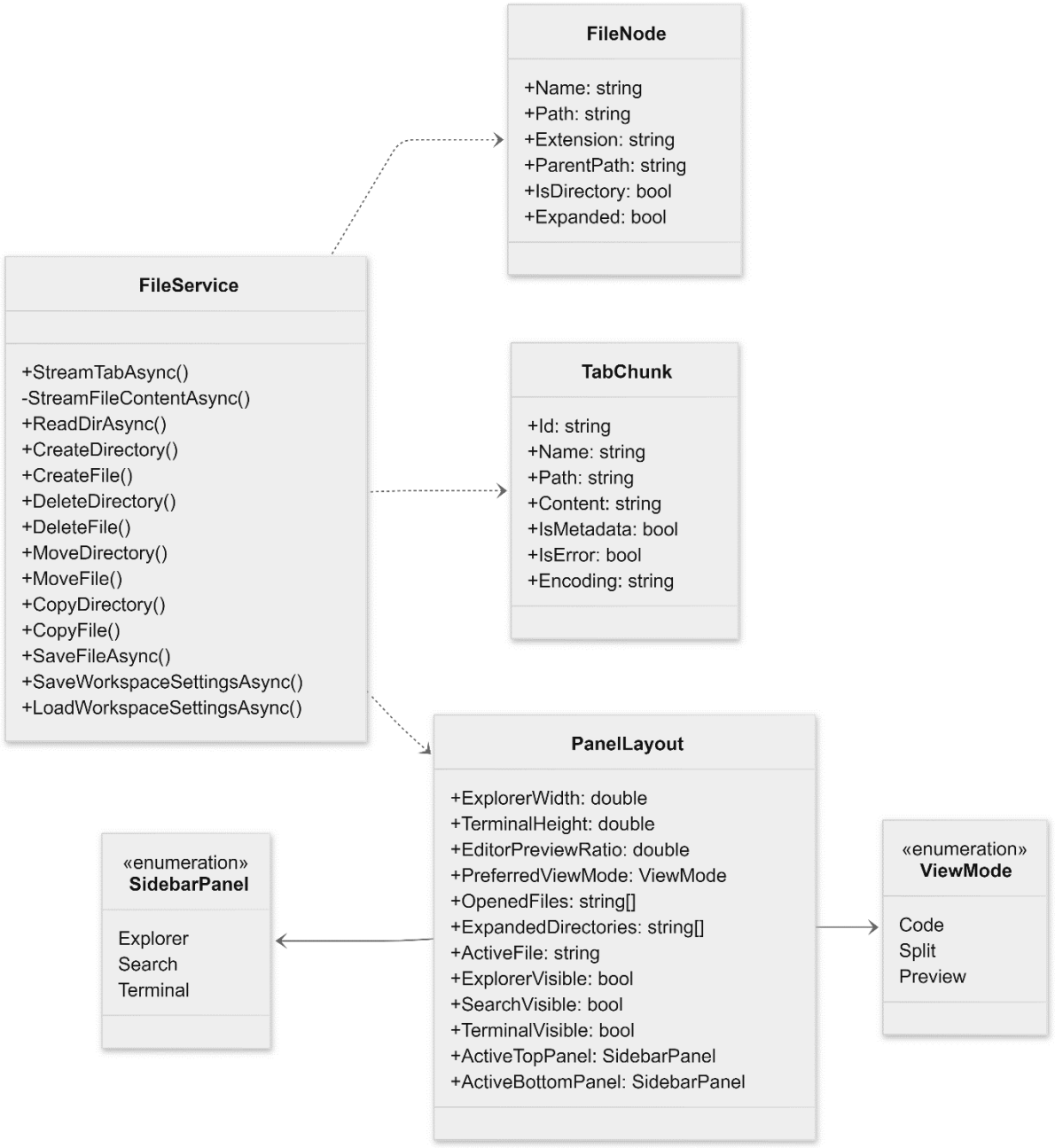


Figure 2.7 UML class diagram of the FileService.cs

2.2.3.2. FileWatcherService

2.2.3.2.1. Watcher Initialisation. Before filesystem events can be monitored, the watcher must be initialised. Therefore, the service provides two dedicated functions, `StartWatching` and `StopWatching`, to control the lifecycle of the watcher.

Since this service only needs to detect changes in the structure of the file tree, the watcher is configured to monitor only four types of filesystem events: renaming, creation, deletion, and error events.

Considering that multiple watchers may be required for different directories, the service maintains a dictionary `_watchers` to map directory paths to their corresponding watchers, as well as another dictionary `_watcherRefCounts` to track how many processes are currently using each watcher.

2.2.3.2.2. Filesystem Events Handling. To handle filesystem events, the service defines dedicated handler functions for each event type, which are `onCreated`, `onDeleted`, `onRenamed`, and `onError`. Because the service is designed to return the paths of dirty directories, the function `MarkDirectoryDirty` is implemented for invocation by the filesystem event handlers.

Due to the way Windows reports low-level file system changes, a single file modification can result in multiple filesystem events being emitted. Therefore, the `MarkDirectoryDirty` function also acts as a normalisation layer that collects discrete filesystem events and performs debouncing and merging.



Figure 2.8 UML class diagram of the FileWatcherService

2.2.3.3. TerminalService

Terminal sessions are usually stateful, interactive, and long-lived. `TerminalService` manages interactive shell sessions using pseudo-terminals (PTYs), ensuring that their lifecycle and instance resources are automatically released.

Considering that there might be multiple terminals in the project, the service maintains a `_terminals` dictionary to track terminal instances and also a `_cts` dictionary to hold the

`CancellationTokenSource` for terminals.

The class has four functions that handles most of the terminal operations, `StartTerminalAsync`, `WriteInputAsync`, `Resize`, and `KillTerminal`.

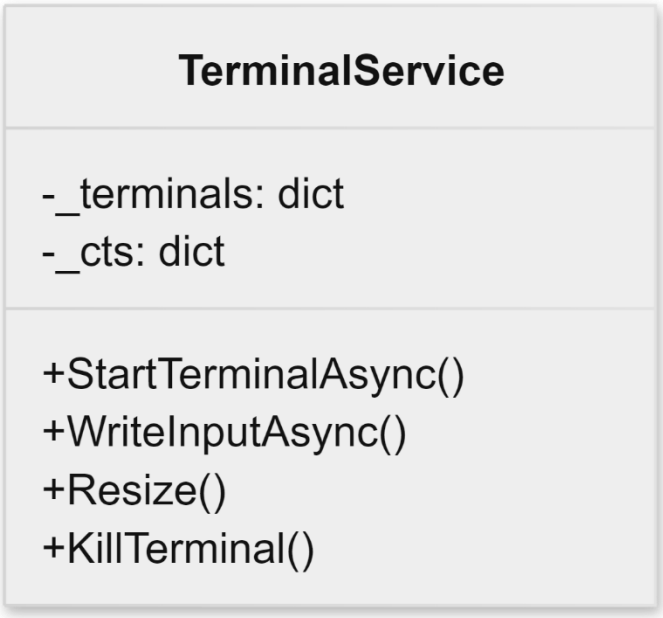


Figure 2.9 UML class diagram of the TerminalService

2.2.3.4. PythonVenvRunner

To let the code within the code block executable, we require a Python service to manage the virtual environment, handle temporary files, execute the code, and stream output to the client.

The structure of this service is similar to that of `TerminalService`, as `Python.exe` is essentially a command-line program.

In addition, the service provides the `GetVenvPythonExecutable` and functions for virtual environment management, as well as the `ExecuteCodeAsync` and `RunProcessAsync` functions for creating temporary files and executing code.



Figure 2.10 UML class diagram of PythonVenvRunner

2.2.4 SignalR Hubs

SignalR [9] Hubs act as explicit boundaries between the Electron client and the kernel's internal logic. They are responsible for:

- Receiving requests from the client
- Translating requests into calls to internal services
- Returning results or streaming data back to the client

Crucially, hubs do not contain core business logic. Instead, they control the bidirectional connection network. Although hubs run inside the kernel process, they are not the kernel itself. The kernel process owns the lifecycle and Hubs are instantiated either per connection or per invocation, ensuring that kernel state is not accidentally tied to the transient lifetime of a hub instance.

2.2.4.1. FileHub

FileHub acts as an interface layer between the client and the `FileService`. It maintains a dictionary `ConnectionWatchedDirs` that maps client identifiers to the directory paths they are watching.

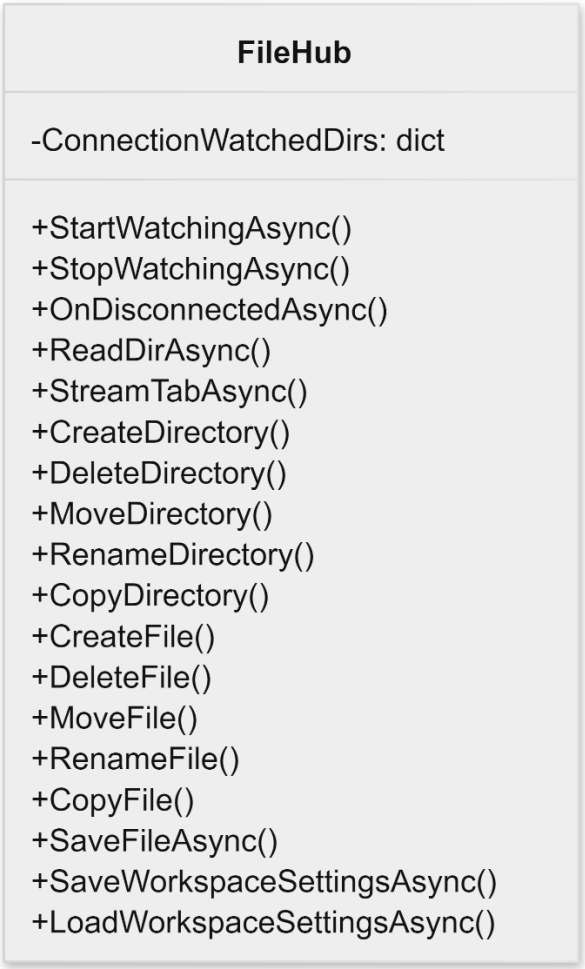


Figure 2.11 UML class diagram of the FileHub

2.2.4.2. PythonHub

`PythonHub` acts as an interface between the client and the `PythonVenvRunner`, exposing encapsulated functions to the client.

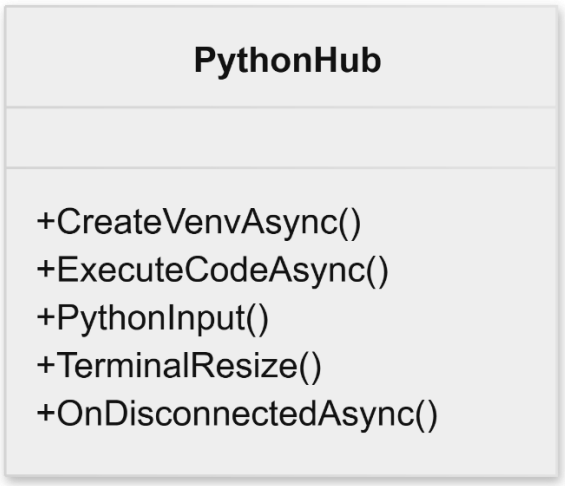


Figure 2.12 UML class diagram of the PythonHub

2.2.4.3. TerminalHub

Similar to `PythonHub`, `TerminalHub` acts as an interface that exposes encapsulated functions.



Figure 2.13 UML class diagram of the TerminalHub

2.3. Frontend Design

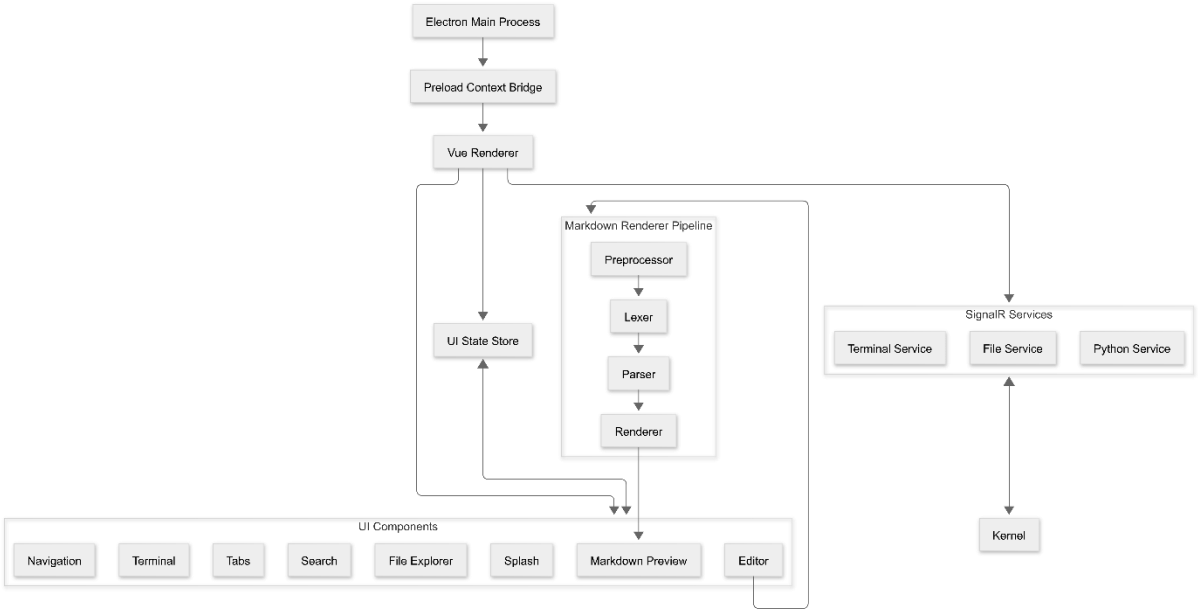


Figure 2.14 High-level structure of the client.

2.3.1 Application Framework Choice

2.3.1.1. WPF

WPF is a mature desktop application framework within the .NET ecosystem, providing deep integration with Windows and strong support for rich user interfaces. While it offers high performance and native access to system resources, its UI development model is relatively rigid and less suited to rapidly evolving, highly interactive interfaces.

2.3.1.2. Qt

Qt is a cross-platform C++ framework widely used for building native desktop applications. It provides excellent performance and control over system resources. However, its reliance on C++ increases development complexity and slows iteration speed, which conflicts with the project's requirement for rapid development and maintainability.

2.3.1.3. Flutter

Flutter is a cross-platform UI framework that renders its own UI using a custom graphics engine with good performance. However, Flutter introduces an additional rendering layer and a less mature desktop ecosystem compared to Electron, which increases integration complexity for system-level desktop applications.

2.3.1.4. Electron

Electron enables desktop application development using web technologies by embedding Chromium and Node.js. It provides excellent cross-platform compatibility and allows reuse of web development tooling. However, Electron introduces higher memory usage due to its bundled runtime, which must be carefully considered in system design.

2.3.1.5. Conclusion

Based on the evaluation above, Electron is selected as the application framework for this project. Electron provides a stable and widely adopted desktop runtime with strong integration capabilities. Despite Its higher memory usage, its development efficiency and ecosystem maturity outweigh the drawbacks.

2.3.2 Frontend UI Framework Choice

2.3.2.1. React

React is a widely adopted UI library with a large ecosystem and strong community support. However, its reliance on additional libraries for state management and routing introduces architectural fragmentation and increases maintenance complexity for medium-sized desktop applications.

2.3.2.2. Angular

Angular is a comprehensive frontend framework providing a full-featured solution with built-in tooling and strong TypeScript integration. While powerful, its steep learning curve and heavy framework structure impose a higher architectural overhead than required for this project.

2.3.2.3. Vue 3

Vue 3 provides a lightweight yet expressive UI framework with a clear and consistent architecture. Its API offers flexible logic organisation while maintaining readability, and its tight integration with TypeScript improves maintainability. These characteristics make Vue 3 well-suited for building complex interfaces with relatively low development overhead.

2.3.2.4. Conclusion

Based on the evaluation above, Electron combined with Vue 3 and TypeScript is selected as the frontend framework stack. This combination provides a balance between development efficiency, architectural clarity, and long-term maintainability.

2.3.3 Electron Main Process

Electron Main Process (`main.ts`) acts as the coordinator, managing the application lifecycle, creating native windows, and launching external .NET/C++ kernels.

As the kernel is a standalone programme, we require functions to spawn `kernel.exe` and manage its lifecycle.

```
function startKernel(): void {
    if (kernelProcess) return;

    const kernelDir = path.join(process.resourcesPath, '.kernel')
    const kernelPath = path.join(kernelDir, 'kernel.exe')

    kernelProcess = spawn(kernelPath, [], {stdio: 'pipe', cwd: kernelDir})

    const onData = (chunk: Buffer) => {
        const msg = chunk.toString()
        const portMatch = msg.match(/SIGNALR_PORT=(\d+)/)
        if (portMatch) {
            kernelPort = parseInt(portMatch[1], 10)
            kernelProcess!.stdout?.off('data', onData)
        }

    }

    kernelProcess.stdout?.on('data', onData)

    kernelProcess.on('exit', (code, signal) => {
        const wasRunning = kernelPort !== null
        kernelProcess = null
        kernelPort = null

        if (wasRunning && mainWindow && !mainWindow.isDestroyed()) {
            setTimeout(() => startKernel(), 1000)
        }
    })
}

function stopKernel(): void {
    if (kernelProcess) {
        kernelProcess.kill()
        kernelProcess = null
        kernelPort = null
    }
}
```

Figure 2.15 Functions to spawn / kill kernel process.

2.3.4 Preload Context Bridge

Preload Script (`preload.ts`): Acts as a secure gateway. It uses `contextBridge` to expose specific, restricted Node.js functionality to the Renderer without compromising security.

```
contextBridge.exposeInMainWorld("nodePath", {
    join: (...args: string[]): string => path.join(...args),
    dirname: (p: string): string => path.dirname(p),
    basename: (p: string, ext?: string): string => path.basename(p, ext)
})
```

Figure 2.16 An example of the contextBridge in the preload script.

2.3.5 Text Editor

Creating a text editor is challenging because text editing involves more than just typing characters on the screen. A real editor must manage text efficiently, display it correctly and respond instantly to user input. It must also be able to handle very large files without slowing down. Even simple actions such as moving the cursor or inserting a character can become complicated when the file is large.

One issue is how text is stored in memory. If the editor stores everything as one long string, the time complexity of inserting or deleting text is O(n), which is not ideal for large documents. Therefore, real editors use special data structures to enable fast editing.

Another issue is displaying the text on the screen. The editor must calculate where each character should appear and handle line wrapping, virtual scrolling and different font widths. To improve performance, editors usually only draw the lines that are visible on the screen, which makes the rendering system more complicated.

Therefore, to complete the project on time, I will use the Monaco Editor [12], which is easy to use and also provides all the features of a modern text editor.

2.3.6 Markdown Pipeline



Figure 2.17 Structure of the Markdown pipeline.

Unlike a General-Purpose Language (GPL) like C, Markdown is a declarative markup language. It lacks the complexities of variable scoping, type systems, or memory management. Consequently, the pipeline is optimized by removing the Semantic Analysis phase and shifting structural identification into Lexer.

According to the CommonMark specification, Markdown elements can be categorised into 3

categories: Container Block, Leaf Block, and Inline.

A Container Block is a block element that can contain other block elements; a Leaf Block is a block element that cannot contain other block elements, it can only contain inline elements; Inline elements can only exist inside a block element, and Inline elements can contain other Inline elements.

To improve development efficiency, the Lexer uses a hybrid of regular expression and state machine architecture. Regular expressions recognise the start of block elements and parse inline elements. The state machine scans the string linearly to parse block elements.

```
const containerBlockRules: Array<[TokenType, RegExp]> = [...]
const leafBlockRules: Array<[TokenType, RegExp]> = [...]
const inlineRules: Array<[TokenType, RegExp]> = [...]

function parseContainerBlock(input: string): Token[] {
    ...
    createToken(...)
    return tokens
}

function parseLeafBlock(input: string): Token[] {
    ...
    createToken(...)
    return tokens
}

function parseInline(input: string): Token[] {
    ...
    createToken(...)
    return tokens
}

function createToken(type: TokenType, ...): Token {
    switch (type) {
        case ContainerBlock:
            return parseContainerBlock(...)
        case LeafBlock:
            return parseLeafBlock(...)
        case Inline:
            return parseInline(...)
    }
}
```

Figure 2.18 Pseudo code of the high-level structure of the Lexer.

2.3.6.1. Stage 1: Preprocessing

The Preprocessor standardises the input string to ensure the Lexer operates on a predictable character set by unifying line endings (CRLF to LF), removing leading and ending empty lines, expanding tabs to spaces and replacing U+0000 character with U+FFFD character.

2.3.6.2. Stage 2: Lexical Analysis

In a traditional compiler, the Lexer only identifies words. In this Markdown pipeline, because there is no Semantic Analysis phase, the Lexer is designed to be “Heavy” - performing much of the work typically reserved for a Parser.

- **Pattern-Based Contextualization:** The Lexer doesn't just find symbols, it identifies Container Block Contexts, Leaf Block Contexts and Inline recursively in order
- **Token Emission:** The Lexer does not emit symbol tokens, but directly emits parsed rich tokens.

```
interface BaseToken<T extends TokenType = TokenType> {
    readonly type: T;
    raw: string;        // The raw Markdown string for this token
    text?: string;      // The text content of the token
    tokens?: Token[];   // Nested tokens
}
```

Figure 2.19 The fundamental structure shared by all tokens.

```
const enum TokenType {
    BODY = 'body',
    HEADING = 'heading',
    PARAGRAPH = 'paragraph',
    CODE_BLOCK = 'code_block',
    INDENTED_CODE_BLOCK = 'indented_code_block',
    CODE_INLINE = 'code_inline',
    BOLD = 'bold',
    ITALIC = 'italic',
    BOLD_ITALIC = 'bold_italic',
    LINK = 'link',
    AUTOLINK = 'autolink',
    IMAGE = 'image',
    ULIST = 'ulist',
    OLIST = 'olist',
    LIST_ITEM = 'list_item',
    BLOCKQUOTE = 'blockquote',
    HR = 'hr',
    ESCAPE = 'escape',
    TEXT = 'text',
    NEWLINE = 'newline',
    SOFTBREAK = 'softbreak',
    HARDBREAK = 'hardbreak',
    HTML = 'html',
}
```

Figure 2.20 Token types used by the Lexer.

2.3.6.3. Stage 3: Parsing

As the “Heavy Lexer” already emits rich tokens, the parser need only recursively expand these tokens, extract their properties and data, assemble them into an Abstract Syntax Tree (AST), and then pass this to the renderer.

2.3.6.4. Stage 4: Rendering

The Renderer traverses the AST using a Depth-First Search (DFS) to generate the final output.

- Each node in the AST triggers a specific rendering function (e.g., `renderBold`, `renderEmph`).
- Since the structure was already validated by the Heavy Lexer and the Light Parser, the Renderer can operate deterministically, ensuring that the generated HTML is always well-formed.
- The HTML block is sanitised here to prevent cross-site scripting (XSS), ensuring that the final markup is safe for the browser to execute.

2.4. Human-Computer Interaction (HCI)

As mentioned in Chapter 1.4.1, the user interface and interaction logic of my application should be aligned with those of the VS Code family of IDEs. This chapter provides screenshots of the UI to demonstrate how this have been achieved.



Figure 2.21 Icon of the application.



Figure 2.22 Splash window of the application.

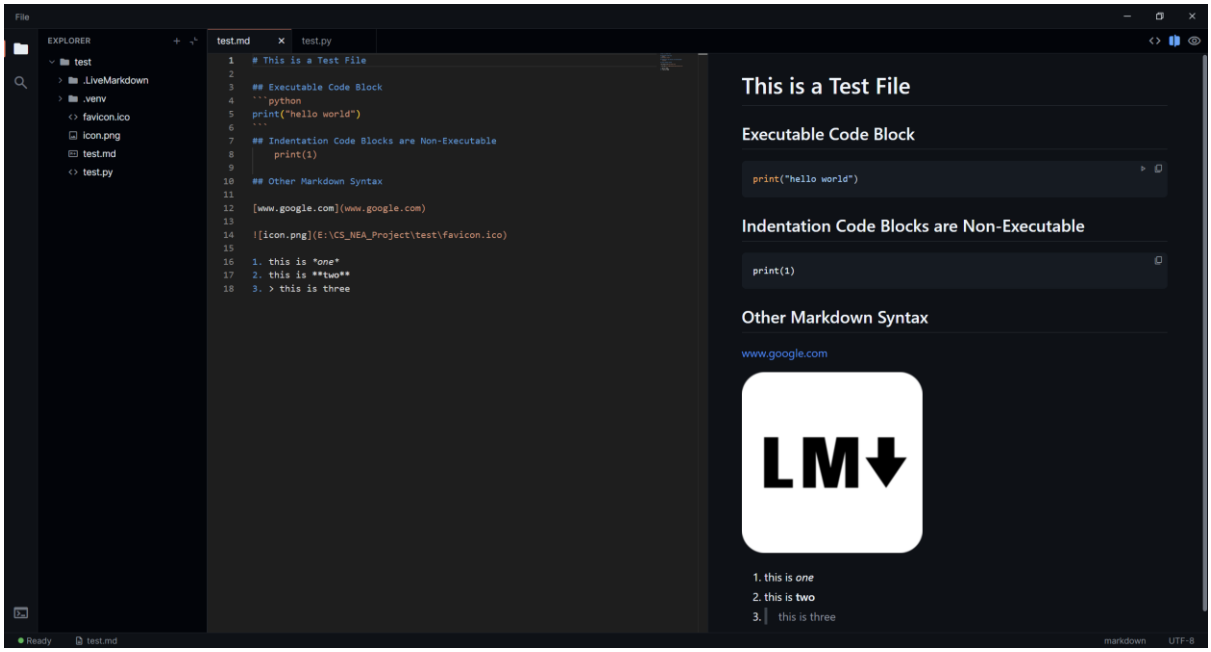


Figure 2.23 Main window of the application.

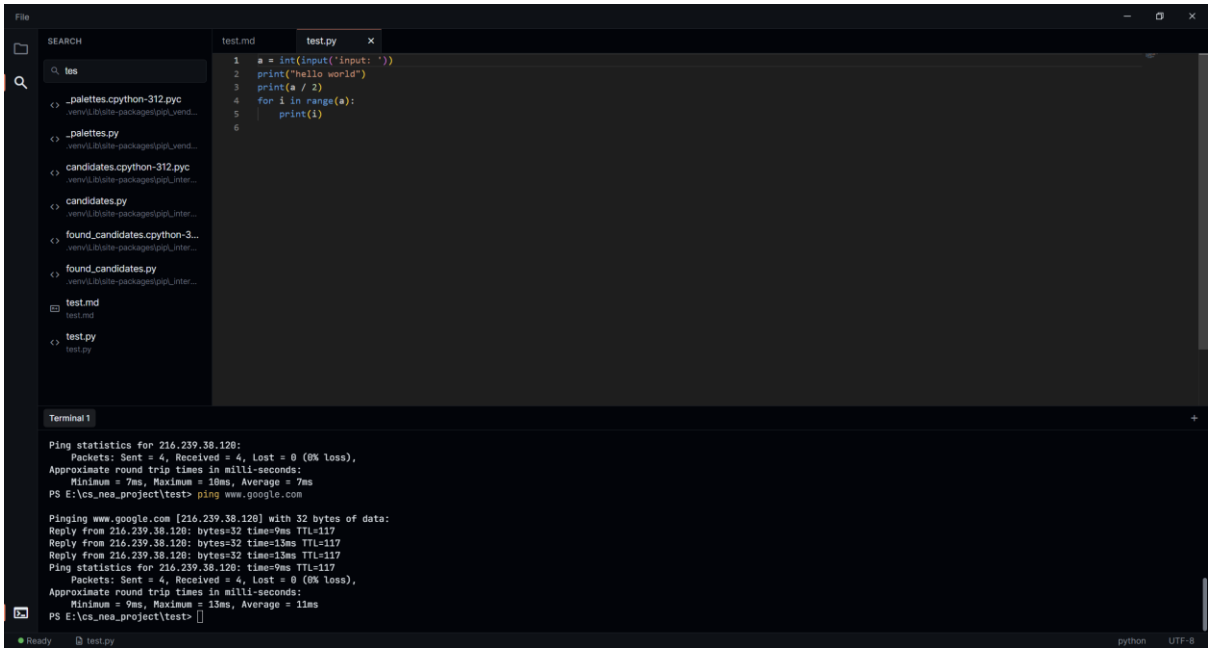


Figure 2.24 Main window of the application.

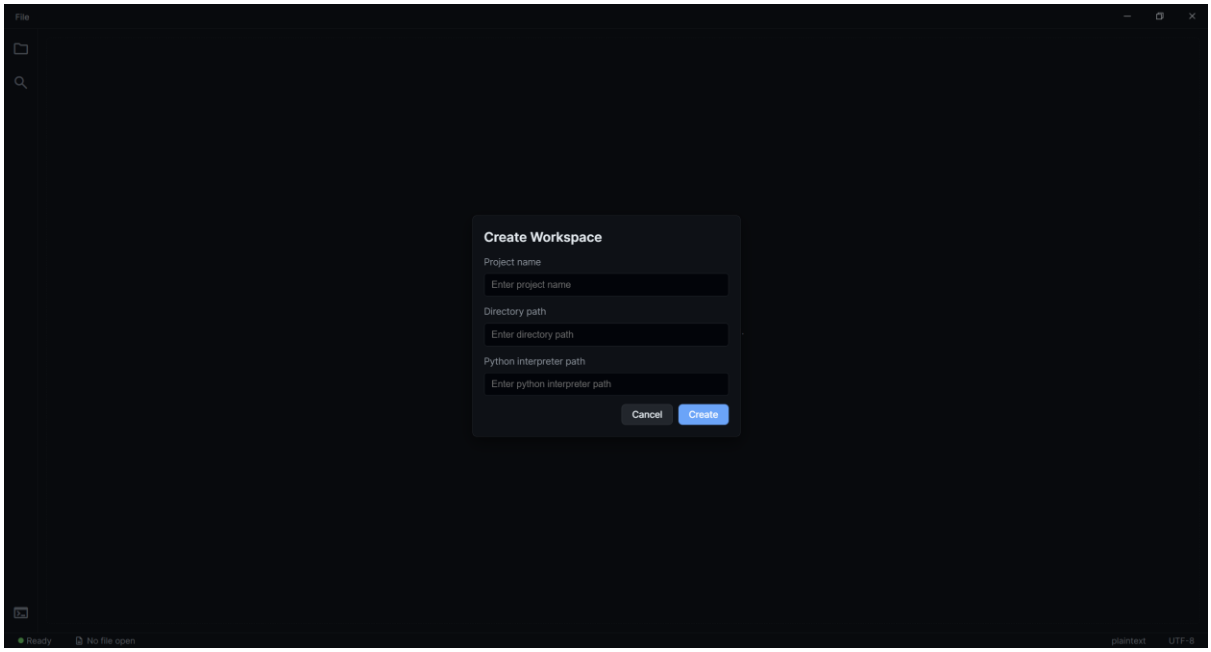


Figure 2.25 Main window of the application.

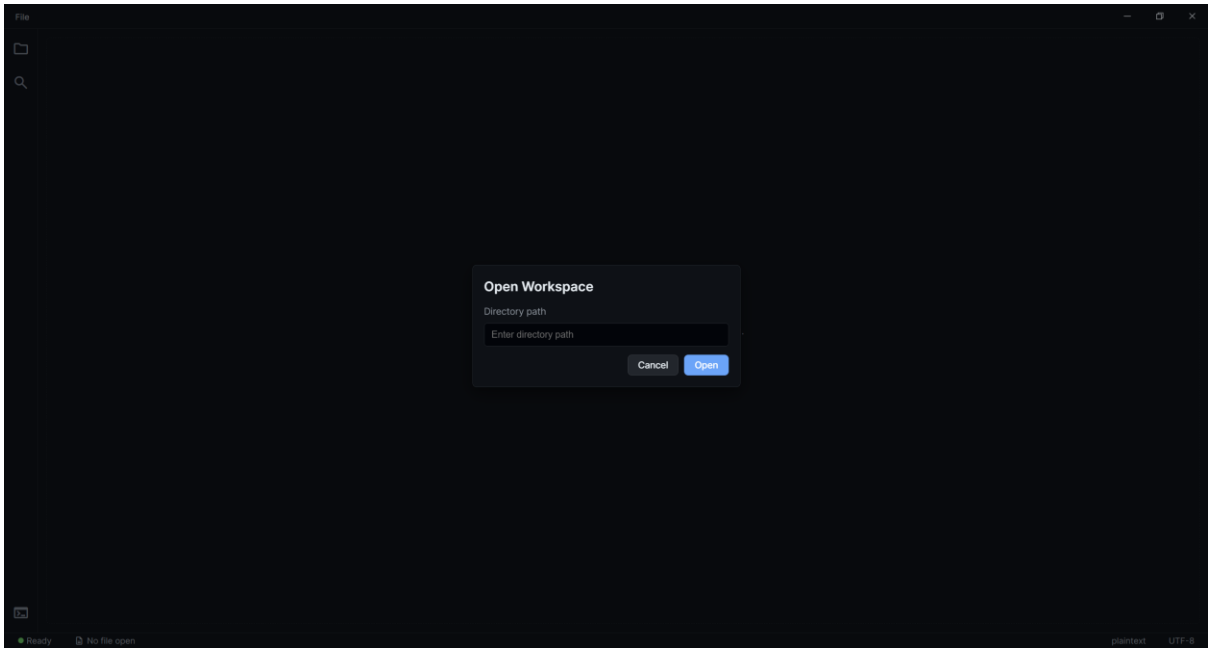


Figure 2.26 Main window of the application.

3. Implementation

The Implementation section only displays complicated code; simpler or self-explanatory code will be found in the source code section.

3.1. Project File Structure

```
.
├── client/
│   ├── electron/
│   │   ├── electron-env.d.ts
│   │   ├── main.ts
│   │   ├── preload.ts
│   │   └── splash.ts
│   ├── public/
│   │   ├── electron-vite.animate.svg
│   │   ├── electron-vite.svg
│   │   ├── favicon.ico
│   │   └── vite.svg
│   ├── src/
│   │   ├── assets/
│   │   │   ├── icon.png
│   │   │   ├── icon.svg
│   │   │   ├── splash.png
│   │   │   └── splash2x.png
│   │   ├── components/
│   │   │   ├── ContextMenu.vue
│   │   │   ├── Editor.vue
│   │   │   ├── FileExplorer.vue
│   │   │   ├── FileTreeNode.vue
│   │   │   ├── MarkdownCodeBlock.vue
│   │   │   ├── MarkdownIndentedCodeBlock.vue
│   │   │   ├── NavigationBar.vue
│   │   │   ├── Preview.vue
│   │   │   ├── SearchPanel.vue
│   │   │   ├── Splash.vue
│   │   │   ├── TabsBar.vue
│   │   │   └── Terminal.vue
│   │   ├── lib/
│   │   │   └── markdown/
│   │   │       ├── lexer.ts
│   │   │       ├── markdown.ts
│   │   │       ├── markdownVue3.ts
│   │   │       ├── parser.ts
│   │   │       ├── preprocessor.ts
│   │   │       ├── renderer.ts
│   │   │       ├── rendererVue3.ts
│   │   │       └── types.ts
│   │   ├── services/
│   │   │   ├── fileService.ts
│   │   │   ├── pythonService.ts
│   │   │   ├── serviceConnection.ts
│   │   │   └── terminalService.ts
│   │   ├── stores/
│   │   │   ├── kernel.ts
│   │   │   └── workspace.ts
│   │   ├── styles/
│   │   │   ├── GithubTerminalTheme.json
│   │   │   ├── github-markdown.css
│   │   │   └── style.css
│   │   ├── types/
│   │   │   └── workspace.ts
```

```
│   │   ├── App.vue
│   │   ├── main.ts
│   │   ├── shims-vue.d.ts
│   │   ├── splash.ts
│   │   └── vite-env.d.ts
│   ├── tests/
│   │   ├── fixtures/
│   │   │   ├── Les Miserables.md
│   │   │   ├── commonmark-0.31.2-filtered.json
│   │   │   └── general-stress-test.md
│   │   ├── markdown.stress.test.ts
│   │   ├── markdown.test.ts
│   │   └── types.ts
│   ├── README.md
│   ├── electron-builder.json5
│   ├── index.html
│   ├── package-lock.json
│   ├── package.json
│   ├── splash.html
│   ├── tsconfig.json
│   ├── tsconfig.node.json
│   └── vite.config.ts
├── kernel/
│   ├── kernel/
│   │   ├── Hubs/
│   │   │   ├── FileHub.cs
│   │   │   ├── PythonHub.cs
│   │   │   └── TerminalHub.cs
│   │   ├── Properties/
│   │   │   └── launchSettings.json
│   │   ├── Services/
│   │   │   ├── FileService.cs
│   │   │   ├── FileWatcherService.cs
│   │   │   ├── PythonVenvRunner.cs
│   │   │   └── TerminalService.cs
│   │   ├── Utils/
│   │   │   └── LogFormatter.cs
│   │   ├── Program.cs
│   │   ├── appsettings.Development.json
│   │   ├── appsettings.json
│   │   ├── kernel.csproj
│   │   └── kernel.http
│   ├── kernel.Tests/
│   │   ├── Services/
│   │   │   ├── FileServiceTest.cs
│   │   │   ├── FileWatcherServiceTests.cs
│   │   │   ├── PythonVenvRunnerTests.cs
│   │   │   ├── TerminalServiceTests.cs
│   │   │   └── TestDoubles.cs
│   │   └── kernel.Tests.csproj
│   └── kernel.sln
└── LICENSE
```

3.2. Kernel

This project is using the .NET 9.0 framework and C# 13.0 language version.

3.2.1 Program.cs

The following streamlined code demonstrates the configuration of the SignalR [9] protocol, JSON serialisation settings, and server initialisation with logging. The complete source code is provided in the source code section.

```
// Application entry point and configuration.
// - Configures minimal web app, CORS and SignalR hubs
// - Sets up console logging with timestamps and colors
// - Maps SignalR hub endpoints
var builder = WebApplication.CreateBuilder(args);
// If no URL is configured, bind Kestrel to an ephemeral local port.
var urlsConfigured = builder.Configuration["urls"] ??
builder.Configuration["ASPNETCORE_URLS"];
if (string.IsNullOrWhiteSpace(urlsConfigured))
{
    builder.WebHost.ConfigureKestrel(options =>
    {
        options.Listen(System.Net.IPAddress.Loopback, 0);
    });
}
// Controllers JSON (only affects MVC)
builder.Services
    .AddControllers()
    .AddJsonOptions(o =>
    {
        o.JsonSerializerOptions.PropertyNamingPolicy =
JsonNamingPolicy.CamelCase;
        o.JsonSerializerOptions.DictionaryKeyPolicy =
JsonNamingPolicy.CamelCase;
        o.JsonSerializerOptions.Converters.Add(new
JsonStringEnumConverter(JsonNamingPolicy.CamelCase));
    });
// SignalR JSON (affects hub payloads)
builder.Services
    .AddSignalR()
    .AddJsonProtocol(o =>
    {
        o.PayloadSerializerOptions.PropertyNamingPolicy =
JsonNamingPolicy.CamelCase;
        o.PayloadSerializerOptions.DictionaryKeyPolicy =
JsonNamingPolicy.CamelCase;
        o.PayloadSerializerOptions.Converters.Add(new
JsonStringEnumConverter(JsonNamingPolicy.CamelCase));
    });
// Configure console logging with timestamps and colors
builder.Logging.ClearProviders();
```

```
builder.Logging.AddSimpleConsole(options =>
{
    options.SingleLine = false;
    options.TimestampFormat = "[HH:mm:ss] ";
    options.ColorBehavior =
Microsoft.Extensions.Logging.Console.LoggerColorBehavior.Enabled;
    options.IncludeScopes = true;
});
builder.Logging.SetMinimumLevel(LogLevel.Information);
// Configure CORS for frontend development
builder.Services.AddCors(options =>
{
    options.AddDefaultPolicy(policy =>
    {
        policy.SetIsOriginAllowed(origin =>
            {
                if (string.IsNullOrEmpty(origin))
                    return false;
                return Uri.TryCreate(origin, UriKind.Absolute, out var uri)
&& uri.IsLoopback;
            })
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials();
    });
});
// Register services
builder.Services.AddSingleton<TerminalService>();
builder.Services.AddSingleton<PythonVenvRunner>();
builder.Services.AddSingleton<FileService>();
builder.Services.AddSingleton<FileWatcherService>();
var app = builder.Build();
app.UseCors();
app.UseRouting();
// Map SignalR hubs
app.MapHub<PythonHub>("/pythonHub");
app.MapHub<TerminalHub>("/terminalHub");
app.MapHub<FileHub>("/fileHub");
// Run application with error handling
try
{
    await app.StartAsync();
    var server = app.Services.GetRequiredService<IServer>();
    var addresses =
server.Features.Get<IServerAddressesFeature>()!.Addresses;
    foreach (var address in addresses)
    {
        var port = new Uri(address).Port;
```

```
80          Console.Out.WriteLine($"SIGNALR_PORT={port}");
81          Console.Out.Flush();
82          app.Logger.LogInformation("Listening on {Address}",
    LogFormatter.ToGreen(address));
83      }
84
85      app.Logger.LogInformation("Starting kernel service (PID: {Pid})",
    LogFormatter.ToYellow(Environment.ProcessId));
86      app.Logger.LogInformation("Environment: {Env}",
    LogFormatter.ToGreen(builder.Environment.EnvironmentName));
87
88      app.Logger.LogInformation("Mapped hub: {Path} -> {Hub}",
    LogFormatter.ToMagenta("/kernelHub"), LogFormatter.ToCyan("PythonHub"));
89      app.Logger.LogInformation("Mapped hub: {Path} -> {Hub}",
    LogFormatter.ToMagenta("/terminalHub"), LogFormatter.ToCyan("TerminalHub"));
90      app.Logger.LogInformation("Mapped hub: {Path} -> {Hub}",
    LogFormatter.ToMagenta("/fileHub"), LogFormatter.ToCyan("FileHub"));
91      await app.WaitForShutdownAsync();
92  }
93  catch (Exception ex)
94  {
95      app.Logger.LogCritical(ex, "Host terminated unexpectedly");
96      throw;
97  }
```

3.2.2 FileHub

The following streamlined code demonstrates the implementation of the `FileHub`. The complete source code is provided in the source code section.

```
1   /// <summary>
2   /// SignalR hub for file system operations.
3   /// Provides streaming file tree reading, file content streaming, workspace
    settings,
4   /// and file system watching for incremental updates.
5   /// </summary>
6   public class FileHub : Hub
7   {
8       private readonly FileService _fileService;
9       private readonly FileWatcherService _fileWatcherService;
10      private readonly IHubContext<FileHub> _hubContext;
11      private readonly ILogger<FileHub> _logger;
12      // Track which directories each connection is watching
13      private static readonly Dictionary<string, HashSet<string>>
    ConnectionWatchedDirs = new();
14      private static readonly Lock Lock = new();
```

Since SignalR hub methods could run concurrently across multiple threads, a synchronise lock

object is required to ensure atomic access to the shared dictionary `ConnectionWatchedDirs`, especially in asynchronous functions [13].

```
15      public FileHub(
16          FileService fileService,
17          FileWatcherService fileWatcherService,
18          IHubContext<FileHub> hubContext,
19          ILogger<FileHub> logger)
20      {
21          _fileService = fileService;
22          _fileWatcherService = fileWatcherService;
23          _hubContext = hubContext;
24          _logger = logger;
25          _fileWatcherService.DirectoryDirty -= OnDirectoryDirty;
26          _fileWatcherService.DirectoryDirty += OnDirectoryDirty;
27      }
28      /// <summary>
29      /// Called by FileWatcherService when a directory change is detected.
30      /// Broadcasts to clients watching that directory.
31      /// </summary>
32      /// <param name="rootPath">
33      /// the top-level directory being watched (the one clients subscribe
    to).
34      /// </param>
35      /// <param name="dirPath">
36      /// dirPath is the specific directory that changed (could be rootPath or
    a subdirectory
37      /// </param>
38      private void OnDirectoryDirty(string rootPath, string dirPath)
39      {
40          var groupName = GetGroupName(rootPath);
41          _hubContext.Clients.Group(groupName).SendAsync("DirectoryDirty",
    dirPath);
42      }
43      /// <summary>
44      /// Start watching a directory for file changes.
45      /// The client will receive FileChanged events for this directory.
46      /// </summary>
47      public async Task StartWatchingAsync(string directoryPath)
```

`Task` is a class from `System.Threading.Tasks` namespaces that represents an asynchronous operation [14].

```
48      {
49          if (string.IsNullOrWhiteSpace(directoryPath))
50          {
51              _logger.LogWarning("Invalid directory path for watching");
52              return;
```

```
        }
        var fullPath = Path.GetFullPath(directoryPath);
        var groupName = GetGroupName(fullPath);
        if (_fileWatcherService.IsWatching(fullPath))
        {
            return;
        }
        // Add this connection to the group
        await Groups.AddToGroupAsync(Context.ConnectionId, groupName);
        // Track which directories this connection is watching
        lock (Lock)
        {
            if (!ConnectionWatchedDirs.TryGetValue(Context.ConnectionId, out
var dirs))
            {
                dirs = [];
                ConnectionWatchedDirs[Context.ConnectionId] = dirs;
            }
            dirs.Add(fullPath);
        }
```

Here, the lock object locks the shared dictionary for atomic data access. The subsequent code follows a similar pattern.

```
        // Start the watcher if not already watching
        _fileWatcherService.StartWatching(fullPath);
            Context.ConnectionId, LogFormatter.ToGreen(fullPath));
    }
    /// <summary>
    /// Stop watching a directory for file changes.
    /// </summary>
    public async Task StopWatchingAsync(string directoryPath)
    {
        if (string.IsNullOrWhiteSpace(directoryPath))
        {
            return;
        }
        var fullPath = Path.GetFullPath(directoryPath);
        var groupName = GetGroupName(fullPath);

        if (!_fileWatcherService.IsWatching(fullPath))
        {
            return;
        }
        // Remove this connection from the group
        await Groups.RemoveFromGroupAsync(Context.ConnectionId, groupName);
        // Update tracking
        lock (Lock)
```

```
        {
            if (ConnectionWatchedDirs.TryGetValue(Context.ConnectionId, out
var dirs))
            {
                dirs.Remove(fullPath);
            }
        }

        _fileWatcherService.StopWatching(fullPath);
            Context.ConnectionId, LogFormatter.ToYellow(fullPath));
    }
    /// <summary>
    /// Called when a client disconnects. Clean up watched directories.
    /// </summary>
    public override async Task OnDisconnectedAsync(Exception? exception)
    {
        lock (Lock)
        {
            ConnectionWatchedDirs.Remove(Context.ConnectionId);
        }
        await base.OnDisconnectedAsync(exception);
    }
```

The `OnDisconnectedAsync` function overrides the base implementation in the `Microsoft.AspNetCore.SignalR` namespace. It removes the `ConnectionId` from the shared dictionary before calling the base implementation.

```
    private static string GetGroupName(string directoryPath) =>
$"watch:{directoryPath}";
    /// <summary>
    /// Read directory contents recursively and stream results.
    /// </summary>
    public IAsyncEnumerable<FileService.FileNode> ReadDirAsync(string
dirPath)
```

`IAsyncEnumerable<T>` is an interface from `System.Collections.Generic` namespace that exposes an enumerator that provides asynchronous iteration over values of a specified type [15].

```
    {
        return _fileService.ReadDirAsync(dirPath);
    }
    /// <summary>
    /// Stream file content for opening in a tab.
    /// </summary>
    public IAsyncEnumerable<FileService.TabChunk> StreamTabAsync(string
filePath)
    {
```

```
        return _fileService.StreamTabAsync(filePath);
    }
    /// <summary>
    /// Create a new directory.
    /// </summary>
    public void CreateDirectory(string dirPath)
    {
        _fileService.CreateDirectory(dirPath);
    }
    public void DeleteDirectory(string dirPath)
    {
        _fileService.DeleteDirectory(dirPath);
    }
    public void MoveDirectory(string sourcePath, string destPath)
    {
        _fileService.MoveDirectory(sourcePath, destPath);
    }
    public void RenameDirectory(string sourcePath, string destPath)
    {
        _fileService.MoveDirectory(sourcePath, destPath);
    }
    public void CopyDirectory(string sourcePath, string destPath)
    {
        _fileService.CopyDirectory(sourcePath, destPath);
    }
    public void CreateFile(string filePath)
    {
        _fileService.CreateFile(filePath);
    }
    public void DeleteFile(string filePath)
    {
        _fileService.DeleteFile(filePath);
    }
    public void MoveFile(string sourcePath, string destPath)
    {
        _fileService.MoveFile(sourcePath, destPath);
    }
    public void RenameFile(string sourcePath, string destPath)
    {
        _fileService.MoveFile(sourcePath, destPath);
    }
    public void CopyFile(string sourcePath, string destPath)
    {
        _fileService.CopyFile(sourcePath, destPath);
    }
    /// <summary>
    /// Save file content from a stream.
    /// </summary>
```

```
178     /// <param name="filePath">The target file path</param>
179     /// <param name="stream">The content stream to write</param>
180     /// <param name="encoding">The encoding to use when saving (default:
    utf-8)</param>
181     public async Task SaveFileAsync(string filePath, ChannelReader<string>
    stream, string encoding = "utf-8")
182     {
183         await _fileService.SaveFileAsync(filePath, stream, encoding);
184     }
```

The `ChannelReader<T>` is a class from `System.Threading.Channels` namespace that provides asynchronous data streaming capabilities [16].

```
185     /// <summary>
186     /// Save workspace settings to disk.
187     /// </summary>
188     public async Task SaveWorkspaceSettingsAsync(string cwd,
    FileService.PanelLayout layout)
189     {
190         await _fileService.SaveWorkspaceSettingsAsync(cwd, layout);
191     }
192     /// <summary>
193     /// Load workspace settings from disk.
194     /// </summary>
195     public async Task<FileService.PanelLayout?>
    LoadWorkspaceSettingsAsync(string cwd)
```

`Task<TResult>` is a class from `System.Threading.Tasks` namespace that represents an asynchronous operation that can return a value [17].

```
196     {
197         return await _fileService.LoadWorkspaceSettingsAsync(cwd);
198     }
199 }
```

3.2.3 Other Hub Implementation

The implementations of `TerminalHub` and `PythonHub` follow the same architectural pattern as `FileHub`, differing only in the APIs they expose. Therefore, they are not discussed in detail here. The corresponding source codes are provided in the source code section.

3.2.4 FileService

The following streamlined code demonstrates the implementation of the `FileService`. The complete source code is provided in the source code section.

```
1  /// <summary>
2  /// Service for file system operations including reading directories,
```

```
/// streaming file content, and managing workspace settings.
/// </summary>
public class FileService(ILogger<FileService> logger)
{
    /// <summary>
    /// JSON serializer options for workspace settings.
    /// </summary>
    private static readonly JsonSerializerOptions JsonSerializerOptions =
new()
    {
        WriteIndented = true,
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
        Converters =
        {
            new JsonStringEnumConverter(JsonNamingPolicy.CamelCase)
        }
    };
    /// <summary>
    /// Stream file content as chunks for efficient loading of large files.
    /// First yields metadata (including detected encoding), then yields
content chunks.
    /// Uses Mozilla Universal Charset Detector to detect file encoding.
    /// </summary>
    public async IAsyncEnumerable<TabChunk> StreamTabAsync(string filePath,
        [EnumeratorCancellation] CancellationToken token = default)
    {
        var tabId = Guid.NewGuid().ToString();
        // Detect file encoding using Mozilla Universal Charset Detector
        var detectedEncoding = "utf-8";
        var result = await CharsetDetector.DetectFromFileAsync(filePath,
token);
```

`CharsetDetector` is a class of the package UtfUnknown, which is a C# port of Mozilla Universal Charset Detector [18].

```
        if (result.Detected?.EncodingName != null)
        {
            // Treat ASCII as UTF-8 for frontend compatibility
            detectedEncoding = result.Detected.EncodingName.Equals("ASCII",
StringComparison.OrdinalIgnoreCase)
                ? "utf-8"
                : result.Detected.EncodingName;
        }
        // First return tab metadata so frontend can create the tab
immediately
        yield return new TabChunk(
            Id: tabId,
            Name: Path.GetFileName(filePath),
```

```
            Path: filePath,
            IsMetadata: true,
            Encoding: detectedEncoding
        );
        await foreach (var chunk in StreamFileContentAsync(tabId, filePath,
detectedEncoding, token))
        {
            yield return chunk;
        }
```

The function returns `TabChunk` as a stream, and the chunks will be assembled in the client.

```
    }
    /// <summary>
    /// Stream file content in chunks using the specified encoding.
    /// </summary>
    private async IAsyncEnumerable<TabChunk> StreamFileContentAsync(string
tabId, string filePath,
        string encodingName, [EnumeratorCancellation] CancellationToken
token = default)
    {
        TabChunk? errorTabChunk = null;
        Encoding encoding;

        try
        {
            encoding = GetEncodingByName(encodingName);
        }
        catch (Exception ex)
        {
            encoding = Encoding.UTF8;
        }

        StreamReader? reader = null;
        try
        {
            reader = new StreamReader(filePath, encoding,
detectEncodingFromByteOrderMarks: true);
        }
        catch (Exception ex)
        {
            errorTabChunk = CreateErrorTabChunk(tabId, ex.Message);
        }
        if (errorTabChunk != null)
        {
            yield return errorTabChunk;
            yield break;
        }
```

```
        using (reader)
        {
            const int chunkSize = 65536; // 64KiB per chunk for efficient streaming
            var buffer = new char[chunkSize];
            int charsRead;
            while ((charsRead = await reader!.ReadAsync(buffer, 0, buffer.Length)) > 0)
            {
                token.ThrowIfCancellationRequested();
                yield return new TabChunk(
                    Id: tabId,
                    Content: new string(buffer, 0, charsRead)
                );
            }
        }
    }

    /// <summary>
    /// Get the Encoding object by name, with fallback to UTF-8.
    /// </summary>
    private static Encoding GetEncodingByName(string encodingName)
    {
        // Common encoding name mappings
        return encodingName.ToLowerInvariant() switch
        {
            "utf-8" or "utf8" => Encoding.UTF8,
            "utf-16le" or "utf-16" => Encoding.Unicode,
            "utf-16be" => Encoding.BigEndianUnicode,
            "utf-32" or "utf-32le" => Encoding.UTF32,
            "ascii" => Encoding.UTF8,
            "iso-8859-1" or "latin1" => Encoding.Latin1,
            _ => Encoding.GetEncoding(encodingName)
```

ASCII is manually treated as UTF-8, since UTF-8 is a superset of the ASCII character set.

```
        };
    }
    /// <summary>
    /// Create an error tab chunk for reporting file access errors.
    /// </summary>
    private static TabChunk CreateErrorTabChunk(string tabId, string errorMessage)
    {
        return new TabChunk(
            Id: tabId,
            Content: errorMessage,
            IsError: true
```

```
        );
    }
    /// <summary>
    /// Read directory contents recursively using BFS.
    /// Streams results as they are discovered.
    /// </summary>
    public async IAsyncEnumerable<FileNode> ReadDirAsync(string dirPath,
        [EnumeratorCancellation] CancellationToken token = default)
    {
        var queue = new Queue<string>();
        queue.Enqueue(dirPath);
        var count = 0;
        while (queue.Count > 0)
        {
            token.ThrowIfCancellationRequested();
            var currentDir = queue.Dequeue();
            IEnumerable<FileSystemInfo> entries;
            try
            {
                entries = new
DirectoryInfo(currentDir).EnumerateFileSystemInfos();
            }
            catch (UnauthorizedAccessException ex)
            {
                logger.LogWarning("Access denied to {Path}: {Msg}", dirPath,
ex.Message);
                yield break; // Exit if root directory cannot be opened
            }
            catch (DirectoryNotFoundException)
            {
                continue;
            }
            foreach (var entry in entries)
            {
                token.ThrowIfCancellationRequested();
                // Yield periodically to avoid blocking
                if (++count % 100 == 0)
                {
                    await Task.Yield();
                }
```

Manually pause the thread to prevent long CPU loops from monopolising it, and allow other works to run.

```
                FileNode? node;
                try
                {
                    node = new FileNode(
```

```
                        Name: entry.Name,
                        Path: entry.FullName,
                        Extension: entry is FileInfo file ? file.Extension :
    string.Empty,
                        IsDirectory: entry is DirectoryInfo,
                        ParentPath: currentDir,
                        Expanded: false
                    );
                }
                catch (Exception ex)
                {
                    continue;
                }
                yield return node;
                // BFS: enqueue directories for further exploration
                if (entry is DirectoryInfo)
                {
                    queue.Enqueue(entry.FullName);
                }
            }
        }
    }
    /// <summary>
    /// Create a directory at the specified path.
    /// </summary>
    /// <param name="dirPath"></param>
    public void CreateDirectory(string dirPath)
    {
        if (string.IsNullOrWhiteSpace(dirPath))
        {
            return;
        }
        // Normalize and validate path
        var fullPath = Path.GetFullPath(dirPath);
        try
        {
            Directory.CreateDirectory(fullPath);
        }
        catch (Exception ex)
        {
        }
    }
    public void CreateFile(string filePath)
    {
        if (string.IsNullOrWhiteSpace(filePath))
        {
            return;
        }
```

```
        // Normalize and validate path
        var fullPath = Path.GetFullPath(filePath);
        try
        {
            using var fs = File.Create(fullPath);
        }
        catch (Exception ex)
        {
        }
    }
    public void DeleteFile(string filePath)
    {
        if (string.IsNullOrWhiteSpace(filePath))
        {
            return;
        }
        // Normalize and validate path
        var fullPath = Path.GetFullPath(filePath);
        try
        {
            if (File.Exists(fullPath))
            {
                File.Delete(fullPath);
            }
        }
        catch (Exception ex)
        {
        }
    }
    public void DeleteDirectory(string dirPath)
    {
        if (string.IsNullOrWhiteSpace(dirPath))
        {
            return;
        }
        // Normalize and validate path
        var fullPath = Path.GetFullPath(dirPath);
        try
        {
            if (Directory.Exists(fullPath))
            {
                Directory.Delete(fullPath, recursive: true);
            }
        }
        catch (Exception ex)
        {
        }
    }
```

```
    public void MoveFile(string oldFilePath, string newFilePath)
    {
        if (string.IsNullOrWhiteSpace(oldFilePath) ||
string.IsNullOrWhiteSpace(newFilePath))
        {
            return;
        }
        // Normalize and validate paths
        var fullOldPath = Path.GetFullPath(oldFilePath);
        var fullNewPath = Path.GetFullPath(newFilePath);
        try
        {
            if (File.Exists(fullOldPath))
            {
                File.Move(fullOldPath, fullNewPath);
                    LogFormatter.ToGreen(fullOldPath),
                    LogFormatter.ToGreen(fullNewPath));
            }
        }
        catch (Exception ex)
        {
        }
    }

    public void MoveDirectory(string oldFilePath, string newFilePath)
    {
        if (string.IsNullOrWhiteSpace(oldFilePath) ||
string.IsNullOrWhiteSpace(newFilePath))
        {
            return;
        }
        // Normalize and validate paths
        var fullOldPath = Path.GetFullPath(oldFilePath);
        var fullNewPath = Path.GetFullPath(newFilePath);
        try
        {
            if (Directory.Exists(fullOldPath))
            {
                Directory.Move(fullOldPath, fullNewPath);
            }
        }
        catch (Exception ex)
        {
        }
    }
    public void CopyFile(string oldFilePath, string newFilePath)
    {
```

```
        if (string.IsNullOrWhiteSpace(oldFilePath) || string.IsNullOrWhiteSpace(newFilePath))
        {
            return;
        }
        // Normalize and validate paths
        var fullOldPath = Path.GetFullPath(oldFilePath);
        var fullNewPath = Path.GetFullPath(newFilePath);
        try
        {
            if (File.Exists(fullOldPath))
            {
                File.Copy(fullOldPath, fullNewPath, overwrite: false);
            }
        }
        catch (Exception ex)
        {
        }
    }

    public void CopyDirectory(string oldDirPath, string newDirPath)
    {
        if (string.IsNullOrWhiteSpace(oldDirPath) || string.IsNullOrWhiteSpace(newDirPath))
        {
            return;
        }
        // Normalize and validate paths
        var fullOldPath = Path.GetFullPath(oldDirPath);
        var fullNewPath = Path.GetFullPath(newDirPath);
        try
        {
            if (Directory.Exists(fullOldPath))
            {
                CopyDirectoryRecursive(fullOldPath, fullNewPath);
            }
        }
        catch (Exception ex)
        {
        }
    }

    private static void CopyDirectoryRecursive(string sourceDir, string destDir)
    {
        Directory.CreateDirectory(destDir);
        foreach (var filePath in Directory.GetFiles(sourceDir))
        {
```

```
            var destFilePath = Path.Combine(destDir,
Path.GetFileName(filePath));
            File.Copy(filePath, destFilePath, overwrite: false);
        }
        foreach (var directoryPath in Directory.GetDirectories(sourceDir))
        {
            var destDirectoryPath = Path.Combine(destDir,
Path.GetFileName(directoryPath));
            CopyDirectoryRecursive(directoryPath, destDirectoryPath);
        }
    }
```

C# doesn't have native asynchronous copy, move, delete, create functions. Calling the function with a task object and putting it into the thread pool to force asynchronous execution will only decrease performance, as the function will still be blocking the thread and thread switching is usually more resource-intensive for CPU. Since operations such as copy are fast and atomic, implementing them synchronously will not affect performance.

```

    /// <summary>
    /// Save file content from a stream to disk.
    /// Uses atomic write with temporary file for data safety.
    /// </summary>
    /// <param name="filePath">The target file path</param>
    /// <param name="stream">The content stream to write</param>
    /// <param name="encodingName">The encoding to use when saving (default:
utf-8)</param>
    /// <param name="ct">Cancellation token</param>
    public async Task SaveFileAsync(string filePath, ChannelReader<string>
stream,
        string encodingName = "utf-8", CancellationToken ct = default)
    {
        if (string.IsNullOrWhiteSpace(filePath))
        {
            return;
        }
        // Normalize and validate path
        var fullPath = Path.GetFullPath(filePath);
        var dir = Path.GetDirectoryName(fullPath);
        if (string.IsNullOrEmpty(dir))
        {
            return;
        }
        Directory.CreateDirectory(dir);
        var tempPath = Path.Combine(dir,
$"{Path.GetFileName(fullPath)}.{Guid.NewGuid():N}.tmp");
```

```
385
386         // Get the encoding to use for saving
387         Encoding encoding;
388         try
389         {
390             encoding = GetEncodingByName(encodingName);
391         }
392         catch (Exception ex)
393         {
394             encoding = Encoding.UTF8;
395         }
396         try
397         {
398             await using (var writer = new StreamWriter(tempPath, false,
    encoding, 65536))
399             {
400                 await foreach (var chunk in stream.ReadAllAsync(ct))
401                 {
402                     await writer.WriteAsync(chunk);
403                 }
404             }
405             // Atomic replace for data safety
406             if (File.Exists(fullPath))
407             {
408                 File.Replace(tempPath, fullPath, destinationBackupFileName:
    null, ignoreMetadataErrors: true);
409             }
410             else
411             {
412                 File.Move(tempPath, fullPath, overwrite: true);
413             }
```

If the process crashes while saving a file, the data in the file is likely to become corrupted. Therefore, create a temporary file and then overwrite the original file is used here to ensure data safety.

```
414         }
415         catch
416         {
417             // Clean up temporary file on failure
418             if (File.Exists(tempPath))
419                 File.Delete(tempPath);
420         }
421     }
422     /// <summary>
423     /// Save workspace settings to a JSON file in the workspace directory.
424     /// </summary>
```

```
425     public async Task SaveWorkspaceSettingsAsync(string cwd, PanelLayout
    layout, CancellationToken ct = default)
426     {
427         if (string.IsNullOrWhiteSpace(cwd))
428         {
429             return;
430         }
431         // Normalize and validate path
432         var fullPath = Path.GetFullPath(cwd);
433         var settingsDir = Path.Combine(fullPath, ".LiveMarkdown");
434         var settingsFile = Path.Combine(settingsDir, "settings.json");
435         Directory.CreateDirectory(settingsDir);
436         var tempPath = Path.Combine(
437             settingsDir,
438             $"{Path.GetFileName(fullPath)}.{Guid.NewGuid():N}.tmp"
439         );
440         try
441         {
442             var json = JsonSerializer.Serialize(
443                 layout,
444                 options: JsonSerializerOptions
445             );
446             await File.WriteAllTextAsync(tempPath, json, Encoding.UTF8, ct);
447             // Atomic replace for data safety
448             if (File.Exists(settingsFile))
449             {
450                 File.Replace(tempPath, settingsFile,
    destinationBackupFileName: null, ignoreMetadataErrors: true);
451             }
452             else
453             {
454                 File.Move(tempPath, settingsFile, overwrite: true);
455             }
456         }
457         catch (Exception ex)
458         {
459             // Clean up temporary file on failure
460             if (File.Exists(tempPath))
461                 File.Delete(tempPath);
462         }
463     }
464     /// <summary>
465     /// Load workspace settings from a JSON file in the workspace directory.
466     /// </summary>
467     public async Task<PanelLayout?> LoadWorkspaceSettingsAsync(string cwd,
    CancellationToken ct = default)
468     {
469         if (string.IsNullOrWhiteSpace(cwd))
```

```
        {
            return null;
        }
        // Normalize and validate path
        var fullPath = Path.GetFullPath(cwd);
        var settingsFile = Path.Join(fullPath, ".LiveMarkdown",
"settings.json");
        if (!File.Exists(settingsFile))
        {
            return null;
        }
        try
        {
            await using var stream = File.OpenRead(settingsFile);
            var layout =
                await JsonSerializer.DeserializeAsync<PanelLayout>(stream,
options: JsonSerializerOptions,
                    cancellationToken: ct);
            return layout;
        }
        catch (Exception ex)
        {
            return null;
        }
    }
}
```

3.2.5 FileWatcherService

The following streamlined code demonstrates the implementation of the `FileWatcherService`. The complete source code is provided in the source code section.

```
/// <summary>
/// Service for watching file system changes and notifying clients.
/// Detects create, delete, and rename operations for incremental directory
updates.
/// </summary>
public class FileWatcherService(ILogger<FileWatcherService> logger) :
IDisposable
{
    private readonly ConcurrentDictionary<string, FileSystemWatcher>
_watchers = new();
    private readonly ConcurrentDictionary<string, int> _watcherRefCounts =
new();
    private readonly ConcurrentDictionary<string, CancellationTokenSource>
_directoryDebounceTokens = new();
    private readonly Lock _watcherLock = new();
    private const int DirectoryDebounceMs = 200;
```

```
12      private volatile bool _disposed;
```

The `volatile` keyword is used here to prevent CPU caching to ensure memory visibility between threads [19].

```
13      /// <summary>
14      /// Event raised when a directory is marked dirty (debounced). Only the
    dirty directory path is provided.
15      /// </summary>
16      public event Action<string, string>? DirectoryDirty;
```

The `Action` object represents a file change event that has been subscribed to by the `FileHub` in order to notify a file change.

```
17      /// <summary>
18      /// Start watching a directory for file system changes.
19      /// </summary>
20      /// <param name="directoryPath">The directory to watch.</param>
21      /// <returns>True if watching started successfully, false
    otherwise.</returns>
22      public bool StartWatching(string directoryPath)
23      {
24          if (string.IsNullOrWhiteSpace(directoryPath))
25          {
26              return false;
27          }
28          var fullPath = Path.GetFullPath(directoryPath);
29          if (!Directory.Exists(fullPath))
30          {
31              return false;
32          }
33          lock (_watcherLock)
34          {
35              // Keep watcher creation/removal and ref-count updates in the
    same critical section
36              // to prevent races that could leak watchers or drive ref counts
    negative.
37              if (_disposed)
38              {
39                  return false;
40              }
41              if (_watchers.ContainsKey(fullPath))
42              {
43                  _watcherRefCounts[fullPath] =
    _watcherRefCounts.TryGetValue(fullPath, out var count) ? count + 1 : 1;
44                  return true;
45              }
```

```
            try
            {
                var watcher = new FileSystemWatcher(fullPath)
                {
                    NotifyFilter = NotifyFilters.FileName
                                 | NotifyFilters.DirectoryName
                                 | NotifyFilters.CreationTime,
                    IncludeSubdirectories = true,
                    EnableRaisingEvents = false
                };
                watcher.Created += OnCreated;
                watcher.Deleted += OnDeleted;
                watcher.Renamed += OnRenamed;
                watcher.Error += OnError;
                if (_watchers.TryAdd(fullPath, watcher))
                {
                    _watcherRefCounts[fullPath] = 1;
                    watcher.EnableRaisingEvents = true;
                    return true;
                }
                watcher.Dispose();
                return false;
            }
            catch (Exception ex)
            {
                return false;
            }
        }
    }
    /// <summary>
    /// Stop watching a directory.
    /// </summary>
    /// <param name="directoryPath">The directory to stop watching.</param>
    public void StopWatching(string directoryPath)
    {
        if (string.IsNullOrWhiteSpace(directoryPath)) return;
        var fullPath = Path.GetFullPath(directoryPath);
        lock (_watcherLock)
        {
            // Serialize ref-count decrement and watcher removal to avoid
disposing a watcher
            // that another thread just incremented.
            if (_disposed) return;
            if (_watchers.TryGetValue(fullPath, out _))
            {
                var remaining = _watcherRefCounts.TryGetValue(fullPath, out
var count) ? Math.Max(count - 1, 0) : 0;
                if (remaining > 0)
```

```
                {
                    _watcherRefCounts[fullPath] = remaining;
                    return;
                }
                _watcherRefCounts.TryRemove(fullPath, out _);
                if (_watchers.TryRemove(fullPath, out var watcher))
                {
                    watcher.EnableRaisingEvents = false;
                    watcher.Created -= OnCreated;
                    watcher.Deleted -= OnDeleted;
                    watcher.Renamed -= OnRenamed;
                    watcher.Error -= OnError;
                    watcher.Dispose();
                }
            }
        }
    }
    /// <summary>
    /// Check if a directory is being watched.
    /// </summary>
    public bool IsWatching(string directoryPath)
    {
        if (string.IsNullOrWhiteSpace(directoryPath)) return false;
        if (_disposed) return false;
        var fullPath = Path.GetFullPath(directoryPath);
        return _watchers.ContainsKey(fullPath);
    }
    private void OnCreated(object sender, FileSystemEventArgs e)
    {
        var parentPath = Path.GetDirectoryName(e.FullPath) ?? string.Empty;
        var rootPath = GetWatcherRootPath(sender);

        MarkDirectoryDirty(rootPath, parentPath);
    }
    private void OnDeleted(object sender, FileSystemEventArgs e)
    {
        var parentPath = Path.GetDirectoryName(e.FullPath) ?? string.Empty;
        var rootPath = GetWatcherRootPath(sender);

        MarkDirectoryDirty(rootPath, parentPath);
    }
    private void OnRenamed(object sender, RenamedEventArgs e)
    {
        var parentPath = Path.GetDirectoryName(e.FullPath) ?? string.Empty;
        var rootPath = GetWatcherRootPath(sender);

        MarkDirectoryDirty(rootPath, parentPath);
```

```
        var oldParent = Path.GetDirectoryName(e.OldFullPath) ??
string.Empty;
        if (!string.Equals(oldParent, parentPath,
StringComparison.OrdinalIgnoreCase))
        {
            MarkDirectoryDirty(rootPath, oldParent);
        }
    }
    private void OnError(object sender, ErrorEventArgs e)
    {
        var exception = e.GetException();
    }
    private static string GetWatcherRootPath(object sender)
    {
        if (sender is FileSystemWatcher watcher)
        {
            return watcher.Path;
        }
        return string.Empty;
    }
    private void MarkDirectoryDirty(string rootPath, string? directoryPath)
    {
        if (string.IsNullOrWhiteSpace(directoryPath)) return;
        if (_disposed) return;
        var key = $"{rootPath}|{directoryPath}";
        var cts = new CancellationTokenSource();
        // Atomically replace any existing token for this directory so only
the newest
        // change can fire (older tokens are canceled / disposed
immediately).
        _directoryDebounceTokens.AddOrUpdate(
            key,
            cts,
            (_, existingCts) =>
            {
                existingCts.Cancel();
                existingCts.Dispose();
                return cts;
            });
        _ = Task.Run(async () =>
        {
            try
            {
                await Task.Delay(DirectoryDebounceMs, cts.Token);
                if (!cts.Token.IsCancellationRequested && !_disposed)
                {
                    // Only the active token is allowed to remove itself and
notify, which
```

```
                    // prevents a race where a stale token fires after a
newer change.
                    if (_directoryDebounceTokens.TryRemove(new
KeyValuePair<string, CancellationTokenSource>(key, cts)))
                    {
                        DirectoryDirty?.Invoke(rootPath, directoryPath);
                    }
                }
            }
            catch (OperationCanceledException)
            {
                // ignore
            }
            finally
            {
                cts.Dispose();
            }
        }, cts.Token);
    }
    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }
```

The watcher has been manually disposed , meaning the object's finalizer doesn't need to be executed by the garbage collector [20].

```
    private void Dispose(bool disposing)
    {
        lock (_watcherLock)
        {
            if (_disposed) return;
            if (disposing)
            {
                // Mark disposed before canceling tokens so event threads
stop scheduling work.
                _disposed = true;
                foreach (var (_, cts) in _directoryDebounceTokens)
                {
                    cts.Cancel();
                    cts.Dispose();
                }
                _directoryDebounceTokens.Clear();
                foreach (var (path, watcher) in _watchers)
                {
                    watcher.EnableRaisingEvents = false;
                    watcher.Dispose();
```

```
                }
                _watchers.Clear();
                _watcherRefCounts.Clear();
            }
            _disposed = true;
        }
    }
}
```

3.3. Client

This Project is using Electron 30.5.1 and TypeScript 5.9.3 with Vue3.

3.3.1 Service Connection

The following streamlined code demonstrates the implementation of the `serviceConnection`. The complete source code is provided in the source code section.

```
/*
 * Ensure that the given SignalR HubConnection is connected.
 * If it is disconnected, it will start the connection.
 * If it is connecting or reconnecting, it will wait until connected.
 * Concurrent calls for the same connection will share the same start promise.
 */
const pending = new WeakMap<HubConnection, Promise<void>>()
```

The `HubConnection` object could be disposed by the garbage collector, so `WeakMap` is used instead of normal `Map [21]`. If a connection is already being initiated, other callers should wait directly for the same Promise to avoid concurrent initiations.

```
export async function ensureServiceConnection(connection: HubConnection): Promise<void> {
    // already connected
    if (connection.state === signalR.HubConnectionState.Connected) {
        return
    }
    // wait if a start is in progress
    const existing = pending.get(connection)
    if (existing) {
        await existing
        return
    }
    // if disconnected, start and track
    if (connection.state === signalR.HubConnectionState.Disconnected) {
        const promise = connection.start().finally(() => {
            pending.delete(connection)
```

```
        })
        pending.set(connection, promise)
        await promise
        return
    }
    // connecting or reconnecting: wait until connected or fails
    await new Promise<void>((resolve, reject) => {
        const onClose = (err?: Error) => {
            cleanup()
            reject(err ?? new Error('Connection closed while waiting'))
        }
        const cleanup = () => {
            connection.off('close', onClose)
        }
        connection.onclose(onClose)
        const check = () => {
            if (connection.state === signalR.HubConnectionState.Connected) {
                cleanup()
                resolve()
            } else if (
                connection.state === signalR.HubConnectionState.Disconnected
            ) {
                cleanup()
                reject(
                    new Error('Connection disconnected while waiting')
                )
            } else {
                setTimeout(check, 50)
```

Check every 50 milliseconds: If it changes to Connected, resolve it. If it changes to Disconnected, reject it. Otherwise, continue waiting.

```
            }
        }
        check()
    })
}
```

4. Source Code

The compiled executable is available in the GitHub repository, the link to which is provided in Appendix A – Source Code Repository.

`electron-env.d.ts`

```
/// <reference types="vite-plugin-electron/electron-env" />
declare namespace NodeJS {
```

```
    interface ProcessEnv {
        /**
         * The built directory structure
         *
         * ```tree
         * ├─┬─┬ dist
         * │ │ └── index.html
         * │ │
         * │ ├─┬ dist-electron
         * │ │ ├── main.js
         * │ │ └── preload.js
         * │
         * ```
         */
        APP_ROOT: string
        /** /dist/ or /public/ */
        VITE_PUBLIC: string
    }
}
// Used in Renderer process, expose in `preload.ts`
declare global {
    interface Window {
        ipcRenderer: import('electron').IpcRenderer
        nodePath: {
            join: (...args: string[]) => string
            dirname: (p: string) => string
            basename: (p: string, ext?: string) => string
            normalize: (p: string) => string
            normalizeDisplay: (p: string) => string
        }
        windowControls: {
            minimize(): void
            maximize(): void
            close(): void
            canClose(): void
            onMaximize(cb: (maximized: boolean) => void): void
            onBeforeClose(cb: () => void): void
        }
        cwd: {
            setCwd(cwd: string): void
            getCwd(): Promise<string>
            setDisplayCwd(cwd: string): void
            getDisplayCwd(): Promise<string>
        }
        kernel: {
            start(): void
            stop(): void
            getPort(): Promise<number | null>
```

```
            getStatus(): Promise<'starting' | 'running' | 'stopped' |
'error'>
            onPort(cb: (port: number) => void): void
            onStatus(cb: (status: string, error?: string) => void): void
        }
    }
}
declare module '*.vue' {
    import type {DefineComponent} from 'vue'
    const component: DefineComponent<{}, {}, any>
    export default component
}
```

electron\main.ts

```
/**
 * Electron Main Process
 *
 * Handles application lifecycle, window management, and IPC communication.
 * Supports both development and production environments.
 */
import {app, BrowserWindow, Menu, ipcMain, shell} from 'electron'
import {fileURLToPath} from 'node:url'
import {spawn, ChildProcess} from 'child_process';
import path from 'node:path'
import Store from 'electron-store'
import windowStateKeeper from 'electron-window-state'
import {createSplashWindow, closeSplashWindow} from './splash'
//
================================================================================
=
// Environment & Path Configuration
//
================================================================================
=
const __dirname = path.dirname(fileURLToPath(import.meta.url))
/**
 * Application directory structure:
 * ├─ dist/              - Vite build output (renderer)
 * │  └── index.html
 * └─ dist-electron/     - Electron build output (main process)
 *    ├── main.js
 *    └── preload.mjs
 */
const APP_ROOT = path.join(__dirname, '..')
const RENDERER_DIST = path.join(APP_ROOT, 'dist')
// Use bracket notation to avoid Vite's define plugin transformation
const VITE_DEV_SERVER_URL = process.env['VITE_DEV_SERVER_URL']
/** Whether the app is running in development mode */
```

```
const isDev = !!VITE_DEV_SERVER_URL
/** Path to public assets (varies between dev and prod) */
const PUBLIC_PATH = isDev ? path.join(APP_ROOT, 'public') : RENDERER_DIST
// Set environment variables for other modules
process.env.APP_ROOT = APP_ROOT
process.env.VITE_PUBLIC = PUBLIC_PATH
//
================================================================================
=
// Persistent Storage
//
================================================================================
=
interface StoreSchema {
    lastCwd?: string
    lastDisplayCwd?: string
}
const store = new Store<StoreSchema>({
    name: 'config',
    defaults: {
        lastCwd: undefined,
        lastDisplayCwd: undefined
    }
})
//
================================================================================
=
// Window State
//
================================================================================
=
let mainWindow: BrowserWindow | null = null
let mainWindowState: ReturnType<typeof windowStateKeeper> | null = null
let shouldMaximize = false
let pendingClose = false
let allowClose = false
let kernelProcess: ChildProcess | null = null
let kernelPort: number | null = null
//
================================================================================
=
// Kernel Management
//
================================================================================
=
/**
 * Start the kernel process and monitor its lifecycle.
 */
```

```
function startKernel(): void {
    if (kernelProcess) {
        console.log('[Kernel] Already running')
        return
    }
    mainWindow?.webContents.send('kernel:status', 'starting')
    // In production, kernel is in resources/.kernel folder
    // In development, it's in the project's kernel folder
    const kernelDir = isDev
        ? path.join(APP_ROOT, 'kernel')
        : path.join(process.resourcesPath, '.kernel')
    const kernelPath = path.join(kernelDir, 'kernel.exe')
    console.log('[Kernel] Starting from:', kernelPath)
    kernelProcess = spawn(kernelPath, [], {stdio: 'pipe', cwd: kernelDir})
    const onData = (chunk: Buffer) => {
        const msg = chunk.toString()
        console.log('[Kernel stdout]', msg)
        const portMatch = msg.match(/SIGNALR_PORT=(\d+)/)
        if (portMatch) {
            kernelPort = parseInt(portMatch[1], 10)
            mainWindow?.webContents.send('kernel:port', kernelPort)
            mainWindow?.webContents.send('kernel:status', 'running')
            kernelProcess!.stdout?.off('data', onData)
        }
    }
    kernelProcess.stdout?.on('data', onData)
    kernelProcess.stderr?.on('data', (chunk) => {
        console.error('[Kernel stderr]', chunk.toString())
    })
    kernelProcess.on('error', (err) => {
        console.error('[Kernel] Process error:', err)
        mainWindow?.webContents.send('kernel:status', 'error', err.message)
        kernelProcess = null
        kernelPort = null
    })
    kernelProcess.on('exit', (code, signal) => {
        console.log(`[Kernel] Exited with code ${code}, signal ${signal}`)
        const wasRunning = kernelPort !== null
        kernelProcess = null
        kernelPort = null
        mainWindow?.webContents.send('kernel:status', 'stopped')
        // Auto-restart if it was running (unexpected exit)
        if (wasRunning && mainWindow && !mainWindow.isDestroyed()) {
            console.log('[Kernel] Unexpected exit, restarting...')
            setTimeout(() => startKernel(), 1000)
        }
    })
}
```

```
/**
 * Stop the kernel process.
 */
function stopKernel(): void {
    if (kernelProcess) {
        kernelProcess.kill()
        kernelProcess = null
        kernelPort = null
    }
}
// IPC handlers for kernel management
ipcMain.on('kernel:start', () => {
    startKernel()
})
ipcMain.on('kernel:stop', () => {
    stopKernel()
})
ipcMain.handle('kernel:get-port', () => {
    return kernelPort
})
ipcMain.handle('kernel:get-status', () => {
    if (kernelProcess) {
        return kernelPort ? 'running' : 'starting'
    }
    return 'stopped'
})
//
==========================================================================
=
// Window Management
//
==========================================================================
=
/**
 * Create the main application window.
 * Handles both development (with dev server) and production (with built
files) modes.
 */
async function createMainWindow(): Promise<void> {
    // Reset close state for new window
    pendingClose = false
    allowClose = false
    // Restore window state
    mainWindowState = windowStateKeeper({
        defaultWidth: 1200,
        defaultHeight: 800,
    })
    shouldMaximize = mainWindowState.isMaximized
```

```
158     mainWindow = new BrowserWindow({
159         x: mainWindowState.x,
160         y: mainWindowState.y,
161         width: mainWindowState.width,
162         height: mainWindowState.height,
163         minWidth: 800,
164         minHeight: 600,
165         title: 'LiveMarkdown',
166         frame: false,
167         icon: path.join(PUBLIC_PATH, 'favicon.ico'),
168         show: false, // Don't show until app:ready
169         backgroundColor: '#0d0d0d', // Prevent white flash
170         webPreferences: {
171             preload: path.join(__dirname, 'preload.mjs'),
172             contextIsolation: true,
173             nodeIntegration: false,
174             sandbox: false,
175         },
176     })
177     // Track window state changes (without auto-restoring maximize)
178     mainWindow.on('resize', () => mainWindowState?.saveState(mainWindow!))
179     mainWindow.on('move', () => mainWindowState?.saveState(mainWindow!))
180     mainWindow.on('close', () => mainWindowState?.saveState(mainWindow!))
181     // Send timestamp to renderer when loaded
182     mainWindow.webContents.on('did-finish-load', () => {
183         const timestamp = new Date().toLocaleString()
184         mainWindow?.webContents.send('main-process-message', timestamp)
185     })
186     // Handle window maximize/unmaximize events
187     mainWindow.on('maximize', () => {
188         mainWindow?.webContents.send('win:maximized', true)
189         mainWindowState?.saveState(mainWindow!)
190     })
191     mainWindow.on('unmaximize', () => {
192         mainWindow?.webContents.send('win:maximized', false)
193         mainWindowState?.saveState(mainWindow!)
194     })
195     // Handle graceful close with save confirmation
196     mainWindow.on('close', (event) => {
197         if (allowClose) return
198         event.preventDefault()
199         if (pendingClose) return
200         pendingClose = true
201         // Notify renderer to save changes before closing
202         mainWindow?.webContents.send('app:before-close')
203     })
204     // Load content based on environment
205     if (isDev) {
```

```
206         await mainWindow.loadURL(VITE_DEV_SERVER_URL!)
207         // Open DevTools in development
208         mainWindow.webContents.openDevTools({mode: 'detach'})
209     } else {
210         await mainWindow.loadFile(path.join(RENDERER_DIST, 'index.html'))
211     }
212 }
213 //
    ================================================================================
    =
214 // IPC Handlers - Window Controls
215 //
    ================================================================================
    =
216 /** Minimize the main window */
217 ipcMain.on('win:minimize', () => {
218     mainWindow?.minimize()
219 })
220 /** Toggle maximize/restore state */
221 ipcMain.on('win:maximize', () => {
222     if (mainWindow?.isMaximized()) {
223         mainWindow.unmaximize()
224     } else {
225         mainWindow?.maximize()
226     }
227 })
228 /** Request window close (triggers save confirmation) */
229 ipcMain.on('win:close', () => {
230     mainWindow?.close()
231 })
232 /** Confirm close after save completion */
233 ipcMain.on('win:can-close', () => {
234     if (!mainWindow) return
235     allowClose = true
236     pendingClose = false
237     mainWindow.close()
238 })
239 // Open external links in the system default browser.
240 ipcMain.on('open-external-url', async (_event, url) => {
241     if (!/^https?:\/\//i.test(url)) return
242     await shell.openExternal(url)
243 })
244 //
    ================================================================================
    =
245 // IPC Handlers - Workspace
```

```
246 //
    ================================================================================
    =
247 /** Get the last used workspace directory */
248 ipcMain.handle('cwd:get', () => {
249     return store.get('lastCwd')
250 })
251 /** Save the current workspace directory */
252 ipcMain.on('cwd:set', (_, cwd: string) => {
253     store.set('lastCwd', cwd)
254 })
255 ipcMain.handle('cwd:get-display', () => {
256     return store.get('lastDisplayCwd')
257 })
258 ipcMain.on('cwd:set-display', (_, cwd: string) => {
259     store.set('lastDisplayCwd', cwd)
260 })
261 //
    ================================================================================
    =
262 // IPC Handlers - Splash
263 //
    ================================================================================
    =
264 /** Close splash and show main window when app is ready */
265 ipcMain.on('app:ready', () => {
266     closeSplashWindow()
267     if (mainWindow) {
268         mainWindow.show()
269         if (shouldMaximize) {
270             mainWindow.maximize()
271         }
272     }
273 })
274 //
    ================================================================================
    =
275 // Application Lifecycle
276 //
    ================================================================================
    =
277 /**
278  * Quit when all windows are closed.
279  * Windows-only application - always quit when window is closed.
280  */
281 app.on('window-all-closed', () => {
282     stopKernel()
283     app.quit()
```

```
    mainWindow = null
})
/**
 * Initialize application when Electron is ready.
 */
app.whenReady().then(async () => {
    // Disable menu in production
    if (!isDev) {
        Menu.setApplicationMenu(null)
    }
    // Show splash first
    createSplashWindow(__dirname, RENDERER_DIST, VITE_DEV_SERVER_URL)
    // Start kernel early
    startKernel()
    await createMainWindow()
    // Log startup info in development
    if (isDev) {
        console.log('[Electron] Main process started')
        console.log('[Electron] Dev server URL:', VITE_DEV_SERVER_URL)
    }
})
/**
 * Handle uncaught exceptions gracefully
 */
process.on('uncaughtException', (error) => {
    console.error('[Electron] Uncaught exception:', error)
})
process.on('unhandledRejection', (reason) => {
    console.error('[Electron] Unhandled rejection:', reason)
})
```

preload.ts

```
import {ipcRenderer, contextBridge} from 'electron'
import path from "node:path";
// --------- Expose some API to the Renderer process ---------
contextBridge.exposeInMainWorld('ipcRenderer', {
    on(...args: Parameters<typeof ipcRenderer.on>) {
        const [channel, listener] = args
        return ipcRenderer.on(channel, (event, ...args) =>
listener(event, ...args))
    },
    off(...args: Parameters<typeof ipcRenderer.off>) {
        const [channel, ...omit] = args
        return ipcRenderer.off(channel, ...omit)
    },
    send(...args: Parameters<typeof ipcRenderer.send>) {
        const [channel, ...omit] = args
        return ipcRenderer.send(channel, ...omit)
```

```
    },
    invoke(...args: Parameters<typeof ipcRenderer.invoke>) {
        const [channel, ...omit] = args
        return ipcRenderer.invoke(channel, ...omit)
    },
    // You can expose other APIs you need here.
    // ...
})
contextBridge.exposeInMainWorld("nodePath", {
    join: (...args: string[]): string => path.join(...args),
    dirname: (p: string): string => path.dirname(p),
    basename: (p: string, ext?: string): string => path.basename(p, ext),
    /**
     * Normalize path and convert to lowercase for storage/comparison (Windows case-insensitive).
     */
    normalize: (p: string): string => {
        let n = path.normalize(p)
        if (n.length > 1 && n.endsWith(path.sep)) {
            n = n.slice(0,-1)
        }
        return n.toLowerCase()
    },
    /**
     * Normalize path preserving original case for display.
     */
    normalizeDisplay: (p: string): string => {
        let n = path.normalize(p)
        if (n.length > 1 && n.endsWith(path.sep)) {
            n = n.slice(0,-1)
        }
        return n
    }
})
contextBridge.exposeInMainWorld('windowControls', {
    minimize: () => ipcRenderer.send('win:minimize'),
    maximize: () => ipcRenderer.send('win:maximize'),
    close: () => ipcRenderer.send('win:close'),
    canClose: () => ipcRenderer.send('win:can-close'),
    appReady: () => ipcRenderer.send('app:ready'),
    onMaximize: (cb: (maximized: boolean) => void) => {
        ipcRenderer.on('win:maximized', (_e, v) => cb(v))
    },
    onBeforeClose: (cb: () => void) => {
        ipcRenderer.on("app:before-close", cb)
    }
})
contextBridge.exposeInMainWorld("cwd", {
```

```
    setCwd: (cwd: string) => ipcRenderer.send("cwd:set", cwd),
    getCwd: () => ipcRenderer.invoke("cwd:get"),
    setDisplayCwd: (cwd: string) => ipcRenderer.send("cwd:set-display", cwd),
    getDisplayCwd: () => ipcRenderer.invoke("cwd:get-display")
})
contextBridge.exposeInMainWorld("kernel", {
    start: () => ipcRenderer.send("kernel:start"),
    stop: () => ipcRenderer.send("kernel:stop"),
    getPort: () => ipcRenderer.invoke("kernel:get-port"),
    getStatus: () => ipcRenderer.invoke("kernel:get-status"),
    onPort: (cb: (port: number) => void) => {
        ipcRenderer.on("kernel:port", (_e, port) => cb(port))
    },
    onStatus: (cb: (status: string, error?: string) => void) => {
        ipcRenderer.on("kernel:status", (_e, status, error) => cb(status, error))
    }
})
```

splash.ts

```
/**
 * Splash Window Module
 */
import { BrowserWindow } from 'electron'
import path from 'node:path'
let splashWindow: BrowserWindow | null = null
export function createSplashWindow(
    dirname: string,
    rendererDist: string,
    devServerUrl: string | undefined
): void {
    splashWindow = new BrowserWindow({
        width: 600,
        height: 338,
        frame: false,
        transparent: true,
        alwaysOnTop: true,
        resizable: false,
        skipTaskbar: true,
        show: true,
        webPreferences: {
            contextIsolation: true,
            nodeIntegration: false,
        },
    })
    splashWindow.center()
    if (devServerUrl) {
```

```
        splashWindow.loadURL(`${devServerUrl}splash.html`)
    } else {
        splashWindow.loadFile(path.join(rendererDist, 'splash.html'))
    }
}
export function closeSplashWindow(): void {
    if (splashWindow && !splashWindow.isDestroyed()) {
        splashWindow.close()
        splashWindow = null
    }
}
```

ContextMenu.vue

```
<script setup lang="ts">
import {ref, onMounted, onUnmounted} from 'vue'
export interface MenuItem {
    label: string
    icon?: string
    action: () => void
    disabled?: boolean
    separator?: boolean
}
interface Props {
    items: MenuItem[]
    x: number
    y: number
}
defineProps<Props>()
const emit = defineEmits<{
    close: []
}>()
const menuRef = ref<HTMLElement | null>(null)
function handleClickOutside(event: MouseEvent): void {
    if (menuRef.value && !menuRef.value.contains(event.target as Node)) {
        emit('close')
    }
}
function handleItemClick(item: MenuItem): void {
    if (!item.disabled && !item.separator) {
        item.action()
        emit('close')
    }
}
onMounted(() => {
    document.addEventListener('click', handleClickOutside)
    document.addEventListener('contextmenu', handleClickOutside)
})
onUnmounted(() => {
```

```
    document.removeEventListener('click', handleClickOutside)
    document.removeEventListener('contextmenu', handleClickOutside)
})
</script>
<template>
    <Teleport to="body">
        <div
            ref="menuRef"
            class="context-menu"
            :style="{ left: `${x}px`, top: `${y}px` }"
        >
            <template v-for="(item, index) in items" :key="index">
                <div v-if="item.separator" class="context-menu-
separator"></div>
                <div
                    v-else
                    class="context-menu-item"
                    :class="{ disabled: item.disabled }"
                    @click="handleItemClick(item)"
                >
                    <span v-if="item.icon" class="material-symbols-outlined
menu-icon">{{ item.icon }}</span>
                    <span class="menu-label">{{ item.label }}</span>
                </div>
            </template>
        </div>
    </Teleport>
</template>
<style scoped>
.context-menu {
    position: fixed;
    background-color: var(--bg-secondary);
    border: 1px solid var(--widget-border-color);
    border-radius: 6px;
    padding: 4px 0;
    min-width: 160px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.4);
    z-index: 10000;
}
.context-menu-item {
    display: flex;
    align-items: center;
    padding: 6px 12px;
    cursor: pointer;
    font-size: 13px;
    color: var(--text-primary);
    transition: background-color 0.1s ease;
}
```

```
.context-menu-item:hover:not(.disabled) {
    background-color: rgba(255, 255, 255, 0.08);
}
.context-menu-item.disabled {
    color: var(--text-muted);
    cursor: not-allowed;
}
.menu-icon {
    font-size: 16px;
    margin-right: 8px;
    font-variation-settings: 'FILL' 0, 'wght' 400, 'GRAD' 0, 'opsz' 16;
}
.menu-label {
    flex: 1;
}
.context-menu-separator {
    height: 1px;
    background-color: var(--widget-border-color);
    margin: 4px 8px;
}
</style>
```

Editor.vue

```
<script setup lang="ts">
import {onMounted, onUnmounted, ref, watch, nextTick} from 'vue'
import {storeToRefs} from 'pinia'
import * as monaco from 'monaco-editor'
import {useWorkspaceStore} from '@/stores/workspace'
const workspace = useWorkspaceStore()
const {code, currentLanguage, editorPaneStyle, showCodePane} =
storeToRefs(workspace)
const editorContainer = ref<HTMLElement | null>(null)
let editor: monaco.editor.IStandaloneCodeEditor | null = null
const EDITOR_OPTIONS: monaco.editor.IStandaloneEditorConstructionOptions = {
    fontSize: 14,
    minimap: {enabled: true},
    automaticLayout: true,
    wordWrap: 'on',
    theme: 'vs-dark'
}
function initializeEditor(): void {
    if (!editorContainer.value) return
    editor = monaco.editor.create(editorContainer.value, {
        value: code.value,
        language: currentLanguage.value.toLowerCase(),
        ...EDITOR_OPTIONS
    })
    editor.onDidChangeModelContent(() => {
```

```
        if (editor) {
            code.value = editor.getValue()
        }
    })
}
// Watch for code changes from outside
watch(code, (newCode) => {
    if (editor && editor.getValue() !== newCode) {
        editor.setValue(newCode)
    }
})
// Watch for language changes
watch(currentLanguage, (newLang) => {
    if (editor) {
        const model = editor.getModel()
        if (model) {
            monaco.editor.setModelLanguage(model, newLang.toLowerCase())
        }
    }
})
// Watch for showCodePane to layout editor
watch(showCodePane, async (visible) => {
    if (visible && editor) {
        await nextTick()
        editor.layout()
    }
})
onMounted(() => {
    initializeEditor()
})
onUnmounted(() => {
    editor?.dispose()
    editor = null
})
</script>
<template>
    <div class="editor-main" :style="editorPaneStyle" v-show="showCodePane">
        <div ref="editorContainer" class="code-editor"></div>
    </div>
</template>
<style scoped>
.editor-main {
    display: flex;
    flex: 1 1 auto;
    min-width: 0;
    overflow: hidden;
}
.code-editor {
```

```
    width: 100%;
    height: 100%;
    min-width: 0;
}
.editor-main :deep(.monaco-editor),
.editor-main :deep(.monaco-editor .overflow-guard) {
    max-width: 100%;
}
</style>
```

FileExplorer.vue

```
<script setup lang="ts">
import {ref} from 'vue'
import {storeToRefs} from 'pinia'
import FileTreeNode from './FileTreeNode.vue'
import ContextMenu, {MenuItem} from './ContextMenu.vue'
import {useWorkspaceStore} from '@/stores/workspace'
import {UIFileNode} from '@/types/workspace'
const workspace = useWorkspaceStore()
const {visibleFileTree, hasWorkspace, rootDirectory} =
storeToRefs(workspace)
// Context menu state
const showContextMenu = ref(false)
const contextMenuX = ref(0)
const contextMenuY = ref(0)
const contextMenuItems = ref<MenuItem[]>([])
const contextMenuNode = ref<UIFileNode | null>(null)
// Input dialog state
const showInputDialog = ref(false)
const inputDialogTitle = ref('')
const inputDialogValue = ref('')
const inputDialogCallback = ref<((value: string) => void) | null>(null)
const inputRef = ref<HTMLInputElement | null>(null)
function handleContextMenu(event: MouseEvent, node: UIFileNode | null): void
{
    event.preventDefault()
    contextMenuNode.value = node
    contextMenuX.value = event.clientX
    contextMenuY.value = event.clientY
    const targetPath = node?.path ?? rootDirectory.value
    if (node === null || node.isDirectory) {
        // Context menu for directory or empty space
        contextMenuItems.value = [
            {
                label: 'New File',
                icon: 'note_add',
                action: () => showNewFileDialog(targetPath)
            },
```

```
            {
                label: 'New Folder',
                icon: 'create_new_folder',
                action: () => showNewFolderDialog(targetPath)
            },
            {separator: true, label: '', action: () => {}},
            {
                label: 'Copy',
                icon: 'content_copy',
                action: () => {
                    if (node) workspace.copyToClipboard(node.path,
node.isDirectory)
                },
                disabled: !node
            },
            {
                label: 'Cut',
                icon: 'content_cut',
                action: () => {
                    if (node) workspace.cutToClipboard(node.path,
node.isDirectory)
                },
                disabled: !node
            },
            {
                label: 'Paste',
                icon: 'content_paste',
                action: () => workspace.pasteFromClipboard(targetPath),
                disabled: !workspace.hasClipboardContent()
            },
            {separator: true, label: '', action: () => {}},
            {
                label: 'Rename',
                icon: 'edit',
                action: () => {
                    if (node) showRenameDialog(node)
                },
                disabled: !node
            },
            {
                label: 'Delete',
                icon: 'delete',
                action: () => {
                    if (node) workspace.deleteFileNode(node.path,
node.isDirectory)
                },
                disabled: !node
            }
```

```
        ]
    } else {
        // Context menu for file
        contextMenuItems.value = [
            {
                label: 'Open',
                icon: 'open_in_new',
                action: () => workspace.openFile(node)
            },
            {separator: true, label: '', action: () => {}},
            {
                label: 'Copy',
                icon: 'content_copy',
                action: () => workspace.copyToClipboard(node.path, false)
            },
            {
                label: 'Cut',
                icon: 'content_cut',
                action: () => workspace.cutToClipboard(node.path, false)
            },
            {separator: true, label: '', action: () => {}},
            {
                label: 'Rename',
                icon: 'edit',
                action: () => showRenameDialog(node)
            },
            {
                label: 'Delete',
                icon: 'delete',
                action: () => workspace.deleteFileNode(node.path, false)
            }
        ]
    }
    showContextMenu.value = true
}
function closeContextMenu(): void {
    showContextMenu.value = false
    contextMenuNode.value = null
}
function showNewFileDialog(parentPath: string): void {
    inputDialogTitle.value = 'New File'
    inputDialogValue.value = ''
    inputDialogCallback.value = (name: string) => {
        if (name.trim()) {
            workspace.createNewFile(parentPath, name.trim())
        }
    }
    showInputDialog.value = true
```

```
    setTimeout(() => inputRef.value?.focus(), 50)
}
function showNewFolderDialog(parentPath: string): void {
    inputDialogTitle.value = 'New Folder'
    inputDialogValue.value = ''
    inputDialogCallback.value = (name: string) => {
        if (name.trim()) {
            workspace.createNewFolder(parentPath, name.trim())
        }
    }
    showInputDialog.value = true
    setTimeout(() => inputRef.value?.focus(), 50)
}
function showRenameDialog(node: UIFileNode): void {
    inputDialogTitle.value = 'Rename'
    inputDialogValue.value = node.name
    inputDialogCallback.value = (name: string) => {
        if (name.trim() && name.trim() !== node.name) {
            workspace.renameFileNode(node.path, name.trim(),
node.isDirectory)
        }
    }
    showInputDialog.value = true
    setTimeout(() => {
        inputRef.value?.focus()
        inputRef.value?.select()
    }, 50)
}
function confirmInputDialog(): void {
    if (inputDialogCallback.value && inputDialogValue.value.trim()) {
        inputDialogCallback.value(inputDialogValue.value)
    }
    closeInputDialog()
}
function closeInputDialog(): void {
    showInputDialog.value = false
    inputDialogValue.value = ''
    inputDialogCallback.value = null
}
function handleInputKeydown(event: KeyboardEvent): void {
    if (event.key === 'Enter') {
        confirmInputDialog()
    } else if (event.key === 'Escape') {
        closeInputDialog()
    }
}
</script>
<template>
```

```
176     <div class="view-content" @contextmenu="handleContextMenu($event,
    null)">
177         <div class="content-title-bar">
178             <div class="title-label">
179                 <span>EXPLORER</span>
180             </div>
181             <div class="title-actions">
182                 <div class="title-action"
    @click="showNewFileDialog(rootDirectory)">
183                     <span class="material-symbols-outlined">add</span>
184                 </div>
185                 <div class="title-action" @click="workspace.toggleExplorer">
186                     <span class="material-symbols-
    outlined">collapse_content</span>
187                 </div>
188             </div>
189         </div>
190         <div v-if="hasWorkspace && visibleFileTree.length" class="file-
    explorer">
191             <FileTreeNode
192                 v-for="node in visibleFileTree"
193                 :key="node.path"
194                 :node="node"
195                 @toggleFolder="workspace.toggleFolder"
196                 @openFile="workspace.openFile"
197                 @contextmenu="handleContextMenu"
198             />
199         </div>
200         <div v-else class="file-explorer empty">
201             <p>Select a workspace to load files.</p>
202         </div>
203         <!-- Context Menu -->
204         <ContextMenu
205             v-if="showContextMenu"
206             :items="contextMenuItems"
207             :x="contextMenuX"
208             :y="contextMenuY"
209             @close="closeContextMenu"
210         />
211         <!-- Input Dialog -->
212         <Teleport to="body">
213             <div v-if="showInputDialog" class="input-dialog-overlay"
    @click.self="closeInputDialog">
214                 <div class="input-dialog">
215                     <div class="input-dialog-
    title">{{ inputDialogTitle }}</div>
216                     <input
217                         ref="inputRef"
```

```
                        v-model="inputDialogValue"
                        type="text"
                        class="input-dialog-input"
                        @keydown="handleInputKeydown"
                    />
                    <div class="input-dialog-actions">
                        <button class="dialog-btn cancel"
@click="closeInputDialog">Cancel</button>
                        <button class="dialog-btn confirm"
@click="confirmInputDialog">OK</button>
                    </div>
                </div>
            </div>
        </Teleport>
    </div>
</template>
<style scoped>
.file-explorer {
    font-family: inherit;
    color: var(--text-primary);
    padding: 0;
    overflow-y: scroll;
    overflow-x: auto;
    height: 100%;
    user-select: none;
    contain: strict;
}
.file-explorer.empty {
    display: flex;
    align-items: center;
    justify-content: center;
    color: var(--text-muted);
    font-size: 13px;
    overflow: hidden;
}
.input-dialog-overlay {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background-color: rgba(0, 0, 0, 0.5);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 1001;
}
.input-dialog {
```

```
    background-color: var(--bg-secondary);
    border: 1px solid var(--widget-border-color);
    border-radius: 8px;
    padding: 16px;
    min-width: 300px;
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.4);
}
.input-dialog-title {
    font-size: 14px;
    font-weight: 500;
    color: var(--text-primary);
    margin-bottom: 12px;
}
.input-dialog-input {
    width: 100%;
    padding: 8px 12px;
    font-size: 13px;
    background-color: var(--bg-primary);
    border: 1px solid var(--widget-border-color);
    border-radius: 4px;
    color: var(--text-primary);
    outline: none;
    box-sizing: border-box;
}
.input-dialog-input:focus {
    border-color: var(--accent-color);
}
.input-dialog-actions {
    display: flex;
    justify-content: flex-end;
    gap: 8px;
    margin-top: 16px;
}
.dialog-btn {
    padding: 6px 16px;
    font-size: 13px;
    border-radius: 4px;
    border: none;
    cursor: pointer;
    transition: background-color 0.1s ease;
}
.dialog-btn.cancel {
    background-color: transparent;
    color: var(--text-secondary);
}
.dialog-btn.cancel:hover {
    background-color: rgba(255, 255, 255, 0.08);
}
```

```
.dialog-btn.confirm {
    background-color: var(--accent-color);
    color: white;
}
.dialog-btn.confirm:hover {
    background-color: var(--accent-color-hover);
}
</style>
```

FileTreeNode.vue

```
<script setup lang="ts">
import {UIFileNode} from "@/types/workspace"
interface Props {
    node: UIFileNode
}
defineProps<Props>()
// Icon mappings for file types
const ICONS = {
    folder: 'folder',
    file: 'code',
    json: 'data_object',
    javascript: 'javascript',
    markdown: 'markdown',
    image: 'image',
    csv: 'csv',
    database: 'database',
    arrowDown: 'keyboard_arrow_down',
    arrowRight: 'keyboard_arrow_right',
} as const
type IconKey = keyof typeof ICONS
// Extension to icon category mapping
const CATEGORY_MAP: Partial<Record<IconKey, string[]>> = {
    image: ['.png', '.jpg', '.jpeg', '.gif', '.svg', '.bmp', '.webp'],
    json: ['.json'],
    javascript: ['.js', '.jsx', '.mjs', '.cjs', '.ts', '.tsx'],
    markdown: ['.md', '.mdx'],
    csv: ['.csv'],
    database: ['.db', '.sqlite', '.sqlite3', '.sql'],
}
// Build extension index for fast lookup
const extensionIndex = new Map<string, IconKey>()
for (const [category, extensions] of Object.entries(CATEGORY_MAP)) {
    for (const ext of extensions) {
        extensionIndex.set(ext, category as IconKey)
    }
}
const emit = defineEmits<{
    toggleFolder: [node: UIFileNode]
```

```
    openFile: [node: UIFileNode]
    contextmenu: [event: MouseEvent, node: UIFileNode]
}>()
/**
 * Handle node click - toggle folder or open file.
 */
function handleClick(node: UIFileNode): void {
    if (node.isDirectory) {
        emit('toggleFolder', node)
    } else {
        emit('openFile', node)
    }
}
/**
 * Handle right-click context menu.
 */
function handleContextMenu(event: MouseEvent, node: UIFileNode): void {
    event.preventDefault()
    event.stopPropagation()
    emit('contextmenu', event, node)
}
/**
 * Get the icon name for a file based on its extension.
 */
function getFileIcon(extension: string): string {
    return ICONS[extensionIndex.get(extension) ?? 'file']
}
</script>
<template>
    <div class="file-node">
        <div
            class="file-node-content"
            @click="handleClick(node)"
            @contextmenu="handleContextMenu($event, node)"
        >
            <span
                v-if="node.isDirectory"
                class="material-symbols-outlined folder-arrow"
            >
                {{ node.expanded ? ICONS.arrowDown : ICONS.arrowRight }}
            </span>
            <span
                v-if="node.isDirectory"
                class="material-symbols-outlined folder-icon"
            >
                {{ ICONS.folder }}
            </span>
            <span
```

```
                v-else
                class="material-symbols-outlined file-icon"
            >
                {{ getFileIcon(node.extension) }}
            </span>
            <span class="file-name">{{ node.name }}</span>
        </div>
        <!-- Recursive rendering for children -->
        <div
            v-if="node.expanded && node.children && node.children.length > 0"
            class="file-children"
        >
            <FileTreeNode
                v-for="child in node.children"
                :key="child.path"
                :node="child"
                @toggle-folder="emit('toggleFolder', $event)"
                @open-file="emit('openFile', $event)"
                @contextmenu="emit('contextmenu', $event, child)"
            />
        </div>
    </div>
</template>
<style scoped>
.file-node {
    margin: 0;
    contain: layout style;
}
.file-node-content {
    display: flex;
    align-items: center;
    padding: 5px 8px 5px 12px;
    cursor: pointer;
    border-radius: 4px;
    transition: background-color 0.1s ease;
    font-size: 13px;
    line-height: 16px;
    height: 26px;
    box-sizing: border-box;
    white-space: nowrap;
    contain: layout style;
}
.file-node-content:hover {
    background-color: rgba(255, 255, 255, 0.06);
}
.file-node-content:active {
    background-color: rgba(255, 255, 255, 0.08);
```

```
}
.folder-arrow {
    font-size: 16px;
    width: 16px;
    height: 16px;
    line-height: 16px;
    margin-right: 2px;
    color: var(--text-muted);
    font-variation-settings: 'FILL' 0, 'wght' 400, 'GRAD' 0, 'opsz' 16;
    flex-shrink: 0;
    transition: transform 0.15s ease;
    text-align: center;
}
.folder-icon {
    font-size: 16px;
    width: 16px;
    height: 16px;
    line-height: 16px;
    margin-right: 6px;
    color: var(--text-secondary);
    font-variation-settings: 'FILL' 1, 'wght' 400, 'GRAD' 0, 'opsz' 16;
    flex-shrink: 0;
    text-align: center;
}
.file-icon {
    font-size: 16px;
    width: 16px;
    height: 16px;
    line-height: 16px;
    margin-left: 18px;
    margin-right: 6px;
    color: var(--text-secondary);
    font-variation-settings: 'FILL' 0, 'wght' 400, 'GRAD' 0, 'opsz' 16;
    flex-shrink: 0;
    text-align: center;
}
.file-name {
    font-weight: 400;
    color: var(--text-primary);
    flex-shrink: 0;
    line-height: 16px;
}
.file-children {
    padding-left: 12px;
}
</style>
```

MarkdownCodeBlock.vue

```vue
<script setup lang="ts">
import {getKernelConnection, executePythonCodeAsync, pythonInputAsync} from "@/services/pythonService.ts";
import {ensureServiceConnection} from "@/services/serviceConnection.ts";
import {onMounted, onUnmounted, ref, nextTick, watch} from "vue";
import {Terminal} from "@xterm/xterm";
import {FitAddon} from "@xterm/addon-fit";
import '@xterm/xterm/css/xterm.css';
import terminalTheme from "@/styles/GithubTerminalTheme.json";
import hljs from 'highlight.js'
import 'highlight.js/styles/github-dark.css'
import {v4 as uuidv4} from "uuid";
import {useWorkspaceStore} from "@/stores/workspace.ts";
import {useKernelStore} from "@/stores/kernel.ts";
const props = defineProps<{ code: string, lang: string }>();
const mdTerminal = ref<HTMLDivElement | null>(null);
const terminal = ref<HTMLDivElement | null>(null); // Renamed to avoid confusion
const codeEl = ref<HTMLElement | null>(null)
const workspace = useWorkspaceStore()
const kernelStore = useKernelStore()
const terminalId: string = uuidv4();
let xterm: Terminal | null = null;
let fitAddon: FitAddon | null = null;
// Define callback reference for easier disposal later
const onOutputReceived = (targetId: string, output: string) => {
    if (terminalId !== targetId) return;
    // 1. Ensure container is visible
    if (mdTerminal.value && mdTerminal.value.style.display !== 'block') {
        mdTerminal.value.style.display = 'block';
        // After container becomes block, must re-fit, otherwise xterm thinks its size is 0
        // Use nextTick to ensure DOM rendering is complete
        nextTick(() => {
            fitAddon?.fit();
        });
    }
    // 2. Write to Xterm
    // Note: Handle line breaks, backend python print defaults to \n, xterm needs \r\n
    if (xterm) {
        // Simple line break fix to prevent stepped output
        const formatted = output.replace(/\n/g, '\r\n');
        xterm.write(formatted);
    }
};
```

```
const onCodeExecutionCompleted = (targetId: string) => {
    if (terminalId !== targetId) return;
    console.log(`[SignalR] Code execution completed for terminal ${terminalId}`);
    if (xterm) {
        xterm.options.cursorBlink = false;
        xterm.options.cursorStyle = 'block';
    }
};
onMounted(async () => {
    // 1. Initialize Terminal UI
    initializeTerminal();
    // Wait for kernel to be running
    if (!kernelStore.isRunning) {
        console.log('Waiting for kernel to start...')
        return
    }
    // 2. Register SignalR listener
    const connection = getKernelConnection()
    await ensureServiceConnection(connection)
    connection.off('CodeOutput');
    connection.on('CodeOutput', onOutputReceived);
    connection.on('CodeExecutionCompleted', onCodeExecutionCompleted);
});
onUnmounted(() => {
    // Cleanup
    if (kernelStore.isRunning) {
        try {
            const conn = getKernelConnection()
            conn.off('CodeOutput', onOutputReceived);
        } catch (e) {
            // Ignore if kernel not running
        }
    }
    xterm?.dispose();
    xterm = null;
});
watch(
    [() => props.code, () => props.lang],
    async () => {
        await nextTick()
        const el = codeEl.value
        if (!el) return
        // remove previous highlight
        delete (el as any).dataset.highlighted
        // hljs highlight
        hljs.highlightElement(el)
    },
```

```
    {immediate: true}
)
function initializeTerminal() {
    xterm = new Terminal({
        fontFamily: 'JetBrains Mono, monospace',
        cursorBlink: true,
        rows: 10,
        cols: 20,
        fontSize: 13,
        lineHeight: 1.2,
        theme: terminalTheme,
        convertEol: true
    });
    fitAddon = new FitAddon();
    xterm.loadAddon(fitAddon);
    xterm.open(terminal.value!);
    // Initial fit might be invalid (because parent container might be hidden), but do it anyway
    fitAddon.fit();
    xterm.onData(async data => {
        await pythonInputAsync(terminalId, data);
    })
}
async function executeCodeBlock() {
    // When clicking execute, show terminal and fit first
    if (mdTerminal.value) {
        mdTerminal.value.style.display = 'block';
        xterm?.clear(); // Clear previous run results
        fitAddon?.fit();
    }
    const venvPath = window.nodePath.join(workspace.rootDirectory, '.venv');
    await executePythonCodeAsync(terminalId, props.code, workspace.pythonInterpreterPath, venvPath)
}
function copyCodeBlock() {
    navigator.clipboard.writeText(props.code).then(() => {
    }).catch(err => {
        console.error('Failed to copy code: ', err)
    })
}
</script>
<template>
    <div class="md-code-wrapper">
        <div class="md-code-actions">
            <div class="md-icon-btn md-execute-btn" @click="executeCodeBlock">
                <span class="material-symbols-outlined">play_arrow</span>
            </div>
```

```
            <div class="md-icon-btn md-copy-btn" @click="copyCodeBlock">
                <span class="material-symbols-outlined">content_copy</span>
            </div>
        </div>
        <pre><code ref="codeEl" :class="`language-${lang}`">{{ code }}</code></pre>
        <div class="md-output-console" id="md-terminal" ref="mdTerminal">
            <div id="terminal" ref="terminal"></div>
        </div>
    </div>
</template>
<style scoped>
pre code.hljs {
    padding: 0;
}
</style>
```

MarkdownIndentedCodeBlock.vue

```
<script setup lang="ts">
import hljs from 'highlight.js'
import 'highlight.js/styles/github-dark.css'
import {nextTick, ref, watch} from "vue";
const props = defineProps<{ code: string }>();
const codeEl = ref<HTMLElement | null>(null)
watch(
    () => props.code,
    async () => {
        await nextTick()
        const el = codeEl.value
        if (!el) return
        // remove previous highlight
        delete (el as any).dataset.highlighted
        // hljs highlight
        hljs.highlightElement(el)
    },
    {immediate: true}
)
function copyCodeBlock() {
    navigator.clipboard.writeText(props.code).then(() => {
        console.log('Code copied to clipboard')
    }).catch(err => {
        console.error('Failed to copy code: ', err)
    })
}
</script>
<template>
    <div class="md-code-wrapper">
        <div class="md-code-actions">
```

```
            <div class="md-icon-btn md-copy-btn" @click="copyCodeBlock">
                <span class="material-symbols-outlined">content_copy</span>
            </div>
        </div>
        <pre><code ref="codeEl" class="nohighlight">{{ code }}</code></pre>
        <div class="md-output-console" id="md-terminal" ref="mdTerminal">
            <div id="terminal" ref="terminal"></div>
        </div>
    </div>
</template>
<style scoped>
pre code.hljs {
    padding: 0;
}
</style>
```

NavigationBar.vue

```
<script setup lang="ts">
import {onBeforeUnmount, onMounted, ref} from 'vue'
import {useWorkspaceStore} from '@/stores/workspace'
import {createDirectory} from "@/services/fileService.ts";
import {createPythonVenvAsync} from "@/services/pythonService.ts";
const workspace = useWorkspaceStore()
const isMenuOpen = ref(false)
const showOpenDialog = ref(false)
const showNewDialog = ref(false)
const openPath = ref('')
const newProjectName = ref('')
const newDirPath = ref('')
const pythonInterpreter = ref('')
const menuRef = ref<HTMLElement | null>(null)
const isMaximized = ref(true)
window.windowControls.onMaximize((maximised: boolean) => {
    isMaximized.value = maximised;
})
function toggleMenu() {
    isMenuOpen.value = !isMenuOpen.value
}
function openExistingWorkspace() {
    isMenuOpen.value = false
    showOpenDialog.value = true
}
function createWorkspace() {
    isMenuOpen.value = false
    showNewDialog.value = true
}
function closeDialogs() {
    showOpenDialog.value = false
```

```
    showNewDialog.value = false
}
async function confirmOpen() {
    const target = openPath.value.trim()
    if (!target) return
    try {
        await workspace.openWorkspace(target)
        openPath.value = ''
        closeDialogs()
    } catch (err) {
        console.error('Failed to open workspace', err)
    }
}
async function confirmCreate() {
    const dirPath = newDirPath.value.trim()
    const projectName = newProjectName.value.trim()
    const directoryPath =
window.nodePath.normalizeDisplay(window.nodePath.join(dirPath, projectName))
    const venvPath = window.nodePath.join(directoryPath, '.venv')
    if (!directoryPath) return
    try {
        await createDirectory(directoryPath)
        await createPythonVenvAsync(venvPath,
pythonInterpreter.value.trim())
        await workspace.createNewWorkspace({
            projectName: newProjectName.value.trim(),
            directoryPath,
            pythonInterpreter: pythonInterpreter.value.trim()
        })
        newProjectName.value = ''
        newDirPath.value = ''
        pythonInterpreter.value = ''
        closeDialogs()
    } catch (err) {
        console.error('Failed to create workspace', err)
    }
}
function handleGlobalClick(e: MouseEvent) {
    if (!isMenuOpen.value) return
    const target = e.target as Node
    if (menuRef.value && !menuRef.value.contains(target)) {
        isMenuOpen.value = false
    }
}
function handleGlobalKey(e: KeyboardEvent) {
    if (e.key === 'Escape') {
        isMenuOpen.value = false
    }
```

```
}
// Window control functions (Placeholder: Hook these up to Electron/Tauri)
const minimizeWindow = () => { window.windowControls.minimize()}
const maximizeWindow = () => { window.windowControls.maximize() }
const closeWindow = () => { window.windowControls.close() }
onMounted(() => {
    window.addEventListener('click', handleGlobalClick)
    window.addEventListener('keydown', handleGlobalKey)
})
onBeforeUnmount(() => {
    window.removeEventListener('click', handleGlobalClick)
    window.removeEventListener('keydown', handleGlobalKey)
})
</script>
<template>
    <div class="app-nav">
        <div class="nav-left">
            <div class="menu-root" ref="menuRef">
                <button class="nav-button" @click="toggleMenu">File</button>
                <div v-if="isMenuOpen" class="menu-dropdown">
                    <div class="menu-item"
@click="openExistingWorkspace">Open</div>
                    <div class="menu-item"
@click="createWorkspace">New</div>
                </div>
            </div>
        </div>
        <div class="nav-right window-controls">
            <div class="window-btn minimize" @click="minimizeWindow"
title="Minimize">
                <svg width="10" height="1" viewBox="0 0 10 1">
                    <path d="M0 0h10v1H0z" fill="currentColor"/>
                </svg>
            </div>
            <div class="window-btn maximize" @click="maximizeWindow"
title="Maximize">
                <svg v-if="isMaximized" width="10" height="10" viewBox="0 0
10 10">
                    <path d="M2.1 0v2H0v8.1h8.2v-2h2V0H2.1zm1.1 1h6v6h-
1V2.1H3.2V1zM1 3h6v6H1V3z" fill="currentColor"/>
                </svg>
                <svg v-else width="10" height="10" viewBox="0 0 10 10">
                    <path d="M1 1h8v8H1V1zm1 1v6h6V2H2z"
fill="currentColor"/>
                </svg>
            </div>
            <div class="window-btn close" @click="closeWindow"
title="Close">
```

```
                <svg width="10" height="10" viewBox="0 0 10 10">
                    <path d="M5 4.293L1.354.646l-.708.708L4.293 5 .646
8.646l.708.708L5 5.707l3.646 3.647.708-.708L5.707 5l3.647-3.646-.708-.708L5
4.293z" fill="currentColor"/>
                </svg>
            </div>
        </div>
    </div>
    <div v-if="showOpenDialog" class="dialog-backdrop"
@click.self="closeDialogs">
        <div class="dialog">
            <h3>Open Workspace</h3>
            <label class="input-group">
                <span>Directory path</span>
                <input v-model="openPath" type="text" placeholder="Enter
directory path" />
            </label>
            <div class="dialog-actions">
                <button @click="closeDialogs">Cancel</button>
                <button class="primary" @click="confirmOpen">Open</button>
            </div>
        </div>
    </div>
    <div v-if="showNewDialog" class="dialog-backdrop"
@click.self="closeDialogs">
        <div class="dialog">
            <h3>Create Workspace</h3>
            <label class="input-group">
                <span>Project name</span>
                <input v-model="newProjectName" type="text"
placeholder="Enter project name" />
            </label>
            <label class="input-group">
                <span>Directory path</span>
                <input v-model="newDirPath" type="text" placeholder="Enter
directory path" />
            </label>
            <label class="input-group">
                <span>Python interpreter path</span>
                <input v-model="pythonInterpreter" type="text"
placeholder="Enter python interpreter path" />
            </label>
            <div class="dialog-actions">
                <button @click="closeDialogs">Cancel</button>
                <button class="primary"
@click="confirmCreate">Create</button>
            </div>
        </div>
```

```
    </div>
</template>
<style scoped>
.app-nav {
    display: flex;
    align-items: center;
    justify-content: space-between;
    height: 35px;
    padding-left: 8px; /* Remove right padding so buttons hit the edge */
    background-color: var(--bg-secondary);
    border-bottom: 1px solid var(--widget-border-color);
    position: relative;
    z-index: 5;
    -webkit-app-region: drag;
}
.app-nav .nav-button,
.app-nav .menu-dropdown,
.app-nav .window-controls {
    -webkit-app-region: no-drag;
}
.nav-left {
    display: flex;
    align-items: center;
    gap: 8px;
}
/* Windows Style Window Controls */
.window-controls {
    display: flex;
    height: 100%;
}
.window-btn {
    display: flex;
    align-items: center;
    justify-content: center;
    width: 46px; /* Standard Windows width */
    height: 100%;
    border: none;
    background: transparent;
    color: var(--text-primary);
    cursor: default;
    transition: background-color 0.1s;
}
.window-btn svg {
    pointer-events: none; /* Let clicks pass through to div */
}
.window-btn:hover {
    background-color: var(--bg-hover); /* Subtle hover for min/max */
}
```

```css
/* Specific Close Button Styles */
.window-btn.close:hover {
    background-color: #e81123; /* Windows standard red */
    color: white;
}
.window-btn.close:active {
    background-color: #b90d1c;
    color: white;
}
/* File Menu Styles */
.menu-root {
    position: relative;
}
.nav-button {
    background: transparent;
    border: none;
    color: var(--text-secondary);
    padding: 4px 8px;
    border-radius: var(--border-radius-sm);
    cursor: default;
    font-size: 12px;
    transition: background-color var(--transition-fast);
}
.nav-button:hover {
    background-color: var(--bg-hover);
    color: var(--text-primary);
}
.menu-dropdown {
    position: absolute;
    top: 28px;
    left: 0;
    min-width: 160px;
    background-color: var(--bg-secondary);
    border: 1px solid var(--widget-border-color);
    box-shadow: var(--shadow-md);
    border-radius: var(--border-radius-sm);
    padding: 4px 0;
    z-index: 10;
}
.menu-item {
    padding: 8px 12px;
    color: var(--text-primary);
    cursor: default;
    font-size: 13px;
    transition: background-color var(--transition-fast);
}
.menu-item:hover {
    background-color: rgba(255, 255, 255, 0.06);
```

```
}
/* Dialog Styles (Unchanged) */
.dialog-backdrop {
    position: fixed;
    inset: 0;
    background: rgba(0, 0, 0, 0.4);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 20;
    padding: 16px;
}
.dialog {
    width: 380px;
    background: var(--bg-secondary);
    border: 1px solid var(--widget-border-color);
    border-radius: var(--border-radius-md);
    box-shadow: var(--shadow-lg);
    padding: 16px;
    color: var(--text-primary);
}
.dialog h3 {
    margin-bottom: 12px;
}
.input-group {
    display: flex;
    flex-direction: column;
    gap: 6px;
    margin-bottom: 12px;
    color: var(--text-secondary);
    font-size: 13px;
}
.input-group input {
    padding: 8px 10px;
    border-radius: var(--border-radius-sm);
    border: 1px solid var(--widget-border-color);
    background: var(--bg-tertiary);
    color: var(--text-primary);
}
.input-group input:focus {
    outline: 1px solid var(--accent-color);
}
.dialog-actions {
    display: flex;
    justify-content: flex-end;
    gap: 8px;
    margin-top: 4px;
}
```

```
</style>
```

Preview.vue

```
<script setup lang="ts">
import {storeToRefs} from 'pinia'
import {useWorkspaceStore} from '@/stores/workspace'
import '@/styles/github-markdown.css'
import 'highlight.js/styles/github-dark.css'
const workspace = useWorkspaceStore()
const {renderedVNode, previewPaneStyle, showPreviewPane} =
storeToRefs(workspace)
</script>
<template>
    <div class="preview-main" :style="previewPaneStyle" v-
show="showPreviewPane">
        <div class="markdown-body">
            <component :is="renderedVNode"/>
        </div>
    </div>
</template>
<style scoped>
.preview-main {
    display: flex;
    flex-direction: column;
    flex: 1 1 auto;
    min-width: 0;
    background-color: var(--bg-secondary);
    overflow: hidden;
}
.markdown-body {
    width: 100%;
    flex: 1;
    display: block;
    padding: 24px 48px;
    overflow-y: auto;
    background-color: var(--bg-secondary);
}
</style>
```

SearchPanel.vue

```
<script setup lang="ts">
import {ref, watch} from 'vue'
import {useWorkspaceStore} from '@/stores/workspace'
import {UIFileNode} from '@/types/workspace'
const workspace = useWorkspaceStore()
const searchQuery = ref('')
const searchResults = ref<UIFileNode[]>([])
// Debounced search
```

```
let searchTimeout: ReturnType<typeof setTimeout> | null = null
watch(searchQuery, (query) => {
    if (searchTimeout) {
        clearTimeout(searchTimeout)
    }
    searchTimeout = setTimeout(() => {
        searchResults.value = workspace.searchFiles(query)
    }, 150)
})
function handleResultClick(node: UIFileNode): void {
    workspace.openFile(node)
}
function getFileIcon(extension: string): string {
    const iconMap: Record<string, string> = {
        '.json': 'data_object',
        '.js': 'javascript',
        '.jsx': 'javascript',
        '.ts': 'javascript',
        '.tsx': 'javascript',
        '.md': 'markdown',
        '.mdx': 'markdown',
        '.png': 'image',
        '.jpg': 'image',
        '.jpeg': 'image',
        '.gif': 'image',
        '.svg': 'image',
        '.csv': 'csv',
        '.db': 'database',
        '.sql': 'database',
    }
    return iconMap[extension] || 'code'
}
/**
 * Get relative path from workspace root for display (preserving original
case).
 */
function getRelativePath(displayPath: string): string {
    const root = workspace.displayRootDirectory
    if (!root) return displayPath
    // Normalize both paths for comparison (lowercase)
    const normalizedPath = window.nodePath.normalize(displayPath)
    const normalizedRoot = window.nodePath.normalize(root)
    if (normalizedPath.startsWith(normalizedRoot + '\\') || normalizedPath
=== normalizedRoot) {
        // Use display path for the result but calculate offset from
normalized paths
        return displayPath.slice(root.length + 1)
    }
```

```
    return displayPath
}
</script>
<template>
    <div class="view-content">
        <div class="content-title-bar">
            <div class="title-label">
                <span>SEARCH</span>
            </div>
        </div>
        <div class="search-panel">
            <div class="search-input-wrapper">
                <span class="material-symbols-outlined search-icon">search</span>
                <input
                    v-model="searchQuery"
                    type="text"
                    placeholder="Search files..."
                    class="search-input"
                />
            </div>
            <div v-if="searchQuery && searchResults.length === 0" class="search-empty">
                <p>No files found</p>
            </div>
            <div v-else-if="searchResults.length > 0" class="search-results">
                <div
                    v-for="result in searchResults"
                    :key="result.path"
                    class="search-result-item"
                    @click="handleResultClick(result)"
                >
                    <span class="material-symbols-outlined file-icon">
                        {{ getFileIcon(result.extension) }}
                    </span>
                    <div class="result-info">
                        <span class="result-name">{{ result.name }}</span>
                        <span class="result-path">{{ getRelativePath(result.displayPath) }}</span>
                    </div>
                </div>
            </div>
            <div v-else class="search-placeholder">
                <p>Type to search files in workspace</p>
            </div>
        </div>
    </div>
```

```
</template>
<style scoped>
.search-panel {
    display: flex;
    flex-direction: column;
    padding: 8px;
    height: calc(100% - 35px);
    overflow: hidden;
}
.search-input-wrapper {
    display: flex;
    align-items: center;
    background-color: var(--bg-secondary);
    border: 1px solid var(--widget-border-color);
    border-radius: var(--border-radius-sm);
    padding: 0 8px;
}
.search-icon {
    font-size: 16px;
    color: var(--text-muted);
    margin-right: 6px;
}
.search-input {
    flex: 1;
    background: transparent;
    border: none;
    color: var(--text-primary);
    font-size: 13px;
    padding: 8px 0;
    outline: none;
}
.search-input::placeholder {
    color: var(--text-muted);
}
.search-results {
    flex: 1;
    overflow-y: auto;
    margin-top: 8px;
}
.search-result-item {
    display: flex;
    align-items: center;
    padding: 6px 8px;
    border-radius: var(--border-radius-sm);
    cursor: pointer;
    transition: background-color var(--transition-fast);
}
.search-result-item:hover {
```

```
    background-color: rgba(255, 255, 255, 0.06);
}
.file-icon {
    font-size: 16px;
    color: var(--text-secondary);
    margin-right: 8px;
    flex-shrink: 0;
    font-variation-settings: 'FILL' 0, 'wght' 400, 'GRAD' 0, 'opsz' 16;
}
.result-info {
    display: flex;
    flex-direction: column;
    min-width: 0;
}
.result-name {
    font-size: 13px;
    color: var(--text-primary);
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}
.result-path {
    font-size: 11px;
    color: var(--text-muted);
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}
.search-empty,
.search-placeholder {
    display: flex;
    align-items: center;
    justify-content: center;
    flex: 1;
    color: var(--text-muted);
    font-size: 13px;
    text-align: center;
}
</style>
```

Splash.vue

```
<script setup lang="ts">
import splashImage from '@/assets/splash.png'
</script>
<template>
    <div class="splash">
        <img :src="splashImage" alt="Loading..." />
    </div>
```

```
</template>
<style scoped>
.splash {
    position: fixed;
    inset: 0;
    display: flex;
    align-items: center;
    justify-content: center;
    background: #0d0d0d;
    z-index: 9999;
}
.splash img {
    max-width: 600px;
    max-height: 338px;
    object-fit: contain;
    -webkit-user-drag: none;
}
</style>
```

TabsBar.vue

```
<script setup lang="ts">
import {storeToRefs} from 'pinia'
import {useWorkspaceStore} from '@/stores/workspace'
import {ViewMode} from '@/types/workspace'
const props = defineProps<{
    viewModeButtons: { value: ViewMode, icon: string, label: string }[]
}>()
const workspace = useWorkspaceStore()
const {tabs, activeTabIndex, isMarkdownTab, currentViewMode} =
storeToRefs(workspace)
function onWheel(e: WheelEvent) {
    const el = e.currentTarget as HTMLElement
    el.scrollLeft += e.deltaY  // Horizontal scroll on vertical wheel
    el.scrollLeft += e.deltaX
}
</script>
<template>
    <div class="tab-bar">
        <div class="tab-list" @wheel="onWheel">
            <div
                v-for="(tab, index) in tabs"
                :key="tab.id"
                class="tab"
                :class="{ active: activeTabIndex === index }"
                @click="workspace.openTab(tab)"
            >
                <span>{{ tab.name }}</span>
```

```
                <div class="tab-action"
@click.stop="workspace.closeTab(tab)">
                    <span class="material-symbols-outlined">close</span>
                </div>
            </div>
        </div>
        <div v-if="isMarkdownTab" class="tab-view-toggle">
            <div
                v-for="btn in props.viewModeButtons"
                :key="btn.value"
                class="tab-view-btn"
                :class="{ active: currentViewMode === btn.value }"
                @click="workspace.setViewMode(btn.value)"
                :title="btn.label"
            >
                <span class="material-symbols-
outlined">{{ btn.icon }}</span>
            </div>
        </div>
    </div>
</template>
<style scoped>
.tab-bar {
    display: flex;
    height: 35px;
    background-color: var(--bg-tertiary);
    border-bottom: 1px solid var(--widget-border-color);
    overflow: hidden;
    align-items: stretch;
}
.tab-list {
    display: flex;
    flex: 1;
    min-width: 0;
    overflow-x: auto;
    scrollbar-width: none;
}
.tab-list::-webkit-scrollbar {
    display: none;
}
.tab-list::-webkit-scrollbar-track {
    background: transparent;
}
.tab-list::-webkit-scrollbar-thumb {
    background: var(--text-muted);
    border-radius: 2px;
}
.tab {
```

```
    display: flex;
    justify-content: space-between;
    align-items: center;
    min-width: 120px;
    max-width: 200px;
    height: 100%;
    padding: 0 4px 0 14px;
    font-size: 13px;
    color: var(--text-secondary);
    background-color: transparent;
    border-right: 1px solid var(--widget-border-color);
    cursor: pointer;
    position: relative;
    transition: background-color var(--transition-fast), color var(--
transition-fast);
    flex-shrink: 0;
}
.tab:hover {
    background-color: rgba(255, 255, 255, 0.03);
    color: var(--text-primary);
}
.tab.active {
    color: var(--text-primary);
    background-color: var(--bg-secondary);
}
.tab.active::after {
    content: '';
    position: absolute;
    bottom: 0;
    left: 0;
    right: 0;
    height: 1px;
    background-color: var(--bg-secondary);
}
.tab.active::before {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    height: 1px;
    background-color: var(--accent-color-secondary);
}
.tab span {
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
    flex: 1;
```

```
}
.tab-action {
    display: flex;
    height: 20px;
    width: 20px;
    margin-left: 4px;
    justify-content: center;
    text-align: center;
    align-items: center;
    cursor: pointer;
    user-select: none;
    border-radius: var(--border-radius-sm);
    color: var(--text-primary);
    opacity: 0;
    transition: opacity var(--transition-fast), background-color var(--
transition-fast), color var(--transition-fast);
}
.tab:hover .tab-action,
.tab.active .tab-action {
    opacity: 1;
}
.tab-action:hover {
    background-color: var(--button-hover);
    color: var(--text-primary);
}
.tab-action span {
    font-size: 16px;
    font-variation-settings: 'FILL' 0, 'wght' 400, 'GRAD' 0, 'opsz' 16;
}
.tab-view-toggle {
    display: flex;
    align-items: center;
    padding: 0 8px;
    gap: 4px;
    background-color: var(--bg-tertiary);
    flex-shrink: 0;
    z-index: 10;
}
.tab-view-btn {
    display: flex;
    align-items: center;
    justify-content: center;
    width: 24px;
    height: 24px;
    border-radius: var(--border-radius-sm);
    background-color: transparent;
    cursor: pointer;
    color: var(--text-secondary);
```

```
    transition: background-color var(--transition-fast), color var(--transition-fast), border-color var(--transition-fast);
    padding: 0;
}
.tab-view-btn:hover {
    color: var(--text-primary);
    border-color: var(--text-muted);
    background-color: var(--button-hover);
}
.tab-view-btn.active {
    color: var(--text-primary);
    border-color: var(--accent-color);
}
.tab-view-btn .material-symbols-outlined {
    font-size: 20px;
    font-variation-settings: 'FILL' 0, 'wght' 500, 'GRAD' 0, 'opsz' 20;
}
.tab-view-btn.active .material-symbols-outlined {
    color: var(--accent-color);
    font-variation-settings: 'FILL' 1, 'wght' 600, 'GRAD' 0, 'opsz' 20;
}
</style>
```

Terminal.vue

```
<script setup lang="ts">
import {nextTick, onMounted, onUnmounted, ref, watch, reactive, computed} from 'vue'
import {storeToRefs} from 'pinia'
import {Terminal as XTerminal} from '@xterm/xterm'
import {FitAddon} from '@xterm/addon-fit'
import '@xterm/xterm/css/xterm.css'
import {useWorkspaceStore} from '@/stores/workspace'
import {useKernelStore} from '@/stores/kernel'
import {
    initializeTerminal,
    terminalInputAsync,
    getTerminalConnection,
    closeTerminalSession
} from '@/services/terminalService.ts'
import {ensureServiceConnection} from "@/services/serviceConnection.ts";
import terminalTheme from '@/styles/GithubTerminalTheme.json'
import {v4 as uuidv4} from 'uuid'
interface TerminalTab {
    id: string
    name: string
    xterm: XTerminal | null
    fitAddon: FitAddon | null
    containerRef: HTMLElement | null
```

```
    fitAddonLoaded?: boolean
    disposed?: boolean
}
const workspace = useWorkspaceStore()
const kernelStore = useKernelStore()
const {showTerminal, layoutState, terminalStyle} = storeToRefs(workspace)
// Terminal tabs management
const terminalTabs = reactive<TerminalTab[]>([])
const activeTabIndex = ref(0)
// Computed active tab ID for v-show binding
const activeTabId = computed(() => terminalTabs[activeTabIndex.value]?.id)
const TERMINAL_CONFIG = {
    fontFamily: 'JetBrains Mono, monospace',
    cursorBlink: true,
    rows: 20,
    fontSize: 13,
    lineHeight: 1,
    letterSpacing: 0,
    theme: terminalTheme,
    convertEol: true,
}
/**
 * Handle terminal output from SignalR.
 */
function handleTerminalOutput(output: string, sessionId?: string): void {
    // Find the tab for this session
    const tab = sessionId
        ? terminalTabs.find(t => t.id === sessionId)
        : terminalTabs[activeTabIndex.value]
    if (tab?.xterm) {
        tab.xterm.write(output)
    }
}
/**
 * Resize terminal to fit container.
 */
function handleResize(): void {
    const activeTab = terminalTabs[activeTabIndex.value]
    if (activeTab?.fitAddon) {
        activeTab.fitAddon.fit()
    }
}
/**
 * Set container ref for a terminal tab.
 */
function setContainerRef(el: HTMLElement | null, tab: TerminalTab): void {
    // Always keep the latest element in case Vue re-creates the host node
    if (el) tab.containerRef = el
```

```
    if (el && !tab.xterm) {
        initializeXtermForTab(tab)
    }
}
/**
 * Create a new terminal tab.
 */
async function createTerminalTab(): Promise<void> {
    const tabId = uuidv4()
    const tabNumber = terminalTabs.length + 1
    const newTab: TerminalTab = {
        id: tabId,
        name: `Terminal ${tabNumber}`,
        xterm: null,
        fitAddon: null,
        containerRef: null
    }
    terminalTabs.push(newTab)
    activeTabIndex.value = terminalTabs.length - 1
    // Container will be set via ref callback
    await nextTick()
}
/**
 * Initialize xterm terminal for a specific tab.
 */
async function initializeXtermForTab(tab: TerminalTab): Promise<void> {
    if (!tab.containerRef) return
    const connection = getTerminalConnection()
    await ensureServiceConnection(connection)
    tab.disposed = false
    tab.fitAddonLoaded = false
    tab.xterm = new XTerminal(TERMINAL_CONFIG)
    tab.fitAddon = new FitAddon()
    tab.xterm.loadAddon(tab.fitAddon)
    tab.fitAddonLoaded = true
    tab.xterm.open(tab.containerRef)
    tab.fitAddon.fit()
    tab.xterm.onData(async data => {
        await terminalInputAsync(tab.id, data)
    })
    await initializeTerminal(tab.id, workspace.displayRootDirectory)
}
function disposeTabTerminal(tab: TerminalTab): void {
    if (tab.disposed) return
    tab.disposed = true
    // Dispose addon only if it was actually loaded for this terminal
instance
    if (tab.fitAddon && tab.fitAddonLoaded) {
```

```
        try {
            tab.fitAddon.dispose()
        } catch { /* noop */
        }
    }
    tab.fitAddon = null
    tab.fitAddonLoaded = false
    if (tab.xterm) {
        try {
            tab.xterm.dispose()
        } catch { /* noop */
        }
    }
    tab.xterm = null
    tab.containerRef = null
}
/**
 * Switch to a terminal tab.
 */
async function switchTab(index: number): Promise<void> {
    if (index < 0 || index >= terminalTabs.length) return
    if (index === activeTabIndex.value) return
    activeTabIndex.value = index
    // Fit the newly visible terminal
    await nextTick()
    const newTab = terminalTabs[index]
    if (newTab?.fitAddon) {
        newTab.fitAddon.fit()
    }
}
/**
 * Close a terminal tab.
 */
async function closeTab(index: number, event: Event): Promise<void> {
    event.stopPropagation()
    if (terminalTabs.length <= 1) return
    const tab = terminalTabs[index]
    if (!tab) return
    await closeTerminalSession(tab.id)
    disposeTabTerminal(tab)
    terminalTabs.splice(index, 1)
    // Adjust active index
    if (activeTabIndex.value >= terminalTabs.length) {
        activeTabIndex.value = terminalTabs.length - 1
    } else if (activeTabIndex.value > index) {
        activeTabIndex.value--
    }
    // Fit the now-active terminal
```

```
    await nextTick()
    handleResize()
}
// Watch: fit terminal when shown
watch(showTerminal, async (newVal) => {
    if (newVal) {
        await nextTick()
        handleResize()
    }
})
// Watch: resize terminal when height changes
watch(() => layoutState.value.terminalHeight, async () => {
    await nextTick()
    handleResize()
})
onMounted(async () => {
    if (!kernelStore.isRunning) {
        console.log('Terminal: Waiting for kernel...')
        return
    }
    const connection = getTerminalConnection()
    await ensureServiceConnection(connection)
    connection.on('TerminalOutput', handleTerminalOutput)
    connection.on('TerminalOutputSession', handleTerminalOutput)
    window.addEventListener('resize', handleResize)
    // Create initial terminal tab
    await createTerminalTab()
})
onUnmounted(async () => {
    if (kernelStore.isRunning) {
        try {
            const conn = getTerminalConnection()
            conn.off('TerminalOutput', handleTerminalOutput)
            conn.off('TerminalOutputSession', handleTerminalOutput)
        } catch (e) {
            // Ignore if kernel not running
        }
    }
    window.removeEventListener('resize', handleResize)
    for (const tab of terminalTabs) {
        disposeTabTerminal(tab)
    }
})
</script>
<template>
    <div class="terminal-container" :style="terminalStyle">
        <div class="terminal-title-bar">
            <div class="terminal-tabs">
```

```
                <div
                    v-for="(tab, index) in terminalTabs"
                    :key="tab.id"
                    class="terminal-tab"
                    :class="{ active: index === activeTabIndex }"
                    @click="switchTab(index)"
                >
                    <span class="tab-name">{{ tab.name }}</span>
                    <span
                        v-if="terminalTabs.length > 1"
                        class="material-symbols-outlined tab-close"
                        @click="closeTab(index, $event)"
                    >close</span>
                </div>
            </div>
            <div class="terminal-actions">
                <div class="terminal-action" @click="createTerminalTab"
title="New Terminal">
                    <span class="material-symbols-outlined">add</span>
                </div>
            </div>
        </div>
        <div class="terminal-hosts">
            <div
                v-for="tab in terminalTabs"
                :key="tab.id"
                class="terminal-host"
                :class="{ active: tab.id === activeTabId }"
                :ref="(el) => setContainerRef(el as HTMLElement, tab)"
            ></div>
        </div>
    </div>
</template>
<style scoped>
.terminal-container {
    display: flex;
    flex-direction: column;
    min-height: 150px;
    max-height: 80vh;
    background-color: var(--bg-tertiary);
    overflow: hidden;
}
.terminal-title-bar {
    display: flex;
    height: 35px;
    padding: 0 8px;
    font-size: 12px;
    align-items: center;
```

```
    justify-content: space-between;
    cursor: default;
    user-select: none;
    flex-shrink: 0;
    background-color: var(--bg-tertiary);
    border-bottom: 1px solid var(--widget-border-color);
}
.terminal-tabs {
    display: flex;
    align-items: center;
    gap: 2px;
    overflow-x: auto;
    flex: 1;
    min-width: 0;
}
.terminal-tab {
    display: flex;
    align-items: center;
    gap: 4px;
    padding: 4px 8px;
    border-radius: 4px;
    cursor: pointer;
    color: var(--text-secondary);
    transition: background-color var(--transition-fast), color var(--
transition-fast);
    white-space: nowrap;
}
.terminal-tab:hover {
    background-color: rgba(255, 255, 255, 0.06);
    color: var(--text-primary);
}
.terminal-tab.active {
    background-color: rgba(255, 255, 255, 0.1);
    color: var(--text-primary);
}
.tab-name {
    font-size: 12px;
    font-weight: 500;
}
.tab-close {
    font-size: 14px;
    padding: 2px;
    border-radius: 4px;
    opacity: 0.6;
    transition: opacity var(--transition-fast), background-color var(--
transition-fast);
}
.tab-close:hover {
```

```
    opacity: 1;
    background-color: rgba(255, 255, 255, 0.1);
}
.terminal-actions {
    display: flex;
    align-items: center;
    gap: 4px;
    flex-shrink: 0;
}
.terminal-action {
    display: flex;
    align-items: center;
    justify-content: center;
    width: 24px;
    height: 24px;
    border-radius: 4px;
    cursor: pointer;
    color: var(--text-secondary);
    transition: background-color var(--transition-fast), color var(--
transition-fast);
}
.terminal-action:hover {
    background-color: rgba(255, 255, 255, 0.1);
    color: var(--text-primary);
}
.terminal-action span {
    font-size: 18px;
}
.terminal-hosts {
    flex: 1;
    display: flex;
    position: relative;
    overflow: hidden;
}
.terminal-host {
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    overflow: hidden;
    background-color: var(--bg-tertiary);
    display: none;
}
.terminal-host.active {
    display: block;
}
.terminal-host :deep(.xterm) {
```

```
    padding: 12px 16px;
    height: 100%;
}
</style>
```

lexer.ts

```ts
import {Token, TokenType} from './types.ts';
import DOMPurify from 'dompurify';
/**
 * Delimiter for emphasis parsing
 */
interface Delimiter {
    type: '*' | '_';
    count: number;
    origCount: number;
    pos: number;
    canOpen: boolean;
    canClose: boolean;
    active: boolean;
}
/**
 * Match between opener and closer delimiters
 */
interface DelimiterMatch {
    openerIdx: number;
    closerIdx: number;
    count: number;
}
/**
 * The Lexer class is responsible for converting the raw Markdown string into a stream of tokens.
 * It uses regular expressions to identify block-level and inline-level elements.
 */
class Lexer {
    /**
     * Rules for block-level elements (headings, lists, code blocks, etc.).
     * Order matters: rules are matched sequentially.
     */
    private containerBlockRules: Array<[TokenType, RegExp]> = [
        [TokenType.BLOCKQUOTE, /^ {0,3}>\s?/],
        [TokenType.ULIST, /^( {0,3})([-*+])(?: {1,4}|\t|$)/],
        [TokenType.OLIST, /^( {0,3})(\d{1,9}[.])(?: {1,4}|\t|$)/],
    ];
    private leafBlockRules: Array<[TokenType, RegExp]> = [
        [TokenType.INDENTED_CODE_BLOCK, /^( {4})(.+)/],
        [TokenType.CODE_BLOCK, /^( {0,3})(`{3,}) *([^\s`]+)?(?:\n|$)/],
        [TokenType.HEADING, /^(#{1,6})\s/],
```

```
        [TokenType.HR, /^ {0,3}((_ *){3,}|(- *){3,}|(\* *){3,})(?:\n|$)/],
        [TokenType.HTML, /^ {0,3}<[A-Za-z!/?][^\n]*\n?/],
    ];
    /**
     * Rules for inline elements (bold, italic, links, etc.).
     * Order matters: more specific patterns should come first.
     * Note: Emphasis (* and _) is handled separately via delimiter-based
parsing
     */
    private inlineRules: Array<[TokenType, RegExp]> = [
        // Escape sequences (must be first to handle escaped characters)
        [TokenType.ESCAPE, /\\([\\`*_{}\[\]()#+\-.!|~])/],
        [TokenType.HARDBREAK, /(?: {2,}|\\)\n/],
        [TokenType.SOFTBREAK, /\n/],
        // Code (highest priority to avoid conflicts)
        [TokenType.CODE_INLINE, /`([^`\n]+)`/],
        // Images (must come before links due to leading !)
        [TokenType.IMAGE, /!\[([^\]]*)]\(([^)\s]+)(?:\s+"([^"]+)")?\)/],
        [TokenType.LINK, /\[([^\]]+)]\(([^)\s]+)(?:\s+"([^"]+)")?\)/],
        [TokenType.AUTOLINK, /<(https?:\/\/[a-zA-Z0-9][\w.-]*(?::[0-
9]+)?(?:\/[^\s>]*)?)>/],
        [TokenType.AUTOLINK, /<([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-
Z]{2,})>/],
        [TokenType.HTML, /<([A-Za-
z][\w:-]*)(?:\s[^>]*)?>([\s\S]*?)<\/\1\s*>|<(?:!--[\s\S]*?--|\/?[A-Za-
z][^>]*?>)/],
    ];
    /**
     * Tokenizes the input string into an array of tokens.
     * This is the main entry point for the Lexer.
     * @param input The raw Markdown string.
     * @returns An array of tokens.
     */
    tokenize(input: string): Token[] {
        const tokens: Token[] = [];
        while (input.length > 0) {
            // Skip empty lines between block elements (CommonMark
compliance)
            const emptyLineMatch = input.match(/^(\n+)/);
            if (emptyLineMatch && tokens.length > 0) {
                input = input.slice(emptyLineMatch[0].length);
                continue;
            }
            let matched: boolean = false;
            for (const [type, regex] of
[...this.containerBlockRules, ...this.leafBlockRules]) {
                const match: RegExpMatchArray | null = input.match(regex);
                if (match) {
```

```
                    input = input.slice(match[0].length);
                    const [token, length] = this.createToken(type, match,
input, {htmlBlock: type === TokenType.HTML});
                    tokens.push(token);
                    input = input.slice(length);
                    matched = true;
                    break;
                }
            }
            if (!matched) {
                const [token, length] =
this.createToken(TokenType.PARAGRAPH, null, input);
                tokens.push(token);
                input = input.slice(length);
            }
        }
        return tokens;
    }
    private parseLeafBlock(input: string): [tokens: Token[], length: number]
{
        const tokens: Token[] = [];
        let totalLength: number = 0;
        while (input.length > 0) {
            for (const [type, regex] of this.leafBlockRules) {
                const match: RegExpMatchArray | null = input.match(regex);
                if (match) {
                    input = input.slice(match[0].length);
                    const [token, length] = this.createToken(type, match,
input, {htmlBlock: type === TokenType.HTML});
                    tokens.push(token);
                    input = input.slice(length);
                    totalLength += match[0].length + length;
                    break;
                }
            }
            const [token, length] = this.createToken(TokenType.PARAGRAPH,
null, input);
            tokens.push(token);
            input = input.slice(length);
            totalLength += length;
        }
        return [tokens, totalLength];
    }
    private parseIndentedCodeBlock(input: string, firstLine: string, indent:
number): [result: string, length: number] {
        let lines: string[] = [];
        let idx: number = 0;
        lines.push(firstLine);
```

```
        while (idx < input.length) {
            const lineEnd: number = input.indexOf('\n', idx);
            const end: number = lineEnd === -1 ? input.length : lineEnd + 1;
            const line: string = input.slice(idx, end);
            if (this.isEmptyLine(line)) {
                lines.push(line);
                idx = end;
                continue;
            }
            let leadingSpaces = 0;
            while (leadingSpaces < line.length && line[leadingSpaces] === '
') leadingSpaces++;
            if (leadingSpaces >= indent) {
                lines.push(line.slice(indent));
                idx = end;
            } else {
                break;
            }
        }
        const code: string = lines.join('').trim();
        return [code, idx];
    }
    private parseCodeBlock(input: string, indent: number, length: number):
[result: string, length: number] {
        let lines: string[] = []
        let idx: number = 0
        const regex = new RegExp(`^ {0,3}\`{${length}}\\s*\\n?$`);
        while (idx < input.length) {
            const lineEnd: number = input.indexOf('\n', idx);
            const end: number = lineEnd === -1 ? input.length : lineEnd + 1;
            const line: string = input.slice(idx, end);
            let leadingSpaces = 0;
            // closing fence: 最多 3 个空格 + ``` + 可选空白 + 换行
            if (regex.test(line)) {
                idx = end;
                break;
            }
            while (leadingSpaces < line.length && line[leadingSpaces] === '
') leadingSpaces++;
            lines.push(line.slice(Math.min(leadingSpaces, indent)));
            idx = end;
        }
        let code: string = lines.join('');
        // Remove the trailing newline if it exists
        // if (code.endsWith('\n')) {
        //     code = code.slice(0, -1);
        // }
        return [code, idx];
```

```
169     }
170     private parseHeading(input: string): [result: string, length: number] {
171         const lineEnd: number = input.indexOf('\n');
172         const end: number = lineEnd === -1 ? input.length : lineEnd + 1;
173         const line: string = input.slice(0, end)
174         return [line.trim(), end];
175     }
176     private parseHtmlBlock(input: string, firstLine: string): [result:
    string, length: number] {
177         const lines: string[] = [firstLine];
178         let idx = 0;
179         while (idx < input.length) {
180             const lineEnd: number = input.indexOf('\n', idx);
181             const end: number = lineEnd === -1 ? input.length : lineEnd + 1;
182             const line: string = input.slice(idx, end);
183             if (this.isEmptyLine(line)) {
184                 break;
185             }
186             lines.push(line);
187             idx = end;
188         }
189         return [lines.join(''), idx];
190     }
191     private parseParagraph(input: string): [result: string, length: number]
    {
192         let lines: string[] = [];
193         let idx: number = 0;
194         while (idx < input.length) {
195             const lineEnd: number = input.indexOf('\n', idx);
196             const end: number = lineEnd === -1 ? input.length : lineEnd + 1;
197             const line: string = input.slice(idx, end);
198             let isBlockElement: boolean = false;
199             for (const [_, regex] of
    [...this.containerBlockRules, ...this.leafBlockRules]) {
200                 if (regex.test(line)) {
201                     isBlockElement = true;
202                     break;
203                 }
204             }
205             // Stop if we hit a block element (but not on first line)
206             if (isBlockElement && lines.length > 0) {
207                 break;
208             }
209             // Stop on blank line (paragraph break)
210             if (this.isEmptyLine(line) && lines.length > 0) {
211                 break;
212             }
213             lines.push(line);
```

```
            idx = end;
            // Stop if this is the first line and it's a block element
            if (isBlockElement && lines.length === 1) {
                break;
            }
        }
        return [lines.join('').trim(), idx];
    }
    private parseBlockquote(input: string): [result: Token[], length:
number] {
        let result: Token[] = [];
        let idx: number = 0;
        let explicits: string[] = [];
        while (idx < input.length) {
            const lineEnd: number = input.indexOf('\n', idx);
            const end: number = lineEnd === -1 ? input.length : lineEnd + 1;
            const line: string = input.slice(idx, end);
            // empty line, stop blockquote
            if (this.isEmptyLine(line)) {
                break;
            }
            const explicit = line.match(/^\s{0,3}>\s?(.*)\n?$/);
            if (explicit) {
                explicits.push(explicit[1] + '\n');
            } else if (idx === 0) {
                explicits.push(line);
            } else {
                result.push(...this.parseLeafBlock(explicits.join(''))[0]);
                explicits = [];
                const parsedLine: Token[] = this.parseLeafBlock(line)[0];
                if (parsedLine[0].type == TokenType.PARAGRAPH &&
result[result.length - 1].type == TokenType.PARAGRAPH) {
                    // Merge with previous paragraph
                    const prevParagraph = result[result.length - 1];
                    prevParagraph.raw += parsedLine[0].raw;
                    prevParagraph.text = (prevParagraph.text || '') +
(parsedLine[0].text || '');
                    prevParagraph.tokens =
this.parseInline(prevParagraph.raw);
                } else {
                    break;
                }
            }
            idx = end;
        }
        if (explicits.length > 0) {
            result.push(...this.parseLeafBlock(explicits.join(''))[0]);
        }
```

```
258         return [result, idx];
259     }
260     /**
261      * Check if a character is Unicode whitespace
262      */
263     private isUnicodeWhitespace(char: string | undefined): boolean {
264         if (!char) return true; // Beginning/end of line counts as
    whitespace
265         return /\s/.test(char);
266     }
267     /**
268      * Check if a character is Unicode punctuation
269      * This includes ASCII punctuation and Unicode punctuation categories
270      */
271     private isUnicodePunctuation(char: string | undefined): boolean {
272         if (!char) return false;
273         // ASCII punctuation
274         if (/[!"#$%&'()*+,\-./:;<=>?@\[\\\]^_`{|}~]/.test(char)) return
    true;
275         // Unicode punctuation categories: Pc, Pd, Pe, Pf, Pi, Po, Ps
276         return /\p{P}/u.test(char);
277     }
278     /**
279      * Check if a delimiter run is left-flanking
280      */
281     private isLeftFlanking(text: string, startIdx: number, length: number,
    _delimChar: string): boolean {
282         const after = text[startIdx + length];
283         const before = text[startIdx - 1];
284         // (1) not followed by Unicode whitespace
285         if (this.isUnicodeWhitespace(after)) return false;
286         // (2a) not followed by a Unicode punctuation character
287         if (!this.isUnicodePunctuation(after)) return true;
288         // (2b) followed by a Unicode punctuation character and
289         // preceded by Unicode whitespace or a Unicode punctuation character
290         return this.isUnicodeWhitespace(before) ||
    this.isUnicodePunctuation(before);
291     }
292     /**
293      * Check if a delimiter run is right-flanking
294      */
295     private isRightFlanking(text: string, startIdx: number, length: number,
    _delimChar: string): boolean {
296         const before = text[startIdx - 1];
297         const after = text[startIdx + length];
298         // (1) not preceded by Unicode whitespace
299         if (this.isUnicodeWhitespace(before)) return false;
300         // (2a) not preceded by a Unicode punctuation character
```

```
        if (!this.isUnicodePunctuation(before)) return true;
        // (2b) preceded by a Unicode punctuation character and
        // followed by Unicode whitespace or a Unicode punctuation character
        return this.isUnicodeWhitespace(after) ||
this.isUnicodePunctuation(after);
    }
    /**
     * Parse emphasis and strong emphasis using delimiter-based parsing
     * according to CommonMark specification
     */
    private parseEmphasis(text: string): Token[] {
        if (!text) return [];
        // First pass: identify all potential delimiter runs
        const delimiters: Delimiter[] = [];
        let i = 0;
        while (i < text.length) {
            const char = text[i];
            if (char === '*' || char === '_') {
                const startPos = i;
                let count = 0;
                while (i < text.length && text[i] === char) {
                    count++;
                    i++;
                }
                const leftFlanking = this.isLeftFlanking(text, startPos,
count, char);
                const rightFlanking = this.isRightFlanking(text, startPos,
count, char);
                let canOpen = false;
                let canClose = false;
                if (char === '*') {
                    canOpen = leftFlanking;
                    canClose = rightFlanking;
                } else { // '_'
                    canOpen = leftFlanking && (!rightFlanking ||
this.isUnicodePunctuation(text[startPos - 1]));
                    canClose = rightFlanking && (!leftFlanking ||
this.isUnicodePunctuation(text[startPos + count]));
                }
                delimiters.push({
                    type: char,
                    count: count,
                    origCount: count,
                    pos: startPos,
                    canOpen: canOpen,
                    canClose: canClose,
                    active: true
                });
```

```
            } else {
                i++;
            }
        }
        // Second pass: match openers with closers
        const matches: DelimiterMatch[] = [];
        for (let closerIdx = 0; closerIdx < delimiters.length; closerIdx++)
{
            const closer = delimiters[closerIdx];
            if (!closer.canClose || !closer.active) continue;
            // Look backwards for a matching opener
            let searchIdx = closerIdx - 1;
            while (closer.count > 0 && closer.active) {
                let matched = false;
                // Look backwards for a matching opener
                for (let openerIdx = searchIdx; openerIdx >= 0; openerIdx--)
{
                    const opener = delimiters[openerIdx];
                    if (!opener.canOpen || !opener.active || opener.type !==
closer.type) continue;
                    // Check the "multiple of 3" rule
                    const canOpenAndClose = opener.canOpen &&
opener.canClose;
                    const closerCanOpenAndClose = closer.canOpen &&
closer.canClose;
                    if (canOpenAndClose || closerCanOpenAndClose) {
                        const totalLength = opener.count + closer.count;
                        if (totalLength % 3 === 0 && opener.count % 3 !== 0
&& closer.count % 3 !== 0) {
                            continue;
                        }
                    }
                    // Prefer strong emphasis (use 2) over regular (use 1)
                    const useCount: number = Math.min(
                        (opener.count >= 2 && closer.count >= 2) ? 2 : 1,
                        opener.count,
                        closer.count
                    );
                    matches.push({openerIdx, closerIdx, count: useCount});
                    opener.count -= useCount;
                    closer.count -= useCount;
                    if (opener.count === 0) opener.active = false;
                    if (closer.count === 0) closer.active = false;
                    matched = true;
                    searchIdx = openerIdx - 1;
                    break;
                }
                if (!matched) {
```

```
                    break;
                }
            }
        }
        // Third pass: build tokens from text and matches
        if (matches.length === 0) {
            // No emphasis found, return as plain text
            return text ? [{
                type: TokenType.TEXT,
                raw: text.replace(/^ +| +$/gm, ''),
                text: text
            }] : [];
        }
        const openerUsage: Record<number, number> = {};
        const closerUsage: Record<number, number> = {};
        // First, assign concrete positions for each match so that
        // opener characters are taken from the right side of the run
        // and closer characters from the left side.
        const matchesWithPositions = matches
            .slice()
            .sort((a, b) => {
                const aClose = delimiters[a.closerIdx].pos;
                const bClose = delimiters[b.closerIdx].pos;
                if (aClose !== bClose) return aClose - bClose;
                const aOpen = delimiters[a.openerIdx].pos;
                const bOpen = delimiters[b.openerIdx].pos;
                return bOpen - aOpen;
            })
            .map(match => {
                const opener = delimiters[match.openerIdx];
                const closer = delimiters[match.closerIdx];
                const openerUsed = openerUsage[match.openerIdx] || 0;
                const closerUsed = closerUsage[match.closerIdx] || 0;
                // Use delimiter characters closest to the content for
openers (from the right)
                // so nested matches sharing a delimiter run allocate the
inner pairs first.
                const openerStart = opener.pos + (opener.origCount -
openerUsed - match.count);
                const openerEnd = openerStart + match.count;
                const closerStart = closer.pos + closerUsed;
                const closerEnd = closerStart + match.count;
                openerUsage[match.openerIdx] = openerUsed + match.count;
                closerUsage[match.closerIdx] = closerUsed + match.count;
                return {...match, openerStart, openerEnd, closerStart,
closerEnd};
            });
        // Sort matches by opener position for token construction
```

```
        matchesWithPositions.sort((a, b) => {
            if (a.openerStart !== b.openerStart) return a.openerStart - b.openerStart;
            return a.closerStart - b.closerStart;
        });
        const result: Token[] = [];
        let currentPos = 0;
        for (const match of matchesWithPositions) {
            const opener = delimiters[match.openerIdx];
            const closer = delimiters[match.closerIdx];
            // Calculate actual positions
            const openerStart = match.openerStart;
            const openerEnd = match.openerEnd;  // Position after matched delimiters
            const closerStart = match.closerStart;
            const closerEnd = match.closerEnd;  // Position after matched delimiters
            // Skip if this match would end at or before the text already emitted
            if (closerEnd <= currentPos) {
                continue;
            }
            // Add text before this emphasis
            if (currentPos < openerStart) {
                const beforeText = text.substring(currentPos, openerStart);
                result.push(...this.parseEmphasis(beforeText));
            }
            // Add unmatched opener delimiters as text
            if (opener.count > 0) {
                result.push({
                    type: TokenType.TEXT,
                    raw: opener.type.repeat(opener.count),
                    text: opener.type.repeat(opener.count)
                });
            }
            // Extract content between opener and closer
            const content = text.substring(openerEnd, closerStart);
            const emphType = match.count === 2 ? TokenType.BOLD : TokenType.ITALIC;
            // Recursively parse the content for nested emphasis
            const innerTokens = content ? this.parseEmphasis(content) : [];
            result.push({
                type: emphType,
                raw: opener.type.repeat(match.count) + content + closer.type.repeat(match.count),
                text: content,
                tokens: innerTokens
            });
```

```
            // Add unmatched closer delimiters as text
            if (closer.count > 0) {
                result.push({
                    type: TokenType.TEXT,
                    raw: closer.type.repeat(closer.count),
                    text: closer.type.repeat(closer.count)
                });
            }
            currentPos = closerEnd;
        }
        // Add remaining text after all matches
        if (currentPos < text.length) {
            const remainingText = text.substring(currentPos);
            result.push(...this.parseEmphasis(remainingText));
        }
        return result;
    }
    /**
     * Parses inline content within a block.
     * @param raw The raw string containing inline elements.
     * @returns An array of inline tokens.
     */
    private parseInline(raw: string): Token[] {
        const tokens: Token[] = [];
        let remaining: string = raw;
        while (remaining.length > 0) {
            let bestMatch: RegExpMatchArray | null = null;
            let bestType: TokenType | null = null;
            let bestIndex: number = Number.MAX_SAFE_INTEGER;
            // Try to match non-emphasis inline rules first
            for (const [type, regex] of this.inlineRules) {
                regex.lastIndex = 0;
                const match = regex.exec(remaining);
                if (match && match.index < bestIndex) {
                    bestMatch = match;
                    bestType = type;
                    bestIndex = match.index;
                }
            }
            if (!bestMatch || bestType === null) {
                tokens.push(...this.parseEmphasis(remaining));
                break;
            }
            if (bestIndex > 0) {
                const prefix = remaining.slice(0, bestIndex);
                tokens.push(...this.parseEmphasis(prefix));
            }
            tokens.push(this.createToken(bestType, bestMatch)[0]);
```

```
            remaining = remaining.slice(bestIndex + bestMatch[0].length);
        }
        return tokens;
    }
    /**
     * Parses an unordered list using line-by-line parsing.
     * An unordered list marker is a -, +, or * character.
     * @param input The input string starting after the first list marker
     * @param initialMarker The marker character (-, +, or *)
     * @param initialIndent The indentation of the first list item
     * @param initialMarkerWidth The width of the first marker including
indentation and following spaces
     * @returns [Token for the list, length consumed]
     */
    private parseUnorderedList(input: string, initialMarker: string,
initialIndent: number, initialMarkerWidth: number): [result: Token, length:
number] {
        const replacedMarker: string =
initialMarker.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
        const bulletMarkerRegex = new RegExp(`^
{0,3}([${replacedMarker}])(?: {1,4}|\\t|$)`);
        let items: Token[] = [];
        let idx: number;
        let loose: boolean = false;
        let currentItemLines: string[] = [];
        let blankLineCount: number = 0;
        let firstItemMarkerWidth: number = initialMarkerWidth;
        // Handle first line (marker already stripped by tokenizer)
        const firstLineEnd: number = input.indexOf('\n', 0);
        const firstLineIdx: number = firstLineEnd === -1 ? input.length :
firstLineEnd + 1;
        const line: string = input.slice(0, firstLineIdx);
        if (initialMarker === '*' || initialMarker === '-') {
            // Escape the special characters for regex
            const escapedMarker = initialMarker.replace(/[*-]/g, '\\$&');
            const HR = new RegExp(`^ *(${escapedMarker} *){2,}(?:\\n|$)`)
            if (HR.test(line)) {
                // It's a horizontal rule, not a list
                return [{
                    type: TokenType.HR,
                    raw: line,
                }, firstLineIdx];
            }
        }
        currentItemLines.push(line);
        idx = firstLineIdx;
        while (idx < input.length) {
            const lineEnd: number = input.indexOf('\n', idx);
```

```
            const end: number = lineEnd === -1 ? input.length : lineEnd + 1;
            const line: string = input.slice(idx, end);
            // Check for blank line
            if (this.isEmptyLine(line)) {
                if (currentItemLines.length > 0) {
                    blankLineCount++;
                    currentItemLines.push(line);
                }
                idx = end;
                continue;
            }
            // Check if this line starts a new list item
            const itemMatch = line.match(bulletMarkerRegex);
            if (itemMatch) {
                // Save previous item if exists
                if (currentItemLines.length > 0) {
                    if (blankLineCount > 0) {
                        loose = true;
                    }
                    const itemContent = currentItemLines.join('');
                    items.push(this.createListItemFromContent(itemContent,
initialIndent));
                    currentItemLines = [];
                    blankLineCount = 0;
                }
                const markerWidth = itemMatch[0].length;
                const lineContent = line.substring(markerWidth);
                currentItemLines.push(lineContent);
                firstItemMarkerWidth = markerWidth;
                idx = end;
            } else {
                // Check if line belongs to current item (indented
continuation)
                if (currentItemLines.length > 0) {
                    const leadingSpaces = line.match(/^ */)?.[0].length ||
0;
                    // Line is part of current item if indented by at least
the marker width
                    // or if it's a continuation after blank lines
                    if (leadingSpaces >= firstItemMarkerWidth ||
blankLineCount > 0) {
                        // Strip indentation equal to marker width
                        const stripped =
line.substring(Math.min(leadingSpaces, firstItemMarkerWidth));
                        currentItemLines.push(stripped);
                        blankLineCount = 0;
                        idx = end;
                    } else {
```

```
                        // Line doesn't belong to this list
                        break;
                    }
                } else {
                    // No current item, list is done
                    break;
                }
            }
        }
        // Save last item
        if (currentItemLines.length > 0) {
            if (blankLineCount > 0) {
                loose = true;
            }
            const itemContent = currentItemLines.join('');
            items.push(this.createListItemFromContent(itemContent,
initialIndent));
        }
        // Mark all items as loose if list is loose
        if (loose) {
            for (const item of items) {
                (item as any).loose = true;
            }
        }
        const raw = input.slice(0, idx);
        return [{
            type: TokenType.ULIST,
            raw: raw,
            loose: loose,
            tokens: items
        }, idx];
    }
    /**
     * Parses an ordered list using line-by-line parsing.
     * An ordered list marker is 1-9 digits followed by . or )
     * @param input The input string starting after the first list marker
     * @param initialMarker The marker string (e.g., "1.", "2)")
     * @param initialIndent The indentation of the first list item
     * @param initialMarkerWidth The width of the first marker including
indentation and following spaces
     * @returns [Token for the list, length consumed]
     */
    private parseOrderedList(input: string, initialMarker: string,
initialIndent: number, initialMarkerWidth: number): [result: Token, length:
number] {
        const delimiter: string = initialMarker.slice(-1); // . or )
        const marker: string = delimiter === '.' ? '\\.' : '\\)'
```

```
        const orderedMarkerRegex = new RegExp(`^
{0,3}(\\d{1,9}[${marker}])(?: {1,4}|\\t|$)`);
        let items: Token[] = [];
        let idx: number;
        let loose: boolean = false;
        let currentItemLines: string[] = [];
        let blankLineCount: number = 0;
        let start: number | undefined = parseInt(initialMarker, 10);
        let firstItemMarkerWidth: number = initialMarkerWidth;
        // Handle first line (marker already stripped by tokenizer)
        const firstLineEnd: number = input.indexOf('\n', 0);
        const firstLineIdx: number = firstLineEnd === -1 ? input.length :
firstLineEnd + 1;
        currentItemLines.push(input.slice(0, firstLineIdx));
        idx = firstLineIdx;
        while (idx < input.length) {
            const lineEnd: number = input.indexOf('\n', idx);
            const end: number = lineEnd === -1 ? input.length : lineEnd + 1;
            const line: string = input.slice(idx, end);
            // Check for blank line
            if (this.isEmptyLine(line)) {
                if (currentItemLines.length > 0) {
                    blankLineCount++;
                    currentItemLines.push(line);
                }
                idx = end;
                continue;
            }
            // Check if this line starts a new list item
            const itemMatch = line.match(orderedMarkerRegex);
            if (itemMatch) {
                // Save previous item if exists
                if (currentItemLines.length > 0) {
                    if (blankLineCount > 0) {
                        loose = true;
                    }
                    const itemContent = currentItemLines.join('');
                    items.push(this.createListItemFromContent(itemContent,
initialIndent));
                    currentItemLines = [];
                    blankLineCount = 0;
                }
                // Start new item - strip the marker and following spaces
                const markerWidth = itemMatch[0].length;
                const lineContent = line.substring(markerWidth);
                currentItemLines.push(lineContent);
                firstItemMarkerWidth = markerWidth;
                idx = end;
```

```
            } else {
                // Check if line belongs to current item (indented continuation)
                if (currentItemLines.length > 0) {
                    const leadingSpaces = line.match(/^ */)?.[0].length || 0;
                    // Line is part of current item if indented by at least the marker width
                    // or if it's a continuation after blank lines
                    if (leadingSpaces >= firstItemMarkerWidth || blankLineCount > 0) {
                        // Strip indentation equal to marker width
                        const stripped = line.substring(Math.min(leadingSpaces, firstItemMarkerWidth));
                        currentItemLines.push(stripped);
                        blankLineCount = 0;
                        idx = end;
                    } else {
                        // Line doesn't belong to this list
                        break;
                    }
                } else {
                    // No current item, list is done
                    break;
                }
            }
        }
        // Save last item
        if (currentItemLines.length > 0) {
            if (blankLineCount > 0) {
                loose = true;
            }
            const itemContent = currentItemLines.join('');
            items.push(this.createListItemFromContent(itemContent, initialIndent));
        }
        // Mark all items as loose if list is loose
        if (loose) {
            for (const item of items) {
                (item as any).loose = true;
            }
        }
        const raw = input.slice(0, idx);
        return [{
            type: TokenType.OLIST,
            raw: raw,
            start: start,
            loose: loose,
```

```
            tokens: items
        }, idx];
    }
    /**
     * Creates a list item token from content string.
     * @param content The content of the list item (marker already stripped)
     * @param indent The indentation of the list marker
     * @returns A ListItemToken
     */
    private createListItemFromContent(content: string, indent: number): Token {
        const isLoose: boolean = /\n\n/.test(content);
        return {
            type: TokenType.LIST_ITEM,
            raw: content,
            indent: indent,
            loose: isLoose,
            tokens: this.tokenize(content.trim())
        };
    }
    /**
     * Factory method to create tokens based on type and regex match.
     * @param type The type of token to create.
     * @param match The regex match result.
     * @param input The raw Markdown string.
     * @param options
     * @returns The created Token.
     */
    private createToken(type: TokenType, match: RegExpMatchArray | null, input: string = '', options: {
        htmlBlock?: boolean
    } = {}): [token: Token, length: number] {
        const raw: string = match ? match[0] : '';
        let marker: string;
        let indent: number;
        let markerWidth: number;
        let [result, length]: [string | Token[], number] = ['', 0];
        switch (type) {
            case TokenType.HEADING:
                [result, length] = this.parseHeading(input);
                return [{
                    type,
                    raw,
                    text: result,
                    depth: match![1].length as 1 | 2 | 3 | 4 | 5 | 6,
                    tokens: this.parseInline(result),
                }, length];
            case TokenType.CODE_BLOCK:
```

```
                [result, length] = this.parseCodeBlock(input, match![1].length, match![2].length);
                return [{
                    type,
                    raw,
                    text: result,
                    lang: match![3] || 'text',
                }, length];
            case TokenType.INDENTED_CODE_BLOCK:
                [result, length] = this.parseIndentedCodeBlock(input, match![2], match![1].length);
                return [{
                    type,
                    raw,
                    text: result,
                }, length];
            case TokenType.CODE_INLINE:
                return [{
                    type,
                    raw,
                    text: match![1],
                }, 0];
            case TokenType.BOLD:
            case TokenType.ITALIC:
            case TokenType.BOLD_ITALIC:
                return [{
                    type,
                    raw,
                    text: match![1],
                    tokens: this.parseInline(match![1]),
                }, 0];
            case TokenType.LINK:
                return [{
                    type,
                    raw,
                    text: match![1],
                    href: match![2],
                    title: match![3],
                    tokens: this.parseInline(match![1]),
                }, 0];
            case TokenType.AUTOLINK:
                return [{
                    type,
                    raw,
                    text: match![1],
                    href: match![1],
                }, 0];
            case TokenType.IMAGE:
```

```
                return [{
                    type,
                    raw,
                    text: match![1],
                    href: match![2],
                    title: match![3],
                    alt: match![1],
                }, 0];
            case TokenType.PARAGRAPH:
                [result, length] = this.parseParagraph(input)
                return [{
                    type,
                    raw,
                    text: result,
                    tokens: this.parseInline(result),
                }, length];
            case TokenType.HTML: {
                const firstLine: string = raw;
                const isBlock: boolean = options.htmlBlock === true;
                const [htmlContent, consumed] = isBlock ?
this.parseHtmlBlock(input, firstLine) : [firstLine, 0];
                const sanitized: string = DOMPurify.sanitize(htmlContent);
                return [{
                    type,
                    raw: htmlContent,
                    text: sanitized,
                    block: isBlock
                }, consumed];
            }
            case TokenType.BLOCKQUOTE:
                [result, length] = this.parseBlockquote(input)
                return [{
                    type,
                    raw,
                    text: '',
                    tokens: result,
                }, length];
            case TokenType.ULIST:
                marker = match![2];
                indent = match![1].length;
                markerWidth = match![0].length;
                return this.parseUnorderedList(input, marker, indent,
markerWidth);
            case TokenType.OLIST:
                marker = match![2];
                indent = match![1].length;
                markerWidth = match![0].length;
```

```
                return this.parseOrderedList(input, marker, indent,
    markerWidth);
            case TokenType.ESCAPE:
                return [{
                    type,
                    raw,
                    text: match![1],
                }, 0];
            case TokenType.NEWLINE:
            case TokenType.SOFTBREAK:
            case TokenType.HARDBREAK:
            case TokenType.HR:
                return [{
                    type,
                    raw,
                }, 0];
            default:
                return [{
                    type: TokenType.TEXT,
                    raw: raw,
                    text: raw,
                }, 0];
        }
    }
    private isEmptyLine(line: string): boolean {
        return /^\s*\n?$/.test(line);
    }
}
export {Lexer};
```

markdown.ts

```
import {Lexer} from './lexer';
import {Parser} from './parser';
import {Renderer} from './renderer.ts';
import {Preprocessor} from "./preprocessor.ts";
/**
 * Main class for the Markdown parser.
 * Orchestrates the parsing process by coordinating the Preprocessor, Lexer,
Parser, and Renderer.
 */
class MarkdownParser {
    private preprocessor = new Preprocessor();
    private lexer = new Lexer();
    private parser = new Parser();
    private renderer = new Renderer();
    /**
     * Converts a Markdown string into an HTML VNode.
     *
```

```
     * The process involves:
     * 1. Preprocessing: Normalizing the input (e.g., line endings).
     * 2. Tokenization (Lexing): Converting text into a stream of tokens.
     * 3. Parsing: Building an Abstract Syntax Tree (AST) from the tokens.
     * 4. Rendering: Traversing the AST to generate HTML.s
     *
     * @param input The raw Markdown string.
     * @returns The generated HTML string.
     */
    render(input: string): string {
        const markdown = this.preprocessor.preprocess(input);
        const tokens = this.lexer.tokenize(markdown);
        const ast = this.parser.parse(tokens);
        return this.renderer.render(ast);
    }
}
export {MarkdownParser};
```

markdownVue3.ts

```
import {Lexer} from './lexer';
import {Parser} from './parser';
import {Renderer} from './rendererVue3.ts';
import {Preprocessor} from "./preprocessor.ts";
import {VNode} from "vue";
/**
 * Main class for the Markdown parser.
 * Orchestrates the parsing process by coordinating the Preprocessor, Lexer, Parser, and Renderer.
 */
class MarkdownParser {
    private preprocessor = new Preprocessor();
    private lexer = new Lexer();
    private parser = new Parser();
    private renderer = new Renderer();
    /**
     * Converts a Markdown string into an HTML VNode.
     *
     * The process involves:
     * 1. Preprocessing: Normalizing the input (e.g., line endings).
     * 2. Tokenization (Lexing): Converting text into a stream of tokens.
     * 3. Parsing: Building an Abstract Syntax Tree (AST) from the tokens.
     * 4. Rendering: Traversing the AST to generate HTML.
     *
     * @param input The raw Markdown string.
     * @returns The generated HTML VNode.
     */
    render(input: string): VNode {
        const markdown = this.preprocessor.preprocess(input);
```

```
        const tokens = this.lexer.tokenize(markdown);
        const ast = this.parser.parse(tokens);
        return this.renderer.render(ast);
    }
}
export {MarkdownParser};
```

parser.ts

```
import {Token, TokenType} from './types.ts';
interface ASTNode {
    type: TokenType;
    children?: ASTNode[];
    value?: string;
    attributes?: Record<string, string|number|boolean>;
}
/**
 * The Parser class is responsible for converting a stream of Tokens into an Abstract Syntax Tree (AST).
 * It processes the tokens generated by the Lexer and structures them into a hierarchy.
 */
class Parser {
    /**
     * Parses an array of tokens into an AST.
     * @param tokens The array of tokens produced by the Lexer.
     * @returns The root node of the AST (usually a BODY node).
     */
    parse(tokens: Token[]): ASTNode {
        const root: ASTNode = {type: TokenType.BODY, children: []};
        for (const token of tokens) {
            const node: ASTNode = this.parseToken(token);
            root.children!.push(node);
        }
        return root;
    }
    /**
     * Extracts attributes from a token to be stored in the AST node.
     * This converts token properties (like depth, href, lang) into a generic attributes object.
     * @param token The token to extract attributes from.
     * @returns A record of attributes or undefined if none exist.
     */
    private extractAttributes(token: Token): Record<string, any> | undefined {
        const entries = Object.entries(token);
        // const attrs = this.attributeKeys.reduce((acc, key) => {
        //     if (token[key] !== undefined) {
        //         acc[key] = token[key];
```

```
        //     }
        //     return acc;
        // }, {} as Record<string, any>);
        if (entries.length === 0) {
            return undefined;
        }
        return Object.fromEntries(entries);
    }
    /**
     * Converts a single Token into an ASTNode.
     * Recursively processes child tokens if they exist.
     * @param token The token to parse.
     * @returns The corresponding ASTNode.
     */
    private parseToken(token: Token): ASTNode {
        // Recursively process nested tokens
        if (token.tokens) {
            return {
                type: token.type,
                children: token.tokens.map(t => this.parseToken(t)),
                attributes: this.extractAttributes(token),
            };
        }
        return {
            type: token.type,
            value: token.text || '',
            attributes: this.extractAttributes(token),
        };
    }
}
export {Parser};
export type {ASTNode}
```

preprocessor.ts

```
/**
 * The Preprocessor class normalizes the input Markdown string before tokenization.
 * This ensures consistent behavior across different operating systems and environments.
 */
class Preprocessor {
    private newlinesPattern = /\r\n|\r|\n/g;
    private tabPattern = /\t/g;
    /**
     * Preprocesses the input string.
     * Normalizes line endings to `\n` and trims empty lines from start and end.
     * @param input The raw Markdown string.
```

```
     * @returns The normalized string.
     */
    preprocess(input: string): string {
        // Normalize newlines to \n
        // Remove leading and trailing empty lines and replace null characters with U+FFFD
        return input.replace(this.newlinesPattern, '\n')
            .replace(this.tabPattern, '    ')
            .replace(/^\n+/, '')
            .replace(/\n+$/, '')
            .replace(/\x00/g, "\uFFFD");
    }
}
export {Preprocessor};
```

renderer.ts

```
import {ASTNode} from './parser';
import {TokenType} from './types';
import DOMPurify from 'dompurify';
function escapeHtml(str: string): string {
    return str
        // .replace(/&/g, '&amp;')
        // .replace(/</g, '&lt;')
        // .replace(/>/g, '&gt;')
        // .replace(/"/g, '&quot;')
        // .replace(/'/g, '&#39;');
}
function escapeAttr(str: string): string {
    return escapeHtml(str);
}
class Renderer {
    /**
     * Map of token types to their corresponding render functions.
     * Each function takes an ASTNode and returns an HTML string.
     */
    private handlers: Map<TokenType, (node: ASTNode) => string> = new Map<TokenType, (node: ASTNode) => string>([
        [TokenType.BODY, (n) => this.renderChildren(n)],
        [TokenType.HEADING, (n) => {
            const depth = n.attributes?.depth ?? 1;
            return `<h${depth}>${this.renderChildren(n)}</h${depth}>`;
        }],
        [TokenType.BOLD, (n) => `<strong>${this.renderChildren(n)}</strong>`],
        [TokenType.ITALIC, (n) => `<em>${this.renderChildren(n)}</em>`],
        [TokenType.BOLD_ITALIC, (n) => `<strong><em>${this.renderChildren(n)}</em></strong>`],
        [TokenType.CODE_BLOCK, (n) => {
```

```
            const lang: string = (n.attributes?.lang || 'plaintext') as string;
            const code = escapeHtml(n.value || '');
            return `<pre><code class="language-${escapeAttr(lang)}">${code}</code></pre>`;
        }],
        [TokenType.INDENTED_CODE_BLOCK, (n) => {
            const code = escapeHtml(n.value || '');
            return `<pre><code>${code}</code></pre>`;
        }],
        [TokenType.CODE_INLINE, (n) => `<code>${escapeHtml(n.value || '')}</code>`],
        [TokenType.LINK, (n) => {
            const href = n.attributes?.href ? ` href="${escapeAttr(n.attributes.href as string)}"` : '';
            const title = n.attributes?.title ? ` title="${escapeAttr(n.attributes.title as string)}"` : '';
            return `<a${href}${title}>${this.renderChildren(n)}</a>`;
        }],
        [TokenType.AUTOLINK, (n) => {
            const href = n.attributes?.href ? escapeAttr(n.attributes.href as string) : '';
            const text = escapeHtml(n.value || href);
            return `<a href="${href}">${text}</a>`;
        }],
        [TokenType.IMAGE, (n) => {
            const src = escapeAttr((n.attributes?.href || '') as string);
            const alt = escapeAttr(n.value || '');
            const title = n.attributes?.title ? ` title="${escapeAttr(n.attributes.title as string)}"` : '';
            return `<img src="${src}" alt="${alt}" loading="lazy"${title}>`;
        }],
        [TokenType.BLOCKQUOTE, (n) => `<blockquote>${this.renderChildren(n)}</blockquote>`],
        [TokenType.ULIST, (node) => `<ul>${this.renderChildren(node)}</ul>`],
        [TokenType.OLIST, (node) => {
            const start = node.attributes?.start;
            const startAttr = start && start !== 1 ? ` start="${start}"` : '';
            return `<ol${startAttr}>${this.renderChildren(node)}</ol>`;
        }],
        [TokenType.LIST_ITEM, (node) => {
            if (node.attributes?.loose) {
                // Loose: contains paragraphs
                return `<li>${this.renderChildren(node)}</li>`;
            }
            // Tight: unwrap paragraphs
```

```
            const children: Array<string> = node.children?.flatMap(c => {
                if (c.type === 'paragraph') {
                    return this.renderChildren(c); // unwrap paragraph
                }
                return this.renderNode(c);
            }) || [escapeHtml(node.value || '')];
            return `<li>${children.join('')}</li>`;
        }],
        [TokenType.NEWLINE, () => ``],
        [TokenType.SOFTBREAK, () => `\n`],
        [TokenType.HARDBREAK, () => `<br>`],
        [TokenType.HR, () => `<hr>`],
        [TokenType.ESCAPE, (n) => escapeHtml(n.value || '')],
        [TokenType.TEXT, (n) => escapeHtml(n.value || '')],
        [TokenType.PARAGRAPH, (n) => `<p>${this.renderChildren(n)}</p>`],
        [TokenType.HTML, (n) => {
            // DOMPurify filtering
            return DOMPurify.sanitize(n.value || '');
        }],
    ]);
    /**
     * Renders the AST into an HTML string.
     * @param ast The root node of the AST.
     * @returns The generated HTML string.
     */
    render(ast: ASTNode): string {
        return this.renderNode(ast);
    }
    /**
     * Renders a single node by looking up its handler.
     * If no handler is found, it falls back to rendering the node's
children.
     * @param node The node to render.
     * @returns The HTML string for the node.
     */
    private renderNode(node: ASTNode): string {
        const handler = this.handlers.get(node.type);
        return handler ? handler(node) : this.renderChildren(node);
    }
    /**
     * Renders the children of a node and joins them into a single string.
     * If the node has no children, it returns the node's value (text
content).
     * @param node The parent node.
     * @returns The concatenated HTML string of the children.
     */
    private renderChildren(node: ASTNode): string {
        if (!node.children || node.children.length === 0) {
```

```
            return escapeHtml(node.value || '');
        }
        return node.children.map(c => this.renderNode(c)).join('');
    }
}
export {Renderer};
```

rendererVue3.ts

```
import {ASTNode} from './parser';
import {TokenType} from './types';
import {Component, h, VNode, Fragment} from 'vue'
import DOMPurify from 'dompurify';
import MarkdownCodeBlock from "@/components/MarkdownCodeBlock.vue";
import MarkdownIndentedCodeBlock from
"@/components/MarkdownIndentedCodeBlock.vue";
/**
 * The Renderer class is responsible for converting the Abstract Syntax Tree
(AST) into an HTML string.
 * It uses a map of handlers to determine how to render each type of node.
 */
class Renderer {
    /**
     * Map of token types to their corresponding render functions.
     * Each function takes an ASTNode and returns an HTML string.
     */
    private handlers: Map<TokenType, (node: ASTNode) => VNode> = new
Map<TokenType, (node: ASTNode) => VNode>([
        [TokenType.BODY, (n) => this.renderChildren(n)],
        [TokenType.HEADING, (n) => h(`h${n.attributes?.depth}`,
this.renderChildren(n))],
        [TokenType.BOLD, (n) => h('strong', this.renderChildren(n))],
        [TokenType.ITALIC, (n) => h('em', this.renderChildren(n))],
        [TokenType.BOLD_ITALIC, (n) => h('strong', h('em',
this.renderChildren(n)))],
        [TokenType.CODE_BLOCK, (n) => h(MarkdownCodeBlock as Component, {
            code: n.value || '', lang: n.attributes?.lang || 'plaintext'
        })],
        [TokenType.INDENTED_CODE_BLOCK, (n) => h(MarkdownIndentedCodeBlock
as Component, {
            code: n.value || ''
        })],
        [TokenType.CODE_INLINE, (n) => h('code', n.value)],
        [TokenType.LINK, (n) => {
            const props: Record<string, any> = { href: n.attributes?.href };
            if (n.attributes?.title) props.title = n.attributes.title;
            return h('a', props, this.renderChildren(n));
        }],
```

```
        [TokenType.AUTOLINK, (n) => h('a', {href: n.attributes?.href},
n.value || '')],
        [TokenType.IMAGE, (n) => {
            const props: Record<string, any> = {
                src: n.attributes?.href || '',
                alt: n.value || '',
                loading: 'lazy'
            };
            if (n.attributes?.title) props.title = n.attributes.title;
            return h('img', props);
        }],
        [TokenType.BLOCKQUOTE, (n) => h('blockquote',
this.renderChildren(n))],
        [TokenType.ULIST, (node) => {
            return h('ul', this.renderChildren(node));
        }],
        [TokenType.OLIST, (node) => {
            const props: Record<string, any> = {};
            const start = node.attributes?.start;
            if (start && start !== 1) props.start = start;
            return h('ol', props, this.renderChildren(node));
        }],
        [TokenType.LIST_ITEM, (node) => {
            if (node.attributes?.loose) {
                // Loose: contains paragraphs
                return h('li', this.renderChildren(node))
            }
            // Tight: unwrap paragraphs
            const children: Array<VNode | string> = node.children?.flatMap(c
=> {
                if (c.type === 'paragraph') {
                    return this.renderChildren(c)  // unwrap paragraph
                }
                return [this.renderNode(c)]
            }) || [node.value || '']
            return h('li', children)
        }],
        [TokenType.NEWLINE, () => h(Fragment, [''])],
        [TokenType.SOFTBREAK, () => h(Fragment, [' '])],
        [TokenType.HARDBREAK, () => h('br')],
        [TokenType.HR, () => h('hr')],
        [TokenType.ESCAPE, (n) => h(Fragment, [n.value || ''])],
        [TokenType.TEXT, (n) => h(Fragment, [n.value || ''])],
        [TokenType.PARAGRAPH, (n) => h('p', this.renderChildren(n)) || ''],
        [TokenType.HTML, (n) => {
            const tag = n.attributes?.block ? 'div' : 'span';
            return h(tag, { innerHTML: DOMPurify.sanitize(n.value || '') });
        }],
```

```
    ]);
    /**
     * Renders the AST into an HTML string.
     * @param ast The root node of the AST.
     * @returns The generated HTML string.
     */
    render(ast: ASTNode): VNode {
        return this.renderNode(ast);
    }
    /**
     * Renders a single node by looking up its handler.
     * If no handler is found, it falls back to rendering the node's children.
     * @param node The node to render.
     * @returns The HTML string for the node.
     */
    private renderNode(node: ASTNode): VNode {
        const handler = this.handlers.get(node.type);
        return handler ? handler(node) : this.renderChildren(node);
    }
    /**
     * Renders the children of a node and joins them into a single string.
     * If the node has no children, it returns the node's value (text content).
     * @param node The parent node.
     * @returns The concatenated HTML string of the children.
     */
    private renderChildren(node: ASTNode): VNode {
        if (!node.children || node.children.length === 0) {
            // Text nodes are also wrapped as VNodes
            return h(Fragment, [node.value || ''])
        }
        const childrenVNodes = node.children.map(c => this.renderNode(c))
        return h(Fragment, childrenVNodes)
    }
}
export {Renderer};
```

types.ts

```
/**
 * Enum representing the different types of tokens supported by the Markdown parser.
 * Using const enum for performance optimization (inlined at compile time).
 */
const enum TokenType {
    BODY = 'body',
    HEADING = 'heading',
    PARAGRAPH = 'paragraph',
```

```
    CODE_BLOCK = 'code_block',
    INDENTED_CODE_BLOCK = 'indented_code_block',
    CODE_INLINE = 'code_inline',
    BOLD = 'bold',
    ITALIC = 'italic',
    BOLD_ITALIC = 'bold_italic',
    LINK = 'link',
    AUTOLINK = 'autolink',
    IMAGE = 'image',
    ULIST = 'ulist',
    OLIST = 'olist',
    LIST_ITEM = 'list_item',
    BLOCKQUOTE = 'blockquote',
    HR = 'hr',
    ESCAPE = 'escape',
    TEXT = 'text',
    NEWLINE = 'newline',
    SOFTBREAK = 'softbreak',
    HARDBREAK = 'hardbreak',
    HTML = 'html',
}
/**
 * Base interface for all tokens.
 * Uses a generic constraint to ensure type safety for the token type.
 */
interface BaseToken<T extends TokenType = TokenType> {
    readonly type: T;
    raw: string;        // The raw Markdown string for this token
    text?: string;      // The text content of the token (if applicable)
    tokens?: Token[];   // Nested tokens (for block tokens that contain
inline tokens)
}
/**
 * Token representing a heading (h1-h6).
 */
interface HeadingToken extends BaseToken<TokenType.HEADING> {
    readonly depth: 1 | 2 | 3 | 4 | 5 | 6;
}
/**
 * Token representing a code block.
 */
interface CodeBlockToken extends BaseToken<TokenType.CODE_BLOCK> {
    readonly lang: string; // The programming language for syntax
highlighting
}
interface IndentedCodeBlockToken extends
BaseToken<TokenType.INDENTED_CODE_BLOCK> {
}
```

```
/**
 * Common interface for tokens that have a link destination (links and images).
 */
interface LinkableToken<T extends TokenType> extends BaseToken<T> {
    readonly href: string;
    readonly title?: string;
}
interface LinkToken extends LinkableToken<TokenType.LINK> {
}
interface AutolinkToken extends LinkableToken<TokenType.AUTOLINK> {
}
interface ImageToken extends LinkableToken<TokenType.IMAGE> {
    readonly alt: string;
}
/**
 * Token representing a list (ordered or unordered).
 */
interface OListToken extends BaseToken<TokenType.OLIST> {
    readonly start?: number;   // Starting number for ordered lists
    readonly loose?: boolean;  // True if list items are separated by newlines (loose list)
}
interface UListToken extends BaseToken<TokenType.ULIST> {
    readonly loose?: boolean;  // True if list items are separated by newlines (loose list)
}
/**
 * Token representing a single item in a list.
 */
interface ListItemToken extends BaseToken<TokenType.LIST_ITEM> {
    readonly indent: number;   // Indentation level
    readonly checked?: boolean; // For task lists (not fully implemented yet)
    readonly loose?: boolean;  // True if this item contains multiple blocks or is in a loose list
}
// Inline text types
type InlineTextType =
    TokenType.TEXT
    | TokenType.BOLD
    | TokenType.ITALIC
    | TokenType.BOLD_ITALIC
    | TokenType.CODE_INLINE
    | TokenType.ESCAPE
    | TokenType.HARDBREAK
    | TokenType.SOFTBREAK;
interface TextToken extends BaseToken<InlineTextType> {
```

```
    readonly type: InlineTextType;
}
// Simple types
type SimpleTokenType =
    TokenType.HR
    | TokenType.BLOCKQUOTE
    | TokenType.PARAGRAPH
    | TokenType.BODY
    | TokenType.NEWLINE
    | TokenType.HTML;
interface SimpleToken extends BaseToken<SimpleTokenType> {
    readonly type: SimpleTokenType;
}
interface HtmlToken extends BaseToken<TokenType.HTML> {
    readonly block?: boolean;
}
// Union type
type Token =
    | HeadingToken
    | CodeBlockToken
    | IndentedCodeBlockToken
    | LinkToken
    | AutolinkToken
    | ImageToken
    | UListToken
    | OListToken
    | ListItemToken
    | TextToken
    | SimpleToken
    | HtmlToken;
export {TokenType};
export type {
    Token,
    BaseToken,
    HeadingToken,
    CodeBlockToken,
    LinkToken,
    AutolinkToken,
    ImageToken,
    UListToken,
    OListToken,
    ListItemToken,
    TextToken,
    SimpleToken,
    HtmlToken
};
```

fileService.ts

```ts
import * as signalR from '@microsoft/signalr'
import {useKernelStore} from '@/stores/kernel'
import {ensureServiceConnection} from '@/services/serviceConnection.ts'
import {FileNode, WorkspaceSettingsDTO, TabChunk} from '@/types/workspace'
let fileServiceConnection: signalR.HubConnection | null = null
let fileServiceConnectionUrl: string | null = null
/**
 * Get or create the SignalR connection for file service operations.
 */
function getConnection(): signalR.HubConnection {
    const kernelStore = useKernelStore()
    const baseUrl = kernelStore.baseUrl
    if (!baseUrl) {
        throw new Error('Kernel not running - no port available')
    }
    const url = `${baseUrl}/fileHub`
    if (!fileServiceConnection || fileServiceConnectionUrl !== url) {
        if (fileServiceConnection) {
            fileServiceConnection.stop().then(() => {
            })
        }
        fileServiceConnection = new signalR.HubConnectionBuilder()
            .withUrl(url)
            .withAutomaticReconnect()
            .build()
        fileServiceConnectionUrl = url
    }
    return fileServiceConnection
}
export {fileServiceConnection}
/**
 * Read directory contents via streaming.
 */
export async function readDirectory(directoryPath: string) {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    return connection.stream<FileNode>('ReadDirAsync', directoryPath)
}
/**
 * Stream file content for opening in a tab.
 */
export async function streamTab(filePath: string) {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    return connection.stream<TabChunk>('StreamTabAsync', filePath)
}
```

```typescript
/**
 * Save file content via streaming.
 * @param filePath The target file path
 * @param content The content to save
 * @param encoding The encoding to use when saving (default: utf-8)
 */
export async function saveTabAsync(filePath: string, content: string, encoding: string = 'utf-8'): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    const subject = new signalR.Subject<string>()
    const chunkSize = 64 * 1024
    const task = connection.invoke("SaveFileAsync", filePath, subject, encoding)
    for (let offset = 0; offset < content.length; offset += chunkSize) {
        subject.next(content.slice(offset, offset + chunkSize))
    }
    subject.complete()
    try {
        await task
        console.log("SaveFileAsync complete")
    } catch (err) {
        console.error("SaveFileAsync error", err)
    }
}
/**
 * Save workspace settings to disk.
 */
export async function saveWorkspaceSettingsAsync(cwd: string, layout: WorkspaceSettingsDTO): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.invoke("SaveWorkspaceSettingsAsync", cwd, layout)
}
/**
 * Load workspace settings from disk.
 */
export async function loadLayoutAsync(dirPath: string): Promise<WorkspaceSettingsDTO | null> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    return await connection.invoke<WorkspaceSettingsDTO | null>("LoadWorkspaceSettingsAsync", dirPath)
}
export async function createDirectory(dirPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("CreateDirectory", dirPath)
```

```
}
export async function deleteDirectory(dirPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("DeleteDirectory", dirPath)
}
export async function moveDirectory(sourcePath: string, destPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("MoveDirectory", sourcePath, destPath)
}
export async function RenameDirectory(sourcePath: string, destPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("RenameDirectory", sourcePath, destPath)
}
export async function copyDirectory(sourcePath: string, destPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("CopyDirectory", sourcePath, destPath)
}
export async function createFile(filePath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("CreateFile", filePath)
}
export async function deleteFile(filePath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("DeleteFile", filePath)
}
export async function moveFile(sourcePath: string, destPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("MoveFile", sourcePath, destPath)
}
export async function renameFile(sourcePath: string, destPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("RenameFile", sourcePath, destPath)
}
export async function copyFile(sourcePath: string, destPath: string): Promise<void> {
```

```
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.send("CopyFile", sourcePath, destPath)
}
/**
 * Start watching a directory for file changes.
 */
export async function startWatching(directoryPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.invoke("StartWatchingAsync", directoryPath)
}
/**
 * Stop watching a directory for file changes.
 */
export async function stopWatching(directoryPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    await connection.invoke("StopWatchingAsync", directoryPath)
}
/**
 * Register a callback for file change events.
 * @returns A function to unregister the callback.
 */
export function onFileChanged(callback: (dirPath: string) => void): () =>
void {
    const connection = getConnection()
    connection.on('DirectoryDirty', callback)
    return () => connection.off('DirectoryDirty', callback)
}
```

pythonService.ts

```
import * as signalR from '@microsoft/signalr'
import { useKernelStore } from '@/stores/kernel'
import { ensureServiceConnection} from "@/services/serviceConnection.ts";
let kernelConnection: signalR.HubConnection | null = null
let kernelConnectionUrl: string | null = null
/**
 * Get or create the SignalR connection for Python code execution.
 */
function getConnection(): signalR.HubConnection {
    const kernelStore = useKernelStore()
    const baseUrl = kernelStore.baseUrl
    if (!baseUrl) {
        throw new Error('Kernel not running - no port available')
    }
    const url = `${baseUrl}/pythonHub`
    if (!kernelConnection || kernelConnectionUrl !== url) {
```

```
        if (kernelConnection) {
            kernelConnection.stop().then(() => {})
        }
        kernelConnection = new signalR.HubConnectionBuilder()
            .withUrl(url)
            .withAutomaticReconnect()
            .build()
        kernelConnectionUrl = url
    }
    return kernelConnection
}
export { kernelConnection, getConnection as getKernelConnection }
export async function createPythonVenvAsync(venvPath: string,
pythonInterpreterPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    try {
        await connection.invoke('CreateVenvAsync', venvPath,
pythonInterpreterPath)
    } catch (err) {
        console.error('Python virtual environment creation failed:', err)
    }
}
export async function executePythonCodeAsync(terminalId: string, code:
string, pythonInterpreterPath: string, venvPath: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    try {
        await connection.invoke('ExecuteCodeAsync', terminalId, code,
pythonInterpreterPath, venvPath)
    } catch (err) {
        console.error('Code execution failed:', err)
    }
}
export async function pythonInputAsync(terminalId: string, data: string):
Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    try {
        await connection.invoke('PythonInput', terminalId, data)
    } catch (err) {
        console.error('Python input failed:', err)
    }
}
```

serviceConnection.ts

```
import * as signalR from '@microsoft/signalr'
import {HubConnection} from '@microsoft/signalr'
```

```
/*
 * Ensure that the given SignalR HubConnection is connected.
 * If it is disconnected, it will start the connection.
 * If it is connecting or reconnecting, it will wait until connected.
 * Concurrent calls for the same connection will share the same start promise.
 */
const pending = new WeakMap<HubConnection, Promise<void>>()
export async function ensureServiceConnection(connection: HubConnection): Promise<void> {
    // already connected
    if (connection.state === signalR.HubConnectionState.Connected) {
        return
    }
    // wait if a start is in progress
    const existing = pending.get(connection)
    if (existing) {
        await existing
        return
    }
    // if disconnected, start and track
    if (connection.state === signalR.HubConnectionState.Disconnected) {
        const promise = connection.start().finally(() => {
            pending.delete(connection)
        })
        pending.set(connection, promise)
        await promise
        return
    }
    // connecting or reconnecting: wait until connected or fails
    await new Promise<void>((resolve, reject) => {
        const onClose = (err?: Error) => {
            cleanup()
            reject(err ?? new Error('Connection closed while waiting'))
        }
        const cleanup = () => {
            connection.off('close', onClose)
        }
        connection.onclose(onClose)
        const check = () => {
            if (connection.state === signalR.HubConnectionState.Connected) {
                cleanup()
                resolve()
            } else if (
                connection.state === signalR.HubConnectionState.Disconnected
            ) {
                cleanup()
                reject(
```

```
                    new Error('Connection disconnected while waiting')
                )
            } else {
                setTimeout(check, 50)
            }
        }
        check()
    })
}
```

terminalService.ts

```
import * as signalR from '@microsoft/signalr'
import { useKernelStore } from '@/stores/kernel'
import { ensureServiceConnection} from "@/services/serviceConnection.ts";
let terminalConnection: signalR.HubConnection | null = null
let terminalConnectionUrl: string | null = null
/**
 * Get or create the SignalR connection for terminal operations.
 */
function getConnection(): signalR.HubConnection {
    const kernelStore = useKernelStore()
    const baseUrl = kernelStore.baseUrl
    if (!baseUrl) {
        throw new Error('Kernel not running - no port available')
    }
    const url = `${baseUrl}/terminalHub`
    if (!terminalConnection || terminalConnectionUrl !== url) {
        if (terminalConnection) {
            terminalConnection.stop().then(() => {})
        }
        terminalConnection = new signalR.HubConnectionBuilder()
            .withUrl(url)
            .withAutomaticReconnect()
            .build()
        terminalConnectionUrl = url
    }
    return terminalConnection
}
export { terminalConnection, getConnection as getTerminalConnection }
/**
 * Initialize a terminal session with a specific session ID.
 */
export async function initializeTerminal(terminalId: string, cwd: string):
Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    try {
        await connection.invoke('TerminalInit', terminalId, cwd)
```

```
    } catch (err) {
        console.error('Terminal initialization failed:', err)
    }
}
/**
 * Send input to a terminal session.
 */
export async function terminalInputAsync(terminalId: string, data: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    try {
        await connection.invoke('TerminalInput', terminalId, data)
    } catch (err) {
        console.error('Terminal input failed:', err)
    }
}
/**
 * Close a terminal session.
 */
export async function closeTerminalSession(terminalId: string): Promise<void> {
    const connection = getConnection()
    await ensureServiceConnection(connection)
    try {
        await connection.invoke('TerminalDisconnect', terminalId)
    } catch (err) {
        console.error('Terminal close session failed:', err)
    }
}
```

kernel.ts

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
export type KernelStatus = 'starting' | 'running' | 'stopped' | 'error'
export const useKernelStore = defineStore('kernel', () => {
    const status = ref<KernelStatus>('stopped')
    const port = ref<number | null>(null)
    const errorMessage = ref<string | null>(null)
    const isRunning = computed(() => status.value === 'running')
    const baseUrl = computed(() => port.value ? `http://localhost:${port.value}` : null)
    function setPort(p: number) {
        port.value = p
    }
    function setStatus(s: KernelStatus, error?: string) {
        status.value = s
        errorMessage.value = error ?? null
```

```
    }
    function reset() {
        status.value = 'stopped'
        port.value = null
        errorMessage.value = null
    }
    return {
        status,
        port,
        errorMessage,
        isRunning,
        baseUrl,
        setPort,
        setStatus,
        reset
    }
})
```

workspace.ts

```
import {computed, reactive, ref, watch} from 'vue'
import {defineStore} from 'pinia'
import {MarkdownParser} from '@markdown/markdownVue3.ts'
import {
    readDirectory,
    streamTab,
    saveTabAsync,
    saveWorkspaceSettingsAsync,
    loadLayoutAsync,
    startWatching,
    stopWatching,
    onFileChanged,
    createFile,
    deleteFile,
    copyFile,
    renameFile,
    createDirectory,
    deleteDirectory,
    copyDirectory,
    RenameDirectory
} from '@/services/fileService'
import {FileNode, WorkspaceSettings, SidebarPanel, Tab, UIFileNode,
ViewMode} from '@/types/workspace'
const md = new MarkdownParser()
// Layout constraints
const DEFAULT_ROOT_DIRECTORY = ''
const MIN_EXPLORER_WIDTH = 160
const MAX_EXPLORER_WIDTH = 520
const MIN_TERMINAL_HEIGHT = 150
```

```
const MAX_TERMINAL_HEIGHT = 900
const MIN_PREVIEW_RATIO = 0.15
const MAX_PREVIEW_RATIO = 0.85
// Auto-save interval in milliseconds (60 seconds)
const AUTO_SAVE_INTERVAL = 60 * 1000
// Language detection by file extension - maps to Monaco editor language identifiers
const LANGUAGE_BY_EXTENSION: Record<string, string> = {
    '.ts': 'typescript',
    '.tsx': 'typescriptreact',
    '.js': 'javascript',
    '.jsx': 'javascriptreact',
    '.json': 'json',
    '.py': 'python',
    '.cs': 'csharp',
    '.cpp': 'cpp',
    '.c': 'c',
    '.java': 'java',
    '.rb': 'ruby',
    '.go': 'go',
    '.rs': 'rust',
    '.html': 'html',
    '.htm': 'html',
    '.css': 'css',
    '.scss': 'scss',
    '.sass': 'scss',
    '.less': 'less',
    '.md': 'markdown',
    '.mdx': 'markdown',
    '.sql': 'sql',
    '.yml': 'yaml',
    '.yaml': 'yaml',
    '.sh': 'shell',
    '.bash': 'shell',
    '.zsh': 'shell',
    '.ps1': 'powershell',
    '.vue': 'vue',
    '.txt': 'plaintext',
    '.xml': 'xml',
    '.php': 'php',
    '.swift': 'swift',
    '.kt': 'kotlin',
    '.kts': 'kotlin',
    '.scala': 'scala',
    '.r': 'r',
    '.lua': 'lua',
    '.perl': 'perl',
    '.pl': 'perl',
```

```
    '.dockerfile': 'dockerfile',
    '.graphql': 'graphql',
    '.gql': 'graphql',
    '.ini': 'ini',
    '.toml': 'toml',
    '.bat': 'bat',
    '.cmd': 'bat',
    '.clj': 'clojure',
    '.coffee': 'coffeescript',
    '.dart': 'dart',
    '.fs': 'fsharp',
    '.fsi': 'fsharp',
    '.fsx': 'fsharp',
    '.handlebars': 'handlebars',
    '.hbs': 'handlebars',
    '.pug': 'pug',
    '.jade': 'pug',
    '.razor': 'razor',
    '.cshtml': 'razor',
    '.vb': 'vb',
    '.m': 'objective-c',
    '.mm': 'objective-c'
}
// Sidebar button configuration
const SIDEBAR_BUTTONS = {
    top: [
        {icon: 'folder', panel: 'explorer' as SidebarPanel},
        {icon: 'search', panel: 'search' as SidebarPanel},
    ],
    bottom: [
        {icon: 'terminal', panel: 'terminal' as SidebarPanel}
    ]
}
// View mode button configuration
const VIEW_MODE_BUTTONS: { value: ViewMode, icon: string, label: string }[]
= [
    {value: 'code', icon: 'code', label: 'Code only'},
    {value: 'split', icon: 'split_scene', label: 'Code & Preview'},
    {value: 'preview', icon: 'visibility', label: 'Preview only'}
]
// Default layout state values
const DEFAULT_LAYOUT_STATE: WorkspaceSettings = {
    explorerWidth: 240,
    terminalHeight: 250,
    editorPreviewRatio: 0.5,
    preferredViewMode: 'split',
    explorerVisible: true,
    searchVisible: false,
```

```
    terminalVisible: false,
    activeTopPanel: 'explorer',
    activeBottomPanel: null,
    openedFiles: [],
    expandedDirectories: new Set<string>(),
    activeFile: ''
}
const fileChangeQueue: Array<{
    dirPath: string;
    resolve: () => void;
    reject: (err: unknown) => void;
}> = [];
let processing = false;
export const useWorkspaceStore = defineStore('workspace', () => {
    // Editor state
    const code = ref('')
    const tabs = ref<Tab[]>([])
    const activeTabIndex = ref(-1)
    const currentViewMode = ref<ViewMode>('split')
    // File tree state
    const fileTree = ref<UIFileNode[]>([])
    const nodes = reactive(new Map<string, FileNode>())
    const nodeVersion = ref(0)
    // Workspace state
    const rootDirectory = ref(DEFAULT_ROOT_DIRECTORY)
    const displayRootDirectory = ref(DEFAULT_ROOT_DIRECTORY) // Original
case for display
    const projectName = ref('')
    const pythonInterpreterPath = ref('')
    // Resize interaction state (not persisted)
    const resizeState = reactive({
        type: null as 'explorer' | 'preview' | 'terminal' | null,
        startX: 0,
        startY: 0,
        startSize: 0,
        containerWidth: 0
    })
    // Unified layout state - all UI panel states in one place
    const layoutState =
reactive<WorkspaceSettings>({...DEFAULT_LAYOUT_STATE})
    // File watcher cleanup function
    let fileWatcherCleanup: (() => void) | null = null
    // Auto-save timer
    let autoSaveTimer: ReturnType<typeof setInterval> | null = null
    // Computed: workspace state
    const hasWorkspace = computed(() => !!rootDirectory.value)
    const hasTabs = computed(() => tabs.value.length > 0)
    /**
```

```
     * Get Monaco editor language identifier from file path or name.
     * Returns the language ID based on file extension, or 'plaintext' if unknown.
     */
    function getLanguageFromPath(filePath: string): string {
        const name = filePath.toLowerCase()
        const extIndex = name.lastIndexOf('.')
        const ext = extIndex >= 0 ? name.slice(extIndex) : ''
        return (ext && LANGUAGE_BY_EXTENSION[ext]) || 'plaintext'
    }
    /**
     * Sort children: folders first, then files, alphabetically within each group.
     */
    function sortChildren(children: UIFileNode[]): UIFileNode[] {
        return children.slice().sort((a, b) => {
            if (a.isDirectory !== b.isDirectory) {
                return a.isDirectory ? -1 : 1
            }
            return a.name.localeCompare(b.name, undefined, {sensitivity: 'base'})
        })
    }
    // Computed: build visible file tree from flat node map
    const visibleFileTree = computed<UIFileNode[]>(() => {
        nodeVersion.value
        if (!rootDirectory.value) {
            return []
        }
        function build(path: string): UIFileNode | null {
            const node = nodes.get(path)
            if (!node) return null
            const children = node.expanded
                ? sortChildren(
                    node.children
                        .map(p => build(p))
                        .filter(Boolean) as UIFileNode[]
                )
                : []
            return {
                name: node.name,
                path: node.path,
                displayPath: node.displayPath,
                extension: node.extension,
                isDirectory: node.isDirectory,
                expanded: node.expanded,
                children
            }
```

```
        }
        const root = build(rootDirectory.value)
        return root ? [root] : []
    })
    // Computed: Markdown rendering
    const renderedVNode = computed(() => md.render(code.value))
    // Computed: active tab and file type detection
    const activeTab = computed(() => tabs.value[activeTabIndex.value] ??
null)
    const isMarkdownTab = computed(() => {
        const name = activeTab.value?.name?.toLowerCase() || ''
        const path = activeTab.value?.path?.toLowerCase() || ''
        return name.endsWith('.md') || path.endsWith('.md')
    })
    // Computed: panel visibility derived from layoutState
    const showExplorer = computed(() =>
        hasWorkspace.value &&
        layoutState.activeTopPanel === 'explorer' &&
        layoutState.explorerVisible
    )
    const showSearch = computed(() =>
        hasWorkspace.value &&
        layoutState.activeTopPanel === 'search' && layoutState.searchVisible
    )
    const showTerminal = computed(() =>
        hasWorkspace.value &&
        layoutState.activeBottomPanel === 'terminal' &&
        layoutState.terminalVisible
    )
    // Computed: editor/preview pane visibility
    const showPreviewPane = computed(() =>
        hasTabs.value && isMarkdownTab.value && currentViewMode.value !==
'code'
    )
    const showCodePane = computed(() =>
        hasTabs.value && (!isMarkdownTab.value || currentViewMode.value !==
'preview')
    )
    // Computed: panel styles
    const explorerStyle = computed(() => ({width:
`${layoutState.explorerWidth}px`}))
    const terminalStyle = computed(() => ({height:
`${layoutState.terminalHeight}px`}))
    // Computed: language detection for status bar and Monaco editor
    // Uses the stored language from the tab, or derives it from file
extension
    const currentLanguage = computed(() => {
        if (activeTab.value?.language) {
```

```
            return activeTab.value.language
        }
        return getLanguageFromPath(activeTab.value?.path ||
activeTab.value?.name || '')
    })
    // Computed: current file encoding for status bar
    const currentEncoding = computed(() => {
        return activeTab.value?.encoding || 'utf-8'
    })
    // Computed: editor/preview pane flex styles
    const editorPaneStyle = computed(() => {
        if (!showCodePane.value) {
            return {display: 'none'}
        }
        if (!showPreviewPane.value) {
            return {flex: 1, minWidth: 0}
        }
        return {flex: layoutState.editorPreviewRatio, minWidth: 0}
    })
    const previewPaneStyle = computed(() => {
        if (!showPreviewPane.value) {
            return {display: 'none'}
        }
        if (!showCodePane.value) {
            return {flex: 1, minWidth: 0}
        }
        return {flex: 1 - layoutState.editorPreviewRatio, minWidth: 0}
    })
    // Watch: sync code editor content with active tab
    watch(activeTab, (tab) => {
        if (!tab) {
            code.value = ''
            applyViewModeForActiveTab()
            return
        }
        code.value = tab?.content || ''
        applyViewModeForActiveTab()
    }, {immediate: true})
    // Watch: mark tab as dirty when code changes
    watch(code, (newCode) => {
        const tab: Tab = activeTab.value
        if (!tab) return
        if (newCode !== tab.content) {
            tab.isDirty = true
            tab.content = newCode
        }
    })
    /**
```

```
     * Clamp a value between min and max bounds.
     */
    function clamp(value: number, min: number, max: number): number {
        return Math.min(Math.max(value, min), max)
    }
    /**
     * Handle top sidebar button click.
     * Toggles panel visibility when clicking the active panel.
     */
    function handleTopButtonClick(index: number): void {
        const panel = SIDEBAR_BUTTONS.top[index]?.panel
        if (!panel) return
        if (layoutState.activeTopPanel === panel) {
            // Toggle visibility for explorer panel
            layoutState.activeTopPanel = null
            if (panel === 'explorer') {
                layoutState.explorerVisible = false
            } else {
                layoutState.searchVisible = false
            }
        } else {
            layoutState.activeTopPanel = panel
            if (panel === 'explorer') {
                layoutState.explorerVisible = true
            } else {
                layoutState.searchVisible = true
            }
        }
    }
    /**
     * Handle bottom sidebar button click.
     * Toggles panel visibility when clicking the active panel.
     */
    function handleBottomButtonClick(index: number): void {
        const panel = SIDEBAR_BUTTONS.bottom[index]?.panel
        if (!panel) return
        if (layoutState.activeBottomPanel === panel) {
            layoutState.activeBottomPanel = null
            if (panel === 'terminal') {
                layoutState.terminalVisible = false
            }
        } else {
            layoutState.activeBottomPanel = panel
            if (panel === 'terminal') {
                layoutState.terminalVisible = true
            }
        }
    }
```

```
    /**
     * Toggle explorer panel visibility.
     */
    function toggleExplorer(): void {
        layoutState.explorerVisible = !layoutState.explorerVisible
        if (!layoutState.explorerVisible) {
            layoutState.activeTopPanel = null
        }
    }
    /**
     * Open a tab and set it as active.
     */
    function openTab(tab: Tab): void {
        const targetIndex = tabs.value.findIndex(t => t.id === tab.id)
        if (targetIndex === -1) return
        activeTabIndex.value = targetIndex
        layoutState.activeFile = tab.path
        code.value = tab.content
        applyViewModeForActiveTab()
    }
    /**
     * Close a tab, saving if dirty.
     */
    async function closeTab(tab: Tab): Promise<void> {
        if (tab.isDirty) {
            try {
                await saveTabAsync(tab.path, tab.content, tab.encoding ||
'utf-8')
                tab.isDirty = false
            } catch (err) {
                console.error('Failed to save tab before closing:',
tab.path, err)
                // Still close the tab even if save failed
            }
        }
        const index = tabs.value.findIndex(t => t.id === tab.id)
        if (index !== -1) {
            tabs.value.splice(index, 1)
            layoutState.openedFiles = tabs.value.map(t => t.displayPath)
            if (tabs.value.length === 0) {
                activeTabIndex.value = -1
                layoutState.activeFile = ''
                code.value = ''
                applyViewModeForActiveTab()
                return
            }
            if (activeTabIndex.value >= index) {
                activeTabIndex.value = Math.max(0, tabs.value.length - 1)
```

```
                layoutState.activeFile = tabs.value[activeTabIndex.value].path
            }
            code.value = tabs.value[activeTabIndex.value]?.content || ''
            applyViewModeForActiveTab()
        }
    }
    /**
     * Save all tabs that have unsaved changes.
     * Uses the tab's detected encoding to preserve the original encoding.
     */
    async function saveDirtyTabs(): Promise<void> {
        for (const tab of tabs.value.filter((t: Tab): boolean => t.isDirty)) {
            try {
                await saveTabAsync(tab.path, tab.content, tab.encoding || 'utf-8')
                tab.isDirty = false
            } catch (err) {
                console.error('Failed to save tab:', tab.path, err)
            }
        }
    }
    /**
     * Start the auto-save timer (60 second interval).
     */
    function startAutoSave(): void {
        stopAutoSave()
        autoSaveTimer = setInterval(async () => {
            try {
                await saveDirtyTabs()
            } catch (err) {
                console.error('Auto-save failed:', err)
            }
        }, AUTO_SAVE_INTERVAL)
    }
    /**
     * Stop the auto-save timer.
     */
    function stopAutoSave(): void {
        if (autoSaveTimer) {
            clearInterval(autoSaveTimer)
            autoSaveTimer = null
        }
    }
    /**
     * Apply the appropriate view mode based on active tab type.
     */
```

```
    function applyViewModeForActiveTab(): void {
        if (!hasTabs.value) {
            currentViewMode.value = 'code'
            return
        }
        currentViewMode.value = isMarkdownTab.value ? layoutState.preferredViewMode : 'code'
    }
    /**
     * Set the view mode for Markdown files.
     */
    function setViewMode(mode: ViewMode): void {
        if (!isMarkdownTab.value) return
        currentViewMode.value = mode
        layoutState.preferredViewMode = mode
    }
    /**
     * Start resizing the explorer panel.
     */
    function startExplorerResize(event: MouseEvent): void {
        if (!showExplorer.value) return
        resizeState.type = 'explorer'
        resizeState.startX = event.clientX
        resizeState.startSize = layoutState.explorerWidth
        attachResizeListeners()
    }
    /**
     * Start resizing the preview panel.
     */
    function startPreviewResize(event: MouseEvent, containerWidth: number): void {
        if (!showPreviewPane.value || !showCodePane.value) return
        resizeState.type = 'preview'
        resizeState.startX = event.clientX
        resizeState.containerWidth = containerWidth
        resizeState.startSize = containerWidth * layoutState.editorPreviewRatio
        attachResizeListeners()
    }
    /**
     * Start resizing the terminal panel.
     */
    function startTerminalResize(event: MouseEvent): void {
        if (!showTerminal.value) return
        resizeState.type = 'terminal'
        resizeState.startY = event.clientY
        resizeState.startSize = layoutState.terminalHeight
        attachResizeListeners()
```

```
    }
    // Resize animation frame ID for smooth dragging
    let resizeAnimationFrame: number | null = null
    /**
     * Attach global resize listeners.
     */
    function attachResizeListeners(): void {
        window.addEventListener('mousemove', handleResizeDrag)
        window.addEventListener('mouseup', stopResize)
        document.body.style.userSelect = 'none'
        document.body.style.cursor = resizeState.type === 'terminal' ? 'row-resize' : 'col-resize'
        // Disable pointer events on iframes to prevent them from capturing mouse events
        document.querySelectorAll('iframe').forEach(iframe => {
            iframe.style.pointerEvents = 'none'
        })
    }
    /**
     * Handle resize drag event with requestAnimationFrame for smoother performance.
     */
    function handleResizeDrag(event: MouseEvent): void {
        if (!resizeState.type) return
        // Cancel any pending animation frame
        if (resizeAnimationFrame !== null) {
            cancelAnimationFrame(resizeAnimationFrame)
        }
        // Use requestAnimationFrame for smoother resizing
        resizeAnimationFrame = requestAnimationFrame(() => {
            if (resizeState.type === 'explorer') {
                const delta = event.clientX - resizeState.startX
                layoutState.explorerWidth = clamp(resizeState.startSize + delta, MIN_EXPLORER_WIDTH, MAX_EXPLORER_WIDTH)
            } else if (resizeState.type === 'preview') {
                const delta = event.clientX - resizeState.startX
                const allowedMin = resizeState.containerWidth * MIN_PREVIEW_RATIO
                const allowedMax = resizeState.containerWidth * MAX_PREVIEW_RATIO
                const newWidth = clamp(resizeState.startSize + delta, allowedMin, allowedMax)
                const safeContainerWidth = Math.max(resizeState.containerWidth, 1)
                layoutState.editorPreviewRatio = clamp(newWidth / safeContainerWidth, MIN_PREVIEW_RATIO, MAX_PREVIEW_RATIO)
            } else if (resizeState.type === 'terminal') {
                const delta = resizeState.startY - event.clientY
```

```
                layoutState.terminalHeight = clamp(resizeState.startSize + delta, MIN_TERMINAL_HEIGHT, MAX_TERMINAL_HEIGHT)
            }
            resizeAnimationFrame = null
        })
    }
    /**
     * Stop resize operation and clean up listeners.
     */
    function stopResize(): void {
        if (!resizeState.type) return
        if (resizeAnimationFrame !== null) {
            cancelAnimationFrame(resizeAnimationFrame)
            resizeAnimationFrame = null
        }
        window.removeEventListener('mousemove', handleResizeDrag)
        window.removeEventListener('mouseup', stopResize)
        document.body.style.userSelect = ''
        document.body.style.cursor = ''
        // Re-enable pointer events on iframes
        document.querySelectorAll('iframe').forEach(iframe => {
            iframe.style.pointerEvents = ''
        })
        resizeState.type = null
    }
    /**
     * Read directory contents recursively via streaming.
     * Preserves the expanded state of existing nodes during updates.
     */
    async function readDirAsync(directoryPath: string, targetMap: Map<string, FileNode>): Promise<void> {
        const normalizedDir = window.nodePath.normalize(directoryPath)
        const displayDir = window.nodePath.normalizeDisplay(directoryPath)
        const tempMap = new Map<string, FileNode>()
        // Preserve expanded state from existing nodes
        const expandedPaths = new Set<string>()
        if (targetMap.has(normalizedDir)) {
            tempMap.set(normalizedDir, {...targetMap.get(normalizedDir)!, displayPath: displayDir, children: []})
        }
        const pendingChildren = new Map<string, string[]>()
        const observable = await readDirectory(normalizedDir)
        await new Promise<void>((resolve, reject) => {
            observable.subscribe({
                next: (node) => {
                    const normalizedPath = window.nodePath.normalize(node.path)
```

```
                    const displayPath =
window.nodePath.normalizeDisplay(node.path)
                    const normalizedParentPath = node.parentPath ?
window.nodePath.normalize(node.parentPath) : null
                    // Preserve expanded state if the node previously
existed
                    const wasExpanded =
layoutState.expandedDirectories.has(normalizedPath)
                    if (wasExpanded) {
                        expandedPaths.add(normalizedPath)
                    }
                    // Destructure to exclude properties we're overriding
                    const {path: _, parentPath: __, children: ___, expanded:
____, ...restNode} = node
                    const normalizedNode: FileNode = {
                        ...restNode,
                        path: normalizedPath,
                        displayPath: displayPath,
                        parentPath: normalizedParentPath,
                        children: [],
                        expanded: wasExpanded
                    }
                    tempMap.set(normalizedPath, normalizedNode)
                    if (normalizedParentPath) {
                        const parent = tempMap.get(normalizedParentPath)
                        if (parent) {
                            parent.children.push(normalizedPath)
                        } else {
                            // Parent not yet loaded, queue this child
                            const list =
pendingChildren.get(normalizedParentPath) ?? []
                            list.push(normalizedPath)
                            pendingChildren.set(normalizedParentPath, list)
                        }
                    }
                    // Check if this node has pending children
                    const waiting = pendingChildren.get(normalizedPath)
                    if (waiting) {
                        const me = tempMap.get(normalizedPath)!
                        me.children.push(...waiting)
                        pendingChildren.delete(normalizedPath)
                    }
                },
                complete: () => {
                    layoutState.expandedDirectories = expandedPaths
                    resolve()
                },
                error: (err) => reject(err)
```

```
            })
        })
        removeSubNodes(normalizedDir, targetMap)
        for (const [path, node] of tempMap.entries()) {
            targetMap.set(path, node)
        }
        nodeVersion.value++
    }
    function removeSubNodes(dirPath: string, targetMap: Map<string, FileNode>): void {
        const normalized = window.nodePath.normalize(dirPath) + '\\'
        for (const key of Array.from(targetMap.keys())) {
            if (key.startsWith(normalized)) {
                targetMap.delete(key);
            }
        }
    }
    /**
     * Stream file content into a tab via streaming.
     * The tab metadata includes detected encoding from the backend.
     * Language is determined from the file extension.
     */
    async function streamTabAsync(filePath: string): Promise<void> {
        const normalizedPath = window.nodePath.normalizeDisplay(filePath)
        const observable = await streamTab(normalizedPath)
        return new Promise((resolve, reject) => {
            observable.subscribe({
                next: (chunk) => {
                    if (chunk.isMetadata) {
                        const tab = {
                            id: chunk.id,
                            name: chunk.name,
                            path: window.nodePath.normalize(chunk.path),
                            displayPath: window.nodePath.normalizeDisplay(chunk.path),
                            content: '',
                            isDirty: false,
                            encoding: chunk.encoding || 'utf-8',
                            language: getLanguageFromPath(chunk.path || chunk.name)
                        }
                        tabs.value.push(tab)
                        layoutState.openedFiles = tabs.value.map(t => t.displayPath)
                        openTab(tab)
                    } else if (chunk.isError) {
                        console.error('Error loading tab:', chunk.content)
                    } else {
```

```
                        const tab = tabs.value.find(t => t.id === chunk.id)
                        if (tab) {
                            tab.content += chunk.content
                            openTab(tab)
                        } else {
                            console.error('Received chunk for unknown tab id:', chunk.id)
                        }
                    }
                },
                complete: () => resolve(),
                error: (err) => {
                    console.error('Stream error:', err)
                    reject(err)
                }
            })
        })
    }
    /**
     * Load the file tree for the workspace root directory.
     */
    async function loadFileTree(): Promise<void> {
        nodes.clear()
        if (!rootDirectory.value) return
        const nodePath = window.nodePath
        const rootName = nodePath?.basename
            ? nodePath.basename(displayRootDirectory.value)
            : displayRootDirectory.value
        nodes.set(rootDirectory.value, {
            name: rootName,
            path: rootDirectory.value,
            displayPath: displayRootDirectory.value,
            extension: '',
            parentPath: null,
            isDirectory: true,
            expanded: true,
            children: []
        })
        await readDirAsync(rootDirectory.value, nodes)
    }
    async function processQueue() {
        if (processing) return;
        processing = true;
        while (fileChangeQueue.length) {
            const {dirPath, resolve, reject} = fileChangeQueue.shift()!;
            try {
                await readDirAsync(dirPath, nodes);
                resolve();
```

```
            } catch (err) {
                reject(err);
            }
        }
        processing = false;
    }
    /**
     * Handle file change events from the backend file watcher.
     * Performs atomic incremental updates to the file tree.
     */
    async function handleFileChange(dirPath: string): Promise<void> {
        const normalizedPath = window.nodePath.normalize(dirPath)
        return new Promise<void>((resolve, reject) => {
            fileChangeQueue.push({dirPath: normalizedPath, resolve,
reject});
            // handling
            void processQueue();
        });
    }
    /**
     * Start watching the workspace directory for file changes.
     */
    async function startFileWatcher(): Promise<void> {
        if (!rootDirectory.value) return
        // Clean up existing watcher
        await stopFileWatcher()
        try {
            // Start watching on the backend
            await startWatching(rootDirectory.value)
            // Listen for file change events
            fileWatcherCleanup = onFileChanged(handleFileChange)
            console.log('File watcher started for:', rootDirectory.value)
        } catch (err) {
            console.error('Failed to start file watcher:', err)
        }
    }
    /**
     * Stop watching the workspace directory for file changes.
     */
    async function stopFileWatcher(): Promise<void> {
        // Unregister event handler
        if (fileWatcherCleanup) {
            fileWatcherCleanup()
            fileWatcherCleanup = null
        }
        // Stop watching on the backend
        if (rootDirectory.value) {
            try {
```

```
                await stopWatching(rootDirectory.value)
                console.log('File watcher stopped for:', rootDirectory.value)
            } catch (err) {
                console.error('Failed to stop file watcher:', err)
            }
        }
    }
    /**
     * Toggle folder expansion state.
     */
    function toggleFolder(node: UIFileNode): void {
        const fileNode = nodes.get(node.path)
        if (fileNode) {
            fileNode.expanded = !node.expanded
            node.expanded = !node.expanded
            if (layoutState.expandedDirectories.has(node.path)) {
                layoutState.expandedDirectories.delete(node.path)
            } else {
                layoutState.expandedDirectories.add(node.path)
            }
        }
    }
    /**
     * Search files by name using the nodes Map.
     * Returns files that match the query (case-insensitive).
     */
    function searchFiles(query: string): UIFileNode[] {
        if (!query.trim()) return []
        const lowerQuery = query.toLowerCase()
        const results: UIFileNode[] = []
        for (const [, node] of nodes) {
            if (!node.isDirectory && node.name.toLowerCase().includes(lowerQuery)) {
                results.push({
                    name: node.name,
                    path: node.path,
                    displayPath: node.displayPath,
                    extension: node.extension,
                    isDirectory: node.isDirectory,
                    expanded: false,
                    children: []
                })
            }
        }
        // Sort results: exact matches first, then alphabetically
        return results.sort((a, b) => {
            const aExact = a.name.toLowerCase() === lowerQuery
```

```
            const bExact = b.name.toLowerCase() === lowerQuery
            if (aExact !== bExact) return aExact ? -1 : 1
            return a.name.localeCompare(b.name, undefined, {sensitivity: 'base'})
        })
    }
    /**
     * Open a file in a new or existing tab.
     */
    async function openFile(node: UIFileNode): Promise<void> {
        if (!rootDirectory.value || node.isDirectory) return
        try {
            const existingTab = tabs.value.find(tab => tab.path === node.path)
            if (existingTab) {
                openTab(existingTab)
            } else {
                await streamTabAsync(node.displayPath)
            }
        } catch (err) {
            console.error('Error opening file:', err)
        }
    }
    /**
     * Reset all tabs to empty state.
     */
    function resetTabs(): void {
        tabs.value = []
        activeTabIndex.value = -1
        code.value = ''
        applyViewModeForActiveTab()
    }
    /**
     * Set the workspace root directory and load file tree.
     */
    async function setWorkspaceDirectory(directoryPath: string): Promise<void> {
        // Stop previous file watcher and auto-save
        try {
            await stopFileWatcher()
        } catch (err) {
            console.error('Failed to stop file watcher:', err)
        }
        stopAutoSave()
        nodes.clear()
        fileTree.value = []
        const trimmedPath = directoryPath.trim()
        rootDirectory.value = window.nodePath.normalize(trimmedPath)
```

```
840         displayRootDirectory.value =
    window.nodePath.normalizeDisplay(trimmedPath)
841         if (!rootDirectory.value) return
842         try {
843             await loadFileTree()
844         } catch (err) {
845             console.error('Failed to load file tree:', err)
846         }
847         // Start file watcher and auto-save for the new directory
848         try {
849             await startFileWatcher()
850         } catch (err) {
851             console.error('Failed to start file watcher:', err)
852         }
853         startAutoSave()
854     }
855     /**
856      * Open a workspace at the given directory.
857      */
858     async function openWorkspace(directoryPath: string): Promise<void> {
859         resetTabs()
860         await setWorkspaceDirectory(directoryPath)
861     }
862     /**
863      * Create a new workspace with the given options.
864      */
865     async function createNewWorkspace(options: {
866         projectName: string,
867         directoryPath: string,
868         pythonInterpreter: string
869     }): Promise<void> {
870         projectName.value = options.projectName
871         pythonInterpreterPath.value = options.pythonInterpreter
872         await openWorkspace(options.directoryPath)
873     }
874     /**
875      * Save workspace settings to disk.
876      */
877     async function saveWorkspaceSettings(): Promise<void> {
878         try {
879             await saveWorkspaceSettingsAsync(rootDirectory.value,
    {...layoutState, expandedDirectories: [...layoutState.expandedDirectories]})
880         } catch (err) {
881             console.error('Failed to save workspace settings:', err)
882         }
883     }
884     /**
885      * Load workspace settings from disk and restore state.
```

```
     */
    async function loadWorkspaceSettings(): Promise<void> {
        try {
            const settings = await loadLayoutAsync(rootDirectory.value)
            if (settings) {
                // Restore layout dimensions
                layoutState.explorerWidth = settings.explorerWidth
                layoutState.terminalHeight = settings.terminalHeight
                layoutState.editorPreviewRatio = settings.editorPreviewRatio
                layoutState.preferredViewMode = settings.preferredViewMode
                // Restore panel visibility state
                layoutState.explorerVisible = settings.explorerVisible ??
true
                layoutState.terminalVisible = settings.terminalVisible ??
false
                layoutState.activeTopPanel = settings.activeTopPanel ??
'explorer'
                layoutState.activeBottomPanel =
settings.activeBottomPanel ?? null
                // Restore opened files
                layoutState.openedFiles = (settings.openedFiles ??
[]).map(window.nodePath.normalizeDisplay)
                layoutState.expandedDirectories = new
Set((settings.expandedDirectories ??
[]).map(window.nodePath.normalizeDisplay))
                layoutState.explorerVisible = (settings.explorerVisible ??
true)
                await openWorkspace(rootDirectory.value)
                for (const filePath of layoutState.openedFiles) {
                    try {
                        await streamTabAsync(filePath)
                    } catch (err) {
                        console.error('Failed to restore tab:', filePath,
err)
                    }
                }
                // Restore active file
                layoutState.activeFile =
window.nodePath.normalize(settings.activeFile ?? '')
                const activeTab = tabs.value.find(t => t.path ===
layoutState.activeFile) || tabs.value[0]
                if (activeTab) {
                    openTab(activeTab)
                }
            }
        } catch (err) {
            console.error('Failed to load workspace settings:', err)
        }
```

```
923     }
924     // Clipboard state for cut/copy/paste operations
925     const clipboard = reactive<{
926         path: string | null,
927         isDirectory: boolean,
928         operation: 'copy' | 'cut' | null
929     }>({
930         path: null,
931         isDirectory: false,
932         operation: null
933     })
934     /**
935      * Create a new file at the specified path.
936      */
937     async function createNewFile(parentPath: string, fileName: string):
    Promise<void> {
938         const filePath = window.nodePath.join(parentPath, fileName)
939         try {
940             await createFile(filePath)
941         } catch (err) {
942             console.error('Failed to create file:', err)
943         }
944     }
945     /**
946      * Create a new folder at the specified path.
947      */
948     async function createNewFolder(parentPath: string, folderName: string):
    Promise<void> {
949         const folderPath = window.nodePath.join(parentPath, folderName)
950         try {
951             await createDirectory(folderPath)
952         } catch (err) {
953             console.error('Failed to create folder:', err)
954         }
955     }
956     /**
957      * Delete a file at the specified path.
958      */
959     async function deleteFileNode(filePath: string, isDirectory: boolean):
    Promise<void> {
960         try {
961             if (isDirectory) {
962                 await deleteDirectory(filePath)
963             } else {
964                 await deleteFile(filePath)
965                 // Close the tab if the file is open
966                 const tab = tabs.value.find(t => t.path ===
    window.nodePath.normalize(filePath))
```

```
                if (tab) {
                    const index = tabs.value.indexOf(tab)
                    tabs.value.splice(index, 1)
                    layoutState.openedFiles = tabs.value.map(t => t.displayPath)
                    if (tabs.value.length === 0) {
                        activeTabIndex.value = -1
                        layoutState.activeFile = ''
                        code.value = ''
                    } else if (activeTabIndex.value >= index) {
                        activeTabIndex.value = Math.max(0, tabs.value.length - 1)
                        layoutState.activeFile = tabs.value[activeTabIndex.value].path
                        code.value = tabs.value[activeTabIndex.value]?.content || ''
                    }
                }
            }
        } catch (err) {
            console.error('Failed to delete:', err)
        }
    }
    /**
     * Rename a file or folder.
     */
    async function renameFileNode(oldPath: string, newName: string, isDirectory: boolean): Promise<void> {
        const parentPath = window.nodePath.dirname(oldPath)
        const newPath = window.nodePath.join(parentPath, newName)
        try {
            if (isDirectory) {
                await RenameDirectory(oldPath, newPath)
            } else {
                await renameFile(oldPath, newPath)
                // Update the tab if the file is open
                const normalizedOldPath = window.nodePath.normalize(oldPath)
                const tab = tabs.value.find(t => t.path === normalizedOldPath)
                if (tab) {
                    tab.path = window.nodePath.normalize(newPath)
                    tab.displayPath = window.nodePath.normalizeDisplay(newPath)
                    tab.name = newName
                    layoutState.openedFiles = tabs.value.map(t => t.displayPath)
                    if (layoutState.activeFile === normalizedOldPath) {
                        layoutState.activeFile = tab.path
```

```
                    }
                }
            }
        } catch (err) {
            console.error('Failed to rename:', err)
        }
    }
    /**
     * Copy a file or folder to clipboard.
     */
    function copyToClipboard(path: string, isDirectory: boolean): void {
        clipboard.path = path
        clipboard.isDirectory = isDirectory
        clipboard.operation = 'copy'
    }
    /**
     * Cut a file or folder to clipboard.
     */
    function cutToClipboard(path: string, isDirectory: boolean): void {
        clipboard.path = path
        clipboard.isDirectory = isDirectory
        clipboard.operation = 'cut'
    }
    /**
     * Paste from clipboard to the specified destination.
     */
    async function pasteFromClipboard(destinationPath: string):
Promise<void> {
        if (!clipboard.path || !clipboard.operation) return
        const fileName = window.nodePath.basename(clipboard.path)
        const destPath = window.nodePath.join(destinationPath, fileName)
        try {
            if (clipboard.isDirectory) {
                if (clipboard.operation === 'copy') {
                    await copyDirectory(clipboard.path, destPath)
                } else {
                    // For cut operation, we need to move (rename) the
directory
                    await RenameDirectory(clipboard.path, destPath)
                }
            } else {
                if (clipboard.operation === 'copy') {
                    await copyFile(clipboard.path, destPath)
                } else {
                    // For cut operation, we need to move (rename) the file
                    const normalizedOldPath =
window.nodePath.normalize(clipboard.path)
                    await renameFile(clipboard.path, destPath)
```

```
                    // Update the tab if the file is open
                    const tab = tabs.value.find(t => t.path === normalizedOldPath)
                    if (tab) {
                        tab.path = window.nodePath.normalize(destPath)
                        tab.displayPath = window.nodePath.normalizeDisplay(destPath)
                        tab.name = fileName
                        layoutState.openedFiles = tabs.value.map(t => t.displayPath)
                        if (layoutState.activeFile === normalizedOldPath) {
                            layoutState.activeFile = tab.path
                        }
                    }
                }
            }
            // Clear clipboard after cut operation
            if (clipboard.operation === 'cut') {
                clipboard.path = null
                clipboard.isDirectory = false
                clipboard.operation = null
            }
        } catch (err) {
            console.error('Failed to paste:', err)
        }
    }
    /**
     * Check if clipboard has content to paste.
     */
    function hasClipboardContent(): boolean {
        return clipboard.path !== null && clipboard.operation !== null
    }
    return {
        // State
        code,
        tabs,
        activeTabIndex,
        currentViewMode,
        layoutState,
        fileTree,
        nodes,
        rootDirectory,
        displayRootDirectory,
        projectName,
        pythonInterpreterPath,
        resizeState,
        // Computed state
        hasWorkspace,
```

```
        hasTabs,
        // Constants
        sidebarButtons: SIDEBAR_BUTTONS,
        viewModeButtons: VIEW_MODE_BUTTONS,
        // Computed getters
        visibleFileTree,
        renderedVNode,
        activeTab,
        isMarkdownTab,
        showExplorer,
        showSearch,
        showTerminal,
        showPreviewPane,
        showCodePane,
        explorerStyle,
        terminalStyle,
        currentLanguage,
        currentEncoding,
        editorPaneStyle,
        previewPaneStyle,
        // Actions
        handleTopButtonClick,
        handleBottomButtonClick,
        toggleExplorer,
        openTab,
        closeTab,
        setViewMode,
        startExplorerResize,
        startPreviewResize,
        startTerminalResize,
        handleResizeDrag,
        stopResize,
        loadFileTree,
        toggleFolder,
        openFile,
        searchFiles,
        saveDirtyTabs,
        applyViewModeForActiveTab,
        resetTabs,
        openWorkspace,
        createNewWorkspace,
        setWorkspaceDirectory,
        saveWorkspaceSettings,
        loadWorkspaceSettings,
        // File watcher actions
        startFileWatcher,
        stopFileWatcher,
        handleFileChange,
```

```
        // Auto-save actions
        startAutoSave,
        stopAutoSave,
        // File operations
        clipboard,
        createNewFile,
        createNewFolder,
        deleteFileNode,
        renameFileNode,
        copyToClipboard,
        cutToClipboard,
        pasteFromClipboard,
        hasClipboardContent
    }
})
```

`github-markdown.css`

This file is publicly available online [22].

`GithubTerminalTheme.json`

```
{
    "black": "#484f58",
    "red": "#ff7b72",
    "green": "#3fb950",
    "yellow": "#d29922",
    "blue": "#58a6ff",
    "magenta": "#bc8cff",
    "cyan": "#39c5cf",
    "white": "#b1bac4",
    "brightBlack": "#6e7681",
    "brightRed": "#ffa198",
    "brightGreen": "#56d364",
    "brightYellow": "#e3b341",
    "brightBlue": "#79c0ff",
    "brightMagenta": "#d2a8ff",
    "brightCyan": "#56d4dd",
    "brightWhite": "#ffffff",
    "background": "#010409",
    "foreground": "#e6edf3",
    "selectionBackground": "#3b5070",
    "cursorAccent": "#e6edf3"
}
```

`styles.css`

```
/* ============================================
   CSS VARIABLES & RESET
   ============================================ */
```

```css
:root {
    /* Border & Spacing */
    --widget-border-color: #21262d;
    --editor-font-size: 14px;
    --editor-line-height: 1.5em;
    --border-radius-sm: 4px;
    --border-radius-md: 6px;
    --border-radius-lg: 8px;
    /* Colors - Dark Theme */
    --accent-color: #58a6ff;
    --accent-color-hover: #79b8ff;
    --accent-color-muted: rgba(88, 166, 255, 0.15);
    --accent-color-secondary: #f78166;
    --accent-color-hover-secondary: #fa9e87;
    --accent-color-muted-secondary: rgba(247, 129, 102, 0.15);
    --bg-hover: rgba(255, 255, 255, 0.05);
    --success-color: #3fb950;
    --warning-color: #d29922;
    --error-color: #f85149;
    /* Background Colors */
    --bg-primary: #161b22;
    --bg-secondary: #0d1117;
    --bg-tertiary: #010409;
    --bg-elevated: #1c2128;
    --bg-overlay: rgba(1, 4, 9, 0.8);
    /* Text Colors */
    --text-primary: #e6edf3;
    --text-secondary: #8b949e;
    --text-muted: #6e7681;
    /* Interactive Elements */
    --button-bg: #21262d;
    --button-hover: #30363d;
    --button-active: #58a6ff;
    --button-border: #363b42;
    /* Shadows */
    --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.3);
    --shadow-md: 0 4px 12px rgba(0, 0, 0, 0.4);
    --shadow-lg: 0 8px 24px rgba(0, 0, 0, 0.5);
    /* Transitions */
    --transition-fast: 0.1s ease;
    --transition-normal: 0.2s ease;
    --transition-slow: 0.3s ease;
}
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

```
html {
    font-size: 16px
}
/* ============================================
   BASE STYLES
   ============================================ */
body {
    font-family: 'Inter', -apple-system, BlinkMacSystemFont, "Segoe UI",
"Noto Sans", Helvetica, Arial, sans-serif;
    background-color: var(--bg-primary);
    height: 100vh;
    overflow: hidden;
    color: var(--text-primary);
    line-height: 1.5;
    -webkit-font-smoothing: antialiased;
    -moz-osx-font-smoothing: grayscale;
}
/* Focus styles for accessibility */
:focus-visible {
    outline: 2px solid var(--accent-color);
    outline-offset: 2px;
}
:focus:not(:focus-visible) {
    outline: none;
}
/* ============================================
   LAYOUT STRUCTURE
   ============================================ */
.window {
    display: flex;
    flex-direction: column;
    height: 100vh;
    width: 100vw;
}
.window-workspace {
    flex: 1;
    display: flex;
    flex-direction: row;
    overflow: hidden;
}
.workspace-wrapper {
    display: flex;
    flex: 1;
    flex-direction: column;
    overflow: hidden;
}
.workspace {
    display: flex;
```

```
    flex: 1;
    overflow: hidden;
    position: relative;
}
.workspace-empty {
    flex: 1;
    display: flex;
    align-items: center;
    justify-content: center;
    color: var(--text-muted);
    background-color: var(--bg-secondary);
    border: 1px dashed var(--widget-border-color);
    margin: 12px;
    border-radius: var(--border-radius-md);
    text-align: center;
    padding: 12px;
}
.workspace-empty p {
    margin: 0;
}
/* ============================================
   SIDEBAR & TOOLBAR
   ============================================ */
.sidebar-panel {
    display: flex;
    flex-direction: column;
    min-width: 160px;
    flex-shrink: 0;
    background-color: var(--bg-tertiary);
    overflow: hidden;
    contain: strict;
}
.view-content {
    display: flex;
    flex-direction: column;
    width: 100%;
    height: 100%;
    flex-shrink: 0;
    background-color: var(--bg-tertiary);
    contain: layout style;
}
.vertical-resizer {
    position: relative;
    width: 8px;
    cursor: ew-resize;
    background-color: transparent;
    flex-shrink: 0;
    z-index: 10;
```

```
    margin-left: -4px;
    margin-right: -4px;
}
.vertical-resizer::after {
    content: '';
    position: absolute;
    top: 0;
    bottom: 0;
    left: 50%;
    width: 1px;
    transform: translateX(-50%);
    background-color: var(--widget-border-color);
}
.tool-bar {
    display: grid;
    grid-template-rows: auto 1fr auto;
    border-right: solid 1px var(--widget-border-color);
    width: 48px;
    min-width: 48px;
    background-color: var(--bg-secondary);
    cursor: default;
}
.general-action {
    grid-row: 1;
    padding-top: 4px;
}
.code-action {
    grid-row: 3;
    padding-bottom: 4px;
}
.badge {
    color: var(--text-muted);
    display: flex;
    width: 48px;
    height: 48px;
    justify-content: center;
    align-items: center;
    cursor: pointer;
    user-select: none;
    position: relative;
    transition: color var(--transition-fast), background-color var(--
transition-fast);
}
.badge:hover {
    color: var(--text-primary);
    background-color: var(--bg-hover);
}
.badge.active {
```

```
    color: var(--text-primary);
    background-color: transparent;
}
.badge.active::before {
    content: '';
    position: absolute;
    left: 0;
    top: 50%;
    transform: translateY(-50%);
    width: 2px;
    height: 24px;
    background-color: var(--accent-color-secondary);
    border-radius: 0 2px 2px 0;
}
.badge span {
    font-size: 24px;
    font-variation-settings: 'FILL' 0, 'wght' 400, 'GRAD' 0, 'opsz' 24;
    transition: font-variation-settings var(--transition-fast);
}
.badge:hover span {
    font-variation-settings: 'FILL' 1, 'wght' 400, 'GRAD' 0, 'opsz' 24;
}
.badge.active span {
    font-variation-settings: 'FILL' 1, 'wght' 400, 'GRAD' 0, 'opsz' 24;
}
/* ============================================
   TITLE BARS & NAVIGATION
   ============================================ */
.title-bar {
    top: 0;
    height: 35px;
    display: flex;
    padding: 8px 12px;
    background-color: var(--bg-tertiary);
    border-bottom: 1px solid var(--widget-border-color);
    gap: 10px;
}
.content-title-bar {
    display: grid;
    grid-template-columns: auto 1fr auto;
    align-items: center;
    height: 35px;
    padding: 0 8px 0 14px;
    cursor: default;
    user-select: none;
    /*border-bottom: 1px solid var(--widget-border-color);*/
    background-color: var(--bg-tertiary);
}
```

```css
.title-label {
    display: block;
    grid-column: 1;
    align-items: center;
    font-size: 11px;
    font-weight: 600;
    letter-spacing: 0.5px;
    text-transform: uppercase;
    color: var(--text-secondary);
}
.title-actions {
    grid-column: 3;
    display: flex;
    flex-direction: row;
    gap: 2px;
}
.title-action {
    display: flex;
    height: 22px;
    width: 22px;
    text-align: center;
    justify-content: center;
    align-items: center;
    cursor: pointer;
    user-select: none;
    border-radius: var(--border-radius-sm);
    color: var(--text-muted);
    transition: background-color var(--transition-fast), color var(--transition-fast);
}
.title-action:hover {
    background-color: var(--button-hover);
    color: var(--text-primary);
}
.title-action span {
    font-size: 18px;
    font-variation-settings: 'FILL' 0, 'wght' 400, 'GRAD' 0, 'opsz' 18;
}
/* ============================================
   BUTTONS & CONTROLS
   ============================================ */
button {
    padding: 6px 14px;
    background-color: var(--button-bg);
    border: 1px solid var(--button-border);
    border-radius: var(--border-radius-md);
    cursor: pointer;
    color: var(--text-primary);
```

```
    font-size: 13px;
    font-weight: 500;
    font-family: inherit;
    transition: background-color var(--transition-fast), border-color var(--
transition-fast), box-shadow var(--transition-fast);
}
button:hover {
    background-color: var(--button-hover);
    border-color: var(--text-muted);
}
button:active {
    background-color: var(--button-active);
    border-color: var(--button-active);
    color: #ffffff;
}
button.primary {
    background-color: var(--accent-color);
    border-color: var(--accent-color);
    color: #ffffff;
}
button.primary:hover {
    background-color: var(--accent-color-hover);
    border-color: var(--accent-color-hover);
}
.file-operations,
.edit-operations,
.format-operations {
    display: flex;
    margin-right: 20px;
    gap: 8px;
}
/* ============================================
   EDITOR CONTAINER & LAYOUT - Partial moved to Editor.vue
   ============================================ */
.editor-container {
    flex: 1;
    display: flex;
    flex-direction: column;
    background-color: var(--bg-secondary);
    min-height: 0;
    min-width: 0;
    overflow: hidden;
}
.editor {
    display: flex;
    flex: 1;
    flex-direction: row;
    overflow: hidden;
```

```
    align-items: stretch;
}
/* ==========================================
   PREVIEW PANEL - Moved to Preview.vue scoped styles
   ========================================== */
.split-resizer {
    position: relative;
    width: 8px;
    cursor: ew-resize;
    background-color: transparent;
    z-index: 10;
    margin-left: -4px;
    margin-right: -4px;
}
.split-resizer::after {
    content: '';
    position: absolute;
    top: 0;
    bottom: 0;
    left: 50%;
    width: 1px;
    transform: translateX(-50%);
    background-color: var(--widget-border-color);
}
/* ==========================================
   TERMINAL - Moved to Terminal.vue scoped styles
   ========================================== */
.horizontal-resizer {
    position: relative;
    height: 8px;
    cursor: ns-resize;
    background-color: transparent;
    flex-shrink: 0;
    z-index: 10;
    margin-top: -4px;
    margin-bottom: -4px;
}
.horizontal-resizer::after {
    content: '';
    position: absolute;
    left: 0;
    right: 0;
    top: 50%;
    height: 1px;
    transform: translateY(-50%);
    background-color: var(--widget-border-color);
}
.terminal-host .xterm {
```

```
    padding: 12px 16px;
    height: 100%;
}
/* ==========================================
   STATUS BAR
   ========================================== */
.status-bar {
    height: 22px;
    background-color: var(--bg-secondary);
    border-top: 1px solid var(--widget-border-color);
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 0 12px;
    font-size: 12px;
    color: var(--text-secondary);
}
.status-bar-left,
.status-bar-right {
    display: flex;
    align-items: center;
    gap: 16px;
}
.status-item {
    display: flex;
    align-items: center;
    gap: 4px;
    padding: 0 8px;
    height: 22px;
    cursor: default;
    transition: background-color var(--transition-fast);
}
.status-item:hover {
    background-color: rgba(255, 255, 255, 0.05);
}
.status-item span.material-symbols-outlined {
    font-size: 14px;
}
.status-indicator {
    width: 8px;
    height: 8px;
    border-radius: 50%;
    background-color: var(--success-color);
}
.status-indicator.warning {
    background-color: var(--warning-color);
}
.status-indicator.error {
```

```
    background-color: var(--error-color);
}
/* ============================================
   SELECTION & INTERACTIVE ELEMENTS
   ============================================ */
::selection {
    background-color: var(--accent-color-muted);
}
/* ============================================
   SCROLLBAR STYLING
   ============================================ */
::-webkit-scrollbar {
    width: 10px;
    height: 10px;
}
::-webkit-scrollbar-track {
    background: transparent;
}
::-webkit-scrollbar-thumb {
    background: var(--text-muted);
    border-radius: 5px;
    border: 2px solid transparent;
    background-clip: content-box;
}
::-webkit-scrollbar-thumb:hover {
    background: var(--text-secondary);
    border: 2px solid transparent;
    background-clip: content-box;
}
::-webkit-scrollbar-corner {
    background: transparent;
}
/* ============================================
   RESPONSIVE DESIGN
   ============================================ */
@media (max-width: 768px) {
    .editor-main,
    .preview-main {
        width: 100%;
    }
    .view-content {
        width: 200px;
        min-width: 160px;
    }
    .tool-bar {
        width: 40px;
        min-width: 40px;
    }
```

```
    .tab {
        min-width: 100px;
    }
}
/* ==========================================
   PREVIEW PANEL HEADER
   ========================================== */
.preview-header {
    display: flex;
    align-items: center;
    height: 35px;
    padding: 0 14px;
    background-color: var(--bg-tertiary);
    border-bottom: 1px solid var(--widget-border-color);
    font-size: 11px;
    font-weight: 600;
    letter-spacing: 0.5px;
    text-transform: uppercase;
    color: var(--text-secondary);
    user-select: none;
}
/* ==========================================
   UTILITY CLASSES
   ========================================== */
.text-muted {
    color: var(--text-muted);
}
.text-secondary {
    color: var(--text-secondary);
}
.text-primary {
    color: var(--text-primary);
}
.text-accent {
    color: var(--accent-color);
}
/* Divider */
.divider {
    width: 1px;
    height: 16px;
    background-color: var(--widget-border-color);
    margin: 0 8px;
}
.md-code-wrapper {
    border-radius: 6px;
    background: #151b23;
    position: relative;
    margin-bottom: 1rem;
```

```
}
.md-code-actions {
    position: absolute;
    top: 2px;
    right: 4px;
    display: flex;
    gap: 4px; /* gap between buttons */
    z-index: 10;
}
.md-icon-btn {
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    padding: 2px;
    border-radius: 4px;
    color: #abb2bf;
    transition: background 0.2s, color 0.2s;
    user-select: none;
}
.md-icon-btn:hover {
    background: var(--button-hover);
    color: #fff;
}
.md-icon-btn span {
    color: var(--text-secondary);
    font-size: 14px;
    font-variation-settings: 'FILL' 0,
    'wght' 400,
    'GRAD' 0,
    'opsz' 20
}
.icon-btn:hover {
    background: rgba(255, 255, 255, 0.1);
    color: #fff;
}
/* icon size */
.material-icons {
    font-size: 20px;
}
/* code */
pre {
    margin: 0;
    /*padding: 16px;*/
    overflow-x: auto;
    color: #abb2bf;
}
/* result output */
```

```css
.md-output-console {
    display: none; /* hidden by default */
    padding: 12px 16px;
    background: transparent;
    border-top: 1px solid var(--widget-border-color);
    color: var(--text-primary);
    font-size: 0.9rem;
    white-space: pre-wrap;
}
.markdown-body .md-output-console p {
    margin-bottom: 0;
}
```

types\workspace.ts

```ts
/**
 * Core file node structure stored in the flat map.
 * Contains the essential file metadata and tree structure pointers.
 */
export interface FileNode {
    name: string              // Display name with original case
    path: string              // Lowercase normalized path for storage/comparison
    displayPath: string       // Original case path for display in UI
    parentPath: string | null
    extension: string
    isDirectory: boolean
    children: string[]        // Paths to child nodes
    expanded: boolean         // UI state for directory expansion
}
/**
 * Derived file node for rendering the tree view.
 * Built from FileNode when needed, contains resolved children.
 */
export interface UIFileNode {
    name: string
    path: string
    displayPath: string       // Original case path for display
    extension: string
    isDirectory: boolean
    children: UIFileNode[]
    expanded: boolean
}
/**
 * Editor tab representing an open file.
 */
export interface Tab {
    id: string
    name: string
```

```
    path: string
    displayPath: string
    content: string
    isDirty: boolean
    encoding: string          // Detected file encoding (e.g., "utf-8",
"utf-16le")
    language: string          // Monaco editor language identifier
}
/**
 * Chunk of data for streaming file content.
 * Used for efficient large file loading.
 */
export interface TabChunk {
    id: string
    name: string
    path: string
    content: string
    isMetadata: boolean
    isError: boolean
    encoding: string          // Detected file encoding
}
/**
 * Editor view mode for Markdown files.
 */
export type ViewMode = 'code' | 'split' | 'preview'
/**
 * Sidebar panel types.
 */
export type SidebarPanel = 'explorer' | 'search' | 'run' | 'terminal' | null
/**
 * Layout state persisted to workspace settings.
 * Contains both UI dimensions and panel visibility state.
 */
export interface WorkspaceSettings {
    // Panel dimensions
    explorerWidth: number
    terminalHeight: number
    editorPreviewRatio: number
    // View preferences
    preferredViewMode: ViewMode
    // Panel visibility state
    explorerVisible: boolean
    searchVisible: boolean
    terminalVisible: boolean
    activeTopPanel: SidebarPanel
    activeBottomPanel: SidebarPanel
    // Open files
    openedFiles: string[]
```

```
    expandedDirectories: Set<string>
    activeFile: string
}
export interface WorkspaceSettingsDTO
    extends Omit<WorkspaceSettings, 'expandedDirectories'> {
    expandedDirectories: string[]
}
```

App.vue

```
<script setup lang="ts">
import {onMounted, ref} from 'vue'
import {storeToRefs} from 'pinia'
import FileExplorer from './components/FileExplorer.vue'
import SearchPanel from './components/SearchPanel.vue'
import TabsBar from './components/TabsBar.vue'
import Editor from './components/Editor.vue'
import Preview from './components/Preview.vue'
import Terminal from './components/Terminal.vue'
import {useWorkspaceStore} from '@/stores/workspace'
import {useKernelStore} from '@/stores/kernel'
import NavigationBar from "@/components/NavigationBar.vue"
const workspace = useWorkspaceStore()
const kernelStore = useKernelStore()
const {
    explorerStyle,
    showExplorer,
    showSearch,
    showTerminal,
    showCodePane,
    showPreviewPane,
    activeTab,
    hasTabs,
    hasWorkspace,
    layoutState
} = storeToRefs(workspace)
const { status: kernelStatus } = storeToRefs(kernelStore)
const sidebarButtons = workspace.sidebarButtons
const viewModeButtons = workspace.viewModeButtons
const editorPane = ref<HTMLElement | null>(null)
/**
 * Start preview pane resize, passing container width to store.
 */
function startPreviewResize(event: MouseEvent): void {
    const width = editorPane.value?.getBoundingClientRect().width || 0
    workspace.startPreviewResize(event, width)
}
/**
 * Check if a sidebar button is active based on layoutState.
```

```
 */
function isTopButtonActive(index: number): boolean {
    const panel = sidebarButtons.top[index]?.panel
    return layoutState.value.activeTopPanel === panel
}
/**
 * Check if a bottom sidebar button is active based on layoutState.
 */
function isBottomButtonActive(index: number): boolean {
    const panel = sidebarButtons.bottom[index]?.panel
    return layoutState.value.activeBottomPanel === panel
}
onMounted(async () => {
    const startTime = Date.now()
    let appReadyCalled = false
    const callAppReady = () => {
        if (appReadyCalled) return
        appReadyCalled = true
        const elapsed = Date.now() - startTime
        const remaining = Math.max(0, 1000 - elapsed)
        setTimeout(() => window.windowControls.appReady(), remaining)
    }
    // Timeout fallback - show main window after 10s even if loading fails
    const timeout = setTimeout(() => {
        console.warn('[App] Loading timeout, showing main window anyway')
        callAppReady()
    }, 10000)
    try {
        // Promise that resolves when kernel port is ready
        const portReady = new Promise<void>((resolve) => {
            window.kernel.onPort((port) => {
                kernelStore.setPort(port)
                resolve()
            })
        })
        window.kernel.onStatus((status, error) => {
            kernelStore.setStatus(status as any, error)
        })
        // Get initial kernel state
        const initialStatus = await window.kernel.getStatus()
        kernelStore.setStatus(initialStatus)
        // Check if port already available, otherwise wait for it
        const initialPort = await window.kernel.getPort()
        if (initialPort) {
            kernelStore.setPort(initialPort)
        } else {
            await portReady
        }
```

```
        // Initialize workspace with last working directory (after kernel port is ready)
        workspace.rootDirectory = await window.cwd.getCwd()
        workspace.displayRootDirectory = await window.cwd.getDisplayCwd()
        await workspace.loadWorkspaceSettings()
        clearTimeout(timeout)
        callAppReady()
    } catch (error) {
        console.error('[App] Loading error:', error)
        clearTimeout(timeout)
        callAppReady()
    }
    // Set up window close handler
    window.windowControls.onBeforeClose(() => {
        workspace.saveDirtyTabs()
        workspace.saveWorkspaceSettings()
        window.cwd.setCwd(workspace.rootDirectory)
        window.cwd.setDisplayCwd(workspace.displayRootDirectory)
        window.windowControls.canClose()
    })
})
</script>
<template>
    <div class="window">
        <NavigationBar/>
        <div class="window-workspace">
            <div class="tool-bar">
                <div class="general-action">
                    <div
                        v-for="(item, index) in sidebarButtons.top"
                        :key="index"
                        class="badge"
                        :class="{ active: isTopButtonActive(index) }"
                        @click="workspace.handleTopButtonClick(index)"
                    >
                        <span class="material-symbols-outlined">{{ item.icon }}</span>
                    </div>
                </div>
                <div class="code-action">
                    <div
                        v-for="(item, index) in sidebarButtons.bottom"
                        :key="index"
                        class="badge"
                        :class="{ active: isBottomButtonActive(index) }"
                        @click="workspace.handleBottomButtonClick(index)"
                    >
```

```
                        <span class="material-symbols-
    outlined">{{ item.icon }}</span>
                    </div>
                </div>
            </div>
            <div class="workspace-wrapper">
                <div class="workspace">
                    <template v-if="hasWorkspace">
                        <div v-show="showExplorer || showSearch"
    class="sidebar-panel" :style="explorerStyle">
                            <FileExplorer v-show="showExplorer"/>
                            <SearchPanel v-show="showSearch"/>
                        </div>
                        <div v-show="showExplorer || showSearch"
    class="vertical-resizer"

    @mousedown="workspace.startExplorerResize"></div>
                        <div class="editor-container">
                            <TabsBar v-if="hasTabs" :view-mode-
    buttons="viewModeButtons"/>
                            <div
                                v-if="hasTabs"
                                class="editor"
                                ref="editorPane"
                            >
                                <Editor/>
                                <div
                                    v-show="showCodePane && showPreviewPane"
                                    class="split-resizer"
                                    @mousedown="startPreviewResize"
                                ></div>
                                <Preview/>
                            </div>
                            <div v-else class="workspace-empty">
                                <p>Select a file from the explorer to start
    editing.</p>
                            </div>
                        </div>
                    </template>
                    <div v-else class="workspace-empty">
                        <p>Select a workspace from File &gt; Open or File
    &gt; New.</p>
                    </div>
                </div>
                <div
                    v-if="hasWorkspace && showTerminal"
                    class="horizontal-resizer"
                    @mousedown="workspace.startTerminalResize"
```

```
                ></div>
                <template v-if="hasWorkspace">
                    <Terminal v-show="showTerminal"/>
                </template>
            </div>
        </div>
        <div class="status-bar">
            <div class="status-bar-left">
                <div class="status-item" :class="'kernel-' + kernelStatus">
                    <span v-if="kernelStatus === 'running'" class="status-
indicator"></span>
                    <span v-else-if="kernelStatus === 'starting'"
class="status-indicator warning"></span>
                    <span v-else class="status-indicator error"></span>
                    <span>{{ kernelStatus === 'running' ? 'Ready' :
kernelStatus === 'starting' ? 'Starting' : 'Kernel'}}</span>
                </div>
                <div class="status-item">
                    <span class="material-symbols-
outlined">description</span>
                    <span>{{ hasTabs ? activeTab?.name : 'No file
open' }}</span>
                </div>
            </div>
            <div class="status-bar-right">
                <div class="status-item">
                    <span>{{ workspace.currentLanguage }}</span>
                </div>
                <div class="status-item">

<span>{{ workspace.currentEncoding.toUpperCase() }}</span>
                </div>
            </div>
        </div>
    </div>
</template>
```

main.ts

```
/**
 * Vue Application Entry Point
 *
 * Initializes the Vue 3 application with Pinia state management
 * and sets up communication with the Electron main process.
 */
import {createApp} from 'vue'
import {createPinia} from 'pinia'
import './styles/style.css'
import 'material-symbols'
```

```
import App from './App.vue'
// ============================================================================
// Environment Detection
// ============================================================================
/**
 * Check if running in development mode.
 * Uses Vite's import.meta.env which is statically replaced at build time.
 */
const isDev = import.meta.env.DEV
/**
 * Check if running in Electron environment.
 * The ipcRenderer is only available when running in Electron.
 */
const isElectron = typeof window !== 'undefined' && !!window.ipcRenderer
// ============================================================================
// Application Initialization
// ============================================================================
/**
 * Create and configure the Vue application instance.
 */
function initializeApp() {
    const app = createApp(App)
    // Configure Pinia state management
    const pinia = createPinia()
    app.use(pinia)
    // Global error handler for Vue errors
    app.config.errorHandler = (error, instance, info) => {
        console.error('[Vue Error]', error)
        console.error('[Vue Error Info]', info)
        // In development, log additional debug information
        if (isDev) {
            console.error('[Vue Error Instance]', instance)
        }
    }
    // Global warning handler (development only)
    if (isDev) {
        app.config.warnHandler = (msg, _instance, trace) => {
            console.warn('[Vue Warning]', msg)
            if (trace) {
                console.warn('[Vue Warning Trace]', trace)
            }
        }
    }
    return app
}
/**
 * Set up IPC communication with Electron main process.
 */
```

```
function setupElectronIPC() {
    if (!isElectron) {
        if (isDev) {
            console.log('[App] Running in browser mode (no Electron IPC)')
        }
        return
    }
    // Listen for messages from the main process
    window.ipcRenderer.on('main-process-message', (_event, message) => {
        if (isDev) {
            console.log('[Electron] Main process message:', message)
        }
    })
    // Log startup information in development
    if (isDev) {
        console.log('[App] Running in Electron mode')
        console.log('[App] Development mode enabled')
    }
}
/**
 * Intercept external link clicks and open them in the system browser.
 */
function setupExternalLinkHandler() {
    const nativeWindowOpen = window.open.bind(window)
    window.open = (url, target, features) => {
        const u = typeof url === 'string' ? url : String(url ?? '')
        if (/^https?:\/\//i.test(u) && isElectron && window.ipcRenderer) {
            window.ipcRenderer.send('open-external-url', u)
            return null
        }
        return nativeWindowOpen(url, target, features)
    }
    const normalizeExternalHref = (raw: any) => {
        if (!raw) return null
        let s = raw.trim()
        // If starts with "www.", prepend https://
        if (/^www\./i.test(s)) s = `https://${s}`
        // If looks like domain but missing protocol, prepend https://
        if (!/^[a-zA-Z][a-zA-Z\d+\-.]*:/.test(s) &&
/^[\w-]+(\.[\w-]+)+([/?#].*)?$/.test(s)) {
            s = `https://${s}`
        }
        try {
            const u = new URL(s)
            if (u.protocol !== 'http:' && u.protocol !== 'https:') return
null
            return u.toString()
        } catch {
```

```
            return null
        }
    }
    const handleLinkEvent = (event: any) => {
        if (event.defaultPrevented) return
        if (event.type === 'click' && event.button !== 0) return
        if (event.type === 'auxclick' && event.button !== 1) return
        const target = event.target
        if (!(target instanceof Element)) return
        const anchor = target.closest('a[href]')
        if (!(anchor instanceof HTMLAnchorElement)) return
        const hrefAttr = anchor.getAttribute('href') || ''
        if (!hrefAttr || hrefAttr.startsWith('#')) return
        const external = normalizeExternalHref(hrefAttr)
        if (!external) return
        event.preventDefault()
        event.stopPropagation()
        if (typeof event.stopImmediatePropagation === 'function') {
            event.stopImmediatePropagation()
        }
        if (isElectron && window.ipcRenderer) {
            window.ipcRenderer.send('open-external-url', external)
        } else {
            nativeWindowOpen(external, '_blank', 'noopener,noreferrer')
        }
    }
    document.addEventListener('click', handleLinkEvent, true)
    document.addEventListener('auxclick', handleLinkEvent, true)
}
/**
 * Mount the application to the DOM.
 */
function mountApp(app: ReturnType<typeof createApp>) {
    const mountPoint = '#app'
    app.mount(mountPoint).$nextTick(() => {
        // Application is now mounted and ready
        setupElectronIPC()
        setupExternalLinkHandler()
        if (isDev) {
            console.log('[App] Application mounted successfully')
        }
    }).then(() => {
    })
}
// ============================================================================
// Bootstrap
// ============================================================================
const app = initializeApp()
```

```
mountApp(app)
```

shims-vue.d.ts

```
declare module '*.vue' {
    import type { DefineComponent } from 'vue'
    const component: DefineComponent<{}, {}, any>
    export default component
}
```

splash.ts

```
import { createApp } from 'vue'
import Splash from './components/Splash.vue'
createApp(Splash).mount('#app')
```

vite-env.d.ts

```
/// <reference types="vite/client" />
interface FileNode {
    name: string
    path: string
    extension: string
    isDirectory: boolean
    children?: FileNode[]
    expanded?: boolean
}
interface Tab {
    id: string
    name: string
    path: string
    content: string
    isDirty: boolean
}
export interface IFileAPI {
    readDir: (dir: string) => Promise<FileNode[]>
    createTab: (name: string, path: string) => Promise<Tab>
}
export interface INodePathAPI {
    join: (...args: string[]) => string
    dirname: (p: string) => string
    basename: (p: string, ext?: string) => string
    normalize: (p: string) => string
    normalizeDisplay: (p: string) => string
}
export interface ITerminalAPI {
    init: () => void
    input: (data: string) => void
    onOutput: (callback: (data: string) => void) => void
}
```

```
declare global {
    interface Window {
        fileAPI: IFileAPI
        nodePath: INodePathAPI
        terminal: ITerminalAPI
        ipcRenderer: {
            on: (channel: string, listener: (event: any, ...args: any[]) =>
void) => void
            send: (channel: string, ...args: any[]) => void
        }
        windowControls: {
            minimize(): void
            maximize(): void
            close(): void
            canClose(): void
            appReady(): void
            onMaximize(cb: (maximized: boolean) => void): void
            onBeforeClose(cb: () => void): void
        }
        cwd: {
            setCwd(cwd: string): void
            getCwd(): Promise<string>
            setDisplayCwd(cwd: string): void
            getDisplayCwd(): Promise<string>
        }
        kernel: {
            start(): void
            stop(): void
            getPort(): Promise<number | null>
            getStatus(): Promise<'starting' | 'running' | 'stopped' |
'error'>
            onPort(cb: (port: number) => void): void
            onStatus(cb: (status: string, error?: string) => void): void
        }
        require?: NodeRequire
    }
}
declare module "*.vue" {
    import { DefineComponent } from "vue";
    const component: DefineComponent<{}, {}, any>;
    export default component;
}
```

markdown.stress.test.ts

```
import { describe, expect, it, vi } from "vitest";
import { readFileSync } from "node:fs";
import { dirname, resolve } from "node:path";
import { performance } from "node:perf_hooks";
```

```
import { fileURLToPath } from "node:url";
import { MarkdownParser } from "@markdown/markdown";
vi.mock("dompurify", () => ({
    default: {
        sanitize: (input: string) => input
    }
}));
const RUNS = 5;
const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);
function runStressCase(fixturePath: string, label: string) {
    const input = readFileSync(resolve(__dirname, fixturePath), "utf-8");
    const durationsMs: number[] = [];
    const memoryDeltaRssBytes: number[] = [];
    let output = "";
    for (let i = 0; i < RUNS; i++) {
        const parser = new MarkdownParser();
        const memoryBefore = process.memoryUsage();
        const startTime = performance.now();
        output = parser.render(input);
        const durationMs = performance.now() - startTime;
        const memoryAfter = process.memoryUsage();
        durationsMs.push(durationMs);
        memoryDeltaRssBytes.push(Math.max(0, memoryAfter.rss -
memoryBefore.rss));
    }
    const avgDurationMs = durationsMs.reduce((sum, value) => sum + value, 0)
/ RUNS;
    const avgRssDeltaMB =
        memoryDeltaRssBytes.reduce((sum, value) => sum + value, 0) / RUNS /
1024 / 1024;
    console.info(
        `[markdown stress] ${label} | runs=${RUNS} |
avgTime=${avgDurationMs.toFixed(2)}ms |
avgRssDelta=${avgRssDeltaMB.toFixed(2)}MB`
    );
    expect(output.length).toBeGreaterThan(0);
    expect(Number.isFinite(avgDurationMs)).toBe(true);
}
describe("Markdown parser stress tests", () => {
    it("times and measures memory for general-stress-test fixture", () => {
        runStressCase("./fixtures/general-stress-test.md", "general-stress-
test.md");
    });
    it("times and measures memory for Les Miserables fixture", () => {
        runStressCase("./fixtures/Les Miserables.md", "Les Miserables.md");
    });
});
```

markdown.test.ts

```ts
import { describe, it, expect } from "vitest";
import cases from "./fixtures/commonmark-0.31.2-filtered.json";
import { MarkdownParser } from "@markdown/markdown";
import type { Case } from "./types";
import { JSDOM } from "jsdom";
const PRESERVE_WHITESPACE = new Set(["PRE", "CODE", "TEXTAREA"]);
const markdown = new MarkdownParser();
function normalizeTextNodes(node: Node) {
    if (node.nodeType === node.TEXT_NODE) {
        const parent = node.parentNode as Element | null;
        if (parent && !PRESERVE_WHITESPACE.has(parent.tagName)) {
            node.nodeValue = node.nodeValue!.replace(/\s+/g, " ").trim();
        }
    }
    node.childNodes.forEach(normalizeTextNodes);
}
function normalizeDom(html: string): HTMLElement {
    const dom = new JSDOM(`<body>${html}</body>`);
    const body = dom.window.document.body;
    normalizeTextNodes(body);
    body.normalize();
    return body;
}
function domEqual(a: string, b: string): boolean {
    const A = normalizeDom(a);
    const B = normalizeDom(b);
    return A.isEqualNode(B);
}
describe.each(cases as Case[])("markdown example $example ($section)", (c)
=> {
    it(`renders example ${c.example}: ${c.markdown.replace(/\n/g, "\\n")}`,
() => {
        const actual = markdown.render(c.markdown);
        const expected = c.html;
        expect(
            domEqual(actual, expected),
            `DOM mismatch\n\nActual:\n${actual}\n\nExpected:\n${expected}`
        ).toBe(true);
    });
});
```

tests\types.ts

```ts
export interface Case {
  markdown: string;
  html: string;
  example: number;
```

```
  start_line: number;
  end_line: number;
  section: string;
}
```

electron-builder.json5

```
// @see - https://www.electron.build/configuration/configuration
// Windows-only application
{
  "$schema": "https://raw.githubusercontent.com/electron-userland/electron-builder/master/packages/app-builder-lib/scheme.json",
  "appId": "com.csnea.client",
  "asar": true,
  "productName": "LiveMarkdown",
  "directories": {
    "output": "release/${version}"
  },
  "files": [
    "dist",
    "dist-electron"
  ],
  "extraResources": [
    {
      "from": "kernel",
      "to": ".kernel",
      "filter": ["**/*"]
    }
  ],
  "win": {
    "target": [
      {
        "target": "nsis",
        "arch": [
          "x64"
        ]
      }
    ],
    "artifactName": "${productName}-Windows-${version}-Setup.${ext}",
    "icon": "public/favicon.ico"
  },
  "nsis": {
    "oneClick": false,
    "perMachine": false,
    "allowToChangeInstallationDirectory": true,
    "deleteAppDataOnUninstall": false
  }
}
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>LiveMarkdown</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.ts"></script>
  </body>
</html>
```

package.json

```
{
  "name": "client",
  "private": true,
  "version": "1.0.0",
  "description": "CS NEA Project",
  "author": "Jonah Gao",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vue-tsc && vite build && electron-builder",
    "preview": "vite preview",
    "test": "vitest run",
    "test:watch": "vitest",
    "start": "electron ."
  },
  "dependencies": {
    "@microsoft/signalr": "^10.0.0",
    "@xterm/addon-fit": "^0.10.0",
    "@xterm/xterm": "^5.5.0",
    "electron-store": "^11.0.2",
    "electron-window-state": "^5.0.3",
    "highlight.js": "^11.11.1",
    "material-symbols": "^0.40.0",
    "monaco-editor": "^0.55.1",
    "pinia": "^3.0.4",
    "uuid": "^13.0.0",
    "vite-plugin-monaco-editor": "^1.1.0",
    "vue": "^3.4.21"
  },
  "devDependencies": {
    "@types/jsdom": "^27.0.0",
```

```
    "@types/markdown-it": "^14.1.2",
    "@vitejs/plugin-vue": "^5.0.4",
    "electron": "^30.0.1",
    "electron-builder": "^24.13.3",
    "jsdom": "^28.0.0",
    "typescript": "^5.2.2",
    "vite": "^5.1.6",
    "vite-plugin-electron": "^0.28.6",
    "vite-plugin-electron-renderer": "^0.14.5",
    "vitest": "^4.0.16",
    "vue-tsc": "^2.0.26"
  },
  "main": "dist-electron/main.js"
}
```

splash.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Loading</title>
</head>
<body>
    <div id="app"></div>
    <script type="module" src="/src/splash.ts"></script>
</body>
</html>
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "module": "ESNext",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "skipLibCheck": true,
    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "preserve",
    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
```

```
    "noFallthroughCasesInSwitch": true,
    "baseUrl": ".",
    "paths": {
      "@markdown/*": ["src/lib/markdown/*"],
      "@/*": ["src/*"]
    }
  },
  "include": ["src", "tests/**/*"],
  "references": [{ "path": "./tsconfig.node.json" }],
}
```

tsconfig.node.ts

```
{
  "compilerOptions": {
    "composite": true,
    "skipLibCheck": true,
    "module": "ESNext",
    "moduleResolution": "bundler",
    "allowSyntheticDefaultImports": true,
    "strict": true
  },
  "include": ["vite.config.ts", "electron"],
  "exclude": ["src"]
}
```

vite.config.ts

```
import {defineConfig} from 'vite'
import path from 'node:path'
import electron from 'vite-plugin-electron/simple'
import vue from '@vitejs/plugin-vue'
import monacoEditorPlugin from 'vite-plugin-monaco-editor'
// The vite-plugin-monaco-editor package uses CommonJS default export
// but TypeScript/ESM import gets the module object. This handles both
cases.
const monacoEditor = (monacoEditorPlugin as unknown as { default: typeof
monacoEditorPlugin }).default || monacoEditorPlugin
// https://vitejs.dev/config/
export default defineConfig({
    build: {
        rollupOptions: {
            input: {
                main: path.join(__dirname, 'index.html'),
                splash: path.join(__dirname, 'splash.html'),
            },
        },
    },
    plugins: [
        vue(),
```

```
        monacoEditor({}),
        electron({
            main: {
                // Shortcut of `build.lib.entry`.
                entry: 'electron/main.ts',
                vite: {
                    build: {
                    },
                },
            },
            preload: {
                // Shortcut of `build.rollupOptions.input`.
                // Preload scripts may contain Web assets, so use the `build.rollupOptions.input` instead `build.lib.entry`.
                input: path.join(__dirname, 'electron/preload.ts'),
                vite: {
                    build: {
                        rollupOptions: {
                            output: {
                                format: 'module', // Use moduleJS format
                                entryFileNames: 'preload.mjs', // Force filename to preload.mjs
                            }
                        }
                    }
                }
            },
            // Ployfill the Electron and Node.js API for Renderer process.
            // If you want to use Node.js in Renderer process, the `nodeIntegration` needs to be enabled in the Main process.
            // See 👉 https://github.com/electron-vite/vite-plugin-electron-renderer
            renderer: process.env.NODE_ENV === 'test'
                // https://github.com/electron-vite/vite-plugin-electron-renderer/issues/78#issuecomment-2053600808
                ? undefined
                : {},
        }),
    ],
    resolve: {
        alias: {
            "@markdown": path.resolve(__dirname, "src/lib/markdown"),
            "@": path.resolve(__dirname, "src"),
        },
    }
})
```

FileHub.cs

```
using System.Threading.Channels;
using kernel.Services;
using kernel.Utils;
using Microsoft.AspNetCore.SignalR;
namespace kernel.Hubs;
/// <summary>
/// SignalR hub for file system operations.
/// Provides streaming file tree reading, file content streaming, workspace settings,
/// and file system watching for incremental updates.
/// </summary>
public class FileHub : Hub
{
    private readonly FileService _fileService;
    private readonly FileWatcherService _fileWatcherService;
    private readonly IHubContext<FileHub> _hubContext;
    private readonly ILogger<FileHub> _logger;
    // Track which directories each connection is watching
    private static readonly Dictionary<string, HashSet<string>> ConnectionWatchedDirs = new();
    private static readonly Lock Lock = new();
    public FileHub(
        FileService fileService,
        FileWatcherService fileWatcherService,
        IHubContext<FileHub> hubContext,
        ILogger<FileHub> logger)
    {
        _fileService = fileService;
        _fileWatcherService = fileWatcherService;
        _hubContext = hubContext;
        _logger = logger;
        _fileWatcherService.DirectoryDirty -= OnDirectoryDirty;
        _fileWatcherService.DirectoryDirty += OnDirectoryDirty;
    }
    /// <summary>
    /// Called by FileWatcherService when a directory change is detected.
    /// Broadcasts to clients watching that directory.
    /// </summary>
    /// <param name="rootPath">
    /// the top-level directory being watched (the one clients subscribe to).
    /// </param>
    /// <param name="dirPath">
    /// dirPath is the specific directory that changed (could be rootPath or a subdirectory
    /// </param>
```

```
43      private void OnDirectoryDirty(string rootPath, string dirPath)
44      {
45          var groupName = GetGroupName(rootPath);
46          _hubContext.Clients.Group(groupName).SendAsync("DirectoryDirty",
    dirPath);
47          _logger.LogDebug("Broadcasted directory dirty to group {Group}:
    {DirPath}", groupName, dirPath);
48      }
49      /// <summary>
50      /// Start watching a directory for file changes.
51      /// The client will receive FileChanged events for this directory.
52      /// </summary>
53      public async Task StartWatchingAsync(string directoryPath)
54      {
55          if (string.IsNullOrWhiteSpace(directoryPath))
56          {
57              _logger.LogWarning("Invalid directory path for watching");
58              return;
59          }
60          var fullPath = Path.GetFullPath(directoryPath);
61          var groupName = GetGroupName(fullPath);
62          if (_fileWatcherService.IsWatching(fullPath))
63          {
64              return;
65          }
66          // Add this connection to the group
67          await Groups.AddToGroupAsync(Context.ConnectionId, groupName);
68          // Track which directories this connection is watching
69          lock (Lock)
70          {
71              if (!ConnectionWatchedDirs.TryGetValue(Context.ConnectionId, out
    var dirs))
72              {
73                  dirs = [];
74                  ConnectionWatchedDirs[Context.ConnectionId] = dirs;
75              }
76              dirs.Add(fullPath);
77          }
78          // Start the watcher if not already watching
79          _fileWatcherService.StartWatching(fullPath);
80          _logger.LogInformation("Client {ConnectionId} started watching:
    {Path}",
81              Context.ConnectionId, LogFormatter.ToGreen(fullPath));
82      }
83      /// <summary>
84      /// Stop watching a directory for file changes.
85      /// </summary>
86      public async Task StopWatchingAsync(string directoryPath)
```

```
    {
        if (string.IsNullOrWhiteSpace(directoryPath))
        {
            return;
        }
        var fullPath = Path.GetFullPath(directoryPath);
        var groupName = GetGroupName(fullPath);

        if (!_fileWatcherService.IsWatching(fullPath))
        {
            return;
        }
        // Remove this connection from the group
        await Groups.RemoveFromGroupAsync(Context.ConnectionId, groupName);
        // Update tracking
        lock (Lock)
        {
            if (ConnectionWatchedDirs.TryGetValue(Context.ConnectionId, out
var dirs))
            {
                dirs.Remove(fullPath);
            }
        }

        _fileWatcherService.StopWatching(fullPath);
        _logger.LogInformation("Client {ConnectionId} stopped watching:
{Path}",
            Context.ConnectionId, LogFormatter.ToYellow(fullPath));
    }
    /// <summary>
    /// Called when a client disconnects. Clean up watched directories.
    /// </summary>
    public override async Task OnDisconnectedAsync(Exception? exception)
    {
        lock (Lock)
        {
            ConnectionWatchedDirs.Remove(Context.ConnectionId);
        }
        _logger.LogInformation("Client disconnected: {ConnectionId}",
Context.ConnectionId);
        await base.OnDisconnectedAsync(exception);
    }
    private static string GetGroupName(string directoryPath) =>
$"watch:{directoryPath}";
    /// <summary>
    /// Read directory contents recursively and stream results.
    /// </summary>
```

```
130     public IAsyncEnumerable<FileService.FileNode> ReadDirAsync(string
    dirPath)
131     {
132         _logger.LogDebug("Reading files in directory: {DirPath}",
    LogFormatter.ToGreen(dirPath));
133         return _fileService.ReadDirAsync(dirPath);
134     }
135     /// <summary>
136     /// Stream file content for opening in a tab.
137     /// </summary>
138     public IAsyncEnumerable<FileService.TabChunk> StreamTabAsync(string
    filePath)
139     {
140         _logger.LogInformation("Creating tab for file: {FilePath}",
    LogFormatter.ToGreen(filePath));
141         return _fileService.StreamTabAsync(filePath);
142     }
143     /// <summary>
144     /// Create a new directory.
145     /// </summary>
146     public void CreateDirectory(string dirPath)
147     {
148         _logger.LogInformation("Creating directory: {DirPath}",
    LogFormatter.ToGreen(dirPath));
149         _fileService.CreateDirectory(dirPath);
150     }
151     public void DeleteDirectory(string dirPath)
152     {
153         _logger.LogInformation("Deleting directory: {DirPath}",
    LogFormatter.ToGreen(dirPath));
154         _fileService.DeleteDirectory(dirPath);
155     }
156     public void MoveDirectory(string sourcePath, string destPath)
157     {
158         _logger.LogInformation("Moving directory from {SourcePath} to
    {DestPath}",
159             LogFormatter.ToGreen(sourcePath),
    LogFormatter.ToGreen(destPath));
160         _fileService.MoveDirectory(sourcePath, destPath);
161     }
162     public void RenameDirectory(string sourcePath, string destPath)
163     {
164         _logger.LogInformation("Renaming directory from {SourcePath} to
    {DestPath}",
165             LogFormatter.ToGreen(sourcePath),
    LogFormatter.ToGreen(destPath));
166         _fileService.MoveDirectory(sourcePath, destPath);
167     }
```

```
168     public void CopyDirectory(string sourcePath, string destPath)
169     {
170         _logger.LogInformation("Copying directory from {SourcePath} to
    {DestPath}",
171             LogFormatter.ToGreen(sourcePath),
    LogFormatter.ToGreen(destPath));
172         _fileService.CopyDirectory(sourcePath, destPath);
173     }
174     public void CreateFile(string filePath)
175     {
176         _logger.LogInformation("Creating file: {FilePath}",
    LogFormatter.ToGreen(filePath));
177         _fileService.CreateFile(filePath);
178     }
179     public void DeleteFile(string filePath)
180     {
181         _logger.LogInformation("Deleting file: {FilePath}",
    LogFormatter.ToGreen(filePath));
182         _fileService.DeleteFile(filePath);
183     }
184     public void MoveFile(string sourcePath, string destPath)
185     {
186         _logger.LogInformation("Moving file from {SourcePath} to
    {DestPath}",
187             LogFormatter.ToGreen(sourcePath),
    LogFormatter.ToGreen(destPath));
188         _fileService.MoveFile(sourcePath, destPath);
189     }
190     public void RenameFile(string sourcePath, string destPath)
191     {
192         _logger.LogInformation("Renaming file from {SourcePath} to
    {DestPath}",
193             LogFormatter.ToGreen(sourcePath),
    LogFormatter.ToGreen(destPath));
194         _fileService.MoveFile(sourcePath, destPath);
195     }
196     public void CopyFile(string sourcePath, string destPath)
197     {
198         _logger.LogInformation("Copying file from {SourcePath} to
    {DestPath}",
199             LogFormatter.ToGreen(sourcePath),
    LogFormatter.ToGreen(destPath));
200         _fileService.CopyFile(sourcePath, destPath);
201     }
202     /// <summary>
203     /// Save file content from a stream.
204     /// </summary>
205     /// <param name="filePath">The target file path</param>
```

```
    /// <param name="stream">The content stream to write</param>
    /// <param name="encoding">The encoding to use when saving (default: utf-8)</param>
    public async Task SaveFileAsync(string filePath, ChannelReader<string> stream, string encoding = "utf-8")
    {
        _logger.LogInformation("Saving file: {FilePath} with encoding: {Encoding}",
            LogFormatter.ToGreen(filePath), encoding);
        await _fileService.SaveFileAsync(filePath, stream, encoding);
    }
    /// <summary>
    /// Save workspace settings to disk.
    /// </summary>
    public async Task SaveWorkspaceSettingsAsync(string cwd, FileService.PanelLayout layout)
    {
        _logger.LogInformation("Saving panel layout to: {LayoutPath}", LogFormatter.ToGreen(cwd));
        await _fileService.SaveWorkspaceSettingsAsync(cwd, layout);
    }
    /// <summary>
    /// Load workspace settings from disk.
    /// </summary>
    public async Task<FileService.PanelLayout?> LoadWorkspaceSettingsAsync(string cwd)
    {
        _logger.LogInformation("Loading panel layout from: {LayoutPath}", LogFormatter.ToGreen(cwd));
        return await _fileService.LoadWorkspaceSettingsAsync(cwd);
    }
}
```

PythonHub.cs

```
using kernel.Services;
using kernel.Utils;
using Microsoft.AspNetCore.SignalR;
namespace kernel.Hubs;
/// <summary>
/// SignalR hub for Python code execution.
/// Manages Python virtual environment and code execution sessions.
/// </summary>
public class PythonHub(PythonVenvRunner pythonService, ILogger<PythonHub> logger) : Hub
{
    public async Task CreateVenvAsync(string venvPath, string systemPythonPath)
    {
```

```
13          logger.LogInformation(
14              "Creating Python virtual environment\n\tVenv Path: {VenvPath}",
15              LogFormatter.ToCyan(venvPath)
16          );
17          await pythonService.CreateVenvAsync(venvPath, systemPythonPath);
18      }
19      /// <summary>
20      /// Execute Python code in a virtual environment.
21      /// </summary>
22      public async Task ExecuteCodeAsync(string terminalId, string userCode,
    string systemPythonPath, string venvPath)
23      {
24          var connectionId = Context.ConnectionId;
25          logger.LogInformation(
26              "Initializing Python execution\n\tTerminal ID:
    {TerminalId}\n\tConnection ID: {ConnectionId}",
27              LogFormatter.ToCyan(terminalId),
28              LogFormatter.ToCyan(connectionId)
29          );
30          await pythonService.ExecuteCodeAsync(connectionId, terminalId,
    userCode, systemPythonPath, venvPath);
31      }
32      /// <summary>
33      /// Send input to a running Python process.
34      /// </summary>
35      public async Task PythonInput(string terminalId, string data)
36      {
37          logger.LogDebug("Received Python input: {Data}",
    LogFormatter.ToYellow(data));
38          await pythonService.WriteInputAsync(terminalId, data);
39      }
40      /// <summary>
41      /// Resize the Python terminal.
42      /// </summary>
43      public void TerminalResize(int cols, int rows)
44      {
45          pythonService.Resize(Context.ConnectionId, cols, rows);
46      }
47      /// <summary>
48      /// Clean up Python session on disconnect.
49      /// </summary>
50      public override Task OnDisconnectedAsync(Exception? exception)
51      {
52          pythonService.KillTerminal(Context.ConnectionId);
53          return base.OnDisconnectedAsync(exception);
54      }
55  }
```

TerminalHub.cs

```
using kernel.Services;
using kernel.Utils;
using Microsoft.AspNetCore.SignalR;
namespace kernel.Hubs;
/// <summary>
/// SignalR hub for terminal operations.
/// Manages PTY terminal sessions for connected clients.
/// </summary>
public class TerminalHub(TerminalService terminalService,
ILogger<TerminalHub> logger) : Hub
{
    /// <summary>
    /// Initialize a new terminal session for the connection.
    /// </summary>
    public async Task TerminalInit(string terminalId, string cwd)
    {
        var connectionId = Context.ConnectionId;
        logger.LogInformation("Initializing terminal for connection:
{ConnectionId}",
            LogFormatter.ToCyan(connectionId));
        await terminalService.StartTerminalAsync(terminalId, connectionId,
"TerminalOutput", cwd);
    }
    /// <summary>
    /// Send input to the terminal.
    /// </summary>
    public async Task TerminalInput(string terminalId, string data)
    {
        logger.LogDebug("Received terminal input: {Data}",
LogFormatter.ToYellow(data));
        await terminalService.WriteInputAsync(terminalId, data);
    }
    /// <summary>
    /// Resize the terminal.
    /// </summary>
    public void TerminalResize(string terminalId, int cols, int rows)
    {
        terminalService.Resize(terminalId, cols, rows);
    }
    /// <summary>
    /// Clean up terminal session on disconnect.
    /// </summary>
    public void TerminalDisconnect(string terminalId)
    {
        terminalService.KillTerminal(terminalId);
    }
```

```
}
```

launchSettings.json

```
{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": false,
      "applicationUrl": "http://127.0.0.1:5238",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": false,
      "applicationUrl": "https://127.0.0.1;http://127.0.0.1:5238",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

FileService.cs

```
using System.Runtime.CompilerServices;
using System.Runtime.Serialization;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Threading.Channels;
using JetBrains.Annotations;
using kernel.Utils;
using UtfUnknown;
namespace kernel.Services;
/// <summary>
/// Service for file system operations including reading directories,
/// streaming file content, and managing workspace settings.
/// </summary>
public class FileService(ILogger<FileService> logger)
{
    /// <summary>
    /// Represents a file or directory node in the file tree.
    /// Used as a DTO for streaming directory contents to the frontend.
    /// </summary>
```

```
    public record FileNode(
        [property: UsedImplicitly] string Name,
        [property: UsedImplicitly] string Path,
        [property: UsedImplicitly] string Extension,
        [property: UsedImplicitly] string ParentPath,
        [property: UsedImplicitly] bool IsDirectory,
        [property: UsedImplicitly] bool Expanded = false
    );
    /// <summary>
    /// Represents a chunk of file content for streaming.
    /// Can be metadata (file info), content (file data), or error.
    /// </summary>
    public record TabChunk(
        [property: UsedImplicitly] string Id,
        [property: UsedImplicitly] string Name = "",
        [property: UsedImplicitly] string Path = "",
        [property: UsedImplicitly] string Content = "",
        [property: UsedImplicitly] bool IsMetadata = false,
        [property: UsedImplicitly] bool IsError = false,
        [property: UsedImplicitly] string Encoding = "utf-8"
    );
    /// <summary>
    /// View mode for the editor/preview split.
    /// </summary>
    [JsonConverter(typeof(JsonStringEnumConverter))]
    public enum ViewMode
    {
        [UsedImplicitly][EnumMember(Value = "code")] Code,
        [UsedImplicitly][EnumMember(Value = "split")] Split,
        [UsedImplicitly][EnumMember(Value = "preview")] Preview
    }
    /// <summary>
    /// Sidebar panel type.
    /// </summary>
    [JsonConverter(typeof(JsonStringEnumConverter))]
    public enum SidebarPanel
    {
        [UsedImplicitly][EnumMember(Value = "explorer")] Explorer,
        [UsedImplicitly][EnumMember(Value = "search")] Search,
        [UsedImplicitly][EnumMember(Value = "terminal")] Terminal
    }
    /// <summary>
    /// Layout configuration for the workspace panels.
    /// Persisted to workspace settings file.
    /// </summary>
    public record PanelLayout(
        double ExplorerWidth,
        double TerminalHeight,
```

```
69          double EditorPreviewRatio,
70          ViewMode PreferredViewMode,
71          string[] OpenedFiles,
72          string[] ExpandedDirectories,
73          string ActiveFile = "",
74          bool ExplorerVisible = true,
75          bool SearchVisible = false,
76          bool TerminalVisible = false,
77          SidebarPanel? ActiveTopPanel = SidebarPanel.Explorer,
78          SidebarPanel? ActiveBottomPanel = null
79      );
80      /// <summary>
81      /// JSON serializer options for workspace settings.
82      /// </summary>
83      private static readonly JsonSerializerOptions JsonSerializerOptions =
    new()
84      {
85          WriteIndented = true,
86          PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
87          Converters =
88          {
89              new JsonStringEnumConverter(JsonNamingPolicy.CamelCase)
90          }
91      };
92      /// <summary>
93      /// Stream file content as chunks for efficient loading of large files.
94      /// First yields metadata (including detected encoding), then yields
    content chunks.
95      /// Uses Mozilla Universal Charset Detector to detect file encoding.
96      /// </summary>
97      public async IAsyncEnumerable<TabChunk> StreamTabAsync(string filePath,
98          [EnumeratorCancellation] CancellationToken token = default)
99      {
100         var tabId = Guid.NewGuid().ToString();
101         // Detect file encoding using Mozilla Universal Charset Detector
102         var detectedEncoding = "utf-8";
103         try
104         {
105             var result = await CharsetDetector.DetectFromFileAsync(filePath,
    token);
106             if (result.Detected?.EncodingName != null)
107             {
108                 // Treat ASCII as UTF-8 for frontend compatibility
109                 detectedEncoding =
    result.Detected.EncodingName.Equals("ASCII",
    StringComparison.OrdinalIgnoreCase)
110                     ? "utf-8"
111                     : result.Detected.EncodingName;
```

```
                logger.LogDebug("Detected encoding for {FilePath}: {Encoding}",
                    LogFormatter.ToGreen(filePath), detectedEncoding);
            }
        }
        catch (Exception ex)
        {
            logger.LogWarning(ex, "Failed to detect encoding for {FilePath}, defaulting to UTF-8", filePath);
        }
        // First return tab metadata so frontend can create the tab immediately
        yield return new TabChunk(
            Id: tabId,
            Name: Path.GetFileName(filePath),
            Path: filePath,
            IsMetadata: true,
            Encoding: detectedEncoding
        );
        await foreach (var chunk in StreamFileContentAsync(tabId, filePath, detectedEncoding, token))
        {
            yield return chunk;
        }
    }
    /// <summary>
    /// Stream file content in chunks using the specified encoding.
    /// </summary>
    private async IAsyncEnumerable<TabChunk> StreamFileContentAsync(string tabId, string filePath,
        string encodingName, [EnumeratorCancellation] CancellationToken token = default)
    {
        TabChunk? errorTabChunk = null;
        Encoding encoding;

        try
        {
            encoding = GetEncodingByName(encodingName);
        }
        catch (Exception ex)
        {
            logger.LogWarning(ex, "Failed to get encoding {EncodingName}, falling back to UTF-8", encodingName);
            encoding = Encoding.UTF8;
        }

        StreamReader? reader = null;
```

```
        try
        {
            reader = new StreamReader(filePath, encoding,
detectEncodingFromByteOrderMarks: true);
        }
        catch (UnauthorizedAccessException ex)
        {
            logger.LogError(ex, "Access denied: {FilePath}",
LogFormatter.ToBrightRed(filePath));
            errorTabChunk = CreateErrorTabChunk(tabId, ex.Message);
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Cannot open file: {FilePath}",
LogFormatter.ToBrightRed(filePath));
            errorTabChunk = CreateErrorTabChunk(tabId, ex.Message);
        }
        if (errorTabChunk != null)
        {
            yield return errorTabChunk;
            yield break;
        }
        using (reader)
        {
            const int chunkSize = 65536; // 64KiB per chunk for efficient
streaming
            var buffer = new char[chunkSize];
            int charsRead;
            while ((charsRead = await reader!.ReadAsync(buffer, 0,
buffer.Length)) > 0)
            {
                token.ThrowIfCancellationRequested();
                yield return new TabChunk(
                    Id: tabId,
                    Content: new string(buffer, 0, charsRead)
                );
            }
        }
    }

    /// <summary>
    /// Get the Encoding object by name, with fallback to UTF-8.
    /// </summary>
    private static Encoding GetEncodingByName(string encodingName)
    {
        // Common encoding name mappings
        return encodingName.ToLowerInvariant() switch
        {
```

```
            "utf-8" or "utf8" => Encoding.UTF8,
            "utf-16le" or "utf-16" => Encoding.Unicode,
            "utf-16be" => Encoding.BigEndianUnicode,
            "utf-32" or "utf-32le" => Encoding.UTF32,
            "ascii" => Encoding.UTF8,
            "iso-8859-1" or "latin1" => Encoding.Latin1,
            _ => Encoding.GetEncoding(encodingName)
        };
    }
    /// <summary>
    /// Create an error tab chunk for reporting file access errors.
    /// </summary>
    private static TabChunk CreateErrorTabChunk(string tabId, string
errorMessage)
    {
        return new TabChunk(
            Id: tabId,
            Content: errorMessage,
            IsError: true
        );
    }
    /// <summary>
    /// Read directory contents recursively using BFS.
    /// Streams results as they are discovered.
    /// </summary>
    public async IAsyncEnumerable<FileNode> ReadDirAsync(string dirPath,
        [EnumeratorCancellation] CancellationToken token = default)
    {
        var queue = new Queue<string>();
        queue.Enqueue(dirPath);
        var count = 0;
        while (queue.Count > 0)
        {
            token.ThrowIfCancellationRequested();
            var currentDir = queue.Dequeue();
            IEnumerable<FileSystemInfo> entries;
            try
            {
                entries = new
DirectoryInfo(currentDir).EnumerateFileSystemInfos();
            }
            catch (UnauthorizedAccessException ex)
            {
                logger.LogWarning("Access denied to {Path}: {Msg}", dirPath,
ex.Message);
                yield break; // Exit if root directory cannot be opened
            }
            catch (DirectoryNotFoundException)
```

```
            {
                continue;
            }
            foreach (var entry in entries)
            {
                token.ThrowIfCancellationRequested();
                // Yield periodically to avoid blocking
                if (++count % 100 == 0)
                {
                    await Task.Yield();
                }
                FileNode? node;
                try
                {
                    node = new FileNode(
                        Name: entry.Name,
                        Path: entry.FullName,
                        Extension: entry is FileInfo file ? file.Extension :
string.Empty,
                        IsDirectory: entry is DirectoryInfo,
                        ParentPath: currentDir,
                        Expanded: false
                    );
                }
                catch (UnauthorizedAccessException)
                {
                    continue; // Skip files/folders without permissions
                }
                catch (PathTooLongException)
                {
                    logger.LogWarning("Path too long: {Name}", entry.Name);
                    continue;
                }
                catch (Exception ex)
                {
                    logger.LogError(ex, "Error reading: {Name}",
entry.Name);
                    continue;
                }
                yield return node;
                // BFS: enqueue directories for further exploration
                if (entry is DirectoryInfo)
                {
                    queue.Enqueue(entry.FullName);
                }
            }
        }
    }
```

```
287     /// <summary>
288     /// Create a directory at the specified path.
289     /// </summary>
290     /// <param name="dirPath"></param>
291     public void CreateDirectory(string dirPath)
292     {
293         if (string.IsNullOrWhiteSpace(dirPath))
294         {
295             logger.LogError("Invalid directory path: {DirPath}",
    LogFormatter.ToBrightRed(dirPath));
296             return;
297         }
298         // Normalize and validate path
299         var fullPath = Path.GetFullPath(dirPath);
300         try
301         {
302             Directory.CreateDirectory(fullPath);
303             logger.LogInformation("Directory created: {DirPath}",
    LogFormatter.ToGreen(fullPath));
304         }
305         catch (Exception ex)
306         {
307             logger.LogError(ex, "Error creating directory: {DirPath}",
    LogFormatter.ToBrightRed(fullPath));
308         }
309     }
310     public void CreateFile(string filePath)
311     {
312         if (string.IsNullOrWhiteSpace(filePath))
313         {
314             logger.LogError("Invalid file path: {FilePath}",
    LogFormatter.ToBrightRed(filePath));
315             return;
316         }
317         // Normalize and validate path
318         var fullPath = Path.GetFullPath(filePath);
319         try
320         {
321             using var fs = File.Create(fullPath);
322             logger.LogInformation("File created: {FilePath}",
    LogFormatter.ToGreen(fullPath));
323         }
324         catch (Exception ex)
325         {
326             logger.LogError(ex, "Error creating file: {FilePath}",
    LogFormatter.ToBrightRed(fullPath));
327         }
328     }
```

```
329     public void DeleteFile(string filePath)
330     {
331         if (string.IsNullOrWhiteSpace(filePath))
332         {
333             logger.LogError("Invalid file path: {FilePath}",
    LogFormatter.ToBrightRed(filePath));
334             return;
335         }
336         // Normalize and validate path
337         var fullPath = Path.GetFullPath(filePath);
338         try
339         {
340             if (File.Exists(fullPath))
341             {
342                 File.Delete(fullPath);
343                 logger.LogInformation("File deleted: {FilePath}",
    LogFormatter.ToGreen(fullPath));
344             }
345             else
346             {
347                 logger.LogWarning("File does not exist: {FilePath}",
    LogFormatter.ToBrightRed(fullPath));
348             }
349         }
350         catch (Exception ex)
351         {
352             logger.LogError(ex, "Error deleting file: {FilePath}",
    LogFormatter.ToBrightRed(fullPath));
353         }
354     }
355     public void DeleteDirectory(string dirPath)
356     {
357         if (string.IsNullOrWhiteSpace(dirPath))
358         {
359             logger.LogError("Invalid directory path: {DirPath}",
    LogFormatter.ToBrightRed(dirPath));
360             return;
361         }
362         // Normalize and validate path
363         var fullPath = Path.GetFullPath(dirPath);
364         try
365         {
366             if (Directory.Exists(fullPath))
367             {
368                 Directory.Delete(fullPath, recursive: true);
369                 logger.LogInformation("Directory deleted: {DirPath}",
    LogFormatter.ToGreen(fullPath));
370             }
```

```
            else
            {
                logger.LogWarning("Directory does not exist: {DirPath}",
LogFormatter.ToBrightRed(fullPath));
            }
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Error deleting directory: {DirPath}",
LogFormatter.ToBrightRed(fullPath));
        }
    }
    public void MoveFile(string oldFilePath, string newFilePath)
    {
        if (string.IsNullOrWhiteSpace(oldFilePath) ||
string.IsNullOrWhiteSpace(newFilePath))
        {
            logger.LogError("Invalid file paths: {OldFilePath},
{NewFilePath}",
                LogFormatter.ToBrightRed(oldFilePath),
                LogFormatter.ToBrightRed(newFilePath));
            return;
        }
        // Normalize and validate paths
        var fullOldPath = Path.GetFullPath(oldFilePath);
        var fullNewPath = Path.GetFullPath(newFilePath);
        try
        {
            if (File.Exists(fullOldPath))
            {
                File.Move(fullOldPath, fullNewPath);
                logger.LogInformation("File renamed from {OldFilePath} to
{NewFilePath}",
                    LogFormatter.ToGreen(fullOldPath),
                    LogFormatter.ToGreen(fullNewPath));
            }
            else
            {
                logger.LogWarning("Source file does not exist:
{OldFilePath}", LogFormatter.ToBrightRed(fullOldPath));
            }
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Error renaming file from {OldFilePath} to
{NewFilePath}",
                LogFormatter.ToBrightRed(fullOldPath),
                LogFormatter.ToBrightRed(fullNewPath));
```

```
412         }
413     }
414 
415     public void MoveDirectory(string oldFilePath, string newFilePath)
416     {
417         if (string.IsNullOrWhiteSpace(oldFilePath) ||
    string.IsNullOrWhiteSpace(newFilePath))
418         {
419             logger.LogError("Invalid directory paths: {OldDirPath},
    {NewDirPath}",
420                 LogFormatter.ToBrightRed(oldFilePath),
421                 LogFormatter.ToBrightRed(newFilePath));
422             return;
423         }
424         // Normalize and validate paths
425         var fullOldPath = Path.GetFullPath(oldFilePath);
426         var fullNewPath = Path.GetFullPath(newFilePath);
427         try
428         {
429             if (Directory.Exists(fullOldPath))
430             {
431                 Directory.Move(fullOldPath, fullNewPath);
432                 logger.LogInformation("Directory renamed from {OldDirPath}
    to {NewDirPath}",
433                     LogFormatter.ToGreen(fullOldPath),
434                     LogFormatter.ToGreen(fullNewPath));
435             }
436             else
437             {
438                 logger.LogWarning("Source directory does not exist:
    {OldDirPath}", LogFormatter.ToBrightRed(fullOldPath));
439             }
440         }
441         catch (Exception ex)
442         {
443             logger.LogError(ex, "Error renaming directory from {OldDirPath}
    to {NewDirPath}",
444                 LogFormatter.ToBrightRed(fullOldPath),
445                 LogFormatter.ToBrightRed(fullNewPath));
446         }
447     }
448     public void CopyFile(string oldFilePath, string newFilePath)
449     {
450         if (string.IsNullOrWhiteSpace(oldFilePath) ||
    string.IsNullOrWhiteSpace(newFilePath))
451         {
452             logger.LogError("Invalid file paths: {OldFilePath},
    {NewFilePath}",
```

```
                LogFormatter.ToBrightRed(oldFilePath),
                LogFormatter.ToBrightRed(newFilePath));
            return;
        }
        // Normalize and validate paths
        var fullOldPath = Path.GetFullPath(oldFilePath);
        var fullNewPath = Path.GetFullPath(newFilePath);
        try
        {
            if (File.Exists(fullOldPath))
            {
                File.Copy(fullOldPath, fullNewPath, overwrite: false);
                logger.LogInformation("File copied from {OldFilePath} to
{NewFilePath}",
                    LogFormatter.ToGreen(fullOldPath),
                    LogFormatter.ToGreen(fullNewPath));
            }
            else
            {
                logger.LogWarning("Source file does not exist:
{OldFilePath}", LogFormatter.ToBrightRed(fullOldPath));
            }
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Error copying file from {OldFilePath} to
{NewFilePath}",
                LogFormatter.ToBrightRed(fullOldPath),
                LogFormatter.ToBrightRed(fullNewPath));
        }
    }

    public void CopyDirectory(string oldDirPath, string newDirPath)
    {
        if (string.IsNullOrWhiteSpace(oldDirPath) ||
string.IsNullOrWhiteSpace(newDirPath))
        {
            logger.LogError("Invalid directory paths: {OldDirPath},
{NewDirPath}",
                LogFormatter.ToBrightRed(oldDirPath),
                LogFormatter.ToBrightRed(newDirPath));
            return;
        }
        // Normalize and validate paths
        var fullOldPath = Path.GetFullPath(oldDirPath);
        var fullNewPath = Path.GetFullPath(newDirPath);
        try
        {
```

```
            if (Directory.Exists(fullOldPath))
            {
                CopyDirectoryRecursive(fullOldPath, fullNewPath);
                logger.LogInformation("Directory copied from {OldDirPath} to
{NewDirPath}",
                    LogFormatter.ToGreen(fullOldPath),
                    LogFormatter.ToGreen(fullNewPath));
            }
            else
            {
                logger.LogWarning("Source directory does not exist:
{OldDirPath}", LogFormatter.ToBrightRed(fullOldPath));
            }
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Error copying directory from {OldDirPath}
to {NewDirPath}",
                LogFormatter.ToBrightRed(fullOldPath),
                LogFormatter.ToBrightRed(fullNewPath));
        }
    }

    private static void CopyDirectoryRecursive(string sourceDir, string
destDir)
    {
        Directory.CreateDirectory(destDir);
        foreach (var filePath in Directory.GetFiles(sourceDir))
        {
            var destFilePath = Path.Combine(destDir,
Path.GetFileName(filePath));
            File.Copy(filePath, destFilePath, overwrite: false);
        }
        foreach (var directoryPath in Directory.GetDirectories(sourceDir))
        {
            var destDirectoryPath = Path.Combine(destDir,
Path.GetFileName(directoryPath));
            CopyDirectoryRecursive(directoryPath, destDirectoryPath);
        }
    }

    /// <summary>
    /// Save file content from a stream to disk.
    /// Uses atomic write with temporary file for data safety.
    /// </summary>
    /// <param name="filePath">The target file path</param>
    /// <param name="stream">The content stream to write</param>
```

```
537     /// <param name="encodingName">The encoding to use when saving (default: utf-8)</param>
538     /// <param name="ct">Cancellation token</param>
539     public async Task SaveFileAsync(string filePath, ChannelReader<string> stream,
540         string encodingName = "utf-8", CancellationToken ct = default)
541     {
542         if (string.IsNullOrWhiteSpace(filePath))
543         {
544             logger.LogError("Invalid file path: {FilePath}", LogFormatter.ToBrightRed(filePath));
545             return;
546         }
547         // Normalize and validate path
548         var fullPath = Path.GetFullPath(filePath);
549         var dir = Path.GetDirectoryName(fullPath);
550         if (string.IsNullOrEmpty(dir))
551         {
552             logger.LogError("Invalid file path: {FilePath}", LogFormatter.ToBrightRed(filePath));
553             return;
554         }
555         Directory.CreateDirectory(dir);
556         var tempPath = Path.Combine(dir, $"{Path.GetFileName(fullPath)}.{Guid.NewGuid():N}.tmp");
557 
558         // Get the encoding to use for saving
559         Encoding encoding;
560         try
561         {
562             encoding = GetEncodingByName(encodingName);
563             logger.LogDebug("Saving file {FilePath} with encoding: {Encoding}",
564                 LogFormatter.ToGreen(fullPath), encodingName);
565         }
566         catch (Exception ex)
567         {
568             logger.LogWarning(ex, "Failed to get encoding {EncodingName}, falling back to UTF-8", encodingName);
569             encoding = Encoding.UTF8;
570         }
571         try
572         {
573             await using (var writer = new StreamWriter(tempPath, false, encoding, 65536))
574             {
575                 await foreach (var chunk in stream.ReadAllAsync(ct))
576                 {
```

```
                    await writer.WriteAsync(chunk);
                }
            }
            // Atomic replace for data safety
            if (File.Exists(fullPath))
            {
                File.Replace(tempPath, fullPath, destinationBackupFileName:
null, ignoreMetadataErrors: true);
            }
            else
            {
                File.Move(tempPath, fullPath, overwrite: true);
            }
            logger.LogInformation("File saved: {FilePath}",
LogFormatter.ToGreen(fullPath));
        }
        catch
        {
            // Clean up temporary file on failure
            if (File.Exists(tempPath))
                File.Delete(tempPath);
        }
    }
    /// <summary>
    /// Save workspace settings to a JSON file in the workspace directory.
    /// </summary>
    public async Task SaveWorkspaceSettingsAsync(string cwd, PanelLayout
layout, CancellationToken ct = default)
    {
        if (string.IsNullOrWhiteSpace(cwd))
        {
            logger.LogError("Invalid layout path: {LayoutPath}",
LogFormatter.ToBrightRed(cwd));
            return;
        }
        // Normalize and validate path
        var fullPath = Path.GetFullPath(cwd);
        var settingsDir = Path.Combine(fullPath, ".LiveMarkdown");
        var settingsFile = Path.Combine(settingsDir, "settings.json");
        logger.LogInformation("Saving settings file: {SettingsFilePath}",
LogFormatter.ToGreen(settingsFile));
        Directory.CreateDirectory(settingsDir);
        var tempPath = Path.Combine(
            settingsDir,
            $"{Path.GetFileName(fullPath)}.{Guid.NewGuid():N}.tmp"
        );
        try
        {
```

```
            var json = JsonSerializer.Serialize(
                layout,
                options: JsonSerializerOptions
            );
            await File.WriteAllTextAsync(tempPath, json, Encoding.UTF8, ct);
            // Atomic replace for data safety
            if (File.Exists(settingsFile))
            {
                File.Replace(tempPath, settingsFile,
destinationBackupFileName: null, ignoreMetadataErrors: true);
            }
            else
            {
                File.Move(tempPath, settingsFile, overwrite: true);
            }
            logger.LogInformation("Layout saved: {LayoutPath}",
LogFormatter.ToGreen(settingsFile));
        }
        catch (Exception ex)
        {
            logger.LogError("Error saving layout: {Error}",
LogFormatter.ToBrightRed(ex.Message));
            // Clean up temporary file on failure
            if (File.Exists(tempPath))
                File.Delete(tempPath);
        }
    }
    /// <summary>
    /// Load workspace settings from a JSON file in the workspace directory.
    /// </summary>
    public async Task<PanelLayout?> LoadWorkspaceSettingsAsync(string cwd,
CancellationToken ct = default)
    {
        if (string.IsNullOrWhiteSpace(cwd))
        {
            logger.LogError("Invalid layout path: {LayoutPath}",
LogFormatter.ToBrightRed(cwd));
            return null;
        }
        // Normalize and validate path
        var fullPath = Path.GetFullPath(cwd);
        var settingsFile = Path.Join(fullPath, ".LiveMarkdown",
"settings.json");
        if (!File.Exists(settingsFile))
        {
            logger.LogWarning("Layout file does not exist: {LayoutPath}",
LogFormatter.ToBrightRed(settingsFile));
            return null;
```

```
661         }
662         try
663         {
664             await using var stream = File.OpenRead(settingsFile);
665             var layout =
666                 await JsonSerializer.DeserializeAsync<PanelLayout>(stream,
    options: JsonSerializerOptions,
667                     cancellationToken: ct);
668             logger.LogInformation("Layout loaded: {LayoutPath}",
    LogFormatter.ToGreen(settingsFile));
669             return layout;
670         }
671         catch (Exception ex)
672         {
673             logger.LogError(ex, "Error loading layout: {LayoutPath}",
    LogFormatter.ToBrightRed(fullPath));
674             return null;
675         }
676     }
677 }
```

FileWatcherService.cs

```
1  using System.Collections.Concurrent;
2  namespace kernel.Services;
3  /// <summary>
4  /// Service for watching file system changes and notifying clients.
5  /// Detects create, delete, and rename operations for incremental directory
   updates.
6  /// </summary>
7  public class FileWatcherService(ILogger<FileWatcherService> logger) :
   IDisposable
8  {
9      private readonly ConcurrentDictionary<string, FileSystemWatcher>
   _watchers = new();
10     private readonly ConcurrentDictionary<string, int> _watcherRefCounts =
   new();
11     private readonly ConcurrentDictionary<string, CancellationTokenSource>
   _directoryDebounceTokens = new();
12     private readonly Lock _watcherLock = new();
13     private const int DirectoryDebounceMs = 200;
14     private volatile bool _disposed;
15     /// <summary>
16     /// Event raised when a directory is marked dirty (debounced). Only the
   dirty directory path is provided.
17     /// </summary>
18     public event Action<string, string>? DirectoryDirty;
19     /// <summary>
20     /// Start watching a directory for file system changes.
```

```
21      /// </summary>
22      /// <param name="directoryPath">The directory to watch.</param>
23      /// <returns>True if watching started successfully, false
    otherwise.</returns>
24      public bool StartWatching(string directoryPath)
25      {
26          if (string.IsNullOrWhiteSpace(directoryPath))
27          {
28              logger.LogWarning("Invalid directory path for watching");
29              return false;
30          }
31          var fullPath = Path.GetFullPath(directoryPath);
32          if (!Directory.Exists(fullPath))
33          {
34              logger.LogWarning("Directory does not exist: {Path}", fullPath);
35              return false;
36          }
37          lock (_watcherLock)
38          {
39              // Keep watcher creation/removal and ref-count updates in the
    same critical section
40              // to prevent races that could leak watchers or drive ref counts
    negative.
41              if (_disposed)
42              {
43                  logger.LogWarning("Attempted to start watching after
    disposal: {Path}", fullPath);
44                  return false;
45              }
46              if (_watchers.ContainsKey(fullPath))
47              {
48                  _watcherRefCounts[fullPath] =
    _watcherRefCounts.TryGetValue(fullPath, out var count) ? count + 1 : 1;
49                  logger.LogDebug("Already watching directory: {Path} (ref
    count incremented)", fullPath);
50                  return true;
51              }
52              try
53              {
54                  var watcher = new FileSystemWatcher(fullPath)
55                  {
56                      NotifyFilter = NotifyFilters.FileName
57                                   | NotifyFilters.DirectoryName
58                                   | NotifyFilters.CreationTime,
59                      IncludeSubdirectories = true,
60                      EnableRaisingEvents = false
61                  };
62                  watcher.Created += OnCreated;
```

```
                watcher.Deleted += OnDeleted;
                watcher.Renamed += OnRenamed;
                watcher.Error += OnError;
                if (_watchers.TryAdd(fullPath, watcher))
                {
                    _watcherRefCounts[fullPath] = 1;
                    watcher.EnableRaisingEvents = true;
                    logger.LogInformation("Started watching directory: {Path}", fullPath);
                    return true;
                }
                watcher.Dispose();
                return false;
            }
            catch (Exception ex)
            {
                logger.LogError(ex, "Failed to start watching directory: {Path}", fullPath);
                return false;
            }
        }
    }
    /// <summary>
    /// Stop watching a directory.
    /// </summary>
    /// <param name="directoryPath">The directory to stop watching.</param>
    public void StopWatching(string directoryPath)
    {
        if (string.IsNullOrWhiteSpace(directoryPath)) return;
        var fullPath = Path.GetFullPath(directoryPath);
        lock (_watcherLock)
        {
            // Serialize ref-count decrement and watcher removal to avoid disposing a watcher
            // that another thread just incremented.
            if (_disposed) return;
            if (_watchers.TryGetValue(fullPath, out _))
            {
                var remaining = _watcherRefCounts.TryGetValue(fullPath, out var count) ? Math.Max(count - 1, 0) : 0;
                if (remaining > 0)
                {
                    _watcherRefCounts[fullPath] = remaining;
                    return;
                }
                _watcherRefCounts.TryRemove(fullPath, out _);
                if (_watchers.TryRemove(fullPath, out var watcher))
                {
```

```
                    watcher.EnableRaisingEvents = false;
                    watcher.Created -= OnCreated;
                    watcher.Deleted -= OnDeleted;
                    watcher.Renamed -= OnRenamed;
                    watcher.Error -= OnError;
                    watcher.Dispose();
                    logger.LogInformation("Stopped watching directory:
{Path}", fullPath);
                }
            }
        }
    }
    /// <summary>
    /// Check if a directory is being watched.
    /// </summary>
    public bool IsWatching(string directoryPath)
    {
        if (string.IsNullOrWhiteSpace(directoryPath)) return false;
        if (_disposed) return false;
        var fullPath = Path.GetFullPath(directoryPath);
        return _watchers.ContainsKey(fullPath);
    }
    private void OnCreated(object sender, FileSystemEventArgs e)
    {
        try
        {
            var parentPath = Path.GetDirectoryName(e.FullPath) ??
string.Empty;
            var rootPath = GetWatcherRootPath(sender);

            MarkDirectoryDirty(rootPath, parentPath);
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Error handling created event for: {Path}",
e.FullPath);
        }
    }
    private void OnDeleted(object sender, FileSystemEventArgs e)
    {
        try
        {
            var parentPath = Path.GetDirectoryName(e.FullPath) ??
string.Empty;
            var rootPath = GetWatcherRootPath(sender);

            MarkDirectoryDirty(rootPath, parentPath);
        }
```

```
151         catch (Exception ex)
152         {
153             logger.LogError(ex, "Error handling deleted event for: {Path}",
    e.FullPath);
154         }
155     }
156     private void OnRenamed(object sender, RenamedEventArgs e)
157     {
158         try
159         {
160             var parentPath = Path.GetDirectoryName(e.FullPath) ??
    string.Empty;
161             var rootPath = GetWatcherRootPath(sender);
162 
163             MarkDirectoryDirty(rootPath, parentPath);
164             var oldParent = Path.GetDirectoryName(e.OldFullPath) ??
    string.Empty;
165             if (!string.Equals(oldParent, parentPath,
    StringComparison.OrdinalIgnoreCase))
166             {
167                 MarkDirectoryDirty(rootPath, oldParent);
168             }
169         }
170         catch (Exception ex)
171         {
172             logger.LogError(ex, "Error handling renamed event for: {Path}",
    e.FullPath);
173         }
174     }
175     private void OnError(object sender, ErrorEventArgs e)
176     {
177         var exception = e.GetException();
178         logger.LogError(exception, "FileSystemWatcher error");
179     }
180     private static string GetWatcherRootPath(object sender)
181     {
182         if (sender is FileSystemWatcher watcher)
183         {
184             return watcher.Path;
185         }
186         return string.Empty;
187     }
188     private void MarkDirectoryDirty(string rootPath, string? directoryPath)
189     {
190         if (string.IsNullOrWhiteSpace(directoryPath)) return;
191         if (_disposed) return;
192         var key = $"{rootPath}|{directoryPath}";
193         var cts = new CancellationTokenSource();
```

```
        // Atomically replace any existing token for this directory so only the newest
        // change can fire (older tokens are canceled / disposed immediately).
        _directoryDebounceTokens.AddOrUpdate(
            key,
            cts,
            (_, existingCts) =>
            {
                existingCts.Cancel();
                existingCts.Dispose();
                return cts;
            });
        _ = Task.Run(async () =>
        {
            try
            {
                await Task.Delay(DirectoryDebounceMs, cts.Token);
                if (!cts.Token.IsCancellationRequested && !_disposed)
                {
                    // Only the active token is allowed to remove itself and notify, which
                    // prevents a race where a stale token fires after a newer change.
                    if (_directoryDebounceTokens.TryRemove(new KeyValuePair<string, CancellationTokenSource>(key, cts)))
                    {
                        DirectoryDirty?.Invoke(rootPath, directoryPath);
                        logger.LogDebug("Directory dirty (debounced): {Dir}", directoryPath);
                    }
                }
            }
            catch (OperationCanceledException)
            {
                // ignore
            }
            finally
            {
                cts.Dispose();
            }
        }, cts.Token);
    }
    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }
```

```
236     private void Dispose(bool disposing)
237     {
238         lock (_watcherLock)
239         {
240             if (_disposed) return;
241             if (disposing)
242             {
243                 // Mark disposed before canceling tokens so event threads
    stop scheduling work.
244                 _disposed = true;
245                 foreach (var (_, cts) in _directoryDebounceTokens)
246                 {
247                     cts.Cancel();
248                     cts.Dispose();
249                 }
250                 _directoryDebounceTokens.Clear();
251                 foreach (var (path, watcher) in _watchers)
252                 {
253                     try
254                     {
255                         watcher.EnableRaisingEvents = false;
256                         watcher.Dispose();
257                         logger.LogDebug("Disposed watcher for: {Path}",
    path);
258                     }
259                     catch (Exception ex)
260                     {
261                         logger.LogError(ex, "Error disposing watcher for:
    {Path}", path);
262                     }
263                 }
264                 _watchers.Clear();
265                 _watcherRefCounts.Clear();
266             }
267             _disposed = true;
268         }
269     }
270 }
```

PythonVenvRunner.cs

```
1  using System.Collections.Concurrent;
2  using System.Diagnostics;
3  using System.Text;
4  using kernel.Hubs;
5  using kernel.Utils;
6  using Microsoft.AspNetCore.SignalR;
7  using Pty.Net;
8  namespace kernel.Services;
```

```
/// <summary>
/// Service for executing Python code in a virtual environment.
/// Responsibilities:
/// - Ensure a virtual environment exists (or create one)
/// - Write user code to a temporary script and run it using the venv python executable
/// - Stream stdout/stderr back to the caller via SignalR
/// - Track the spawned PTY so it can be resized, written to, or killed
/// </summary>
public class PythonVenvRunner(ILogger<PythonVenvRunner> logger, IHubContext<PythonHub> hubContext)
{
private readonly ConcurrentDictionary<string, IPtyConnection> _terminals = new();
private readonly ConcurrentDictionary<string, CancellationTokenSource> _cts = new();
/// <summary>
/// Ensure a virtual environment exists at the specified path.
/// Creates one if it doesn't exist.
/// </summary>
public async Task CreateVenvAsync(string venvPath, string systemPythonPath)
{
// Check if venv already exists
if (Directory.Exists(venvPath) && File.Exists(Path.Combine(venvPath, "pyvenv.cfg")))
{
logger.LogInformation("Virtual environment already exists at {VenvPath}", LogFormatter.ToGreen(venvPath));
return;
}
logger.LogInformation("Creating virtual environment at {VenvPath} using system python: {SystemPython}",
LogFormatter.ToGreen(venvPath), LogFormatter.ToYellow(systemPythonPath));
var startInfo = new ProcessStartInfo
{
FileName = systemPythonPath,
Arguments = $"-m venv \"{venvPath}\"",
RedirectStandardOutput = true,
RedirectStandardError = true,
UseShellExecute = false,
CreateNoWindow = true
};
using var process = Process.Start(startInfo);
if (process == null)
{
logger.LogError("Failed to start process to create virtual environment.");
throw new Exception("Failed to start process to create virtual environment.");
}
```

```
50  await process.WaitForExitAsync();
51  if (process.ExitCode != 0)
52  {
53  var error = await process.StandardError.ReadToEndAsync();
54  logger.LogError("Failed to create virtual environment: {Error}",
    LogFormatter.ToBrightRed(error));
55  throw new Exception($"Failed to create virtual environment: {error}");
56  }
57  logger.LogInformation("Virtual environment created successfully at
    {VenvPath}", LogFormatter.ToGreen(venvPath));
58  }
59  /// <summary>
60  /// Execute Python code in a virtual environment.
61  /// </summary>
62  public async Task ExecuteCodeAsync(string connectionId, string terminalId,
    string userCode, string systemPythonPath,
63  string venvPath)
64  {
65  KillTerminal(terminalId);
66  await CreateVenvAsync(venvPath, systemPythonPath);
67  var venvPythonExe = GetVenvPythonExecutable(venvPath);
68  // Create temporary script file
69  var tempScriptPath = Path.Combine(Path.GetTempPath(),
    $"kernel_script_{Guid.NewGuid()}.py");
70  await File.WriteAllTextAsync(tempScriptPath, userCode, Encoding.UTF8);
71  logger.LogInformation(
72  "Executing Python code in terminal {TerminalId} (connection {ConnectionId}).
    Script: {ScriptPath}",
73  LogFormatter.ToCyan(terminalId), LogFormatter.ToCyan(connectionId),
    LogFormatter.ToGreen(tempScriptPath));
74  try
75  {
76  await RunProcessAsync(connectionId, terminalId, venvPythonExe,
    tempScriptPath);
77  }
78  catch (Exception ex)
79  {
80  logger.LogError(ex, "Error executing code for connection {ConnectionId},
    terminal {TerminalId}",
81  LogFormatter.ToBrightRed(connectionId),
    LogFormatter.ToBrightRed(terminalId));
82  await hubContext.Clients.Client(connectionId).SendAsync("ReceiveResult",
    $"[Exception]: {ex.Message}");
83  }
84  }
85  /// <summary>
86  /// Get the path to the Python executable in the virtual environment.
87  /// </summary>
```

```
88  private static string GetVenvPythonExecutable(string venvPath)
89  {
90  // Windows and Unix venv layout differs
91  var winPath = Path.Combine(venvPath, "Scripts", "python.exe");
92  var unixPath = Path.Combine(venvPath, "bin", "python");
93  return OperatingSystem.IsWindows() ? winPath : unixPath;
94  }
95  /// <summary>
96  /// Run a Python script and stream output back to the client.
97  /// </summary>
98  private async Task RunProcessAsync(string connectionId, string terminalId,
    string venvPythonPath, string scriptPath)
99  {
100 logger.LogInformation("Spawning venv python: {PythonExe} {Script}",
101 LogFormatter.ToYellow(venvPythonPath),
102 LogFormatter.ToGreen(scriptPath));
103 var options = new PtyOptions
104 {
105 App = venvPythonPath,
106 Name = "xterm-color",
107 Rows = 30,
108 Cols = 80,
109 CommandLine = [scriptPath],
110 Cwd             =             Path.GetDirectoryName(scriptPath)             ??
    Environment.GetFolderPath(Environment.SpecialFolder.UserProfile),
111 Environment = new Dictionary<string, string>
112 {
113 { "PYTHONUNBUFFERED", "1" }
114 }
115 };
116 using (logger.BeginScope(new Dictionary<string, object>
117 { ["TerminalId"] = terminalId, ["ConnectionId"] = connectionId }))
118 {
119 try
120 {
121 var terminal = await PtyProvider.SpawnAsync(options, CancellationToken.None);
122 _terminals[terminalId] = terminal;
123 var cts = new CancellationTokenSource();
124 _cts[terminalId] = cts;
125 logger.LogInformation("Spawned     PTY     for     terminal     {TerminalId}",
    LogFormatter.ToCyan(terminalId));
126 var processExitedTcs = new TaskCompletionSource<int>();
127 terminal.ProcessExited += (_, exitCode) =>
128 {
129 logger.LogInformation("Process exited for terminal {TerminalId} with exit code
    {ExitCode}",
130 LogFormatter.ToCyan(terminalId), exitCode);
131 processExitedTcs.TrySetResult(exitCode.ExitCode);
```

```
};
// Start background task to read output
_ = Task.Run(async () =>
{
var buffer = new byte[4096];
await using var stream = terminal.ReaderStream;
try
{
while (!cts.Token.IsCancellationRequested)
{
var bytesRead = await stream.ReadAsync(buffer, cts.Token);
if (bytesRead == 0) break;
var output = Encoding.UTF8.GetString(buffer, 0, bytesRead);
logger.LogDebug("[Python {TerminalId}] Output: {Output}",
LogFormatter.ToCyan(terminalId),
LogFormatter.ToGreen(output));
await hubContext.Clients.Client(connectionId)
.SendAsync("CodeOutput", terminalId, output, cancellationToken: cts.Token);
}
}
catch (OperationCanceledException)
{
logger.LogDebug("Read loop cancelled for terminal {TerminalId}",
LogFormatter.ToCyan(terminalId));
}
catch (Exception ex)
{
logger.LogError(ex, "Unexpected error reading PTY for terminal {TerminalId}",
LogFormatter.ToBrightRed(terminalId));
}
// Wait for process to actually exit before cleanup
try
{
using var timeoutCts = new CancellationTokenSource(TimeSpan.FromSeconds(5));
using var linkedCts = CancellationTokenSource.CreateLinkedTokenSource(cts.Token,
timeoutCts.Token);
await processExitedTcs.Task.WaitAsync(linkedCts.Token);
}
catch (OperationCanceledException)
{
logger.LogDebug("Timed out waiting for process exit event for terminal
{TerminalId}",
LogFormatter.ToCyan(terminalId));
}
logger.LogInformation("PTY read loop ending for terminal {TerminalId},
cleaning up",
LogFormatter.ToCyan(terminalId));
```

```
// Clean-up the temporary script file
if (File.Exists(scriptPath))
{
try
{
File.Delete(scriptPath);
logger.LogDebug("Deleted temporary script: {ScriptPath}",
LogFormatter.ToGreen(scriptPath));
}
catch (Exception ex)
{
logger.LogWarning("Failed to delete temporary script {ScriptPath}: {Error}",
LogFormatter.ToGreen(scriptPath), LogFormatter.ToBrightRed(ex.Message));
}
}
// Notify frontend that execution is finished
await hubContext.Clients.Client(connectionId)
.SendAsync("CodeExecutionCompleted",     terminalId,     cancellationToken:
CancellationToken.None);
if   (_terminals.TryGetValue(terminalId,   out   var   currentTerminal)   &&
currentTerminal == terminal)
{
KillTerminal(terminalId);
}
else
{
// Not the owner (a new process took over), just dispose local resources
terminal.Dispose();
cts.Dispose();
}
}, cts.Token);
}
catch (Exception ex)
{
logger.LogError(ex, "Failed to spawn PTY for terminal {TerminalId}",
LogFormatter.ToBrightRed(terminalId));
throw;
}
}
}
/// <summary>
/// Write input to a running Python process.
/// </summary>
public async Task WriteInputAsync(string terminalId, string input)
{
if (_terminals.TryGetValue(terminalId, out var terminal))
{
```

```
logger.LogDebug("Writing input to python terminal {TerminalId}: {InputPreview}",
LogFormatter.ToCyan(terminalId),
LogFormatter.ToYellow(input.Length > 64 ? input[..64] + "..." : input));
var data = Encoding.UTF8.GetBytes(input);
await terminal.WriterStream.WriteAsync(data, 0, data.Length);
await terminal.WriterStream.FlushAsync();
}
}
/// <summary>
/// Resize a Python terminal.
/// </summary>
public void Resize(string terminalId, int cols, int rows)
{
if (_terminals.TryGetValue(terminalId, out var terminal))
{
logger.LogInformation("Resizing python PTY {TerminalId} to {Cols}x{Rows}",
LogFormatter.ToCyan(terminalId),
LogFormatter.ToYellow(cols), LogFormatter.ToYellow(rows));
terminal.Resize(cols, rows);
}
}
/// <summary>
/// Kill a Python terminal and clean up resources.
/// </summary>
public void KillTerminal(string terminalId)
{
if (_terminals.TryRemove(terminalId, out var terminal))
{
logger.LogInformation("Disposing python PTY {TerminalId}",
LogFormatter.ToCyan(terminalId));
terminal.Dispose();
}
if (_cts.TryRemove(terminalId, out var cts))
{
logger.LogInformation("Cancelling python PTY read loop for {TerminalId}",
LogFormatter.ToCyan(terminalId));
cts.Cancel();
cts.Dispose();
}
}
}
```

TerminalService.cs

```
using System.Collections.Concurrent;
using System.Runtime.InteropServices;
using System.Text;
using kernel.Hubs;
```

```
using kernel.Utils;
using Microsoft.AspNetCore.SignalR;
using Pty.Net;
namespace kernel.Services;
/// <summary>
/// Service for managing pseudo-terminal (PTY) sessions for connected clients.
/// Responsibilities:
/// - Spawn terminal processes (PowerShell/bash) using PtyProvider
/// - Relay output from the PTY to a SignalR client output route
/// - Accept input and control commands (resize, kill)
/// - Track active terminals and cancellation tokens per connection
/// </summary>
public class TerminalService(ILogger<TerminalService> logger, IHubContext<TerminalHub> hubContext)
{
    private readonly ConcurrentDictionary<string, IPtyConnection> _terminals = new();
    private readonly ConcurrentDictionary<string, CancellationTokenSource> _cts = new();
    /// <summary>
    /// Start a terminal session for a client.
    /// </summary>
    /// <param name="terminalId">Terminal instance ID</param>
    /// <param name="connectionId">SignalR connection ID</param>
    /// <param name="outputRoute">SignalR client method name to invoke with output</param>
    /// <param name="cwd">Working directory for the terminal</param>
    /// <param name="commandLine">Optional arguments passed to the shell executable</param>
    public async Task StartTerminalAsync(string terminalId, string connectionId, string outputRoute, string cwd,
        string[]? commandLine = null)
    {
        var shell = RuntimeInformation.IsOSPlatform(OSPlatform.Windows) ? "powershell.exe" : "bash";
        var options = new PtyOptions
        {
            App = shell,
            Name = "xterm-color",
            Rows = 50,
            Cols = 80,
            CommandLine = commandLine ?? [],
            Cwd = cwd,
            Environment = new Dictionary<string, string>()
        };
        using (logger.BeginScope(new Dictionary<string, object> { ["TerminalId"] = terminalId }))
```

```
        {
            logger.LogInformation("Starting terminal for connection {TerminalId} (shell={Shell}, cwd={Cwd})",
                LogFormatter.ToCyan(terminalId), LogFormatter.ToYellow(shell), LogFormatter.ToGreen(options.Cwd));
            try
            {
                var terminal = await PtyProvider.SpawnAsync(options, CancellationToken.None);
                _terminals.TryAdd(terminalId, terminal);
                var cts = new CancellationTokenSource();
                _cts.TryAdd(terminalId, cts);
                // Start background task to read terminal output and relay to client
                _ = Task.Run(async () =>
                {
                    var buffer = new byte[4096];
                    await using var stream = terminal.ReaderStream;
                    try
                    {
                        while (!cts.Token.IsCancellationRequested)
                        {
                            var bytesRead = await stream.ReadAsync(buffer, cts.Token);
                            if (bytesRead == 0) break;
                            var output = Encoding.UTF8.GetString(buffer, 0, bytesRead);
                            logger.LogDebug(
                                "[Terminal {TerminalId}, Output Route {OutputRoute}] Output: {Output}",
                                LogFormatter.ToCyan(terminalId), LogFormatter.ToMagenta(outputRoute),
                                LogFormatter.ToGreen(output));
                            await hubContext.Clients.Client(connectionId)
                                .SendAsync(outputRoute, output, cancellationToken: cts.Token);
                        }
                    }
                    catch (OperationCanceledException)
                    {
                        logger.LogDebug("Read loop cancelled for connection {TerminalId}",
                            LogFormatter.ToCyan(terminalId));
                    }
                    catch (Exception ex)
                    {
                        logger.LogError(ex, "Unexpected error reading from PTY for connection {TerminalId}",
```

```
                            LogFormatter.ToBrightRed(terminalId));
                    }
                    finally
                    {
                        logger.LogInformation(
                            "Terminal read loop ended for connection {TerminalId}, performing cleanup",
                            LogFormatter.ToCyan(terminalId));
                        KillTerminal(terminalId);
                    }
                }, cts.Token);
            }
            catch (Exception ex)
            {
                logger.LogError(ex, "Failed to spawn terminal for connection {TerminalId}",
                    LogFormatter.ToBrightRed(terminalId));
                throw;
            }
        }
    }
    /// <summary>
    /// Write input to a terminal.
    /// </summary>
    public async Task WriteInputAsync(string terminalId, string input)
    {
        if (_terminals.TryGetValue(terminalId, out var terminal))
        {
            logger.LogDebug("Writing input to terminal {TerminalId}: {InputPreview}",
                LogFormatter.ToCyan(terminalId),
                LogFormatter.ToYellow(input.Length > 32 ? input[..32] + "..." : input));
            var data = Encoding.UTF8.GetBytes(input);
            await terminal.WriterStream.WriteAsync(data);
            await terminal.WriterStream.FlushAsync();
        }
        else
        {
            logger.LogWarning("Attempted to write to unknown terminal {TerminalId}",
                LogFormatter.ToBrightRed(terminalId));
        }
    }
    /// <summary>
    /// Resize a terminal (columns x rows).
    /// </summary>
    public void Resize(string terminalId, int cols, int rows)
```

```
124     {
125         if (_terminals.TryGetValue(terminalId, out var terminal))
126         {
127             logger.LogInformation("Resizing terminal {ConnectionId} to
    {Cols}x{Rows}",
128                 LogFormatter.ToCyan(terminalId),
    LogFormatter.ToYellow(cols), LogFormatter.ToYellow(rows));
129             terminal.Resize(cols, rows);
130         }
131         else
132         {
133             logger.LogWarning("Resize requested for unknown terminal
    {ConnectionId}",
134                 LogFormatter.ToBrightRed(terminalId));
135         }
136     }
137     /// <summary>
138     /// Kill a terminal and clean up resources.
139     /// </summary>
140     public void KillTerminal(string terminalId)
141     {
142         try
143         {
144             if (_terminals.TryRemove(terminalId, out var terminal))
145             {
146                 logger.LogInformation("Disposing terminal for connection
    {ConnectionId}",
147                     LogFormatter.ToCyan(terminalId));
148                 terminal.Dispose();
149             }
150             if (_cts.TryRemove(terminalId, out var cts))
151             {
152                 logger.LogInformation("Cancelling read loop for connection
    {ConnectionId}",
153                     LogFormatter.ToCyan(terminalId));
154                 cts.Cancel();
155                 cts.Dispose();
156             }
157         }
158         catch (Exception ex)
159         {
160             logger.LogError(ex, "Error while killing terminal
    {TerminalId}",terminalId);
161         }
162     }
163 }
```

LogFormatter.cs

```cs
using JetBrains.Annotations;
namespace kernel.Utils;
/// <summary>
/// Provides ANSI color formatting for console log output.
/// Used to colorize log messages for better readability.
/// </summary>
public static class LogFormatter
{
    // ANSI escape codes for colors
    private const string Reset = "\e[0m";
    private const string Red = "\e[31m";
    private const string Green = "\e[32m";
    private const string Yellow = "\e[33m";
    private const string Blue = "\e[34m";
    private const string Magenta = "\e[35m";
    private const string Cyan = "\e[36m";
    private const string White = "\e[37m";
    private const string BrightRed = "\e[91m";
    private const string BrightGreen = "\e[92m";
    private const string BrightYellow = "\e[93m";
    private const string BrightBlue = "\e[94m";
    private const string BrightMagenta = "\e[95m";
    private const string BrightCyan = "\e[96m";
    /// <summary>Format value with white color.</summary>
    [UsedImplicitly]
    public static string ToWhite(object value) => $"{White}{value}{Reset}";
    /// <summary>Format value with red color.</summary>
    [UsedImplicitly]
    public static string ToRed(object value) => $"{Red}{value}{Reset}";
    /// <summary>Format value with green color.</summary>
    [UsedImplicitly]
    public static string ToGreen(object value) => $"{Green}{value}{Reset}";
    /// <summary>Format value with yellow color.</summary>
    [UsedImplicitly]
    public static string ToYellow(object value) =>
$"{Yellow}{value}{Reset}";
    /// <summary>Format value with blue color.</summary>
    [UsedImplicitly]
    public static string ToBlue(object value) => $"{Blue}{value}{Reset}";
    /// <summary>Format value with magenta color.</summary>
    [UsedImplicitly]
    public static string ToMagenta(object value) =>
$"{Magenta}{value}{Reset}";
    /// <summary>Format value with cyan color.</summary>
    [UsedImplicitly]
    public static string ToCyan(object value) => $"{Cyan}{value}{Reset}";
```

```
    /// <summary>Format value with bright red color (for errors).</summary>
    [UsedImplicitly]
    public static string ToBrightRed(object value) =>
$"{BrightRed}{value}{Reset}";
    /// <summary>Format value with bright green color.</summary>
    [UsedImplicitly]
    public static string ToBrightGreen(object value) =>
$"{BrightGreen}{value}{Reset}";
    /// <summary>Format value with bright yellow color.</summary>
    [UsedImplicitly]
    public static string ToBrightYellow(object value) =>
$"{BrightYellow}{value}{Reset}";
    /// <summary>Format value with bright blue color.</summary>
    [UsedImplicitly]
    public static string ToBrightBlue(object value) =>
$"{BrightBlue}{value}{Reset}";
    /// <summary>Format value with bright magenta color.</summary>
    [UsedImplicitly]
    public static string ToBrightMagenta(object value) =>
$"{BrightMagenta}{value}{Reset}";
    /// <summary>Format value with bright cyan color.</summary>
    [UsedImplicitly]
    public static string ToBrightCyan(object value) =>
$"{BrightCyan}{value}{Reset}";
}
```

appsettings.Development.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Debug",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

kernel.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
    <PropertyGroup>
        <TargetFramework>net9.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
        <PlatformTarget>x64</PlatformTarget>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="JetBrains.Annotations" Version="2025.2.4" />
        <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="9.0.5" />
        <PackageReference Include="Microsoft.AspNetCore.SignalR" Version="1.2.0" />
        <PackageReference Include="Microsoft.CodeAnalysis.CSharp.Scripting" Version="4.14.0" />
        <PackageReference Include="Microsoft.PowerShell.SDK" Version="7.5.3" />
        <PackageReference Include="Pty.Net" Version="0.3.94-pre" />
        <PackageReference Include="System.Management.Automation" Version="7.5.3" />
        <PackageReference Include="UTF.Unknown" Version="2.6.0" />
    </ItemGroup>
    <Target Name="CopyPtyNativeFilesToPublishDir" AfterTargets="Publish">
        <ItemGroup>
            <PtyNativeFilesOs64 Include="$(OutputPath)os64\**\*.*" />
            <PtyNativeFilesOs86 Include="$(OutputPath)os86\**\*.*" />
        </ItemGroup>
        <MakeDir Directories="$(PublishDir)os64;$(PublishDir)os86" />
        <Copy SourceFiles="@(PtyNativeFilesOs64)" DestinationFiles="@(PtyNativeFilesOs64->'$(PublishDir)os64\%(RecursiveDir)%(Filename)%(Extension)')" />
        <Copy SourceFiles="@(PtyNativeFilesOs86)" DestinationFiles="@(PtyNativeFilesOs86->'$(PublishDir)os86\%(RecursiveDir)%(Filename)%(Extension)')" />
    </Target>
</Project>
```

Program.cs

```
using System.Text.Json;
using System.Text.Json.Serialization;
using kernel.Hubs;
using kernel.Services;
using kernel.Utils;
using Microsoft.AspNetCore.Hosting.Server;
using Microsoft.AspNetCore.Hosting.Server.Features;
```

```
// Application entry point and configuration.
// - Configures minimal web app, CORS and SignalR hubs
// - Sets up console logging with timestamps and colors
// - Maps SignalR hub endpoints
var builder = WebApplication.CreateBuilder(args);
// If no URL is configured, bind Kestrel to an ephemeral local port.
var urlsConfigured = builder.Configuration["urls"] ??
builder.Configuration["ASPNETCORE_URLS"];
if (string.IsNullOrWhiteSpace(urlsConfigured))
{
    builder.WebHost.ConfigureKestrel(options =>
    {
        options.Listen(System.Net.IPAddress.Loopback, 0);
    });
}
// Controllers JSON (only affects MVC)
builder.Services
    .AddControllers()
    .AddJsonOptions(o =>
    {
        o.JsonSerializerOptions.PropertyNamingPolicy =
JsonNamingPolicy.CamelCase;
        o.JsonSerializerOptions.DictionaryKeyPolicy =
JsonNamingPolicy.CamelCase;
        o.JsonSerializerOptions.Converters.Add(new
JsonStringEnumConverter(JsonNamingPolicy.CamelCase));
    });
// SignalR JSON (affects hub payloads)
builder.Services
    .AddSignalR()
    .AddJsonProtocol(o =>
    {
        o.PayloadSerializerOptions.PropertyNamingPolicy =
JsonNamingPolicy.CamelCase;
        o.PayloadSerializerOptions.DictionaryKeyPolicy =
JsonNamingPolicy.CamelCase;
        o.PayloadSerializerOptions.Converters.Add(new
JsonStringEnumConverter(JsonNamingPolicy.CamelCase));
    });
// Configure console logging with timestamps and colors
builder.Logging.ClearProviders();
builder.Logging.AddSimpleConsole(options =>
{
    options.SingleLine = false;
    options.TimestampFormat = "[HH:mm:ss] ";
    options.ColorBehavior =
Microsoft.Extensions.Logging.Console.LoggerColorBehavior.Enabled;
    options.IncludeScopes = true;
```

```
});
builder.Logging.SetMinimumLevel(LogLevel.Information);
// Configure CORS for frontend development
builder.Services.AddCors(options =>
{
    options.AddDefaultPolicy(policy =>
    {
        policy.SetIsOriginAllowed(origin =>
            {
                if (string.IsNullOrEmpty(origin))
                    return false;
                return Uri.TryCreate(origin, UriKind.Absolute, out var uri)
&& uri.IsLoopback;
            })
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials();
    });
});
// Register services
builder.Services.AddSingleton<TerminalService>();
builder.Services.AddSingleton<PythonVenvRunner>();
builder.Services.AddSingleton<FileService>();
builder.Services.AddSingleton<FileWatcherService>();
var app = builder.Build();
app.UseCors();
app.UseRouting();
// Map SignalR hubs
app.MapHub<PythonHub>("/pythonHub");
app.MapHub<TerminalHub>("/terminalHub");
app.MapHub<FileHub>("/fileHub");
// Run application with error handling
try
{
    await app.StartAsync();
    var server = app.Services.GetRequiredService<IServer>();
    var addresses =
server.Features.Get<IServerAddressesFeature>()!.Addresses;
    foreach (var address in addresses)
    {
        var port = new Uri(address).Port;
        Console.Out.WriteLine($"SIGNALR_PORT={port}");
        Console.Out.Flush();
        app.Logger.LogInformation("Listening on {Address}",
LogFormatter.ToGreen(address));
    }

```

```
    app.Logger.LogInformation("Starting kernel service (PID: {Pid})",
LogFormatter.ToYellow(Environment.ProcessId));
    app.Logger.LogInformation("Environment: {Env}",
LogFormatter.ToGreen(builder.Environment.EnvironmentName));

    app.Logger.LogInformation("Mapped hub: {Path} -> {Hub}",
LogFormatter.ToMagenta("/kernelHub"), LogFormatter.ToCyan("PythonHub"));
    app.Logger.LogInformation("Mapped hub: {Path} -> {Hub}",
LogFormatter.ToMagenta("/terminalHub"), LogFormatter.ToCyan("TerminalHub"));
    app.Logger.LogInformation("Mapped hub: {Path} -> {Hub}",
LogFormatter.ToMagenta("/fileHub"), LogFormatter.ToCyan("FileHub"));
    await app.WaitForShutdownAsync();
}
catch (Exception ex)
{
    app.Logger.LogCritical(ex, "Host terminated unexpectedly");
    throw;
}
```

FileServiceTests.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Threading.Channels;
using System.Threading.Tasks;
using kernel.Services;
using Microsoft.Extensions.Logging.Abstractions;
using Xunit;
namespace kernel.Tests.Services;
public class FileServiceTests
{
    private static FileService CreateService() =>
new(NullLogger<FileService>.Instance);
    private static FileService.PanelLayout CreateLayout(string activeFile)
    {
        return new FileService.PanelLayout(
            ExplorerWidth: 240,
            TerminalHeight: 160,
            EditorPreviewRatio: 0.5,
            PreferredViewMode: FileService.ViewMode.Split,
            OpenedFiles: ["README.md", activeFile],
            ExpandedDirectories: ["docs", "src"],
            ActiveFile: activeFile,
            ExplorerVisible: false,
            SearchVisible: true,
            TerminalVisible: true,
            ActiveTopPanel: FileService.SidebarPanel.Search,
```

```
            ActiveBottomPanel: FileService.SidebarPanel.Terminal
        );
    }
    [Fact]
    public async Task StreamTabAsync_ReturnsMetadataThenContent()
    {
        using var tempDir = new TempDirectory();
        var filePath = Path.Combine(tempDir.Path, "sample.txt");
        await File.WriteAllTextAsync(filePath, "Hello",
TestContext.Current.CancellationToken);
        var service = CreateService();
        var chunks = await
TestHelpers.ToListAsync(service.StreamTabAsync(filePath,
TestContext.Current.CancellationToken));
        Assert.True(chunks.Count >= 2);
        Assert.True(chunks[0].IsMetadata);
        var contentBuilder = new StringBuilder();
        for (var i = 1; i < chunks.Count; i++)
        {
            contentBuilder.Append(chunks[i].Content);
        }
        Assert.Equal("Hello", contentBuilder.ToString());
    }
    [Fact]
    public async Task StreamTabAsync_ReturnsErrorChunkWhenFileMissing()
    {
        using var tempDir = new TempDirectory();
        var filePath = Path.Combine(tempDir.Path, "missing.txt");
        var service = CreateService();
        var chunks = await
TestHelpers.ToListAsync(service.StreamTabAsync(filePath,
TestContext.Current.CancellationToken));
        Assert.True(chunks.Count >= 2);
        Assert.True(chunks[0].IsMetadata);
        Assert.True(chunks[1].IsError);
        Assert.False(string.IsNullOrWhiteSpace(chunks[1].Content));
    }
    [Fact]
    public async Task ReadDirAsync_ReturnsEntriesForExistingDirectory()
    {
        using var tempDir = new TempDirectory();
        var filePath = Path.Combine(tempDir.Path, "file.md");
        var subDir = Path.Combine(tempDir.Path, "nested");
        Directory.CreateDirectory(subDir);
        await File.WriteAllTextAsync(filePath, "data",
TestContext.Current.CancellationToken);
        var service = CreateService();
```

```
        var nodes = await
TestHelpers.ToListAsync(service.ReadDirAsync(tempDir.Path,
TestContext.Current.CancellationToken));
        Assert.Contains(nodes, node => node.Path == filePath
&& !node.IsDirectory);
        Assert.Contains(nodes, node => node.Path == subDir &&
node.IsDirectory);
    }
    [Fact]
    public async Task ReadDirAsync_ReturnsEntriesFromNestedDirectory()
    {
        using var tempDir = new TempDirectory();
        var subDir = Path.Combine(tempDir.Path, "nested");
        Directory.CreateDirectory(subDir);
        var childFile = Path.Combine(subDir, "child.txt");
        await File.WriteAllTextAsync(childFile, "child",
TestContext.Current.CancellationToken);
        var service = CreateService();
        var nodes = await
TestHelpers.ToListAsync(service.ReadDirAsync(tempDir.Path,
TestContext.Current.CancellationToken));
        Assert.Contains(nodes, node => node.Path == childFile
&& !node.IsDirectory);
    }
    [Fact]
    public void CreateDirectory_CreatesDirectory()
    {
        using var tempDir = new TempDirectory();
        var directoryPath = Path.Combine(tempDir.Path, "new-dir");
        var service = CreateService();
        service.CreateDirectory(directoryPath);
        Assert.True(Directory.Exists(directoryPath));
    }
    [Fact]
    public void CreateDirectory_HandlesWhitespacePath()
    {
        var service = CreateService();
        var exception = Record.Exception(() => service.CreateDirectory("
"));
        Assert.Null(exception);
    }
    [Fact]
    public void DeleteFile_RemovesExistingFile()
    {
        using var tempDir = new TempDirectory();
        var filePath = Path.Combine(tempDir.Path, "delete.txt");
        File.WriteAllText(filePath, "delete");
        var service = CreateService();
```

```
        service.DeleteFile(filePath);
        Assert.False(File.Exists(filePath));
    }
    [Fact]
    public async Task SaveFileAsync_OverwritesExistingFile()
    {
        using var tempDir = new TempDirectory();
        var filePath = Path.Combine(tempDir.Path, "save.txt");
        await File.WriteAllTextAsync(filePath, "old",
TestContext.Current.CancellationToken);
        var service = CreateService();
        await service.SaveFileAsync(filePath,
TestHelpers.CreateChannelReader("new"), ct:
TestContext.Current.CancellationToken);
        Assert.Equal("new", await File.ReadAllTextAsync(filePath,
TestContext.Current.CancellationToken));
    }
    [Fact]
    public async Task SaveFileAsync_HandlesWhitespacePath()
    {
        var service = CreateService();
        var exception = await Record.ExceptionAsync(() =>
            service.SaveFileAsync(" ",
TestHelpers.CreateChannelReader("data"), ct:
TestContext.Current.CancellationToken));
        Assert.Null(exception);
    }
    [Fact]
    public async Task SaveWorkspaceSettingsAsync_WritesSettingsFile()
    {
        using var tempDir = new TempDirectory();
        var service = CreateService();
        var layout = CreateLayout("active.md");
        await service.SaveWorkspaceSettingsAsync(tempDir.Path, layout,
TestContext.Current.CancellationToken);
        var settingsFile = Path.Combine(tempDir.Path, ".LiveMarkdown",
"settings.json");
        Assert.True(File.Exists(settingsFile));
    }
}
internal sealed class TempDirectory : IDisposable
{
    public TempDirectory()
    {
        Path = System.IO.Path.Combine(System.IO.Path.GetTempPath(),
Guid.NewGuid().ToString("N"));
        Directory.CreateDirectory(Path);
    }
```

```
    public string Path { get; }
    public void Dispose()
    {
        if (Directory.Exists(Path))
        {
            Directory.Delete(Path, recursive: true);
        }
    }
}
internal static class TestHelpers
{
    public static async Task<List<T>> ToListAsync<T>(IAsyncEnumerable<T>
source)
    {
        var list = new List<T>();
        await foreach (var item in source)
        {
            list.Add(item);
        }
        return list;
    }
    public static ChannelReader<string> CreateChannelReader(params string[]
chunks)
    {
        var channel = Channel.CreateUnbounded<string>();
        foreach (var chunk in chunks)
        {
            channel.Writer.TryWrite(chunk);
        }
        channel.Writer.TryComplete();
        return channel.Reader;
    }
}
```

FileWatcherServiceTests.cs

```
using System;
using System.Collections.Concurrent;
using System.IO;
using System.Linq;
using System.Threading.Tasks;
using kernel.Services;
using Microsoft.Extensions.Logging.Abstractions;
using Xunit;
namespace kernel.Tests.Services;
public class FileWatcherServiceTests
{
    private static FileWatcherService CreateService() =>
new(NullLogger<FileWatcherService>.Instance);
```

```
    [Fact]
    public void StartWatching_WhitespacePath_ReturnsFalse()
    {
        using var service = CreateService();
        Assert.False(service.StartWatching(" "));
    }
    [Fact]
    public void StartWatching_MissingDirectory_ReturnsFalse()
    {
        using var service = CreateService();
        var path = Path.Combine(Path.GetTempPath(), "missing_" +
Guid.NewGuid().ToString("N"));
        Assert.False(service.StartWatching(path));
    }
    [Fact]
    public void StartWatching_ExistingDirectory_ReturnsTrueAndIsWatching()
    {
        using var tempDir = new TempDirectory();
        using var service = CreateService();
        var started = service.StartWatching(tempDir.Path);
        Assert.True(started);
        Assert.True(service.IsWatching(tempDir.Path));
    }
    [Fact]
    public void StartWatching_SameDirectoryTwice_RequiresTwoStops()
    {
        using var tempDir = new TempDirectory();
        using var service = CreateService();
        Assert.True(service.StartWatching(tempDir.Path));
        Assert.True(service.StartWatching(tempDir.Path));
        service.StopWatching(tempDir.Path);
        Assert.True(service.IsWatching(tempDir.Path));
        service.StopWatching(tempDir.Path);
        Assert.False(service.IsWatching(tempDir.Path));
    }
    [Fact]
    public void StopWatching_UnknownDirectory_DoesNotThrow()
    {
        using var service = CreateService();
        var exception = Record.Exception(() =>
service.StopWatching(@"C:\definitely-not-watched"));
        Assert.Null(exception);
    }
    [Fact]
    public void Dispose_StopsAllWatchers()
    {
        using var tempDir = new TempDirectory();
        var service = CreateService();
```

```
59          Assert.True(service.StartWatching(tempDir.Path));
60          service.Dispose();
61          Assert.False(service.IsWatching(tempDir.Path));
62      }
63      [Fact]
64      public void StartWatching_AfterDispose_ReturnsFalse()
65      {
66          using var tempDir = new TempDirectory();
67          var service = CreateService();
68          service.Dispose();
69          Assert.False(service.StartWatching(tempDir.Path));
70      }
71      [Fact]
72      public async Task CreatedEvent_RaisesDirectoryDirtyForParent()
73      {
74          using var tempDir = new TempDirectory();
75          using var service = CreateService();
76          var events = new ConcurrentBag<(string Root, string Directory)>();
77          service.DirectoryDirty += (root, dir) => events.Add((root, dir));
78          Assert.True(service.StartWatching(tempDir.Path));
79          var filePath = Path.Combine(tempDir.Path, "created.txt");
80          await File.WriteAllTextAsync(filePath, "x",
    TestContext.Current.CancellationToken);
81          var fired = await WaitUntilAsync(() => events.Any(e => e.Root ==
    tempDir.Path && e.Directory == tempDir.Path));
82          Assert.True(fired);
83      }
84      [Fact]
85      public async Task DeletedEvent_RaisesDirectoryDirtyForParent()
86      {
87          using var tempDir = new TempDirectory();
88          using var service = CreateService();
89          var events = new ConcurrentBag<(string Root, string Directory)>();
90          service.DirectoryDirty += (root, dir) => events.Add((root, dir));
91          Assert.True(service.StartWatching(tempDir.Path));
92          var filePath = Path.Combine(tempDir.Path, "deleted.txt");
93          await File.WriteAllTextAsync(filePath, "x",
    TestContext.Current.CancellationToken);
94          await Task.Delay(300, TestContext.Current.CancellationToken);
95          File.Delete(filePath);
96          var fired = await WaitUntilAsync(() => events.Any(e => e.Root ==
    tempDir.Path && e.Directory == tempDir.Path));
97          Assert.True(fired);
98      }
99      [Fact]
100     public async Task
    RenamedAcrossDirectories_RaisesDirtyForOldAndNewParents()
101     {
```

```
        using var tempDir = new TempDirectory();
        using var service = CreateService();
        var oldDir = Path.Combine(tempDir.Path, "old");
        var newDir = Path.Combine(tempDir.Path, "new");
        Directory.CreateDirectory(oldDir);
        Directory.CreateDirectory(newDir);
        var oldFile = Path.Combine(oldDir, "move.txt");
        var newFile = Path.Combine(newDir, "move.txt");
        await File.WriteAllTextAsync(oldFile, "x",
TestContext.Current.CancellationToken);
        var events = new ConcurrentBag<(string Root, string Directory)>();
        service.DirectoryDirty += (root, dir) => events.Add((root, dir));
        Assert.True(service.StartWatching(tempDir.Path));
        await Task.Delay(300, TestContext.Current.CancellationToken);
        File.Move(oldFile, newFile);
        var fired = await WaitUntilAsync(() =>
            events.Any(e => e.Root == tempDir.Path && e.Directory == oldDir)
&&
            events.Any(e => e.Root == tempDir.Path && e.Directory ==
newDir));
        Assert.True(fired);
    }
    private static async Task<bool> WaitUntilAsync(Func<bool> condition, int
timeoutMs = 5000)
    {
        var start = Environment.TickCount64;
        while (Environment.TickCount64 - start < timeoutMs)
        {
            if (condition()) return true;
            await Task.Delay(50, TestContext.Current.CancellationToken);
        }
        return false;
    }
}
```

PythonVenvRunnerTests.cs

```
using System;
using System.Collections.Concurrent;
using System.IO;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using kernel.Hubs;
using kernel.Services;
using Microsoft.Extensions.Logging.Abstractions;
using Pty.Net;
using Xunit;
namespace kernel.Tests.Services;
```

```
public class PythonVenvRunnerTests
{
    private static PythonVenvRunner CreateService(out FakeHubContext<PythonHub> hubContext)
    {
        hubContext = new FakeHubContext<PythonHub>();
        return new PythonVenvRunner(NullLogger<PythonVenvRunner>.Instance, hubContext);
    }
    [Fact]
    public async Task CreateVenvAsync_WhenVenvExists_DoesNotThrow()
    {
        using var tempDir = new TempDirectory();
        var venvPath = Path.Combine(tempDir.Path, "venv");
        Directory.CreateDirectory(venvPath);
        await File.WriteAllTextAsync(Path.Combine(venvPath, "pyvenv.cfg"), "home = test", TestContext.Current.CancellationToken);
        var service = CreateService(out _);
        var exception = await Record.ExceptionAsync(() => service.CreateVenvAsync(venvPath, "python"));
        Assert.Null(exception);
    }
    [Fact]
    public async Task CreateVenvAsync_InvalidPythonPath_Throws()
    {
        using var tempDir = new TempDirectory();
        var service = CreateService(out _);
        var venvPath = Path.Combine(tempDir.Path, "venv");
        await Assert.ThrowsAnyAsync<Exception>(() =>
            service.CreateVenvAsync(venvPath, @"Z:\missing\python.exe"));
    }
    [Fact]
    public async Task CreateVenvAsync_ProcessFailure_Throws()
    {
        using var tempDir = new TempDirectory();
        var service = CreateService(out _);
        var venvPath = Path.Combine(tempDir.Path, "venv");
        await Assert.ThrowsAnyAsync<Exception>(() =>
            service.CreateVenvAsync(venvPath, "powershell.exe"));
    }
    [Fact]
    public async Task WriteInputAsync_UnknownTerminal_DoesNotThrow()
    {
        var service = CreateService(out _);
        var exception = await Record.ExceptionAsync(() => service.WriteInputAsync("missing", "abc"));
        Assert.Null(exception);
    }
```

```
    [Fact]
    public async Task WriteInputAsync_KnownTerminal_WritesBytesAndFlushes()
    {
        var service = CreateService(out _);
        var terminal = new FakePtyConnection { WriterStream = new TrackingStream() };
        var terminals = ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string, IPtyConnection>>(service, "_terminals");
        terminals["t1"] = terminal;
        await service.WriteInputAsync("t1", "hello");
        var writer = Assert.IsType<TrackingStream>(terminal.WriterStream);
        Assert.Equal("hello", Encoding.UTF8.GetString(writer.ToArray()));
        Assert.True(writer.FlushCalled);
    }
    [Fact]
    public void Resize_KnownTerminal_CallsResize()
    {
        var service = CreateService(out _);
        var terminal = new FakePtyConnection();
        var terminals = ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string, IPtyConnection>>(service, "_terminals");
        terminals["t1"] = terminal;
        service.Resize("t1", 120, 40);
        Assert.Equal(120, terminal.LastCols);
        Assert.Equal(40, terminal.LastRows);
    }
    [Fact]
    public void KillTerminal_WithEntries_DisposesAndCancels()
    {
        var service = CreateService(out _);
        var terminal = new FakePtyConnection();
        var cts = new CancellationTokenSource();
        var terminals = ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string, IPtyConnection>>(service, "_terminals");
        var ctsMap = ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string, CancellationTokenSource>>(service, "_cts");
        terminals["t1"] = terminal;
        ctsMap["t1"] = cts;
        service.KillTerminal("t1");
        Assert.True(terminal.Disposed);
        Assert.True(cts.IsCancellationRequested);
        Assert.False(terminals.ContainsKey("t1"));
        Assert.False(ctsMap.ContainsKey("t1"));
    }
```

```
    [Fact]
    public void KillTerminal_UnknownTerminal_DoesNotThrow()
    {
        var service = CreateService(out _);
        var exception = Record.Exception(() =>
service.KillTerminal("missing"));
        Assert.Null(exception);
    }
    [Fact]
    public async Task ExecuteCodeAsync_WhenSpawnFails_SendsReceiveResult()
    {
        using var tempDir = new TempDirectory();
        var venvPath = Path.Combine(tempDir.Path, "venv");
        Directory.CreateDirectory(venvPath);
        await File.WriteAllTextAsync(Path.Combine(venvPath, "pyvenv.cfg"),
"home = test", TestContext.Current.CancellationToken);
        var service = CreateService(out var hubContext);
        await service.ExecuteCodeAsync("conn1", "t1", "print('hi')",
"ignored-python", venvPath);
        var client = hubContext.ClientsImpl.GetClient("conn1");
        Assert.Contains(client.Invocations, i => i.Method ==
"ReceiveResult");
    }
    [Fact]
    public async Task ExecuteCodeAsync_WhenCreateVenvFails_Throws()
    {
        using var tempDir = new TempDirectory();
        var service = CreateService(out var hubContext);
        var venvPath = Path.Combine(tempDir.Path, "venv");
        await Assert.ThrowsAnyAsync<Exception>(() =>
            service.ExecuteCodeAsync("conn1", "t1", "print('x')",
@"Z:\missing\python.exe", venvPath));
        var client = hubContext.ClientsImpl.GetClient("conn1");
        Assert.DoesNotContain(client.Invocations, i => i.Method ==
"ReceiveResult");
    }
}
```

TerminalServiceTests.cs

```
using System;
using System.Collections.Concurrent;
using System.IO;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using kernel.Hubs;
using kernel.Services;
using Microsoft.Extensions.Logging.Abstractions;
```

```
using Pty.Net;
using Xunit;
namespace kernel.Tests.Services;
public class TerminalServiceTests
{
    private static TerminalService CreateService(out
FakeHubContext<TerminalHub> hubContext)
    {
        hubContext = new FakeHubContext<TerminalHub>();
        return new TerminalService(NullLogger<TerminalService>.Instance,
hubContext);
    }
    [Fact]
    public async Task WriteInputAsync_UnknownTerminal_DoesNotThrow()
    {
        var service = CreateService(out _);
        var exception = await Record.ExceptionAsync(() =>
service.WriteInputAsync("missing", "abc"));
        Assert.Null(exception);
    }
    [Fact]
    public async Task WriteInputAsync_KnownTerminal_WritesBytesAndFlushes()
    {
        var service = CreateService(out _);
        var terminal = new FakePtyConnection { WriterStream = new
TrackingStream() };
        var terminals =
ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string,
IPtyConnection>>(service, "_terminals");
        terminals["t1"] = terminal;
        await service.WriteInputAsync("t1", "hello");
        var writer = Assert.IsType<TrackingStream>(terminal.WriterStream);
        Assert.Equal("hello", Encoding.UTF8.GetString(writer.ToArray()));
        Assert.True(writer.FlushCalled);
    }
    [Fact]
    public void Resize_UnknownTerminal_DoesNotThrow()
    {
        var service = CreateService(out _);
        var exception = Record.Exception(() => service.Resize("missing",
100, 50));
        Assert.Null(exception);
    }
    [Fact]
    public void Resize_KnownTerminal_CallsResize()
    {
        var service = CreateService(out _);
        var terminal = new FakePtyConnection();
```

```
51          var terminals =
    ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string,
    IPtyConnection>>(service, "_terminals");
52          terminals["t1"] = terminal;
53          service.Resize("t1", 120, 40);
54          Assert.Equal(120, terminal.LastCols);
55          Assert.Equal(40, terminal.LastRows);
56      }
57      [Fact]
58      public void KillTerminal_UnknownTerminal_DoesNotThrow()
59      {
60          var service = CreateService(out _);
61          var exception = Record.Exception(() =>
    service.KillTerminal("missing"));
62          Assert.Null(exception);
63      }
64      [Fact]
65      public void KillTerminal_WithTerminalAndCts_DisposesAndCancels()
66      {
67          var service = CreateService(out _);
68          var terminal = new FakePtyConnection();
69          var cts = new CancellationTokenSource();
70          var terminals =
    ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string,
    IPtyConnection>>(service, "_terminals");
71          var ctsMap =
    ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string,
    CancellationTokenSource>>(service, "_cts");
72          terminals["t1"] = terminal;
73          ctsMap["t1"] = cts;
74          service.KillTerminal("t1");
75          Assert.True(terminal.Disposed);
76          Assert.True(cts.IsCancellationRequested);
77          Assert.False(terminals.ContainsKey("t1"));
78          Assert.False(ctsMap.ContainsKey("t1"));
79      }
80      [Fact]
81      public void KillTerminal_WithOnlyTerminal_DisposesTerminal()
82      {
83          var service = CreateService(out _);
84          var terminal = new FakePtyConnection();
85          var terminals =
    ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string,
    IPtyConnection>>(service, "_terminals");
86          terminals["t1"] = terminal;
87          service.KillTerminal("t1");
88          Assert.True(terminal.Disposed);
89          Assert.False(terminals.ContainsKey("t1"));
```

```
    }
    [Fact]
    public void KillTerminal_WithOnlyCts_CancelsAndDisposesToken()
    {
        var service = CreateService(out _);
        var cts = new CancellationTokenSource();
        var ctsMap = ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string, CancellationTokenSource>>(service, "_cts");
        ctsMap["t1"] = cts;
        service.KillTerminal("t1");
        Assert.True(cts.IsCancellationRequested);
        Assert.False(ctsMap.ContainsKey("t1"));
    }
    [Fact]
    public void KillTerminal_WhenTerminalDisposeThrows_DoesNotThrow()
    {
        var service = CreateService(out _);
        var terminal = new FakePtyConnection { ThrowOnDispose = true };
        var terminals = ReflectionTestHelper.GetPrivateField<ConcurrentDictionary<string, IPtyConnection>>(service, "_terminals");
        terminals["t1"] = terminal;
        var exception = Record.Exception(() => service.KillTerminal("t1"));
        Assert.Null(exception);
    }
    [Fact]
    public async Task StartTerminalAsync_InvalidWorkingDirectory_Throws()
    {
        var service = CreateService(out _);
        var invalidPath = Path.Combine(Path.GetTempPath(), "kernel_missing_" + Guid.NewGuid().ToString("N"));
        var exception = await Record.ExceptionAsync(() =>
            service.StartTerminalAsync("t1", "conn1", "TerminalOutput", invalidPath));
        Assert.NotNull(exception);
    }
}
```

TestDoubles.cs

```
#nullable enable
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.IO;
using System.Reflection;
using System.Threading;
using System.Threading.Tasks;
```

```
using Microsoft.AspNetCore.SignalR;
using Pty.Net;
using Xunit;
namespace kernel.Tests.Services;
internal sealed class FakeHubContext<THub> : IHubContext<THub> where THub :
Hub
{
    public FakeHubClients ClientsImpl { get; } = new();
    public IHubClients Clients => ClientsImpl;
    public IGroupManager Groups { get; } = new FakeGroupManager();
}
internal sealed class FakeHubClients : IHubClients
{
    private readonly ConcurrentDictionary<string, FakeClientProxy> _clients
= new();
    private readonly FakeClientProxy _broadcastClient = new();
    public FakeClientProxy GetClient(string connectionId) =>
_clients.GetOrAdd(connectionId, _ => new FakeClientProxy());
    public IClientProxy All => _broadcastClient;
    public IClientProxy AllExcept(IReadOnlyList<string>
excludedConnectionIds) => _broadcastClient;
    public IClientProxy Client(string connectionId) =>
GetClient(connectionId);
    public IClientProxy Clients(IReadOnlyList<string> connectionIds) =>
_broadcastClient;
    public IClientProxy Group(string groupName) => _broadcastClient;
    public IClientProxy GroupExcept(string groupName, IReadOnlyList<string>
excludedConnectionIds) => _broadcastClient;
    public IClientProxy Groups(IReadOnlyList<string> groupNames) =>
_broadcastClient;
    public IClientProxy User(string userId) => _broadcastClient;
    public IClientProxy Users(IReadOnlyList<string> userIds) =>
_broadcastClient;
}
internal sealed class FakeGroupManager : IGroupManager
{
    public Task AddToGroupAsync(string connectionId, string groupName,
CancellationToken cancellationToken = default) =>
        Task.CompletedTask;
    public Task RemoveFromGroupAsync(string connectionId, string groupName,
        CancellationToken cancellationToken = default) =>
Task.CompletedTask;
}
internal sealed class FakeClientProxy : IClientProxy
{
    public ConcurrentQueue<HubInvocation> Invocations { get; } = new();
    public Task SendCoreAsync(string method, object?[] args,
CancellationToken cancellationToken = default)
```

```
    {
        Invocations.Enqueue(new HubInvocation(method, args,
cancellationToken));
        return Task.CompletedTask;
    }
}
internal sealed record HubInvocation(string Method, object?[] Args,
CancellationToken CancellationToken);
internal sealed class TrackingStream : MemoryStream
{
    public bool FlushCalled { get; private set; }
    public override void Flush()
    {
        FlushCalled = true;
        base.Flush();
    }
    public override Task FlushAsync(CancellationToken cancellationToken)
    {
        FlushCalled = true;
        return base.FlushAsync(cancellationToken);
    }
}
internal sealed class FakePtyConnection : IPtyConnection
{
    public Stream ReaderStream { get; set; } = new MemoryStream();
    public Stream WriterStream { get; init; } = new TrackingStream();
    public int Pid => 1234;
    public int ExitCode { get; private set; }
    public bool Disposed { get; private set; }
    public bool KillCalled { get; private set; }
    public int LastCols { get; private set; }
    public int LastRows { get; private set; }
    public bool ThrowOnDispose { get; init; }
    public event EventHandler<PtyExitedEventArgs>? ProcessExited;
    public bool WaitForExit(int timeout) => true;
    public void Kill()
    {
        KillCalled = true;
    }
    public void Resize(int cols, int rows)
    {
        LastCols = cols;
        LastRows = rows;
    }
    public void Dispose()
    {
        if (ThrowOnDispose)
        {
```

```
            throw new InvalidOperationException("dispose failure");
        }
        Disposed = true;
    }
}
internal static class ReflectionTestHelper
{
    public static TField GetPrivateField<TField>(object target, string fieldName)
    {
        var field = target.GetType().GetField(fieldName, BindingFlags.Instance | BindingFlags.NonPublic);
        Assert.NotNull(field);
        return (TField)field.GetValue(target)!;
    }
}
```

kernel.Tests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>

    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="JetBrains.Annotations" Version="2025.2.4" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.0.1" />
    <PackageReference Include="xunit.v3" Version="3.2.2" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="..\kernel\kernel.csproj" />
  </ItemGroup>

</Project>
```

5. Testing

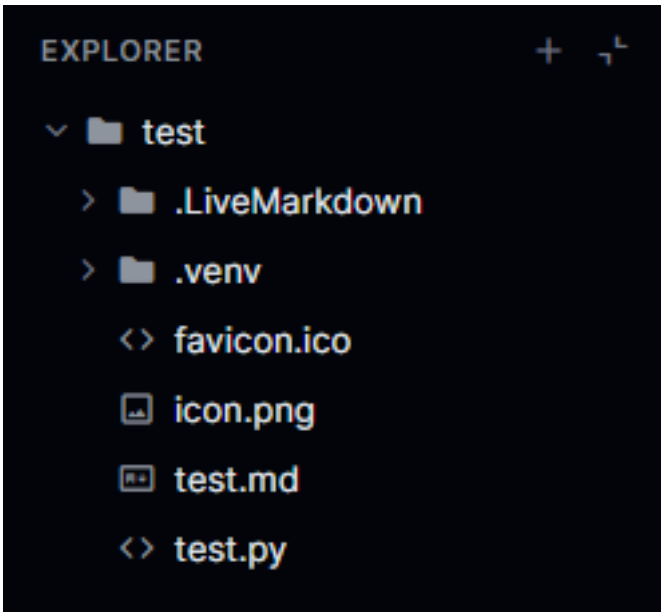


Figure 5.1 This screenshot demonstrates that the file explorer is functional.



Figure 5.2 This screenshot demonstrates that the project-wide file search is functional.

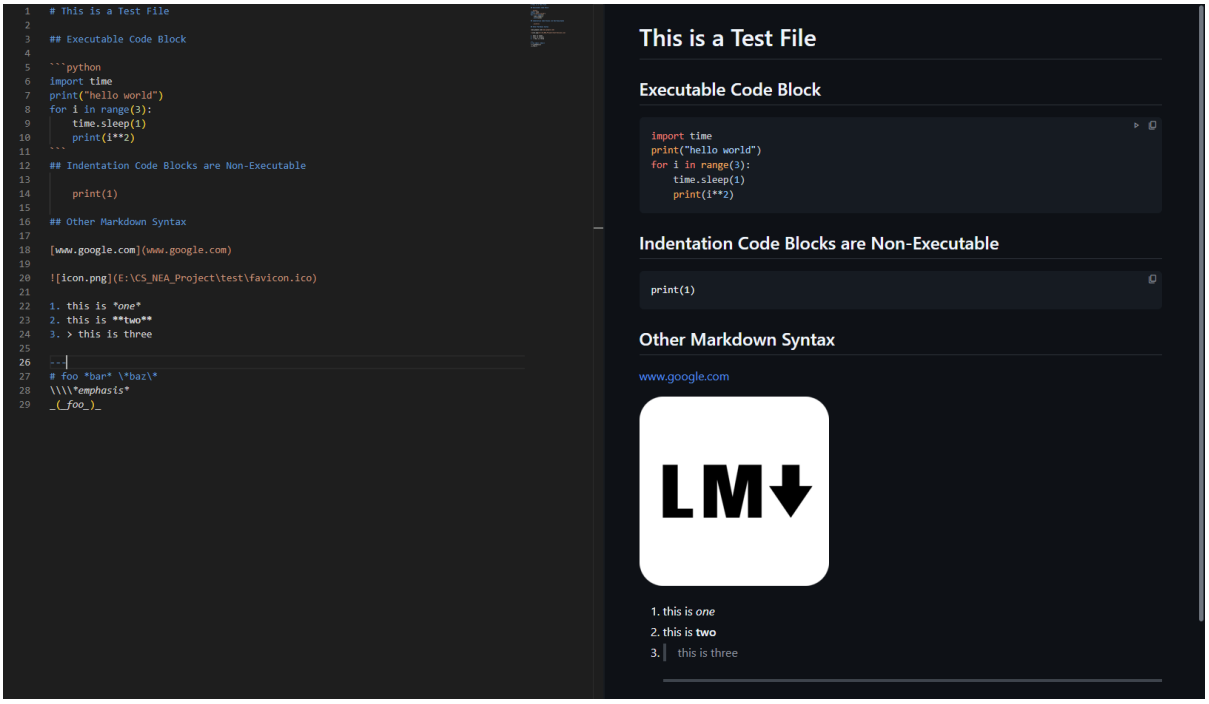


Figure 5.3 This screenshot demonstrates that Markdown is correctly parsed and rendered.



Figure 5.4 This screenshot demonstrates that the code block is executable.



Figure 5.5 This screenshot demonstrates that the terminal is functional.

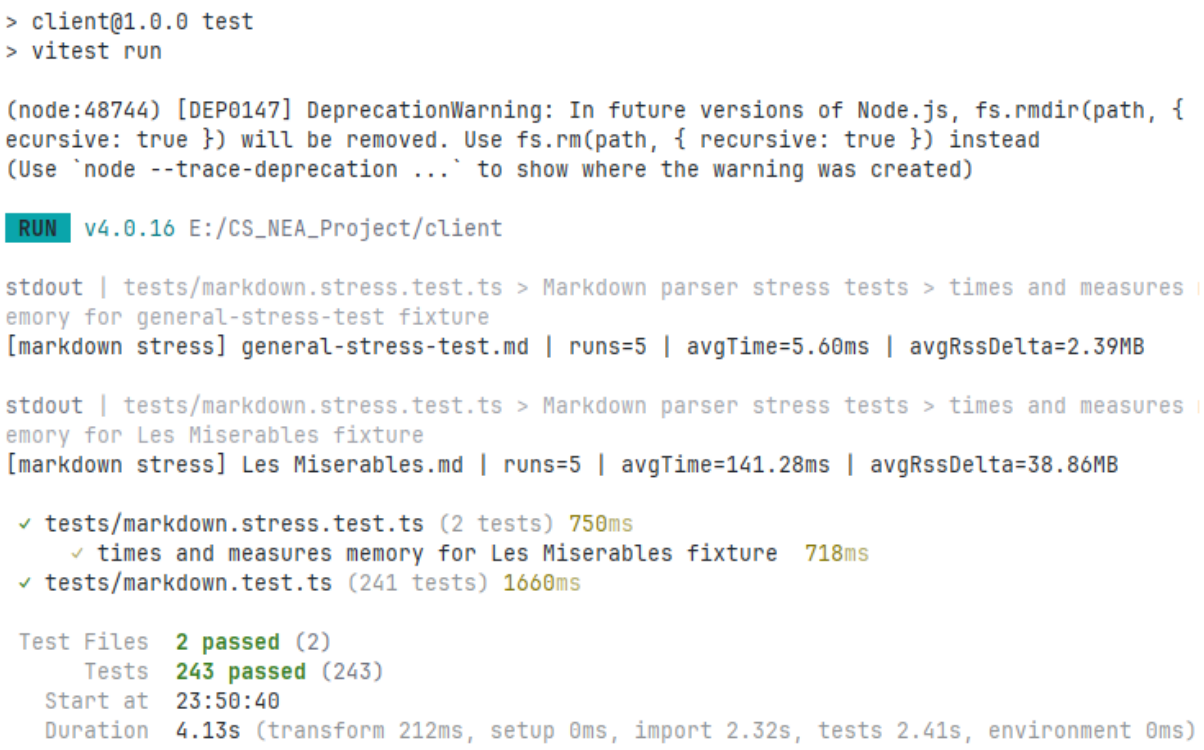


Figure 5.6 Unit and stress tests for the Markdown Parser.

Description	Test Case	Excepted	Status

Mixed u-list markers should parse as separate lists	`- foo`
	`- bar`
	`+ baz`
		`<ul><li>foo</li><li>bar</li></ul>`
		`<ul><li>baz</li></ul>`
			Passed

Heading with tab after the # is correctly parsed. Inline emphasis is parsed and escaped asterisks	`#  foo  *bar*` `\*baz\*`	`<h1>foo <em>bar</em> *baz*</h1>`	Passed

remain literal text.			
Escaped backslashes are rendered as literal backslash. Emphasis after escape sequence is correctly parsed.	`\\\\*emphasis*`	`<p>\\<em>emphasis</em></p>`	Passed

Blockquote is parsed as a standalone block element.	`foo`
	`> bar`
		`<p>foo</p>`
		`<blockquote>`
		`<p>bar</p>`
		`</blockquote>`
			Passed

Unmatched nested emphasis delimiters are treated as literal text.	`*(*foo)`	`<p>*(*foo)</p>`	Passed
Nested emphasis produces nested `<em>` elements.	`_(_foo_)_`	`<p><em>(<em>foo</em>)</em></p>`	Passed
Nested emphasis and strong emphasis within a single emphasis span are parsed correctly.	`*f**b**z*`	`<p><em>f<strong>b</strong>z</em></p>`	Passed
Code span is parsed correctly	`` ` a` ``	`<p><code> a</code></p>`	Passed
Whitespace between link and destination prevents parsing	`[link] (/uri)`	`p>[link] (/uri)</p>\n`	Passed
Unicode characters preserved when parsing	`Foo xpϱv`	`p>Foo xpϱv</p>`	Passed

The stress suite (`client/tests/markdown.stress.test.ts`) runs each fixture 5 times, measuring:

- Average parse time (avgTime)
- Average RSS memory delta (avgRssDelta)

Fixture	Characters	Runs	Avg. Time	Avg. RSS Delta
`general-stress-test.md`	30370	5	5.60ms	2.39MB
`Les Miserables.md`	3188558	5	141.28ms	38.86MB

More information about the tests can be found in `client/tests/` folder in the repository.

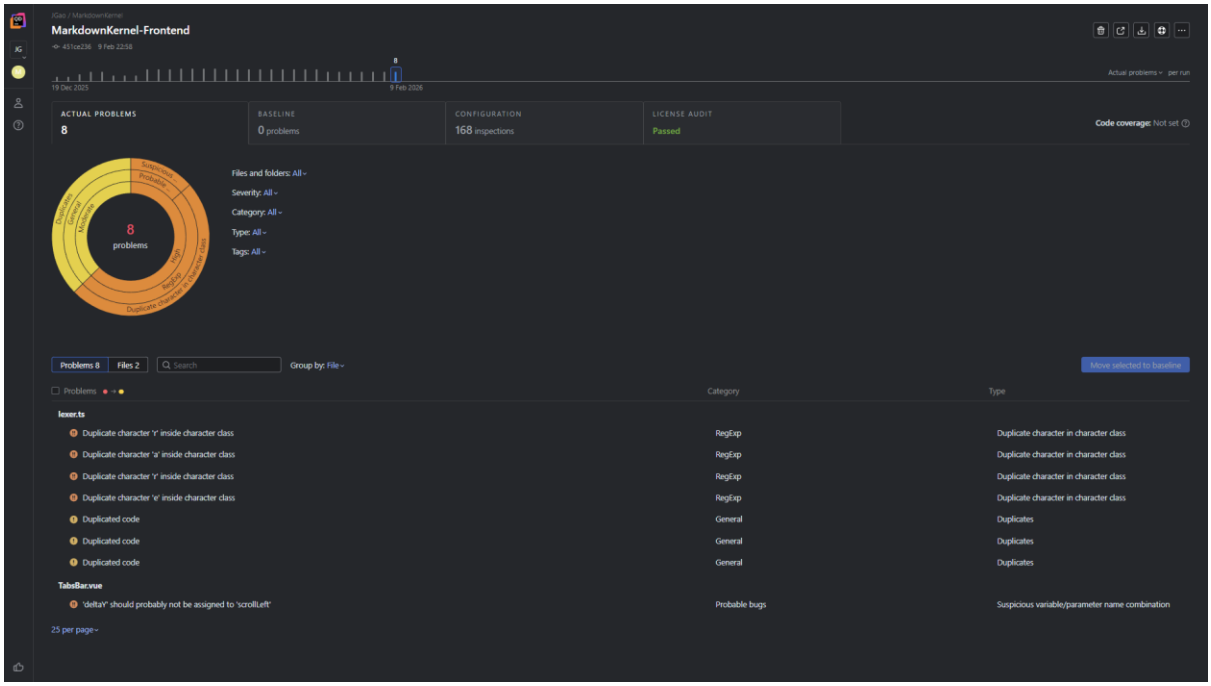


Figure 5.7 Qodana Code Analysis report for the client [23].

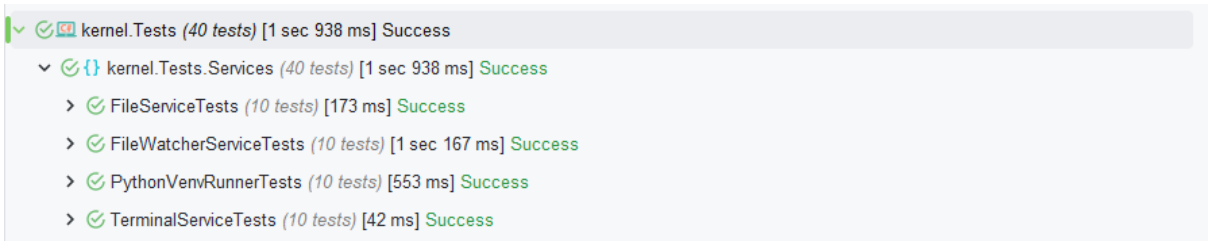


Figure 5.8 Unit tests for the kernel.

Description	Test Case	Expected	Status
File streaming	`StreamTabAsync_ReturnsMetadataThenContent`	First chunk is metadata; remaining chunks reconstruct file content.	Passed
Directory reading nested	`ReadDirAsync_ReturnsEntriesFromNestedDirectory`	Includes nested child file in results.	Passed
Start watcher invalid path	`StartWatching_WhitespacePath_ReturnsFalse`	Returns false.	Passed
File created event	`CreatedEvent_RaisesDirectoryDirtyForParent`	DirectoryDirty raised for root/parent directory.	Passed
Rename across dirs event	`RenamedAcrossDirectories_RaisesDirtyForOldAndNewParents`	DirectoryDirty raised for both old and new parent directories.	Passed
Terminal input	`WriteInputAsync_KnownTerminal_WritesBytesAndFlushes`	Writes UTF-8 bytes and flushes writer stream.	Passed
Python invalid interpreter	`CreateVenvAsync_InvalidPythonPath_Throws`	Throws exception.	Passed
Save file	`SaveFileAsync_OverwritesExistingFile`	Existing file content is overwritten.	Passed

Terminal kill terminal+cts	`KillTerminal_WithTerminalAndCts_DisposesAndCancels`	Disposes terminal, cancels CTS, removes both entries.	Passed
Start terminal invalid cwd	`StartTerminalAsync_InvalidWorkingDirectory_Throws`	Throws exception for bad working directory.	Passed

More information about the tests can be found in `kernel/kernel.Tests/` folder in the repository.

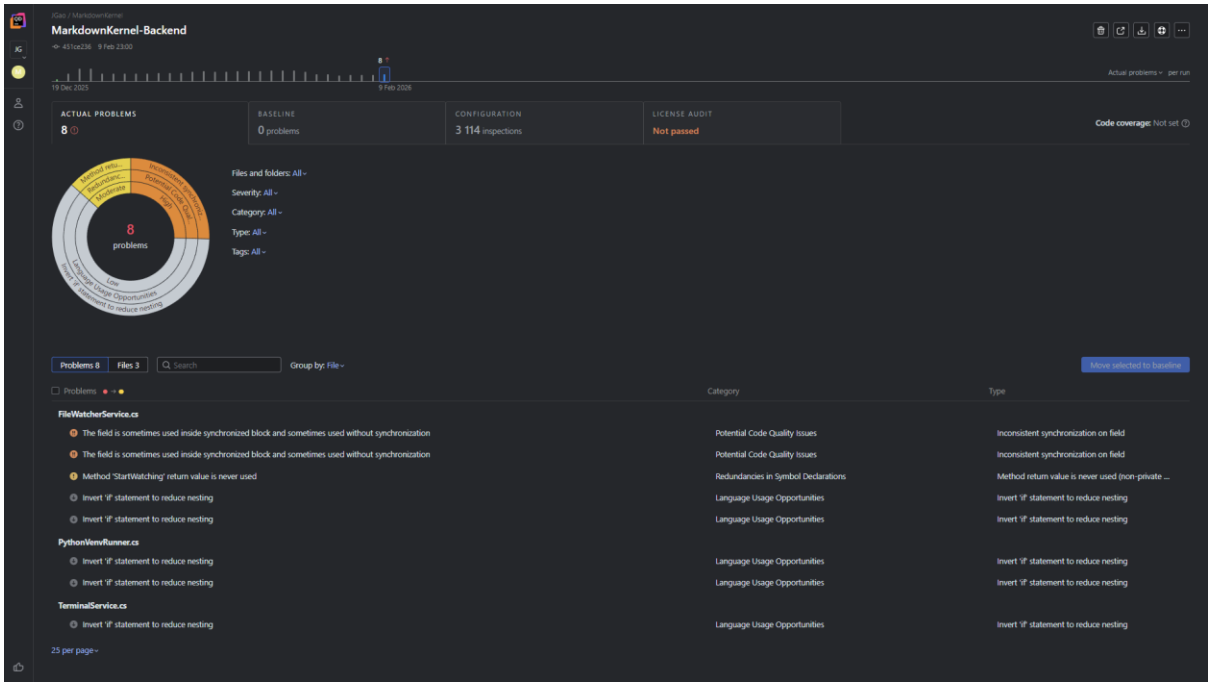


Figure 5.9 Qodana Code Analysis report for the kernel. Licence audit failed since Qodana could not verify the Pty.Net licence from Azure DevOps. [24].

6. Evaluation

6.1. What I Have Achieved

In summary, I have developed a Markdown editor capable of executing code, achieving thread isolation between the frontend and kernel. The editor features interactive GUI, basic file management, virtual environment creation, customised Markdown parsing, code execution, syntax highlighting, built-in terminals, and all the features mentioned in Chapter 1.8, thereby providing the core functionality of a basic IDE.

6.2. What Can Be Improved

1. **GUI** – Instead of creating my own GUI, I should fork the VS Code source code and

modify it to incorporate my Markdown parser and kernel. This would result in a more streamlined UI and a better user experience.

2. **Pty.Net** – Instead of using the deprecated Pty.Net package, I should use the native Windows ConPTY API directly and create my own read/write pipes to implement a pseudo-terminal. This would avoid the DLL dependency issues associated with Pty.Net.
3. **File Selection Dialogue** – Currently, my project uses a basic dialogue box for opening files and selecting directories, in which users must enter the file path directly. To improve the user experience, I should use Electron's built-in `dialog` module to bring up a file selection dialogue.
4. **Markdown Parser** – Due to a lack of experience, my Markdown parser employs a hybrid approach, combining regular expressions and state machines, and the parsing of inline elements relies solely on regular expressions. While this reduces the length of the code, it significantly increases the complexity of the regular expressions, making them difficult to debug and limiting the scalability of the functionality. To minimise side effects and make parsing more deterministic, I should rewrite my parser using a pure state machine architecture to achieve full compatibility with the CommonMark specification.

6.3. User Feed Back

I invited some student volunteers to try out my Markdown editor and distributed a questionnaire to gather some feedback.

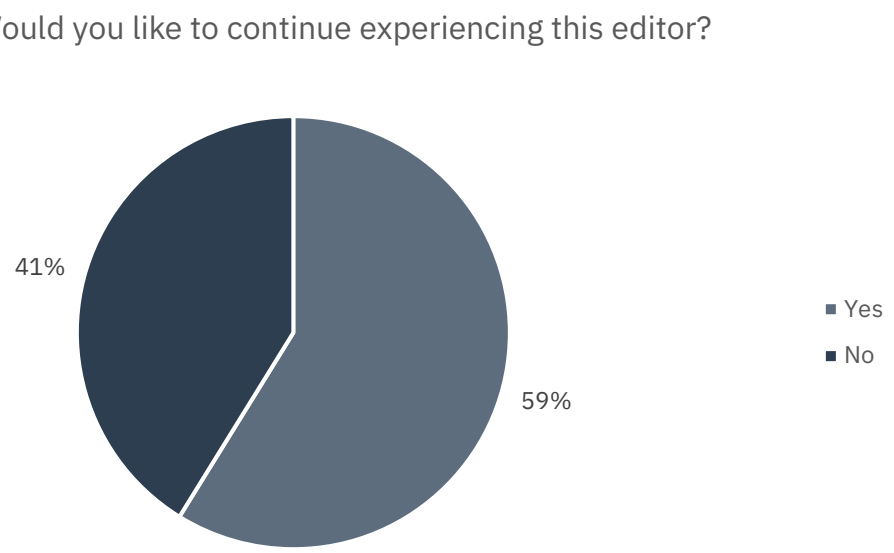


Figure 6.1 This chart illustrates the proportion of students who are willing to continue experiencing this Markdown editor

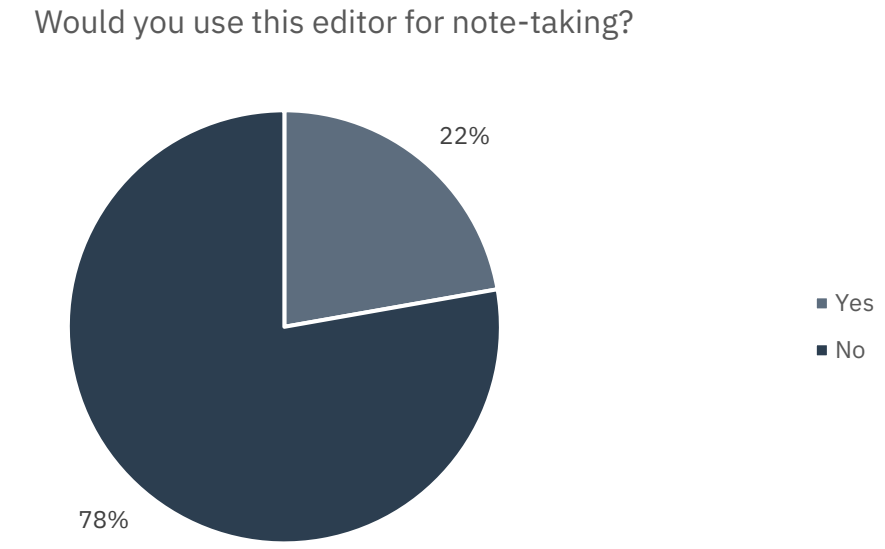


Figure 6.2 This chart illustrates the proportion of students who are interested in continuing their experience with this editor and are willing to use it for their daily note-taking.

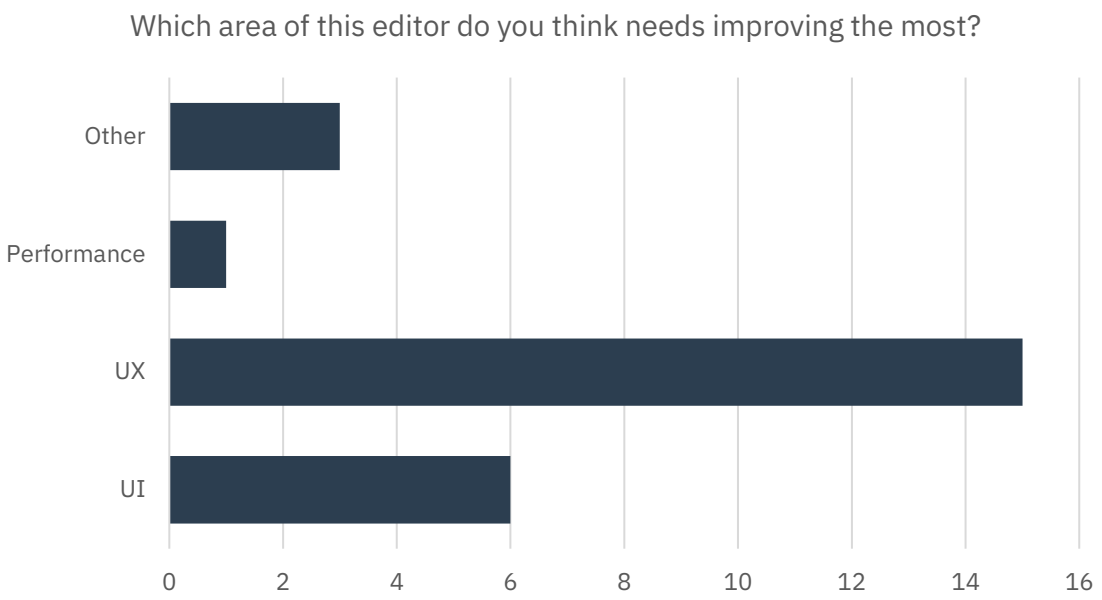


Figure 6.3 This chart shows the distribution of students by their suggestions for improving the editor.

These figures show that, although most people are willing to continue using this editor, they do not feel that its features are robust enough to support their daily work. User experience is the area of functionality that needs the most improvement. The data of the questionnaire support my analysis presented in Chapter 6.2.

Use of AI

This document has been proofread using DeepL's grammar checker.

Due to updates on GitHub Copilot, I have lost the majority of my chat history and agent sessions. Therefore, I can only indicate which parts of the code use AI without providing screenshots as evidence.

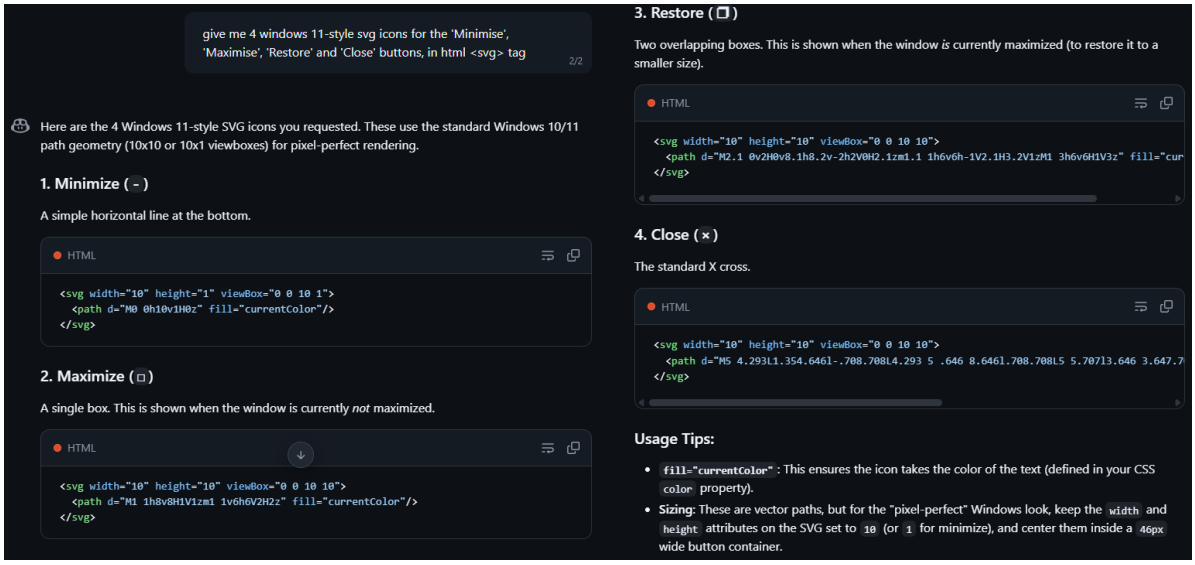


Figure c Model: Gemini 3 Pro, Platform: GitHub Copilot

The files / lines listed below were written with the assist of AI. Models: Claude 4 Opus, Claude 4.5 Opus, Claude 3.7 Sonnet (thinking), Claude 4 Sonnet, Gemini 3 Pro, GPT-5.1-Codex-Max, GPT-5.2-Codex, GPT-5.3-Codex. Platform: GitHub Copilot, GitHub Copilot CLI.

- `ContextMenu.vue`
- `Editor.vue`
- `FileExplorer.vue`
- `NavigationBar.vue`
- `SearchPanel.vue`, Lines 76-88, 115-210
- `TabsBar.vue`, Lines 37-48, 53-212
- `Terminal.vue`, Lines 292-417
- `workspace.ts`, Lines 164-170, 199-206, 337-389, 512-609, 770-785, 883-888, 1030-1038, 1127-1196
- `style.css`
- `App.vue`, Lines 40-59, 131-227,
- `client/main.ts`, Lines 22, 28, 96-160

- `markdown.test.ts`, Lines 12-21
- `TestDoubles.cs`

Bibliography

[1] J. M. a. J. Atwood, “CommonMark Spec,” [Online]. Available: https://spec.commonmark.org/. [Accessed 6 3 2026].

[2] GitHub, “GitHub Flavored Markdown Spec,” [Online]. Available: https://github.github.com/gfm/. [Accessed 6 3 2026].

[3] Obsidian, “Obsidian Flavoured Markdown,” [Online]. Available: https://help.obsidian.md/obsidian-flavored-markdown. [Accessed 6 3 2026].

[4] Microsoft, “Visual Studio Code,” [Online]. Available: https://code.visualstudio.com/. [Accessed 6 3 2026].

[5] Project Juptyer, “Project Jupyter,” [Online]. Available: https://jupyter.org/. [Accessed 5 3 2026].

[6] D. Terzioglu, “jupymd,” [Online]. Available: https://github.com/d-eniz/jupymd. [Accessed 6 3 2026].

[7] G. Poore, “codebraid-preview-vscode,” [Online]. Available: https://github.com/gpoore/codebraid-preview-vscode. [Accessed 6 3 2026].

[8] J. MacFarlane, “Pandoc,” [Online]. Available: https://pandoc.org/MANUAL.html. [Accessed 6 3 2026].

[9] Microsoft, “SignalR,” [Online]. Available: https://github.com/signalR/SignalR. [Accessed 6 3 2026].

[10] Python Software Foundation, “Python,” [Online]. Available: https://www.python.org/. [Accessed 6 3 2026].

[11] Python Software Foundation, “cpython,” [Online]. Available:

https://github.com/python/cpython. [Accessed 9 3 2026].

[12] Microsoft, “monaco-editor,” [Online]. Available: https://github.com/microsoft/monaco-editor. [Accessed 25 3 2026].

[13] Microsoft, “The lock statement - ensure exclusive access to a shared resource,” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/lock. [Accessed 16 2 2026].

[14] Microsoft, “Task Class,” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.task?view=net-9.0. [Accessed 16 2 2026].

[15] Microsoft, “IAsyncEnumerable<T> Interface,” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.iasyncenumerable-1?view=net-9.0. [Accessed 16 2 2026].

[16] Microsoft, “ChannelReader<T> Class,” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/system.threading.channels.channelreader-1?view=net-9.0. [Accessed 16 2 2026].

[17] Microsoft, “Task<TResult> Class,” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/system.threading.tasks.task-1?view=net-9.0. [Accessed 16 2 2026].

[18] 304NotModified, “UTF-unknown,” [Online]. Available: https://github.com/CharsetDetector/UTF-unknown. [Accessed 26 2 2026].

[19] Microsoft, “volatile keyword,” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/volatile. [Accessed 26 2 2026].

[20] Microsoft, “GC.SuppressFinalize(Object) Method,” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/system.gc.suppressfinalize?view=net-9.0. [Accessed 26 2 2026].

[21] Mozilla, “WeakMap,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/WeakMap. [Accessed 26 2 2026].

[22] sindresorhus, “github-markdown-css 5.9.0,” [Online]. Available:

https://github.com/sindresorhus/github-markdown-css/releases/tag/v5.9.0. [Accessed 28 2 2026].

[23] JetBrains, “Qodana,” [Online]. Available: https://www.jetbrains.com/qodana/. [Accessed 3 3 2026].

[24] Microsoft, “Pty.Net 0.3.94-pre,” [Online]. Available: https://dev.azure.com/azure-public/vside/_artifacts/feed/msft_consumption/NuGet/Pty.Net/overview/0.3.94-pre. [Accessed 3 3 2026].

Appendix A – Source Code Repository

The source code for this project is available at the following GitHub repository:

https://github.com/Jonah-Gao/LiveMarkdown

The version used for this report corresponds to release v1.0.2.